ETH zürich

ETH Library

Perceptive Locomotion for Legged Robots in Rough Terrain

Doctoral Thesis

Author(s):
Fankhauser, Péter

Publication date:
2018

Permanent link:
https://doi.org/https://doi.org/10.3929/ethz-b-000284254

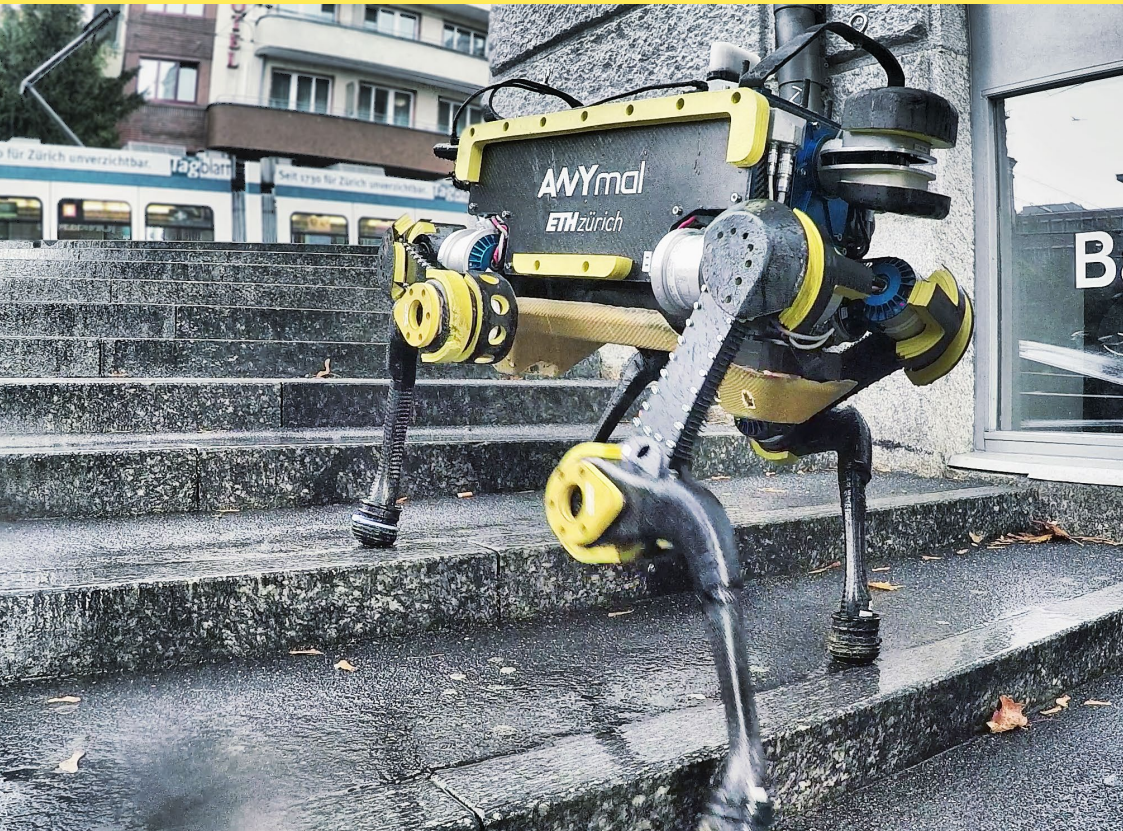
Rights / license:
In Copyright - Non-Commercial Use Permitted

This page was generated automatically upon download from the ETH Zurich Research Collection.
For more information, please consult the Terms of use.

2018
DISS. ETH NO. 24849

Péter Fankhauser

Perceptive Locomotion for Legged Robots in Rough Terrain



DISS. ETH NO. 24849

Perceptive Locomotion for Legged Robots in Rough Terrain

A thesis submitted to attain the degree of

DOCTOR OF SCIENCES of ETH ZURICH

(Dr. sc. ETH Zurich)

presented by

PÉTER FANKHAUSER

MSc ETH in Robotics, Systems and Control

born on 09.01.1987

citizen of Eggiwil BE, Switzerland

accepted on the recommendation of

Prof. Dr. Roland Y. Siegwart, ETH Zurich
Prof. Dr. Marco Hutter, ETH Zurich
Dr. Jerry Pratt, Florida Institute for Human & Machine Cognition

2018

Autonomous Systems Lab & Robotic Systems Lab
Institute for Robotics and Intelligent Systems
ETH Zurich
Switzerland

© 2018 Péter Fankhauser. All rights reserved.

Abstract

Robotic technologies will continue to enter new applications in addition to automated manufacturing and logistics. Once mobile robots can also operate outside of today's special facilities, they have the potential to relieve us of dirty and dangerous labor in various areas. However, for this purpose, these machines will need to be able to navigate autonomously in complex natural, urban, and industrial settings. This thesis addresses the development of locomotion skills for legged robots in challenging environments. Our work focuses on perceptive locomotion where exteroceptive sensing of the surrounding is exploited to plan and control the robots motion. This enables quadrupedal robots to negotiate rough terrain through carefully selected contacts.

In this work, we evaluate different sensing technologies and analyze their performance for local dense terrain mapping on a mobile robot. We include special conditions such as close range objects and the influence of ambient light as we find them in real-world applications. By modeling the error characteristics of the sensors, the robot can judge the quality of the resulting terrain reconstruction.

As the robot moves, the surrounding is continuously mapped to capture new areas and update regions which have changed. We contribute with a mapping framework that models the terrain from a robot-centric perspective. To this end, we present a novel approach for the error propagation from the robot's state estimation to the representation of the map. This formulation allows for robust and high-rate local mapping that is independent of a global localization method.

We introduce our approach to locomotion planning, which finds safe footholds along with collision-free swing-leg motions, leveraging the generated terrain map. A nonlinear optimization finds postures that respect kinematic and stability constraints. We experimentally verify this work with torque-controllable quadrupedal robots that autonomously traverse obstacles, such as rubble, steps, gaps, and stairs without prior knowledge of the scene or any addi-

tional equipment. The locomotion planner re-plans its motion at every step in real-time, to cope with disturbances and dynamic environments.

For the control of the legged robot, we contribute architecturally to the versatile and task-oriented motion execution. This method enables the robust tracking of motion plans, even with significant mismatches between the models and reality. In addition to rough terrain locomotion, we demonstrate the integration of our method for applications, such as whole-body stair climbing, manipulation, jumping, docking, inspection, payload delivery, dancing, and more.

Our approach is thoroughly validated with the quadrupedal robot ANYmal in realistic long-term missions for autonomous industrial inspection and search and rescue. Finally, we extend our work with the design and implementation of a collaborative navigation framework for ground and flying robots. The ground vehicle utilizes the data captured by the flying robot to navigate uncharted environments efficiently.

Zusammenfassung

Robotiktechnologien werden neben der automatisierten Herstellung und Logistik zunehmend Einzug in neue Anwendungsbereiche halten. Sobald mobile Roboter in der Lage sind, auch ausserhalb von heutigen Spezialanlagen zu agieren, haben sie das Potential, uns bei der Bewältigung von verschiedensten schmutzigen und gefährlichen Arbeiten zu entlasten. Dazu müssen die Roboter aber die Fähigkeit besitzen, autonom in komplexen, natürlichen, sowie urbanen und industriellen Umgebungen zu navigieren. Diese Dissertation beschäftigt sich mit der Entwicklung der Bewegungsfähigkeit vierbeiniger Roboter in anspruchsvollem Gelände. Der Fokus liegt dabei auf der wahrnehmenden Fortbewegung, in welcher die Umgebungserfassung für die Planung und Regelung der Bewegung genutzt wird. Dies ermöglicht vierbeinigen Robotern, sich mit Hilfe von einzelnen Kontaktpunkten über unwegsamen Untergrund zu bewegen.

In der vorliegenden Arbeit wurden verschiedene Sensortechnologien evaluiert und deren Eignung für die lokale und flächendeckende Erfassung des Untergrundes mit einem mobilen Roboter analysiert. Dabei wurden Sonderbedingungen, wie Objekte im Nahbereich und der Einfluss des Umgebungslichts, wie man sie häufig in realistischen Einsätzen vorfindet, in Betracht gezogen. Mit Hilfe der Modellierung der Fehlercharakteristik der Sensoren kann der Roboter die Qualität der resultierenden Untergrundrekonstruktion abschätzen.

Während sich der Roboter bewegt, wird der Boden kontinuierlich erfasst, um neue und sich verändernde Bereiche abzubilden. In der vorliegenden Arbeit wird die Umgebungswahrnehmung aus einer roboterzentrischen Perspektive modelliert. Dazu wird ein neuartiger Ansatz für die Fehlerfortpflanzung des Zustandsschätzers des Roboters in die Repräsentation der Karte dargelegt. Dieses Vorgehen ermöglicht ein zuverlässiges, schnelles und flächendeckendes, lokales Kartographieren, welches unabhängig von einer globalen Lokalisierungsmethode ist.

Die Bewegungsplanung basiert auf der Umgebungskarte, wodurch der Roboter sichere Fusspunkte finden und kollisionsfreie Beinbewegungen ausführen kann. Dabei wird demonstriert, wie ein kraftgeregelter, vierbeiniger Roboter Hindernisse wie Trümmer, Stufen, Lücken und Treppen autonom überwinden kann, ohne dabei auf vorherige Informationen über die Umgebung und externe Hilfsmittel zurückzugreifen. Der Bewegungsplaner aktualisiert die Bewegung stetig, um auf externe Störungen und auf sich verändernde Umgebungen reagieren zu können.

Für die Regelung des Laufroboters trägt die vorliegende Arbeit mit einem System für die vielseitige und aufgabenorientierte Bewegungsausführung bei. Dadurch können Bewegungsabläufe auch unter Einfluss von signifikanten Diskrepanzen zwischen Modell und Realität robust gesteuert werden. Zusätzlich zur Fähigkeit, sich in unwegsamem Gelände fortzubewegen, ist der Roboter damit in der Lage, Treppen hoch und runter zu klettern, Objekte zu manipulieren, Nutzlasten auszurichten und abzuladen, zu springen, zu tanzen und sich anzudocken.

Die Forschungsergebnisse wurden im Rahmen von realitätsnahen und umfangreichen Missionen zur autonomen Inspektion von Industrieanlagen und für Such- und Rettungseinsätze validiert. Ergänzende Ausführungen zeigen die Konzeption und Implementation eines Systems für die kollaborative Navigation von Flug- und Bodenrobotern. Damit ist es möglich, die durch die Flugroboter aufgezeichneten Daten mit Bodenrobotern zu nutzen, um effizient in unbekannten Umgebungen zu navigieren.

Acknowledgments

In 2013, I joined the legged robotics group at the ETH Zurich's Autonomous Systems Lab as the fifth person on the team. A few years later, the group has grown to dozens of people, and research is striving at a tremendous pace. We also had the chance to found and grow a company, and today, our robots are being used by groups around the world for research and commercial applications. This experience has been the most exciting and rewarding journey for me, and I'm eternally grateful to all the super talented, hard-working, and generous colleagues and friends who made this happen.

I foremost thank my advisors, Prof. Roland Siegwart and Prof. Marco Hutter, for the invaluable support, trust, and freedom they have given me during this work. I am also very grateful to Dr. Jerry Pratt for his work as co-examiner and for having me work with his fantastic team at the Florida Institute for Human & Machine Cognition (IHMC). Special thanks go to Michael Blösch for his commitment and mathematical support. I further thank Christian Gehring, Dario Bellicoso, and Marko Bjelonic for the close collaboration, fruitful discussions, and their patient help with the experiments.

This work would not have been possible without the great contributions of the lab members at the Robotic Systems Lab and the Autonomous Systems Lab, namely Samuel Bachmann, Karen Bodie, Remo Diethelm, Renaud Dubé, Marcin Dymczyk, Abel Gawel, Fabian Günther, Mark Höpflinger, Gabriel Hottiger, Jemin Hwangbo, Linus Isler, Dominic Jud, Ralph Kästner, Hendrik Kolvenbach, Philipp Krüsi, Andreas Lauber, Philipp Leemann, Thomas Schneider, Hannes Sommer, Vassilios Tsounis, Lorenz Wellhausen, and Martin Wermelinger. I would also like to express my gratitude to the technical staff, Stefan Bertschi, Markus Bühler, Dario Fenner, Bruno Kaufmann, Amir Melzer, Konrad Meyer and to the administrative staff Luciana Borsatti, Cornelia Della Casa, and Maria Trodella.

During this work, I also had the chance to supervise undergraduate and graduate students to whom I'm grateful for their trust and commitment. I especially

thank Tanja Baumann, Thomas Bi, Fabian Blöchliger, David Gonon, Fabian Jenelten, Roman Käslin, Takahiro Miki, Diego Rodriguez, Joel Roos, Marco Sütterlin, Lucas Wagner, and Georg Wiedebach for their extraordinary work.

I would also like to express my gratitude to the external researchers, Dominik Belter (Poznan University of Technology), Ron Fearing and Duncan Haldane (University of California, Berkeley), and Alessandro Giusti (Dalle Molle Institute for Artificial Intelligence, IDSIA), with whom I had the chance to collaborate.

Furthermore, I thank all the employees at ANYbotics, and especially Hanspeter Fässler, for their support and patience.

My most profound gratitude goes to my parents, Ria and Ueli, my brothers, Tamás and Máté, and my partner Adriana for unconditionally supporting me in everything I do. Finally, I would like to shout-out to Enzo, Fabio, Fredrik, Jérôme, Joël, Michael, Remo, and Simon for their lasting friendship.

Zurich, August 2018

Financial Support

Grateful acknowledgments go to the Swiss National Science Foundation (SNF, project 200021_149427 / 1), the National Centre of Competence in Research Robotics, and the Gebert Rüf Stiftung.

This work has been conducted as part of ANYmal Research, a community to advance legged robotics.

Contents

List of Symbols xi

Acronyms xiii

1 Introduction 1
- 1.1 Motivation and Objectives . . . 2
- 1.2 State of the Art . . . 4
- 1.3 Approach and Contributions . . . 7
- 1.4 Robotic Systems . . . 10
- 1.5 Thesis Organization . . . 11

2 Evaluation and Modeling of Range Sensors 15
- 2.1 Related Work . . . 16
- 2.2 Overview of Range Sensors . . . 17
  - 2.2.1 Evaluation Criteria . . . 18
  - 2.2.2 Structured-Light Depth Cameras . . . 20
  - 2.2.3 Laser Range Sensors . . . 22
  - 2.2.4 Time-of-Flight Cameras . . . 23
  - 2.2.5 Active Stereo Cameras . . . 24
- 2.3 Evaluation and Modeling of the Kinect v2 Sensor . . . 25
  - 2.3.1 Systematic Errors & Calibration . . . 26
  - 2.3.2 Indoor Evaluation . . . 27
  - 2.3.3 Outdoor Evaluation . . . 31
- 2.4 Analysis and Conclusions . . . 34
  - 2.4.1 Noise Analysis . . . 35
  - 2.4.2 Density Analysis . . . 36
  - 2.4.3 Conclusions . . . 38
- 2.5 Summary . . . 40

3 Terrain Mapping 41
- 3.1 Related Work . . . 42

3.2 Method . . . 44
3.2.1 Definitions . . . 45
3.2.2 Map Update from Range Measurements . . . 46
3.2.3 Map Update from Robot Motion . . . 48
3.2.4 Map Fusion . . . 51
3.2.5 Extension to Dynamic Environments . . . 52
3.3 Results . . . 53
3.3.1 Evaluation in Simulation . . . 53
3.3.2 Evaluation on Hardware . . . 54
3.3.3 Comparison . . . 57
3.4 Summary . . . 57

4 Legged Motion Control **59**
4.1 Related Work . . . 60
4.2 Interface Definition . . . 61
4.2.1 Command Structure . . . 63
4.2.2 Leg Motions in Joint Space . . . 63
4.2.3 Leg Motions in Cartesian Space . . . 64
4.2.4 Base Motions . . . 65
4.2.5 Output and Integration of Free Gait . . . 66
4.3 Implementation . . . 67
4.3.1 Base Motion Control . . . 67
4.3.2 Swing Leg Control . . . 68
4.3.3 State Transitions . . . 70
4.3.4 Contact Transitions . . . 70
4.3.5 Reference Frames . . . 72
4.3.6 Software Tools . . . 72
4.4 Results . . . 73
4.4.1 Introductory Examples . . . 74
4.4.2 Motion Templates . . . 77
4.4.3 Motion Planning . . . 82
4.5 Summary . . . 84

5 Legged Locomotion Planning **87**
5.1 Related Work . . . 88
5.2 Overview of the Approach . . . 90
5.3 Map Processing . . . 92
5.3.1 Surface Normal Vectors . . . 92
5.3.2 Foothold Classification . . . 93

5.3.3 Signed Distance Field . . . 94
5.4 Map-Based Motion Planning . . . 94
5.4.1 Footstep Generation . . . 95
5.4.2 Footstep Optimization . . . 97
5.4.3 Pose Optimization . . . 98
5.4.4 Swing Leg Planning . . . 101
5.4.5 Integration with Motion Control . . . 103
5.5 Results . . . 104
5.5.1 Steps . . . 104
5.5.2 Inclined Surfaces . . . 106
5.5.3 Dynamic Environments . . . 107
5.5.4 Stairs . . . 107
5.5.5 Robustness . . . 108
5.6 Discussion . . . 109
5.7 Summary . . . 112

6 Collaborative Navigation for Flying and Walking Robots 113
6.1 Related Work . . . 115
6.2 Methods . . . 116
6.2.1 Localization and Mapping . . . 117
6.2.2 Legged State Estimation . . . 118
6.2.3 Terrain Mapping . . . 118
6.2.4 Traversability Estimation . . . 119
6.2.5 Navigation Planning . . . 119
6.2.6 Path Execution . . . 120
6.3 Results . . . 120
6.3.1 Localization and Mapping . . . 121
6.3.2 Elevation Mapping . . . 122
6.3.3 Traversability Estimation and Navigation Planning . . . 123
6.4 Concurrent Operation of Walking and Flying Robots . . . 123
6.4.1 Localization and Mapping . . . 125
6.4.2 Legged State Estimation . . . 125
6.4.3 Mapping and Navigation Planning . . . 127
6.5 Summary . . . 127

7 Conclusion & Future Directions 129
7.1 Evaluation and Modeling of Range Sensors . . . 130
7.2 Terrain Mapping . . . 131
7.3 Legged Motion Control . . . 132

7.4 Legged Locomotion Planning 133
7.5 Collaborative Navigation 133
7.6 Outlook 134

A Appendix **137**
A.1 Computation of Relative Position Uncertainties 138
A.1.1 State Reduction 138
A.1.2 Covariance Transformation 138
A.2 Pose Optimization 141
A.2.1 Gradient 141
A.2.2 Hessian 141
A.2.3 Inequality Constraints 142
A.3 $\Phi(\boldsymbol{a})^\mathsf{T}\boldsymbol{b}$ Derivates 143
A.3.1 Gradient 143
A.3.2 Hessian 143

List of Figures **145**

List of Tables **147**

Bibliography **149**

Curriculum Vitae **165**
List of Publications 166
List of Videos 171
List of Software 173
List of Supervised Student Projects 174

List of Symbols

α	sunlight incidence angle
B	base frame
$\boldsymbol{C}$	rotation matrix (direction cosine matrix)
c	collision field
D	derivative gain
δ	angle difference between to measurement beams / funnel angle
d	angular or areal density of measurements / distance
f	angular or areal flux of measurements
ε	distance threshold
$\boldsymbol{e}$	unit vector
$\boldsymbol{F}$	force vector
h	elevation map height
$\boldsymbol{h}$	non-linear terms (Coriolis, centrifugal, and gravitational forces)
I	inertial frame / integral gain
$\boldsymbol{I}$	identity matrix
$\boldsymbol{J}$	Jacobian
M	map frame
$\boldsymbol{M}$	mass matrix
μ	friction coefficient
$\boldsymbol{n}$	surface normal vector
Φ	orientation
$\boldsymbol{P}$	projection matrix
P	proportional gain
p	height measurement
φ	roll angle
ψ	yaw angle
q	joint position

$\boldsymbol{q}$	state vector
$\boldsymbol{R}$	regularizer matrix
R	search region
$\boldsymbol{r}$	position vector
S	sensor frame
s	foothold score
σ	standard deviation
σ_L	standard deviation of lateral measurement noise
σ_z	standard deviation of axial measurement noise
$\boldsymbol{\Sigma}$	variance-covariance matrix
$\boldsymbol{T}$	torque vector
τ	joint torque
θ	measurement beam incidence angle / pitch angle
t	time
$\boldsymbol{u}$	stacked spline knot position vector

Acronyms

2.5D	two-and-a-half-dimensional
2D	two-dimensional
3D	three-dimensional
API	application programming interface
ARGOS	Autonomous Robot for Gas and Oil
CDF	cumulative density function
CHOMP	covariant Hamiltonian optimization for motion planning
COM	center of mass
COP	center of pressure
DOF	degree of freedom
DRC	DARPA Robotics Challenge
EKF	extended Kalman filter
ERL	European Robotics League
ETH	Eidgenössische Technische Hochschule
FOV	field of view
GPS	Global Positioning System
GPU	graphics processing unit
HAA	hip abduction/adduction
HAL	hardware abstraction layer
HFE	hip flexion/extension
ICP	iterative closest point
IHMC	Florida Institute for Human & Machine Cognition
IMU	inertial measurement unit
IP	Ingress Protection
IR	infrared
IT	integration time
KFE	knee flexion/extension

L2	Learning Locomotion
PCA	principal component analysis
PDF	probability density function
PID	proportional-integral-derivative
QP	quadratic programming
RAM	random-access memory
RF	right fore
RH	right hind
RMS	root-mean-squared
ROS	Robot Operating System
RRT	rapidly-exploring random tree
SDF	signed distance field
SEA	series-elastic actuator
SLAM	simultaneous localization and mapping
SP	support polygon
SQP	sequentiell quadratic programming
TOF	time-of-flight
UAV	unmanned aerial vehicle
VMC	virtual model control
WLAN	wireless local area network
YAML	YAML Ain't Markup Language
ZMP	zero-moment point

1

Introduction

The domestication of animals has accompanied us for thousands of years. Their unique capabilities have allowed humanity to accomplish a range of work differently or more efficiently and safely in areas such as agriculture, transportation, hunting, and searching and retrieving. While modern vehicles such as tractors and cars have replaced many of the traditional uses of animals, they are still unparalleled in their performance to move over difficult terrain. Even though the wheel was invented more than 5000 years ago, a significant drawback of wheeled (and tracked) locomotion over legged locomotion has remained: Wheels require a large diameter to overcome obstacles and have the tendency to slip and dig in on loose ground. As the rolling motion describes a contact path, it requires continuous stable ground support. In comparison, locomotion with legs can adapt naturally to the terrain through deliberate stepping supported by single contacts points. Furthermore, legs allow the control of the interaction forces with the ground to adjust the motion to tilted and slippery surfaces.

While legged locomotion holds promising advantages, it is much harder to solve in comparison to the powerful simplicity of wheels. Even after 50 years of research (Frank, 1968; McGhee, 1967), autonomous walking machines are not yet used in practical applications. Today, it remains a challenge to design and control legged systems that can move by themselves through an environment. However, ongoing advancements of technologies in other fields accelerate the progress. Increasingly more capable and light-weight materials, actuators, sensors, and processors are available from the development of personal computers, mobile phones, gaming consoles, cars, 3D printing, and more. These recent developments give hope that practical legged machines might be a reality sooner than later. Even if legged robots do not yet reach the speed, elegance, and endurance of their natural counterparts, they hold the potential to help us

Figure 1.1 The work described in this thesis is aimed at the perceptive legged locomotion in complex environments. The photograph shows a typical target scene consisting of steps, gaps, obstacles, and stairs. At the 2017 World Petroleum Congress the robot ANYmal was presented to showcase the autonomous inspection of oil and gas sites.

in many areas and might prevent the need to expose humans and animals to dangerous or harmful situations.

1.1 Motivation and Objectives

A key feature enabling legged robots to be used in real-world applications is their ability to locomote over complex terrain. For example, in an urban search and rescue mission, the robot might be required to climb over rubble or fit through narrow spaces. For industrial work, the robot needs to navigate through complex environments consisting of steps, gaps, stairs, and others (see Figure 1.1). Interestingly, a lot of the existing work on legged robots is focussed on the locomotion on flat ground. It is surprising because it is only in difficult terrain where legged locomotion gains advantages over wheeled locomotion. This focus is likely due to the complexity that is involved in building and programming a robot that can negotiate complex terrain. In flat, obstacle-free environments, footholds can be freely chosen to stabilize the desired motion. In contrast, footholds are sparse in rough terrain and require careful selection and planning. Here, it is the terrain that dictates the motion of the system. Figure 1.2 illustrates the underlying challenges that need to be considered when seeking to maximize the mobility of the robot through rough terrain. This maneuverability requires knowledge of the environment and the localization of the robot relative to it. For the motion, stability, kinematic reachability of footholds, torque and speed limitations of the actuators, self-collisions, and undesired contact with the ground must be considered.

The goal of our work is to enable legged robots to locomote over rough terrain to reach places where classical wheeled and tracked robots cannot go. We

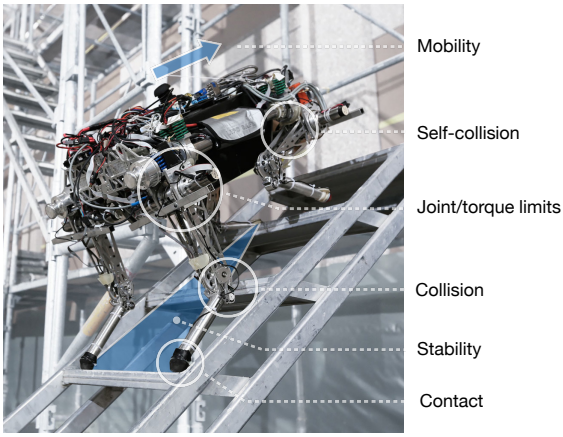


Figure 1.2 Rough terrain locomotion with legged robots poses unique challenges. Accurate knowledge of the terrain is needed to choose stable contact points that support the mobility of the system. During the motion, the robot must find collision-free motions that ensure stability while respecting the constraints concerning kinematic and joint torque limitations. The figure shows StarlETH, one of the robots used to verify the approaches of this work.

characterize this environment to consist of obstacles with a maximum height of $\sim$50 % of the robot's leg length. With a robot size of $\sim$0.5 m, this enables to move through a human-made environment such as shown in Figure 1.1. Our research focusses on quadrupedal robots, although many of the approaches should be adaptable to robots with more or fewer legs (e.g., hexapods, bipeds). The reason for our choice is that four-legged robots form a well-balanced trade-off between stability and complexity. With three legs statically supporting the base, one leg is free to move to the next foothold location. While we mostly work with static walking gaits (only one leg is in motion per step), the methods should be extendible to dynamic gaits with multiple swing legs at a time. To be able to reason about the upcoming environment, our objective is to tightly integrate exteroceptive perception in our approach. This allows the robot to work with a geometric reconstruction of the environment to find a suitable motion. Given a target position or desired velocity, we expect the robot to generate the motion to overcome the terrain autonomously. This motion generation is supposed to act in a local sense, similar to driving a car or horseback riding. We leave it to a navigation framework to generate a feasible global trajectory or to the user to guide the robot with desired velocities. As we aim for practical applications of our work, the system cannot rely on any external equipment or prior information. This means that the

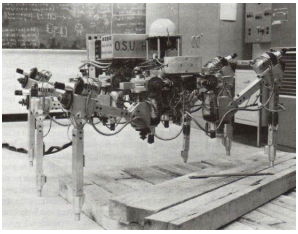
robot is computationally and power-wise self-contained and the environment has to be perceived as the robot moves. Furthermore, we expect non-ideal conditions, external disturbances, and dynamic environments, all of which the system needs to cope with.

1.2 State of the Art

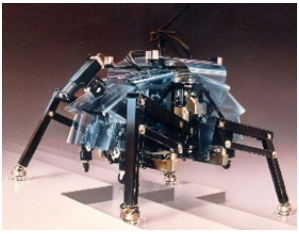
Prior work has taken different approaches to rough terrain locomotion. One method is to perceive the environment through haptics and reactively adapt to the terrain on a step-by-step basis. Klein, Olson, and Pugh (1983) presented the *OSU hexapod* (Figure 1.3a) walking with a tripod and parallelogram gait over obstacles while maintaining a desired body orientation with the help of an inclinometer. Active compliance was used to distribute the loading forces among the legs measured with force sensors on the feet. The *TITAN* series (Figure 1.3b Hirose, 1984; Hirose and Kunieda, 1991) demonstrated quadrupedal locomotion over non-level terrain and could even climb stairs. Tactile sensors on the feet were used to sense the ground and obstacles to approach a viable foothold. For example, the leg would be raised step-wise higher upon sensing an obstacle. Another approach to overcome unperceived terrain changes was presented for the *Tekken* and *Tekken2* quadrupeds (Fukuoka, Kimura, and Cohen, 2003; Kimura, Fukuoka, and Cohen, 2007). These robots have mechanically compliant elements in their legs and are controlled with neural networks and reflexive behaviors to traverse rough surfaces. Due to the lack of a look-ahead feature, these reactive approaches must lift the legs high at every step to overcome obstacles which make them over-conservative and slow.

Introducing optical sensing, the hexapod *Ambler* (Figure 1.3c) was equipped with a laser-range sensor to scan the terrain around the robot from which the next foothold would be chosen (Krotkov and Simmons, 1996). Operators provided the overall path, and the control system of the vehicle adjusted its height to maintain the body level and ground clearance. This approach is limited to obstacles that can be overcome without body adaption and requires experienced operators to guide the robot along an obstacle-free path. Due to computational and hardware constraints, the achieved average speed of 0.6 cm/s was rather low.

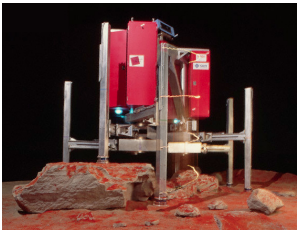
Fully-integrated, rough terrain locomotion was later presented for the robots *RHex* (Rusu et al., 2009), *BigDog* (Wooden et al., 2010), *LS3* (Figure 1.3d, Bajracharya et al., 2013), the *DLR Crawler* (Figure 1.3e, Stelzer, Hirschmuller,



(a) OSU Hexapod

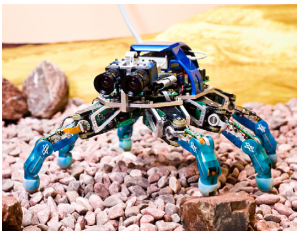


(b) TITAN IV

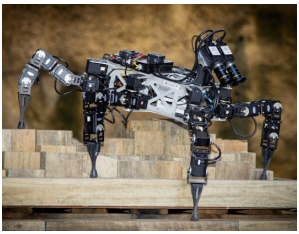


(c) Ambler

(d) LS3



(e) DLR Crawler

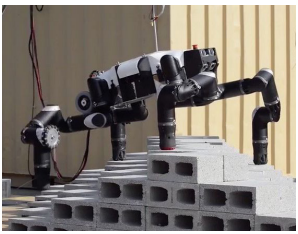


(f) Weaver

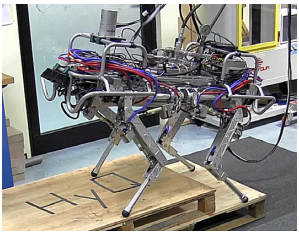
(g) LittleDog

(h) LAURON V

(i) Messor II



(j) RoboSimian



(k) HyQ

(l) SpotMini

Figure 1.3 Research in the last thirty years has demonstrated various approaches for legged robots to conquer rough terrain.

and Gorner, 2012), and the hexapod *Weaver* (Figure 1.3f, Bjelonic et al., 2017). These contributions focus on mission-level path planning and gait parameter adaptations. Using an online generated terrain map, the goal is to automatically command the robot's stepping controller towards a target while ensuring that only ground is covered which the controller can handle reactively. While the terrain information is used to adapt parameters such as the stepping height, they do not fully exploit information from the leg kinematics and ignore foothold selection. This approach prevents the robots from safely climbing high obstacles (relative to their size) or crossing gaps.

In contrast, a few recent approaches use deliberate foothold planning to overcome higher obstacles (proactive approach). The work by Kolter and Ng (2009) proposed a terrain mapping and motion planning framework for the *LittleDog* (Figure 1.3g, Murphy et al., 2010) quadruped based on stereo vision and iterative closest point (ICP) matching. The authors only briefly address the issues of computational resources, as most of the processing is performed off-board. Other notable exceptions are the contributions by Roennau et al. (2009) and Belter, Łabęcki, and Skrzypczyński (2016). The former allows the robot *LAURON IV* (Figure 1.3h) to perceive the terrain with a time-of-flight (TOF) camera and to choose an appropriate foothold in the next step. The latter work extended this concept for the *Messor II* hexapod robot (Figure 1.3i) to an RRT-based multi-step planner, preventing dead-lock situations. Both of these legged robots are maneuvered through joint position control, which makes the success of the locomotion execution dependent on highly accurate terrain information.

In the recently conducted DARPA Robotics Challenge (DRC) (2012–2015, Pratt and Manzo, 2013) various teams worked with legged platforms (mostly humanoids) which were required to drive a vehicle, open doors, walk over rubble piles, climb stairs, and compete in various other tasks. While many participants worked on autonomous climbing methods, most of them fell back on remote control for walking over challenging terrain or avoided the obstacles all together (e.g. Johnson et al., 2017; Karumanchi et al., 2017; Stentz et al., 2015; Stumpf et al., 2014; Zucker et al., 2015, Figure 1.3j). Even though the rules specified the environment of the competition, the teams assessed a failure from autonomous locomotion as too high-risk. This observation emphasizes the difficulty of robust, rough terrain locomotion.

First steps towards deliberate foothold planning based on online mapping were also shown in different contributions on the hydraulically powered (tethered) quadrupedal robot *HyQ* (Figure 1.3k, Semini et al., 2011). Through force-

controllable joints, these works demonstrate the importance of compliant motion control when dealing with the uncertainty of onboard state estimation and perception. The works range from heuristic approaches (Focchi et al., 2018; Havoutis et al., 2013; Winkler et al., 2015), which can be run real-time, to model-based approaches (Aceituno-Cabezas et al., 2018; Mastalli et al., 2017b), whose computational times prohibit online planning with today's available today hardware. This highlights the complexity in finding formulations that generate versatile motions while complying with the computational restrictions.

One of the most advanced rough terrain locomotion skills was demonstrated by Boston Dynamics for the quadrupedal robot *SpotMini* (2016, Figure 1.3l). Although little information about this technology is available, demonstrations have revealed automatic stepping and pose adaptation based on an environment map gathered from stereo vision.

1.3 Approach and Contributions

This thesis summarizes our research and development of rough terrain locomotion capabilities for legged robots. Our work comprises the entire pipeline from the environmental perception, mapping, and interpretation to the planning and control of the motion of the robot. With our goal to employ these features in real-world applications, *robust design* is an overarching driver of our work. The robustness of an approach is the ability to perform unaffected by uncertain inputs and environmental conditions. For every component of our contribution, we aimed at applying three principles:

P1 Simplifying: The number of components and their complexity is reduced to a minimum.

P2 Incorporating uncertainty: Error sources are understood and incorporated into the design.

P3 Embedding redundancies: Missing data causes graceful degradation and not a complete failure.

Thoroughly working with this methodology has allowed us to verify our contributions on four-legged platforms in a series of experiments and several practical applications. Most notably, we deployed our work for the participation in the ARGOS Challenge (2013–2017, Fankhauser et al., 2016a; Hutter et al., 2017a)

for the autonomous inspection of oil and gas sites and in the ERL Emergency Robots competition (2017, Bellicoso et al., 2018b) for emergency response scenarios. In both scenarios, the robot was required to sense its environment, detect obstacles, and navigate over difficult terrain including sand, rubble, steps, inclines, gaps, and stairs.

The remainder of this section describes the approach and contributions focussed on each part of this work.

Evaluation and Modeling of Range Sensors

Exteroceptive perception enables the robot to perceive its surroundings. In this work, we focus on sensors which allow the creation of a reconstruction of the close-range terrain of the robot. We contribute with a qualitative and quantitive comparison of four different sensor technologies. We analyze their applicability for dense mapping on a mobile robot regarding performance (such as resolution, field of view, noise) and non-performance criteria (such as weight, size, power consumption). With the introduction of the *scan flux* metric, sensors with different resolutions and scan rates can be compared to evaluate the data available for the terrain reconstruction. Given the specifications of a sensor and its mounting point, we present a model to approximate the expected resolution of the terrain reconstruction, which can be used to find a desired trade-off between map definition and update rate. For the Kinect v2 TOF sensor, we provide an analysis with particular focus on problematic situations such as outdoor usage in overcast or direct sunlight and measurements of tilted surfaces and close range objects. Additionally, we contribute an empirically derived three-dimensional noise model for the Kinect v2 sensor as a prerequisite for stochastic terrain mapping (P2).

Terrain Mapping

Our terrain mapping approach unifies the error model of the range sensor with the uncertainty information provided by the state estimation of the robot. The primary motivation is to reflect the errors of the robot's pose estimate in the representation of the map (P2). Importantly, our state estimation (Bloesch et al., 2012, 2013) does not rely on exteroceptive but on proprioceptive (inertial and kinematic) sensing only. As we are interested in close range mapping, this approach allows us to circumvent the dependency on an absolute localization method such as GPS, scan matching, or visual localization (P1). This means that localization data is incorporated if available, but the mapping continues to function if external localization degrades or is not available (P3).

We contribute a novel mathematical formulation of a mapping framework, which models the terrain as a 2.5D elevation map from the local perspective of the robot. Our results show how this approach incorporates drift of the proprioceptive state estimation in the representation of the map uncertainty. Evaluations on simulated data, a comparison to ground-truth measurements, and a comparison to other approaches demonstrate the effectiveness of the presented method. Finally, we extend our work to dynamic environments with ray tracing-based visibility checking to enable the map to adapt to moving objects quickly.

Legged Motion Control

For motion control, our main contribution is the design of a simple but powerful abstraction layer for the formulation and robust execution of whole-body motions. The interface allows for controlling generic motions involving base trajectories and leg motions in joint and Cartesian end-effector space. Through this flexible setup and free choice of a reference frame, whole-body motions can be executed in the way most appropriate for the task (P1). The motion definitions are tracked with our implementation of a robot motion controller which is designed for fully torque-controllable robots. In combination with the presented contact handling strategy, this approach allows for accurate and robust control even under significant disturbances (P2). This is a crucial prerequisite to executing motion plans in real-world situations where only partial and error-affected information is available. We demonstrate the effectiveness of our approach in a series of results involving rough-terrain locomotion, whole-body motion planning, basic manipulation, synchronized dancing to live music, dynamic maneuvers such as jumping and rearing, and a range of specialized motions for docking, inspection, and payload delivery.

Legged Locomotion Planning

We combine our work on terrain mapping and motion control in a rough-terrain locomotion planner. Reducing the problem to its essentials, we present an approach that runs fully onboard in real-time without any previous knowledge of the environment or external equipment (P1). For every footstep, the desired motion is re-planned to account for disturbances and new information from the terrain map (P2). At its core, our contribution is based on a non-linear optimization for the base pose and the swing leg motion. Footholds are selected based on a geometrical interpretation of the terrain, taking the uncertainty of the mapping process into account (P2). We utilize the torque-control of the robot to enable locomotion over inclined surfaces by estimating

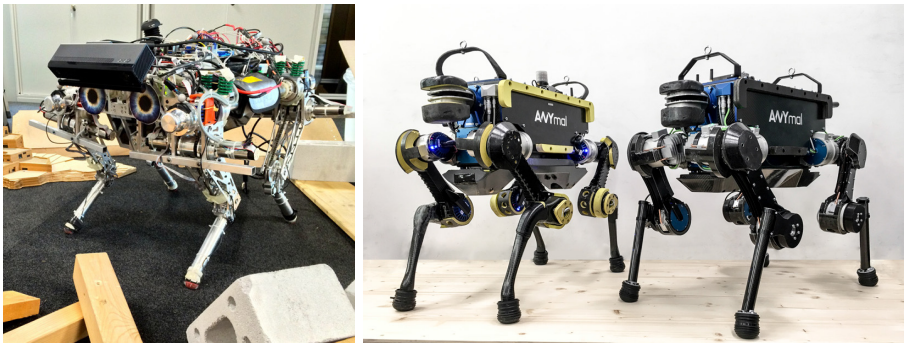
the terrain normals and adapting the ground reaction force accordingly. In the case when map data is missing, our approach transitions from proactive planning to reactive locomotion based on haptic feedback (P3). We present a series of experiments of climbing steps, inclines, and stairs. Verifying the robustness, we demonstrate navigation to a global target under disturbances such as changing and moving environments.

Collaborative Navigation

We conclude this work by embedding the rough terrain locomotion capabilities in an extended navigation framework. To this end, the legged robot is accompanied by a flying robot to showcase collaborative navigation. The agents use their complementary features regarding viewpoint, speed, and payload capability to navigate through an unknown environment as a heterogeneous team efficiently. The method leverages the flying robots onboard monocular camera to create both a map of visual features for simultaneous localization and mapping and a dense representation of the environment as an elevation map. This shared knowledge from the flying platform enables the walking robot to localize itself against the global map, and plan a global path to the goal by interpreting the elevation map in terms of traversability. While following the planned path, the absolute pose corrections are fused with the legged state estimation and the elevation map is continuously updated with distance measurements from an onboard range sensor. This allows the legged robot to safely navigate towards the goal while taking any changes in the environment into account. In this setup, our approach is independent of external localization, relative observations between the robots, and does not require an initial guess about the pose of the robots. The presented methods are fully integrated, and we demonstrate their capabilities in an experiment.

1.4 Robotic Systems

In the course of this dissertation, we have verified our contributions on the quadrupedal robots *StarlETH* (Hutter et al., 2012) and different evolutions of *ANYmal* (Hutter et al., 2016, 2017a). Both robots are symmetric in their design and have three rotational joints per leg in sequence as hip abduction/adduction (HAA), hip flexion/extension (HFE), and knee flexion/extension (KFE). The actuation is based on electrically driven series-elastic actuators (SEAs) (Pratt and Williamson, 1995), providing accurate position and torque control. ANYmal weights 28–35 kg depending on the payload and has a leg length of 50–56 cm depending on the version. The integrated *ANYdrive*



(a) StarlETH (b) ANYmal

Figure 1.4 The worked presented in this thesis has been evaluated on the quadrupedal robots StarlETH (Hutter et al., 2012) and different generations of ANYmal (Hutter et al., 2016). These robots are electrically driven and can be torque-controlled in all joints. The latest versions of ANYmal are developed by ANYbotics and are impact robust and dust and water ingress protected (IP67).

actuators (Hutter et al., 2016) drive ANYmal's joints providing a maximum torque of 40 Nm and a maximum velocity of 12 rad/s. Note that the shanks of ANYmal are built in an *L*-shaped form to increase the clearance when climbing onto high objects, facilitating the shin-hitting problem. For both robots, the onboard PCs perform all computation, and WLAN communication is used for monitoring and operator input. The robots carry batteries which allows them to operate power-autonomously for 2–3 h. Contact detection is implemented either through force sensors in the rubber feet or by thresholding the difference between desired and actual contact force estimated through the joint torques. The proprioceptive state estimation (legged odometry) was developed by Bloesch et al. (2012, 2013). The work is based on an extended Kalman filter (EKF) fusing kinematic measurements from the joint encoders, ground contact measurements, and inertial measurements from the onboard inertial measurement unit (IMU) at 400 Hz. The filter takes no assumption on any terrain shape and allows for reliable estimation of the robot's pose with an average error of 1–5 % of the traveled distance even on rough terrain.

1.5 Thesis Organization

This introduction is followed by the evaluation of different range sensors for the application mobile dense terrain mapping in Chapter 2. This chapter also describes our work on the analysis and modeling of the Kinect v2 sensor. In Chapter 3 we derive the formulation of our elevation mapping framework and

show how it represents a terrain model from a robot-centric perspective. Our approach to command and control the robot's motion is described in Chapter 4. A series of results demonstrates the broad range of applications of the contributed framework. Our locomotion planner for rough terrain navigation is presented in Chapter 5 and concluded with results and an analysis. Finally, an outlook for further applications is given in Chapter 6 with the collaborative navigation of flying and walking robots, before concluding this work in Chapter 7.

During this work, we have published several videos demonstrating the achieved results. The videos are listed at the end of the thesis and an online collection is available at

`https://purl.org/pfankhauser/phd-videos`

We indicate individual videos in this document with (V). Similarly, we have open-sourced various parts of our software and reference them with (S) to the description at the end of this dissertation.

In this thesis, we use following notations adopted from Bloesch et al. (2016):

- Scalar values are written in regular letters s, vectors in bold lower case letters $\boldsymbol{v}$, and matrices in bold upper case letters $\boldsymbol{M}$.
- For a vector ${}_A\boldsymbol{r}_{BC}$, the left subscript (A) indicates the coordinate frame in which the vector is represented and the subscripts on the right indicate the start (B) and end point (C) of the vector.
- The term $\Phi_{BA} \in \mathrm{SO}(3)$ is used to represent the relative orientation of a coordinate system B w.r.t. the coordinate system A. This expression is implementation-agnostic and could be realized with for example a unit quaternion or rotation matrix.
- Applied to a vector ${}_A\boldsymbol{r}_{BC}$, the rotation Φ_{BA} maps the representation of the vector from frame A to frame B, e.g., ${}_B\boldsymbol{r}_{BC} = \Phi_{BA}({}_A\boldsymbol{r}_{BC})$.
- The operator $\circ$ is used to indicate a concatenation of rotations, e.g., $\Phi_{CA} = \Phi_{CB} \circ \Phi_{BA}$.
- The term Φ_I is used to denote the identity and Φ^{-1} the inverse rotation.
- The boxminus operation is used to adopt the function of subtraction of rotations as $\boxminus : \mathrm{SO}(3) \times \mathrm{SO}(3) \to \mathbb{R}^3, \quad \Phi_1, \Phi_2 \mapsto \log(\Phi_1 \circ \Phi_2^{-1})$.

- We define the mapping for a rotation matrix $\boldsymbol{C}: \mathrm{SO}(3) \rightarrow \mathbb{R}^{3\times3}$ such that $\boldsymbol{\Phi}(\boldsymbol{r}) = \boldsymbol{C}(\boldsymbol{\Phi})\boldsymbol{r}$.
- The skew-symmetric matrix of a three-dimensional vector $\boldsymbol{v} \in \mathbb{R}^3$ is denoted as $\boldsymbol{v}^\times \in \mathbb{R}^{3\times3}$. It satisfies $\boldsymbol{v}^\times\boldsymbol{u} = \boldsymbol{v}\times\boldsymbol{u} \;\; \forall\, \boldsymbol{v},\boldsymbol{u} \in \mathbb{R}^3$.
- The planar projection of a three-dimensional vector $\boldsymbol{v} = (v_x, v_y, v_z) \in \mathbb{R}^3$ along the z-axis is denoted as $\overline{\boldsymbol{v}} = (v_x, v_y) \in \mathbb{R}^2$.

2

Evaluation and Modeling of Range Sensors

This chapter incorporates material from the following publication:

Fankhauser, P., Bloesch, M., Rodriguez, D., Kaestner, R., Hutter, M., and Siegwart, R. (2015). "Kinect v2 for Mobile Robot Navigation: Evaluation and Modeling". In: *International Conference on Advanced Robotics (ICAR)*. IEEE, pp. 388–394

Mobile robot navigation requires a robot to perceive its surroundings while moving through previously unseen environments. In robotics, exteroceptive range sensors are used to identify the distance from the robot to various points in the scene. Out of this collection of measurement points, a model is created to reconstruct the shape of the environment. Several technologies exist for exteroceptive range sensing based on ultrasonic, laser, structured-light, camera, capacitive and magnetic measurements (Siciliano and Khatib, 2016). All these technologies exhibit trade-offs and different sensors and combinations thereof are used for different applications.

During this work, we have employed structured-light, time-of-flight, stereo depth cameras, and laser range sensors. This chapter gives an overview of the applied range sensing technologies and provides a comparison based on various evaluation criteria. For each different sensing technology, we focus on one device to exemplify its performance and applicability for our use-case (Section 2.2). Because these sensors have different update frequencies and areas they observe, we introduce the terms *scan density* and *scan flux* to quantify the data stream available for the resulting terrain reconstruction. Our evaluation incorporates performance data as well as factors concerning weight, size, and power consumption regarding the application on a mobile robot. Essential for outdoor operation, we also examine the operability of the technology in low-light and sunlight situations.

For further processing of the range data, we are interested in the noise distribution of the depth measurements. We build on existing models where available and perform a detailed analysis of the *Microsoft Kinect v2* sensor (Section 2.3). To this end, we present a model of the noise characteristics of the Kinect v2 depth sensor in both *axial* and *lateral* direction. We empirically derive our noise model as a function of the measurement distance and the inclination angle of the observed surface. While we can characterize indoor and overcast situations with the same approach, we extend our model by the incidence angle for direct sunlight. Additionally, an analysis of the minimum measurement range for different surfaces is presented.

The sensor evaluation is concluded with a unifying analysis of the scan noise, density, and flux properties (Section 2.4). We show how to approximate the resulting terrain reconstruction's density and reversely, how to modify the sensor's resolution for the desired reconstruction density. Finally, we discuss the unique trade-offs for each technology and highlight the current developments.

2.1 Related Work

Many studies have been conducted involving the evaluation and comparison of range sensors. However, these contributions mostly focus on one sensor technology or even one specific sensor. For structured-light depth cameras, Cruz, Lucio, and Velho (2012) and El-Iaithy, Huang, and Yeh (2012) give an overview on the different applications of the *Microsoft Kinect* sensor and Devereux et al. (2011) evaluate its use for environmental mapping on a humanoid robot with special consideration of motion blur. The works by Khoshelham and Elberink (2012), Mallick, Das, and Majumdar (2014), and Nguyen, Izadi, and Lovell (2012) have performed analyses of the error characteristics and provide noise models. For time-of-flight sensors, Horaud et al. (2016) give an overview of different devices and Foix, Alenya, and Torras (2011) and Fürsattel et al. (2016) compare different sensors regarding measurement accuracy and precision. A similar comparison was published by Pomerleau et al. (2012) for several laser range sensors. We find a lot of work on (passive) stereo vision such as the contributions of Rankin, Huertas, and Matthies (2005) for obstacle detection and Stefanik et al. (2011) for aerial large scale mapping. For active stereo cameras, Keselman et al. (2017) present a performance analysis of *Intel RealSense R200* family and highlight some applications. Some contributions have been made to compare different sensor technologies. For example, Amon and Fuhrmann (2014) and Breuer, Bodensteiner, and Arens (2014) compare the performance of the original Microsoft Kinect (structured-light) with the newer Kinect v2

(time-of-flight). Other comparisons focus on specific applications or conditions, such as Rafibakhsh et al. (2012) for scanning of construction sites, Zug et al. (2012) for indoor obstacle detection, localization, and two-dimensional mapping, Matthies et al. (1998) for obstacle detection for a rough terrain vehicle, and Tretyakov and Linder (2011) for smoky conditions. We contribute over these existing comparisons with an evaluation of four different sensor technologies specific to the application of mobile short-range, dense mapping. Our work takes the special requirements into account that arise when working with a mobile robot in realistic indoor and outdoor environments.

For the analysis of the Microsoft Kinect v2 depth sensor, several authors have reported about the performance in terms of systematic and non-systematic measurement errors (Breuer, Bodensteiner, and Arens, 2014; Pinto et al., 2015; Yang et al., 2015; Zennaro, 2014). Butkiewicz (2014), Lachat et al. (2015), and Zennaro (2014) have also exemplarily shown that the Kinect v2 is able to capture data for shadow and direct sunlight situations. Other analyzed aspects include the influence of surface color and roughness on the depth measurements (Breuer, Bodensteiner, and Arens, 2014; Lachat et al., 2015; Zennaro, 2014), pre-heating of the device (Breuer, Bodensteiner, and Arens, 2014; Lachat et al., 2015), calibration (Butkiewicz, 2014; Lachat et al., 2015), and measurements through water (Butkiewicz, 2014). While these contributions present encouraging results for the use of the Kinect v2 sensor for mobile mapping, the error characteristics are limited to depth measurements in the axial direction and for a single angle of a planar surface. Also, no numerical analyses for overcast and sunlight situations were given. Our work adds to these findings with an extended noise model taking into account different measurement incidence angles and different indoor and outdoor environmental conditions and an analysis of the minimum range of the sensor.

2.2 Overview of Range Sensors

Every application has different requirements on the selection of range sensors, and in general, there is no ‘best sensor’ for all applications. In our evaluation, we focus on the technical data and characteristics that are important for the short range perception of rough terrain for a mobile robot. We assume that one sensor is mounted on the base of the robot (height $\sim$0.5 m) with the viewpoint in the direction of the main motion as illustrated in Figure 2.1.

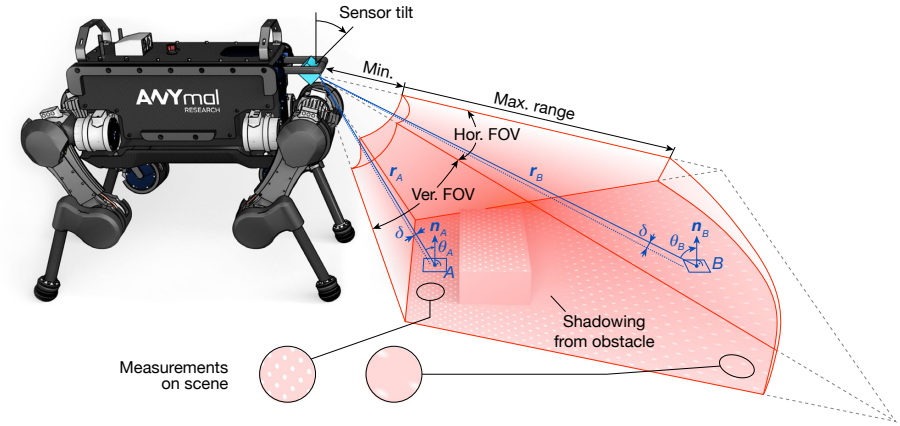


Figure 2.1 This figure illustrates the characteristics for a typical range sensor mounted in front of the robot pointing downwards. Objects within the minimum and maximum range of the sensor are detected. Assuming a equal angle differences of the measurements δ (constant angular density), the setup leads to an uneven distribution of measurements in the scene: Areas close to the sensor (*A*) are measured at a higher density than areas more distant (*B*).

2.2.1 Evaluation Criteria

For the evaluation of the range sensor, we differentiate between performance and non-performance criteria. The performance criteria describe the properties of the sensing process, and the non-performance criteria specify the hardware and implementation characteristics. For the evaluation of the performance criteria, we work with the following definitions (see Figure 2.1):

Field of view (FOV) defines the angular area of the perceivable field of the sensor. It is specified in its horizontal (h) and vertical (v) components. A large FOV is preferential for the application on a mobile robot as a larger region of the environment can be perceived.

Resolution is the number of measurement points taken in a single scan. For image sensor-based scanners, it is specified as a horizontal (image width) and vertical (image height) resolution. Accordingly, the total number of measurement points equals the product of the horizontal and vertical resolution. A higher resolution means that more information is captured which is vital for the accurate reconstruction of the terrain.

Density defines the number of sample points per angle difference within the FOV. When specified in deg^{-1} for the horizontal or vertical component,

it is the number of sample points per degree of the FOV. The density is related to the FOV and resolution as

$$\text{Resolution} = \text{FOV} \times \text{Density}, \tag{2.1}$$

for the horizontal or vertical component. Combining the horizontal and vertical density, we get the number of sample points per solid angle with the unit rad^{-2}, deg^{-2}, or sr^{-1} (steradian), allowing us to compare sensors with different FOVs and resolutions. For our use-case, the density is an essential characteristic to judge the definition of the resulting terrain reconstruction.

Rate specifies the number of full scans taken per second. By *full* scan we mean the scanning procedure to cover the entire FOV. For image sensor-based scanners this corresponds to the frame rate. For moving/rotating sensors, the rate corresponds to the number of periodic motions per second. A high rate is preferable for mobile robotic applications as it allows the robot to move at higher speeds without running into areas where no measurements have been taken yet. In dynamic environments, a rate determines how quickly changes in the environment can be perceived. Additionally, a high rate can be advantageous as redundant measurements can be used to infer a more accurate representation through filtering.

Flux combines the density with the rate. We define the flux as the number of sample points per solid angle per unit time. In other words, the scan flux tells us about how many points are measured within a two-dimensional angle increment in a particular time span. The flux is computed as

$$\text{Flux} = \text{Density (Horizontal)} \times \text{Density (Vertical)} \times \text{Rate} \tag{2.2}$$

and is given in $\text{rad}^{-2}\text{s}^{-1}$ or $\text{deg}^{-2}\text{s}^{-1}$. With this definition, the flux allows us to compare sensors with different densities and rates, as a low-density high-rate sensor and a high-density low-rate sensor can have the same flux. This is important for our application as it tells us about the amount of the data available for the terrain reconstruction.

Operating range defines the distances that are perceivable by the sensor and might vary for different object materials and environmental conditions. For local terrain perception on a mobile robot, measurements $>3\,\text{m}$ can be considered as long range. A low minimum range is important as

objects close to the robot could collide and should not 'disappear' as they fall outside of the operating range of the sensor.

Noise characterizes the non-systematic errors in the measurement process. The noise typically varies over the measurement distance, incident angle, and environmental conditions. A lower noise profile is favorable for a precise terrain reconstruction.

Ambient light resistance specifies if a sensor is suitable to be used in special lighting conditions such as in low-light environments and direct and in-direct sunlight situations. While range sensor measurement performance often degrades in these situations, a non-resistant sensor will output none or faulty data, which limits the application of the robot.

Based on our experience, these performance characteristics are the most essential for the short-range mapping with a mobile robot. While we include the noise resulting from non-systematic errors inherent to the measurement process, we do not consider systematic errors, as they depend on the calibration method. Furthermore, we also exclude the depth resolution (step size between possible measurement ranges), as we have not experienced any deficiencies in this regard in our usage. We also neglect the influence of smoke, fog, rain and translucent materials such as glass.

For the non-performance criteria, we can compare the sensor regarding their suitability to be mounted and implemented on a mobile robot. Ideally, a sensor should be as small and lightweight as possible and sealed against ingress. Low mechanical complexity is desirable, and moving parts should be avoided because they are often fragile and might require additional protection against collisions. On an electrical side, we prefer sensors with a low power consumption that are ideally supplied through the same interface as the communication. Furthermore, the price can be an important decision factor, especially if multiple sensors are mounted on a robot. We do not investigate the shock & vibration ratings as we have not experienced any issues in our application.

2.2.2 Structured-Light Depth Cameras

Structured-light depth sensors project a known light pattern onto the environment. When observing the scene from a different viewpoint than the projector, the light pattern appears distorted by the three-dimensional surface of the objects. Analyzing this distortion, i.e. the disparity from the original projected

Table 2.1 Range sensors specifications

	PrimeSense Carmine 1.08	**Hokuyo UTM-30LX-EW (rotating)**	**Microsoft Kinect v2**	**Intel RealSense ZR300**
Release year	2012	2012	2013	2017
Technology	Structured-light	Laser range	Time-of-flight	(Assisted) Stereo
Field of view (h×v)	57.5° × 45.0°	270.0° × 270.0°	70.6° × 60.0°	59.0° × 46.0°
Resolution	640 × 480	1080 × 40	512 × 424	480 × 360 [d]
Density	11.1 × 10.7 $\deg^{-2}$	4.0 × 0.22 $\deg^{-2}$ [b]	7.3 × 7.1 $\deg^{-2}$	8.1 × 7.8 $\deg^{-2}$
Rate (full scan)	30 Hz [a]	1 Hz [b]	30 Hz	60 Hz
Flux	3562 $\deg^{-2}s^{-1}$	0.9 $\deg^{-2}s^{-1}$	1537 $\deg^{-2}s^{-1}$	3820 $\deg^{-2}s^{-1}$
Operating range	0.8–3.5 m	0.1–30 m	0.5–4.5 m	0.5–2.8 m [e]
Sunlight resistance	No	Yes	(Yes)	Yes
Low-light resistance	Yes	Yes	Yes	Yes
Dimensions (w×d×h)	180 × 35 × 25 mm	62 × 62 × 87.5 mm	249 × 66 × 67 mm	155 × 9 × 35 mm
Mass (without cables)	160 g	336 g [c]	970 g	85 g
Ingress protection	Closed enclosure	IP67	Venting slots	Slot around lens
Moving parts	—	Mirror and servo	—	—
Data interface	USB 2.0/3.0	Ethernet	USB 3.0	USB 3.0
Supply voltage	USB powered (5 V)	12 V DC	12 V DC	USB powered (5 V)
Max. power cons.	2.25 W	8 W	15 W	5.3 W [f]
Price	200 USD	5300 USD	200 USD	290 USD

[a] 60 Hz with a depth image resolution of 320 px × 240 px
[b] With a servo motor speed of 3.14 rad/s (30 rpm)
[c] For sensor and servo motor without mounting
[d] Support for also 640 × 480 resolution but with worse depth quality (Keselman et al., 2017)
[e] Upper range ∼30 m for passive stereo (Keselman et al., 2017)
[f] For module (1.9 W) and peripheral power (3.4 W)

pattern, structured-light sensors can reconstruct the geometry of the environment (Zanuttigh et al., 2016).

This technology is used by the *Microsoft Kinect* (2010) and by the *ASUS Xtion* (2011) which use the same depth sensors developed by *PrimeSense*, who made it available as *Carmine 1.08*. The sensors flood the scene with an infrared (IR) dot pattern to infer the geometry. The specifications of the PrimeSense Carmine 1.08 are summarized in Table 2.1. This sensor is prevalent in robotics due to its fast and high-quality dense depth images. The sensor is easily integrated on a robot thanks to its small dimensions, low weight, low power usage, and the simple communication and power interface over USB. Because of the relatively small FOV of the sensor, often multiple sensors are mounted on the robot pointing in different directions to cover larger areas of the environment (Rafibakhsh et al., 2012). Although the minimum perceivable range is specified by the manufacturer as 0.8 m, the sensor can in practice often measure objects up to 0.5 m. PrimeSense also offers with the *Carmine 1.09* a short range version of the sensor with a measurement range of 0.35–3.0 m. The strongest limitation of the structured-light technology used in these sensors is that they do not function reliably in direct and indirect sunlight (El-Iaithy, Huang, and Yeh, 2012). In outdoor situations, the projected IR light has to compete with the ambient light which has orders of magnitude higher power, rendering the pattern invisible.

2.2.3 Laser Range Sensors

Ranges can be measured by estimating the distance that light travels from the sensor to the object. Laser range sensors (laser rangefinders/LIDAR sensors) use this principle to determine the traveled distance of a laser beam. A laser impulse is transmitted from the emitter onto the object and is partially reflected to the detector. The emitter and detector often use phase or amplitude modulation to determine the signal transit time and compute the target distance from this. Typically, one or multiple laser beams are swept over the scene to receive a three-dimensional scan.

Popular choices for laser range sensors for mobile robots are the devices by *Velodyne* such as the *HDL-32E* (2012) and *VLP-16 PUCK* (2014). These models feature a rotating array of 32 and 16 lasers with a vertical FOV of 40° and 30°, respectively. This setup makes these sensors suitable for localization and collision avoidance but limiting for the application of dense terrain reconstruction, if they are not equipped with an addition sweeping mechanism.

As an alternative, researchers often use single-beam laser range sensors and rotate the device around an additional axis with a motor (e.g., Johnson et al., 2017; Schwarz et al., 2017; Stentz et al., 2015). The mirror inside these sensors reflects the laser beam in different directions and thus creates a two-dimensional scan slice. The additional rotation sweeps the scan slice over the scene and allows assembling a point cloud of the environment. In our work, we are using the *Hokuyo UTM-30LX-EW* laser range sensor mounted with a custom mechanism on a *Robotis Dynamixel MX-64* servo motor (Hutter et al., 2017a). The key specifications of this sensor/servo combination are given in Table 2.1. A critical configuration in this setup is the rotational velocity of the servo motor. Lower velocities lead to high-resolution but slow scans, while higher velocities give fast but sparse scans. We set a velocity such that a full rotation of the sensors is achieved in 2 s. With a full rotation of the sensor, the scene is swept twice by the scanner generating a full scan at a rate of 1 Hz. The two rotation axes (mirror and servo motor) provide a large FOV, and the operating range of the sensor spans the entire range interesting for local mapping. The large FOV and upper range of 30 m also qualifies for the usage for localization and large scale mapping and monitoring (Hutter et al., 2017b; Pomerleau, 2013). Important for outside usage, this laser range sensor is fully resilient against direct and indirect sunlight and is ingress protected. However, this setup has a low scan rate because it acquires the data sequentially with the sweeping motion of a single laser beam. This also leads to an asymmetric density and a scan flux orders of magnitude lower than other scanning approaches. Note that while the density can be increased with the choice of lower servo speed (at the cost of lower scan rates), the flux is independent of the rotational velocity of the servo. Furthermore, the system needs to compensate for the motion of the robot during scanning, and the rotation mechanism adds complexity to construction, electrical connecting, and calibration.

2.2.4 Time-of-Flight Cameras

Similar to laser range sensors (Section 2.2.3), time-of-flight (TOF) cameras determine the distance to objects by the transit time of light pulses. Opposed to sequentially scanning the environment with one or an array of lasers, TOF cameras feature a sensor chip with a matrix of timing circuits. A strobed infrared light illuminates the scene, the light is reflected by obstacles, and the time of flight for each pixel is registered by a IR camera. This type of measurement allows these sensors to capture the entire scene at once.

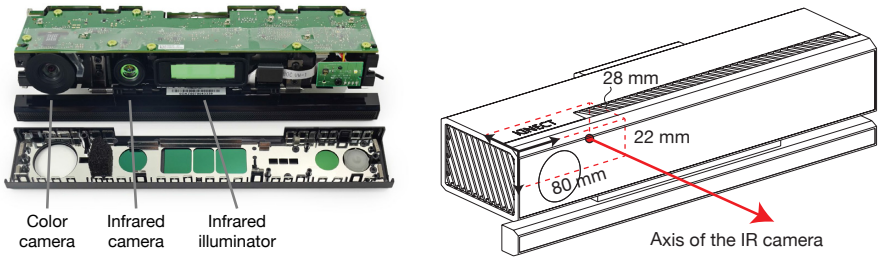
Many different manufacturers exist and an overview is given by Foix, Alenya, and Torras (2011) and by Fürsattel et al. (2016) for the latest available sensors. We have been working with the *Microsoft Kinect for Windows v2* (Sell and O'Connor, 2014), which in comparison to other TOF cameras provides accurate, low-noise depth measurements at a high resolution (Fürsattel et al., 2016). An overview of the technical specifications of the Kinect v2 is given in Table 2.1. This sensor delivers depth measurements at a density, rate, and operating range which is well suited for local, dense terrain mapping. However, obstacles close to the sensor cause the camera to over-saturate from the active IR illumination, depending on the brightness of the observed surface. Similarly, sunlight can cause interference problems with the IR camera, causing unreliable or no depth measurements depending on the target distance, surface angle, and environmental light intensity and incidence angle. A more detailed analysis of the over-saturation and interference limitations is given in Section 2.3. As TOF sensors flood the entire scene with IR light, they use a lot of power when compared to other sensor types, which requires appropriate heat dissipation. In the case of the Kinect v2, this leads to a relatively bulky and heavy device with separate data and power connections and venting slots for airflow.[1]

2.2.5 Active Stereo Cameras

Stereo camera depth estimation is based on the triangulation of a target from two different viewpoints. The distance to the target can be determined by knowing the relative position between the two cameras (baseline) of a stereo sensor and identifying a common point in the two images. Identifying a common point is based on the existence of texture in the scene. In darkness, and when ground and walls have a homogeneous appearance, the stereo matching procedure can fail. To improve over these limitations, active stereo vision uses a projected light pattern to aid the correspondence problem. Different from the light pattern in structured-light depth cameras (see Section 2.2.2), the pattern does not have to be known.

Recently, *Intel* developed with the *RealSense* a family of active stereo vision depth sensors. They use a IR laser projector and two IR cameras to output depth images together with color images from an RGB camera. We have been working with the *RealSense ZR300* (see details in Table 2.1), which outputs high-resolution depth images at a high rate. The minimum range of the sensors

[1] In addition to the TOF camera, the Kinect v2 also features i.a. a color camera and an array of microphones.



(a) Structure of the Kinect v2 sensor

(b) Optical center of the infrared camera

Figure 2.2 The Kinect v2 senses depth by measuring the time it takes for light to travel from the infrared illuminator to the object and back to the infrared camera (a). Estimated optical center of the infrared camera. The axis of the IR camera is assumed to follow perpendicularly through the front plate. Image of Kinect v2 adapted from iFixit (2013).

is defined by the search range of the stereo matching algorithm (Keselman et al., 2017). The maximum range for the active stereo vision is limited by the visibility of light pattern. Although specified at 2.8 m, the maximum range is often more dependent on the illumination. If the light pattern is not detectable in the image (for example due to sunlight), the depth estimation method falls back on passive stereo matching which can reach up to 30 m for well-textured objects (Keselman et al., 2017). Well suited for mobile robots, the RealSense devices are relatively lightweight, small, and USB-powered.

2.3 Evaluation and Modeling of the Kinect v2 Sensor

The Microsoft Kinect v2 was developed as video game console accessory featuring a state-of-the-art TOF depth sensor (Section 2.2.4) as shown in Figure 2.2a. Integrating this device in a robotic system requires a calibration of the sensor and an analysis of the error characteristic of the measurements. Depth measurements from TOF cameras are corrupted by systematic, reproducible errors and non-systematic, random errors (noise). We describe our work for the calibration and analysis of systematic errors in Section 2.3.1. Section 2.3.2 presents the results of the performance analysis and noise modeling for indoor situations. Building on the same experimental setup, Section 2.3.3 extends the analysis for outdoor situations with overcast and direct sunlight conditions. In the course of this work, we have used the open-source Kinect v2 driver by Blake, Echtler, and Kerl (2013–2018) and its associated ROS packages contributed by Wiedemeyer (2014–2018).

2.3.1 Systematic Errors & Calibration

In the following, we analyze the systematic errors that remain after a calibration procedure. Different types of systematic errors have been studied for TOF cameras (Foix, Alenya, and Torras, 2011), and we set the focus on the *depth distortion* and the *amplitude-related errors*. We neglect *integration-time-related errors*, as the integration time (IT) cannot be selected by the user for the Kinect v2. Before taking measurements, the device was run for more than 20 min to account for *temperature-related errors* of the retrieved data (Breuer, Bodensteiner, and Arens, 2014). We calibrated the intrinsic parameters of the IR camera with the help of a checkerboard and the software from Wiedemeyer (2014–2018) which uses the *OpenCV* (Bradski, 2000) calibration routine. The average reprojection error after calibration was below 0.5 px.

We estimate the depth distortion as the difference between the real distance and the mean measured depth. For this, the depth sensor was positioned to perpendicularly face a white wall at several distances. Depth measurements were taken from a small patch at the image center and averaged over 100 samples. The real distance was measured with a tape measure with an accuracy of ±1 mm. Figure 2.3a shows the resulting distance offsets for the range 0.7–2.75 m. The mean of the offset distances was used to determine the position of the coordinate system associated with the depth measurements along the camera axis (Figure 2.2b). The measured depth distortion was found to oscillate between ±6 mm, which is often referred to as *wiggling error* of TOF cameras (Foix, Alenya, and Torras, 2011; Lefloch et al., 2013).

Amplitude-related errors typically occur due to inconsistencies of the infrared light reflected from objects. This can be caused by non-uniform illumination and scenes with objects at different distances and surfaces with different reflectivities. We analyzed the amplitude-related errors by comparing time-averaged data from the sensor facing a white wall with data of a plane fit. The data were averaged over 100 depth images of the same scene, and a plane was fit using a linear least squares regression. Figure 2.3b shows the error for each pixel compared to the reference plane. Our measurements show that the systematic depth error depends on the distance to the obstacle and the image region. The error at the image center is in the range of ±6 mm and increases towards the edges up to 30 mm for a distance of 2.0 m. We can expect additional amplitude-related errors up to 60 mm due to the influence of different materials and colors (Lachat et al., 2015).

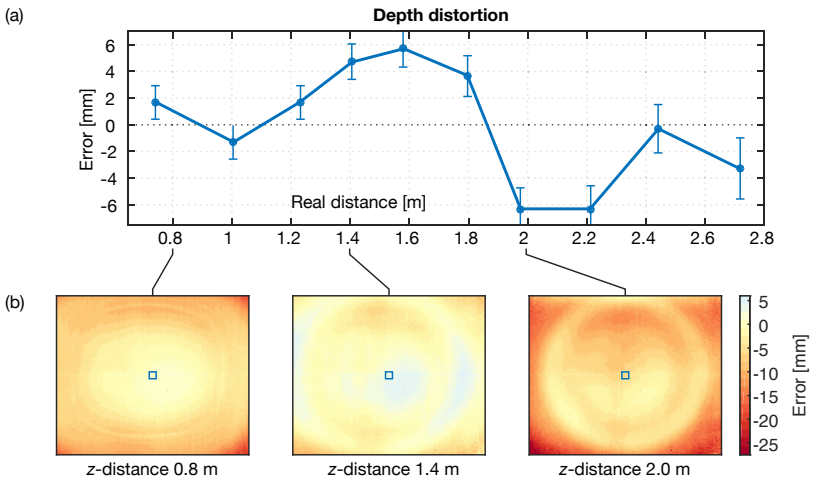


Figure 2.3 The depth distortion error describes the offset between true distance and the corresponding averaged depth measurement (a). The whiskers depict one standard deviation σ_z above and below the mean of the data. The depth distortion follows an oscillatory pattern over the range of distances. This *wiggling effect* appears due to irregularities in the modulation process of the illuminator. The colored depth errors in (b) show the difference between an averaged snapshot of a flat wall and a reference plane. Depth overestimation can be observed due to low illumination at the image borders.

2.3.2 Indoor Evaluation

Further processing of the measurement data requires characterization of non-systematic errors with a suitable statistical model. To take full advantage of the sensor, we are interested in a three-dimensional noise distribution of the depth measurements. For this purpose, we empirically derived the parameters of a noise model for the *axial* (along the sensor camera-axis) and *lateral* (in directions perpendicular to the camera-axis) errors.

Experimental Setup

Our experimental setup is identical to the setup proposed in (Nguyen, Izadi, and Lovell, 2012). We placed a rotatable planar target in front of the Kinect v2 sensor such that it appears in the center of the image (Figure 2.4). Measurements were taken at different z-distances and for varying rotation angles θ (Figure 2.4a) of the target around the vertical axis (image y-axis). For a static setup, the axial noise parameter was determined as the standard deviation for each pixel's depth measurement (Figure 2.4b). The lateral noise parame-

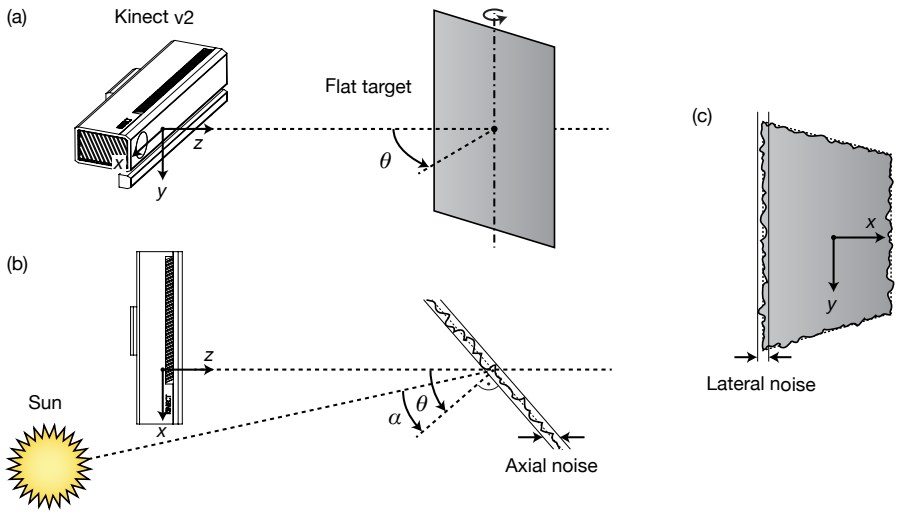


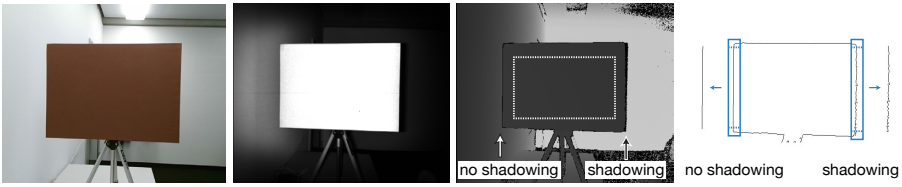
Figure 2.4 The axial and lateral noise of the depth sensor is evaluated in an experimental setup where the sensor is pointed at a flat target at different distances z and angles θ (a). The axial noise is estimated from the variation in time of measurements along the z-axis as shown in the top view (b). The lateral noise is extracted from the variation of the pixels along the edge of the target in the depth image plane as depicted in the frontal view on the target (c). Illustration based on Nguyen, Izadi, and Lovell (2012).

ter was obtained from measurements at pixels along the vertical edges of the planar target (Figure 2.4c).

Figure 2.5 shows the appearance of the target in the color (a), infrared (b), and depth image (c) produced by the Kinect v2. We automated the process of extracting the noise parameters by applying a sequence of filters (thresholding, erosion, and canny edge detection) to find corners of the planar target. For the axial noise, we chose an inscribed rectangle with defined margins as interest region to assure that the measurements were not further corrupted by edge effects (see Figure 2.5c). For the lateral noise, the edge pixels were extracted by setting regions of interest near to edges of the target and then cropping the upper and lower parts (see Figure 2.5d).

Noise Modeling

The target was placed in front of the camera at varying distances between 0.7 m and 3.1 m in 0.3 m increments. For each target distance, the angle of incidence was varied between 0° and 75° with a step size of 15°. To ensure a similar coverage of the target in the resulting fields of view, different target



(a) Color image (b) Infrared image (c) Depth image (d) Extraction of lat. noise

Figure 2.5 Example data from the Kinect v2 with view on the target from the color camera (a), infrared camera (b) and the depth map (c). Axial noise is extracted from a region inside the target (c) and lateral noise from the cropped left/right borders of the target (d). Different noise magnitudes can be observed for the left and right edge of the target (c, d) due to shadowing of the infrared emitter (left: no shadowing, less noise / right: shadowing, more noise).

sizes were used depending on the target distance: A2 (420 mm × 594 mm) for distances 2.2–3.1 m, A3 (297 mm × 420 mm) for distances 1.9–1.0 m, and A4 (210 mm × 297 mm) for a distance of 0.7 m. For each specific target angle and distance, the noise was determined for each pixel from a series of 100 depth images.

The illuminator and the IR camera are located at slightly different positions on the Kinect v2 (see Figure 2.2a). This can lead to the effect that shadows become visible in the depth image due to occlusion of the IR light emitter by foreground objects. In Figure 2.5c, the left edge of the target shows no shadowing, while at the right edge, shadowing is apparent. We observed that measurements originating from the shadowing-affected edge are considerably noisier. For this reason, we differentiate our analysis of the lateral noise with respect to the presence of shadowing.

Figure 2.6 illustrates the standard deviations of axial errors over a range of z-distances and target rotation angles θ. As shown by the noise distributions (for target angle $\theta = 0°$), the spread of the axial noise can be approximated well using a normal distribution. The axial noise standard deviation σ_z increases quadratically with the z-distance and is $\sigma_z < 4\,\text{mm}$ for angles $0° < \theta < 45°$, but it increases quickly for angles $\theta > 45°$. This increase for larger angles can be partly explained by the increased influence of the lateral noise on the axial noise: For larger incidence angles on a planar surface, errors in the bearing of the depth measurements have a stronger influence on the depth measurement

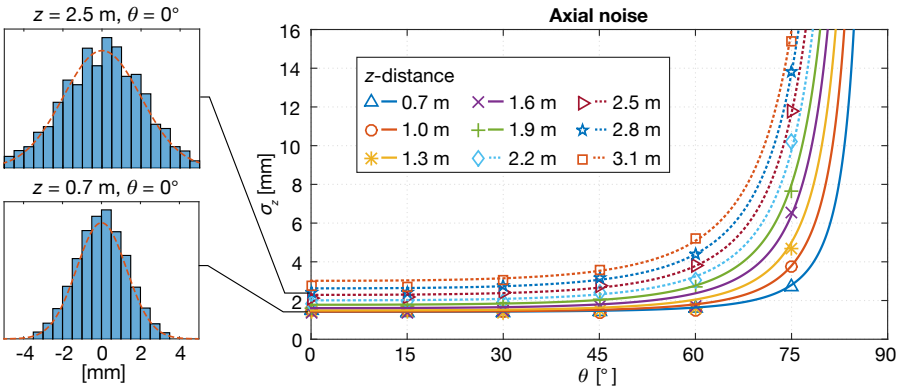


Figure 2.6 Measurements and fitted models for the axial noise σ_z. The noise increases quadratically with the measurement distance z and approaches infinity for angles towards $\theta = 90°$. The histograms illustrate the spread of the of distribution for two sample points and the fitted normal distributions (using empirical standard deviations) are plotted as a dashed line.

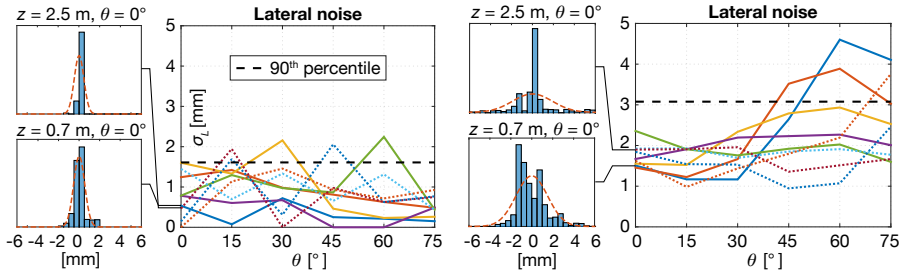
itself. We approximate the standard deviation of the axial noise as a quadratic function of $z\,[\mathrm{m}]$ as

$$\sigma_z(z,\theta)[\mathrm{mm}] = 1.5 - 0.5z + 0.3z^2 + 0.1z^{\frac{3}{2}} \frac{\theta^2}{(\frac{\pi}{2} - \theta)^2}\,, \tag{2.3}$$

with a root-mean-squared (RMS) error of the regression of 0.002 mm. Similarly to the model made in (Nguyen, Izadi, and Lovell, 2012), we added a hyperbolic term depending on $\theta\,[\mathrm{rad}]$ to account for the growing noise as θ approaches 90°, and manually determined the factor $z^{\frac{3}{2}}$.

Figure 2.7 depicts the lateral error characteristics for edges with and without shadowing effect. As shown, no apparent tendency for the lateral noise σ_L can be observed. Therefore, we use a conservative estimate of the lateral noise over the entire range of z and θ for further applications. Based on the 90^{th} percentile of the standard deviations, conservative values of the standard deviation of the lateral noise are

$$\sigma_L = 1.6\,\mathrm{mm} \quad \text{for edges without shadowing,} \tag{2.4}$$
$$\sigma_L = 3.1\,\mathrm{mm} \quad \text{for edges with shadowing.} \tag{2.5}$$



(a) Without shadowing

(b) With shadowing

Figure 2.7 The characteristics for the lateral noise σ_L without (a) and with shadowing influence (b) are shown. As no clear tendency arises, the 90th percentile is proposed as conservative value for further applications.

Short-Range Evaluation

At short distances, the active illumination may cause the IR sensor's cells to over-saturate. We examined the short-range measurement characteristics of the Kinect v2 with an IR bright and dark target surface. The obtained depth measurements are categorized into three groups: valid data, no data (detected by the sensor), and outliers (determined during post-processing). As shown in Figure 2.8, the surface reflectivity has a strong influence on the short-range measurement performance. With a bright surface, the Kinect v2 reports considerable amounts of missing data and outliers for distances $z < 0.7\,\text{m}$. The sensor struggles to detect over-saturation at times which can be observed as an outlier transient between valid and no data. Measurements on a dark surface at short distances are less error-prone, and the Kinect v2 works reliably down to distances of $z = 0.25\,\text{m}$.

2.3.3 Outdoor Evaluation

External high-intensity infrared light makes it difficult for TOF sensors to differentiate the active infrared signal from the external light. For outdoor navigation, we are interested in the depth measurement performance in overcast and direct sunlight situations.

Experimental Setup

We studied the Kinect v2 depth measurement performance in the presence of sunlight using the same approach as for our indoor experiments (Section 2.3.2). Assuming an ideal diffusely reflecting surface of the target, we can model the

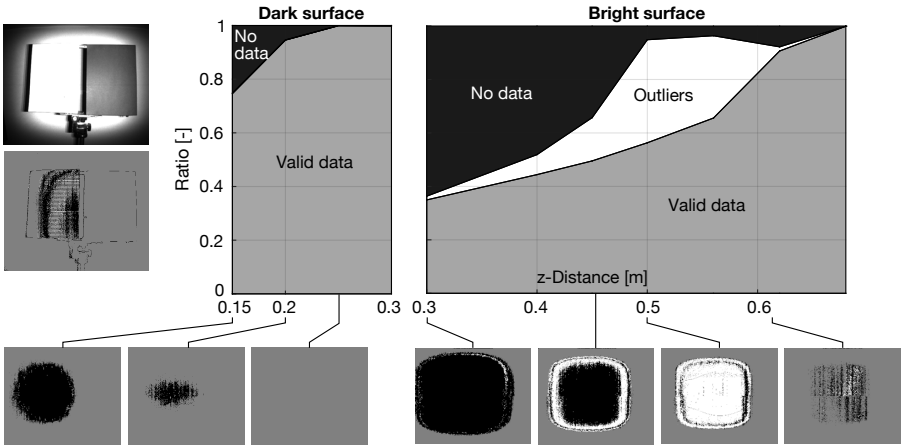


Figure 2.8 At short distances, over-saturation of the camera by the active IR illumination can be observed, leading to invalid measurements. The ratio between *valid data*, *no data*, and *outliers*, plotted for IR bright and dark surfaces at short distances. Observing dark surfaces, the Kinect v2 sensor is able to reliably measure distances for $z > 0.25\,\text{m}$ due to the lower reflectiveness. In the presence of bright surfaces, measurements are corrupted for distances up to 0.7 m.

influence of sunlight by describing the angle of incidence with α as depicted in Figure 2.4b. To simplify the measurements, we set up the experiments such that the sun was directly behind the sensor, i.e., $\alpha = \theta$. The intensity of the environmental light on the target was measured with a lux meter and ranged 1–12 kLux for the overcast and 85–95 kLux for the direct sunlight condition.

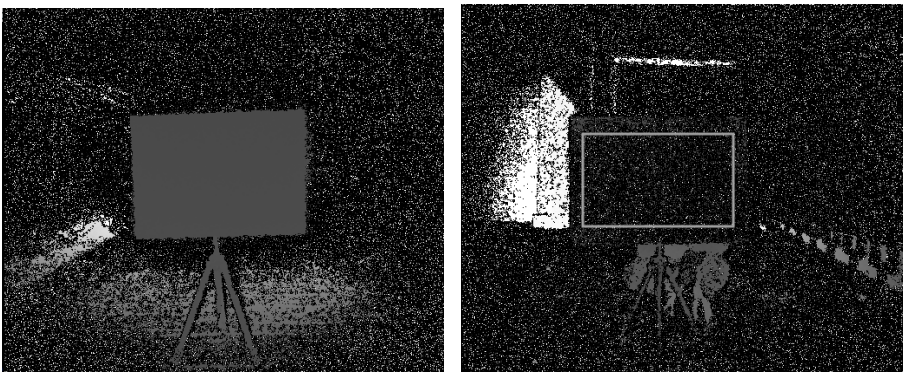
Noise Characterization

For the overcast situation, we are able to capture the main characteristics of the noise amplitude by adapting the coefficients of the indoor model (2.3):

$$\sigma_z(z,\theta)[\text{mm}] = 2.5 - 0.7z + 0.9z^2 + 0.5z^{\frac{3}{2}} \frac{\theta^2}{(\frac{\pi}{2} - \theta)^2} \,, \tag{2.6}$$

with a resulting RMS regression error of 0.55 mm (compared to 0.002 mm for indoors). We found that ranges between 1.0–2.8 m could reliably be measured, whereas measurements outside of this range were considered as unreliable and therefore omitted from the noise model fitting. For the lateral noise, we again found the 90^{th} percentile of the standard deviation as a fixed value with

$$\sigma_L = 6.8\,\text{mm} \,, \tag{2.7}$$



(a) Overcast (b) Direct sunlight

Figure 2.9 The comparison between overcast (1–12 kLux) (a) and direct sunlight (85–95 kLux) (b) reveals a significant reduction of data quality in sunlight for the same target distance $z = 1.6\,\mathrm{m}$.

for both cases with and without shadowing. Our measurements indicate that, by exposing the Kinect v2 to overcast light, the axial and lateral noise in the depth measurements must be expected to increase by factor 2–3.

During direct sunlight exposure, a strong influence of the sensor-to-target distance and the sunlight incidence angle α was observed. Figure 2.9 illustrates the difference between overcast and direct sunlight environments. While 100 % valid measurements could be acquired at a distance of $z = 1.6\,\mathrm{m}$ in overcast weather (Figure 2.9a), only little valid data was captured using the same setup in direct sunlight (Figure 2.9b). The rapid degradation of the measurement data for increasing target distances is shown in Figure 2.10 together with example measurements for an angle of $\theta = 15°$. For distances $z < 1.0\,\mathrm{m}$, reliable measurements could be acquired even under perpendicular sunlight illumination of the target plane. The amount of valid data dropped for increased z-distances until almost no data could be gathered for distances $z > 1.9\,\mathrm{m}$. We also observed that the sunlight incidence angle considerably influences the measurement performance. In particular, we found that a more direct incidence (maximum at $\alpha = 0°$) caused stronger degradation of the data than less direct incidence (such as $\alpha = 75°$).

This behavior is also reflected in the noise amplitude characteristics. Figure 2.11 shows the axial standard deviation σ_z as a function of the target angle $\alpha = \theta$. In general, the measurement noise for direct sunlight environments is roughly an order of magnitude higher than for indoor scenes. Since

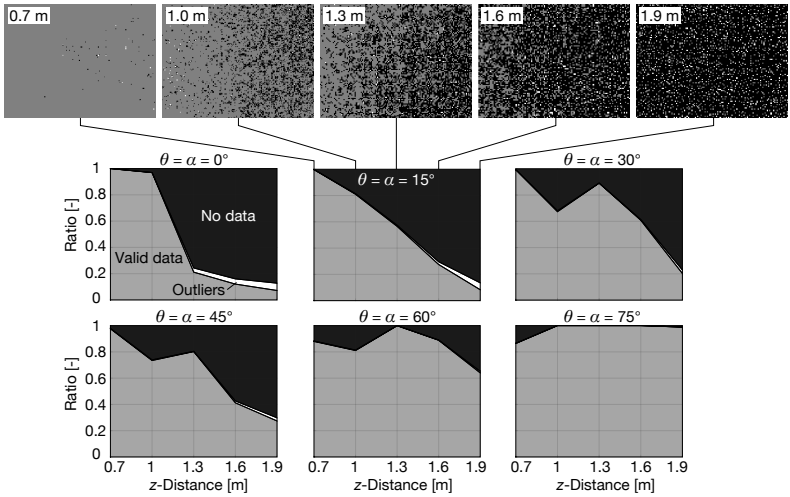


Figure 2.10 The ratio between *valid data*, *no data*, and *outliers* is illustrated for the direct sunlight scene over a range of distances z and angles $\theta = \alpha$. Higher z-distances and more direct sunlight on the target (small α) decrease the reliability of the Kinect v2 to take valid measurements. Sample images of the target plane at angle $\theta = 15°$ indicate the rapid increase in the amount of invalid measurements for increasing distances.

we have determined the sensor's noise characteristics without environmental light influence (Figure 2.6), we may argue that the increase in noise is due to the presence of sunlight. As suggested by Lambert's cosine law, the intensity observed by the target (assumed to be ideally diffusely reflecting) is proportional to the cosine of the incidence angle α. We extend our existing model accordingly by introducing the dependency on $\cos\alpha$ as

$$\sigma_z(z,\theta,\alpha)[\text{mm}] = 28 - 38z + 2.0z^2 + 0.3z^{\frac{3}{2}}\frac{\theta^2}{(\frac{\pi}{2}-\theta)^2} + 42z^2\cos\alpha\,, \qquad (2.8)$$

where the factor z^2 was determined manually. The resulting model describes the noise characteristics with an RMS regression error of 4.4 mm.

2.4 Analysis and Conclusions

Comparing the features and characteristics of the depth sensors introduced in Section 2.2, significant differences can be observed for performance and non-performance criteria. In the following, we analyze the introduced sensors for their measurement noise and scan density and conclude the sensor selection.

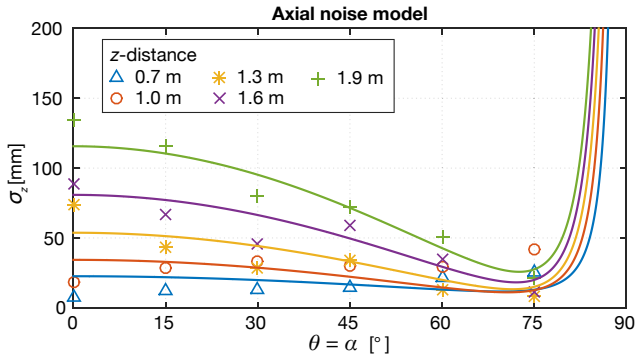


Figure 2.11 Plot of the empirically determined axial noise standard deviation σ_z for direct sunlight. To simplify the experiments, we took measurements with the sun located directly behind the sensor, such that $\alpha = \theta$. The noise magnitude increases with the measurement distance z and for lower incidence angles $\alpha = \theta$. The plotted lines correspond to an extended model fit which takes the sunlight angle α into account.

2.4.1 Noise Analysis

Manufacturers often give inconsistent specifications on the precision of the sensors. To compare the sensors regarding their noise characteristics, we forego the manufacturer's specifications and rely our analysis on empirically derived noise models created from experimentally collected data. Additionally, these noise models lay the foundation for further processing of the measurement data in a statistical process. Figure 2.12 shows the axial and lateral noise models for the discussed sensors for an incidence angle $\theta = 45°$ (see Figure 2.1). This angle between measurement ray and object is representative for the center measurement point of a 45° downwards tilted sensor. We have derived the sensor noise model for the Kinect v2 in Section 2.3. The model for the Carmine 1.08 is taken from Nguyen, Izadi, and Lovell (2012), the model for the UTM-30LX-EW from Pomerleau et al. (2012), and the one for the RealSense ZR300 from Takahiro, Fankhauser, and Bjelonic (2017) for the onboard default stereo matching. The noise is for all sensors in the same order of magnitude. For short distances $z \leq 2.0\,\text{m}$, the axial noise standard deviation is $\sigma_z < 20\,\text{mm}$, except for the Kinect v2 in direct sunlight. For higher distances $z > 2.0\,\text{m}$, the measurements of the RealSense ZR300 suffer the most from a substantial noise increase. For the lateral noise, we find for all sensors a standard deviation $\sigma_L < 7\,\text{mm}$ again with the highest increase from the RealSense ZR300's measurements.

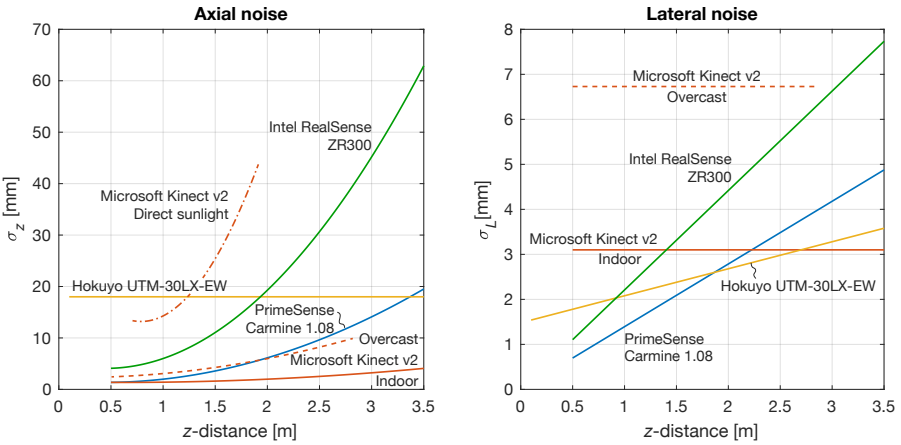


Figure 2.12 The axial and lateral noise models are plotted for the various sensors for an incidence angle between measurement beam and surface normal of $\theta = 45°$. These noise models are empirically derived and used for evaluation and as statistical model in our mapping framework (Chapter 3).

2.4.2 Density Analysis

As discussed in Section 2.2.1, a high scan flux is needed to perceive a dense reproduction of the terrain as the robot navigates through the environment. While the angular density d_{ang} and angular flux f_{ang} of sensor is based on the number of sample points per angle difference, we use the *areal density* d_{areal} and *areal flux* f_{areal} based on the number of sample points per area in the scene given in m^{-2} and $\text{m}^{-2}\text{s}^{-1}$, respectively. Given the angular density d_{ang} of a sensor's measurements and the sensor's position relative to the perceived objects, we can compute the local areal density d_{areal} and areal flux f_{areal} of the measurements on the object. For a point P with position vector $\boldsymbol{r}_{SP}$ from the sensor's optical center S to the point (see Figure 2.1), the areal density can be approximated by

$$d_{\text{areal}}(\boldsymbol{r}_{SP}, \boldsymbol{n}_P) = \underbrace{\frac{|\cos\theta_P|}{\|\boldsymbol{r}_{SP}\|_2^2}}_{k} \; d_{\text{ang}} \; , \tag{2.9}$$

$$\theta_P = \cos^{-1}\left(\frac{\boldsymbol{r}_{SP} \cdot \boldsymbol{n}_P}{\|\boldsymbol{r}_{SP}\|_2}\right) \tag{2.10}$$

with θ_P the angle between the vector $\boldsymbol{r}_{SP}$ and the local surface normal $\boldsymbol{n}_P$. This approximation assumes a constant angular density d_{ang} for a sensor and

neglects any optical influences such as lens distortion. Similarly, the areal flux can be computed with

$$f_{\text{areal}}(\boldsymbol{r}_{SP}, \boldsymbol{n}_P) = k(\boldsymbol{r}_{SP}, \boldsymbol{n}_P)\, f_{\text{ang}} \,, \tag{2.11}$$

with k the factor given in (2.9).

Figure 2.13 shows a top view of one full scan of flat ground for the rotating UTM-30LX-EW laser range sensor ($f_{\text{ang}} = 0.9\,\text{deg}^{-2}\text{s}^{-1}$) and the RealSense ZR300 assisted stereo camera ($f_{\text{ang}} = 3820\,\text{deg}^{-2}\text{s}^{-1}$). In this comparison, we use the RealSense ZR300 as a placeholder for the sensor frame based devices, as the Carmine 1.08 and Kinect v2 have a similar angular flux f_{ang}. Using (2.9) and (2.11) we can estimate the areal density d_{areal} and areal flux f_{areal} for the two sensors on the ground with surface normal vector ${}_I\boldsymbol{n} = (0, 0, 1)^\mathsf{T}$ in the inertial frame I. The two sensor setups have a very different performance regarding the number of measurements on the terrain. The entire area covered by RealSense ZR300 is densely covered by measurement points with an averaged areal density $d_{\text{areal}} = 2.4\,\text{cm}^{-2}$ for even the most distant point (1.7 m distance to sensor). With the high rate of the RealSense ZR300, this means that an area of 1 cm × 1 cm is measured 142 times per second and even 2560 times for the closest point in front of the robot (distance 0.7 m). This stands in stark contrast to the measurements' density of the rotating UTM-30LX-EW sensor. Assuming that the measurements are equally spread, it takes 1.9 s to cover every 1 cm × 1 cm cell close to the sensor and 22 s for an area with 1.7 m distance to the sensor. In reality, the asymmetric density of this scanning method leads to dense scan lines which are spread sparsely over the scene (Figure 2.13), increasing the time for a densely covered terrain even more.

Considering the vast amount of data collected by the sensor frame based devices such as the RealSense ZR300, the filtering and mapping algorithms are often computationally saturated and start dropping scans. Instead of using the full resolution scans at lower rates, we suggest working with lower resolution data at a higher rate. This allows the robot to perceive a changing environment with less delay. Inverting the calculations of (2.9) and (2.1), we can derive a resolution corresponding to a desired areal density of the measurement points in the scene. A good trade-off between rate and resolution is to choose the areal density such that areas at the size of the map discretization are covered with at least one measurement for the entire FOV. For example, with the RealSense ZR300 tilted downwards at 45°, we would choose a reso-

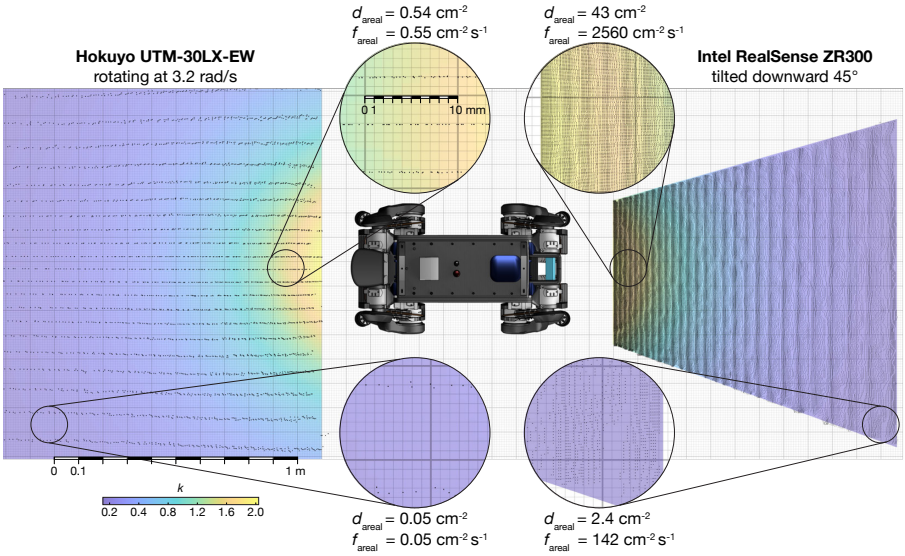


Figure 2.13 The figure shows the range measurement on flat ground for one full scan of the rotating Hokuyo UTM-30LX-EW laser scanner (left) and the Intel RealSense ZR300 active stereo camera (right). Note that the perceived area for the UTM-30LX-EW sensor is wider and was cropped for this illustration. The highlighted parts show close-ups of the areas with maximum and minimum areal density d_{areal} and flux f_{areal}, which varies greatly for the two different sensors.

lution of 312×234 for $1\,\text{cm} \times 1\,\text{cm}$ map discretization cells or 156×117 for $2\,\text{cm} \times 2\,\text{cm}$ cells.

2.4.3 Conclusions

For the application of dense short-range mapping on a mobile robot, no sensor excels in all the evaluation criteria. We have been successfully using all the introduced sensor on our robots during the development work. While all sensors are applicable, recommendations can be given for specific use cases.

Structured-Light Depth Cameras

The popularity of structured-light depth sensor such as the PrimeSense Carmine 1.08 is justified because these products provide high-quality, dense depth images from a small, light-weight, low-power, and cheap device. While excellent for indoor applications, the sunlight sensitivity of these sensors excludes them for robots operating outdoors. However, future developments with higher illumination power might lift this constraint.

Time-of-Flight Cameras

Recent developments for TOF cameras have increased their resolution to be on a par with other sensor technologies and devices like to Microsoft Kinect v2 provide excellent range measurements. Our evaluations (Section 2.3.3) have shown that considerable data degradation regarding increased noise, number of outliers, and amount of missing data has to be expected. If a good noise model is of importance, such as in the case of accurate terrain reconstruction, additional provisions have to be made to estimate the influence of the ambient light. Additionally, the Kinect v2 is the heaviest and largest device with the highest power consumption compared to the other sensors. While still relatively heavy and high-power, newer devices such as the *Fotonic G-Series* (2017) are promising because of their smaller size and IP67 ingress protection.

Laser Range Sensors

Laser range sensors such as the Hokuyo UTM-30LX-EW provide a large measurement range and strong resilience to ambient lighting. When mounted on a rotational axis, this setup provides a large FOV also covering the environment on the side and above the robot. This makes a rotating LIDAR also attractive for localization and monitoring, enabling the usage of one sensor for multiple tasks. However, the orders of magnitude lower density compared to the other sensors (Section 2.4.2) severely limits the speed of the robot and increases the reaction time to changes in the environment. In comparison to the static setup of other sensors, the rotation mechanisms (mirror and servo motor) are less elegant as they add complexity to construction, electrical connection, and calibration. In the future, solid state LIDAR array sensors such as the *Velodyne Velarray* (2018) might resolve some of these drawbacks.

Active Stereo Cameras

Similar to the structured-light depth cameras, active stereo cameras can be built as a small, light-weight, and low-power devices such as the Intel RealSense ZR300. The main difference is that the active stereo vision technology gracefully adapts to environments with strong ambient IR light if sufficient visual structures are available. We have found that this transition from active to passive stereo sensing is mostly unnoticeable as natural, urban, and outdoor industrial environments are typically visually highly structured. We were also able to considerably improve the stereo depth estimation over the default algorithms running on the device by running custom stereo matching software

(Hirschmuller, 2008) on the host computer at the cost of using more computational power. Compared to the other sensors, the RealSense ZR300 are slightly more corrupted by noise (Section 2.4.1), especially at higher ranges. This requires a stochastic estimation framework to yield accurate terrain maps. When lowering the resolution of the two IR camera images as discussed in Section 2.4.2, we get the positive effect of a shortened minimum range (assuming equal stereo matching range) and increased accuracy (Keselman et al., 2017). The latest products such as the *RealSense D435* (2017) show promising developments with a better signal quality and an increased FOV.

2.5 Summary

Different range sensing technologies exist which all have their unique advantages. We have evaluated four different (structured-light, laser range, time-of-flight, active stereo) technologies regarding their applicability for short-range terrain perception with a mobile robot. Defining the terms scan density and flux allowed us to compare the data stream for sensors with different FOV and measurement frequencies. We have presented a model to approximate the expected resolution of the terrain reconstruction which can be used to find the desired trade-off between map definition and update rate. For the Microsoft Kinect v2 TOF sensor, we have given an analysis of its performance at close ranges and outdoors in overcast and direct sunlight situations. With the derived three-dimensional noise models, we provide the prerequisites for stochastic terrain mapping approaches.

3

Terrain Mapping

This chapter incorporates material from the following publications:

- Fankhauser, P., Bloesch, M., and Hutter, M. (2018). "Probabilistic Terrain Mapping for Mobile Robots with Uncertain Localization". *IEEE Robotics and Automation Letters (RA-L)* 3.4, pp. 3019–3026
- Fankhauser, P., Bloesch, M., Gehring, C., Hutter, M., and Siegwart, R. (2014). "Robot-Centric Elevation Mapping with Uncertainty Estimates". In: *International Conference on Climbing and Walking Robots (CLAWAR)*, pp. 433–440
- Fankhauser, P. and Hutter, M. (2016). "A Universal Grid Map Library: Implementation and Use Case for Rough Terrain Navigation". In: *Robot Operating System (ROS): The Complete Reference (Volume 1)*. Ed. by A. Koubaa. Springer International Publishing. Chap. 5, pp. 99–120

Mobile robots require knowledge of the surrounding to safely and efficiently navigate through an environment. Since we are concerned with the planning of the robot's motion over and around obstacles, we are setting our focus on elevation/height maps, which simplify the terrain as a surface. These maps can be represented as 2.5D grid maps, where each cell in the grid holds a height value. Because we are working with a moving system, the sweeping motion of the range sensor over the terrain and the associated measurement errors must be carefully accounted for (Smith, Self, and Cheeseman, 1987).

Much work has focused on obtaining a *globally*-consistent map (Thrun, 2002), where an accurate absolute localization is presumed. These methods are often sensitive to environmental conditions such as sufficient lighting, the availability of visual or geometric features, or external signals such as GPS. Following our approach for robustification (Section 1.3), our focus lies on retrieving a *local*

map of the environment of the robot, independent of a global localization method. In this case, the mapping relies on the *relative* localization based on proprioceptive sensing (kinematic and inertial measurements) such as wheeled or legged odometry. Bloesch et al. (2013) have shown that using proprioceptive sensing only, the position and yaw-rotation of a robot remain unobservable. This means that the estimates of the position and yaw-rotation drift over time in comparison to their real values.[1]

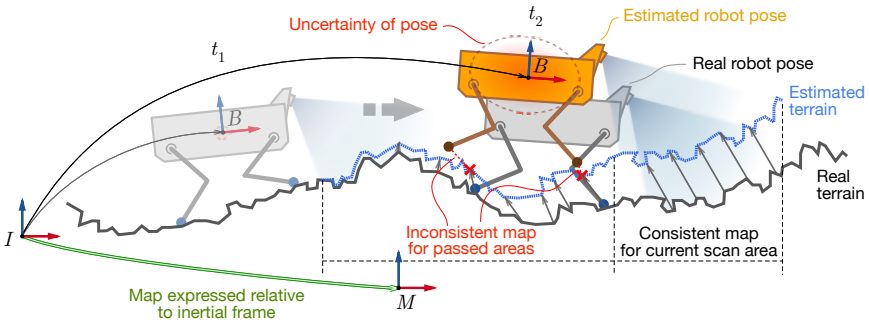
In a classical, *world-centric* mapping approach, the terrain map is associated with an inertial frame. Depending on whether uncertainties are considered or not, the use of drifting state estimation in a world-centric formulation will inevitably lead to either blurry or inconsistent maps (see Figure 3.1a). To account for the pose estimate drift probabilistically, we introduce a formulation which is expressed in a *robot-centric* formulation as shown in Figure 3.1b. In this formulation, the terrain map is associated with the current pose of the robot. At any time, the robot-centric elevation map is a local representation of the surrounding terrain, meaning that the observed regions in front of the robot have the highest accuracy, while 'older', previously seen parts of the map accumulate uncertainty and 'melt' into each other. This approach is similar to the following observation: Imagine a person walking towards a goal in a well-visible environment with obstacles. If at one point the person closes their eyes (or the environment visibility is limited), they will continue to walk confidently towards the goal. However, with every step, they will get less certain about the position of the obstacles and the goal, and will thus move more cautiously.

This chapter introduces probabilistic, local terrain mapping for a mobile robot with uncertain localization from proprioceptive sensing. Based on a robot-centric formulation, our approach allows for the consistent generation of elevation maps by incorporating the drift of a proprioceptive state estimation. We relate to existing work in Section 3.1, describe the method in Section 3.2, and show simulation and hardware-based results and a comparison to other work in Section 3.3.

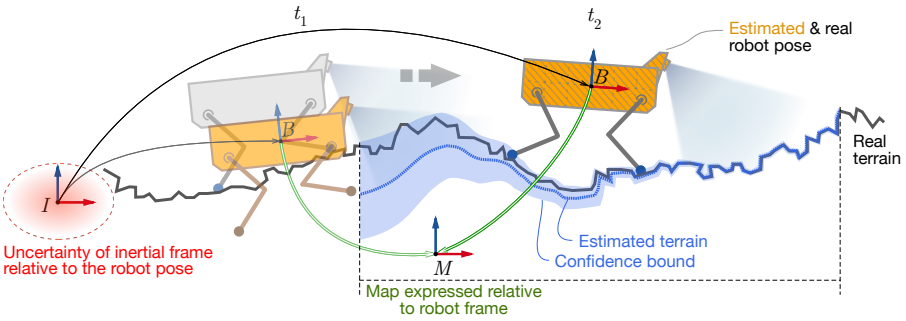
3.1 Related Work

In the context of rough terrain locomotion, many approaches (e.g. Belter, Labcki, and Skrzypczynski, 2012; Kuindersma et al., 2016; Mastalli et al.,

[1] However, the tilt rotation of the robot (pitch and roll) are observable due to the observability of the direction of gravity with an inertial measurement unit (IMU).



(a) World-centric perspective (classical approach)



(b) Robot-centric perspective (proposed approach)

Figure 3.1 A mapping procedure can be expressed from different perspectives: In a world-centric perspective ((a), classical approach), the robot base B and the map M are defined relative to the inertial frame I. As the robot moves (motion from time t_1 to t_2), errors in the pose estimation accumulate (drifting) and the real and the estimated robot poses diverge. When continuously assembling a map from distance measurements, these errors lead to inconsistent representation of the terrain which can cause faulty execution of motion plans and unwanted collision with the environment. In a robot-centric perspective ((b), proposed approach), the map M is defined expressed relative to the robot B at each time instance (depicted for time t_2). The accumulated pose error is propagated to the map data, enabling a consistent representation including estimates of the map confidence bound. This makes reliable navigation in rough terrain for a robot with uncertain localization possible.

2017b; Roennau et al., 2009) rely on a grid-based, 2.5D height map representation of the terrain. While this type of map comes with certain limitations when compared to full 3D maps (such as Hornung et al., 2013), its simple structure and better scalability allow for efficient data access and processing.

Early work on the generation of elevation maps for mobile robots was presented by Herbert et al. (1989) and Kweon and Kanade (1992). Both authors used feature matching algorithms to find the corresponding transformation between multiple scans to build a composite elevation map, but they did not address the issues of error propagation as a result of inaccuracies during matching.

Cremean and Murray (2005) developed an approach to fuse range measurements with uncertainties into a height map. When a range measurement is taken, cells that fall into the region of the measurement are updated based on previously stored data and the uncertainty of the measurement. Cells that do not receive measurement updates are left unchanged. This approach relies on absolute position measurements from GPS and is therefore unsuitable for many applications.

In the approaches presented by Belter, Labcki, and Skrzypczynski (2012) and Wooden et al. (2010), a local terrain map is used that surrounds the robot and is moved along with its motion. While we use a similar setup, their approach relies on an accurate and reliable pose tracking algorithm and does not address the issue of a drifting pose estimation.

Our approach is similar to the work by Kleiner and Dornhege (2007), where the elevation map is updated based on the motion of the robot. In their work, the uncertainty of the robot's position and orientation is reflected in the map by linearly growing the variance of the height estimate based on the accumulated distance and angle. This approach conservatively merges an approximation of the pose uncertainty into the height variance without taking the effect of in-plane uncertainty into account.

Conceptually, our work is similar to the idea presented by Stroom et al. (1999) for treatment planning in radiotherapy: They include the geometrical uncertainties caused by the translation and rotation of the organs in a geometrical model to calculate the planning target volume.

3.2 Method

In the following, we describe a method to obtain probabilistic elevation maps in a robot-centric formulation. A definition of the coordinate frames is given

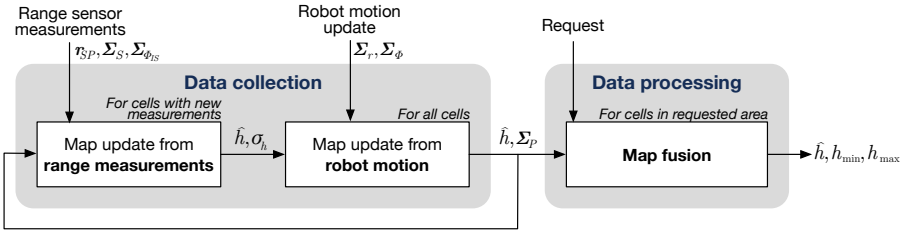


Figure 3.2 The robot-centric elevation mapping is structured in a real-time *data collection* and a parallelized *data processing* step. The data collection consists of the processing steps of the range sensor measurements and the robot motion updates to create a probabilistic terrain map. When the map is needed for further processing, the map fusion process creates a consistent estimate of the terrain map together with an uncertainty interval.

in Section 3.2.1 and the mapping algorithms are described in the remainder of this section.

The processing steps of our elevation mapping method are illustrated in Figure 3.2. In the *data collection* part, the *map update from range measurements* (Section 3.2.2) method processes the range sensor data (Chapter 2) and updates all map cells to which new measurement are associated. As the robot moves, the uncertainty of the robot's motion is propagated to the map data to comply with the robot-centric formulation. This is accomplished in the *map update from robot motion* (Section 3.2.3) for all cells in the map. These two processing steps are used to create a robot-centric elevation map in real-time. Because the computational costs rise for an increasing map size, we limit the size of the map to the local environment of the robot.

Whenever the elevation map is required for further processing, such as foothold planning or collision checking, a request triggers the *map fusion* (Section 3.2.4) in the *data processing* part of the approach. In this step, the map data is used to compute an lower and upper confidence bound for each cell based on its data and the data from its neighboring cells. The requested map is typically a submap of the entire map, which reduced the computationally intensive map fusion to the desired area. To continue the processing of new range measurements during the fusion step, the data processing part can be run in parallel to the data collection.

3.2.1 Definitions

To describe the framework, we introduce four coordinate frames, namely the inertial frame I, the robot base frame B, the sensor frame S, and the map

frame M (Figure 3.3). The inertial frame I is fixed to the environment, and we assume that the real terrain is stationary relative to this frame. The base frame B and sensor frame S are body-fixed frames attached to the center of the base and the distance sensor, respectively. We assume that there exists a known transformation $(\boldsymbol{r}_{BS}, \boldsymbol{\Phi}_{BS})$ between base frame B and distance sensor frame S. With this, we can transform any formulation between the two frames B and S, and we use them interchangeably in the following formulations.

The base frame B is related to the inertial frame I through the translation $\boldsymbol{r}_{IB}$ and rotation $\boldsymbol{\Phi}_{IB}$. This transformation is obtained through the pose estimation (Bloesch et al., 2013) and characterized by the six-dimensional pose covariance matrix $\boldsymbol{\Sigma}_{IB} = \mathrm{cov}(\boldsymbol{r}_{IB}, \boldsymbol{\Phi}_{IB})$. We can split the rotation between the inertial and base frame $\boldsymbol{\Phi}_{IB}$ as

$$\boldsymbol{\Phi}_{IB} = \boldsymbol{\Phi}_{I\tilde{B}}(\psi) \circ \boldsymbol{\Phi}_{\tilde{B}B}(\theta, \varphi) \ , \tag{3.1}$$

where $\boldsymbol{\Phi}_{I\tilde{B}}(\psi)$ describes the rotation around the vertical axis $\boldsymbol{e}_z^I$ with yaw angle ψ for the frame $\tilde{B}$ and $\boldsymbol{\Phi}_{\tilde{B}B}(\theta, \varphi)$ describes the tilt rotation between $\tilde{B}$ and B with pitch and roll angles θ and φ, respectively.

Finally, the elevation map frame M is defined through a relation to the base frame B (or sensor frame S) with translation $\boldsymbol{r}_{BM}$ and rotation $\boldsymbol{\Phi}_{BM}$, which are specified by the user. We choose the rotation $\boldsymbol{\Phi}_{BM}$ such that the z-axis of the map frame M and the inertial frame I remain always aligned ($\boldsymbol{e}_z^I = \boldsymbol{e}_z^M$). The last degree of freedom, which corresponds to the yaw angle ψ between I and M (rotation around $\boldsymbol{e}_z^I$), is chosen to match the yaw angle between I and B. The three-dimensional position of a grid cell i of the map is given by $P_i = (x_i, y_i, \hat{h}_i)$ where x_i and y_i are defined by position of grid cell i, and $\hat{h}_i$ is the estimated height of the terrain at cell i.

3.2.2 Map Update from Range Measurements

New measurements from the distance sensor are handled as points in space and are mapped to the elevation map. This results in a new height measurement $\tilde{p}$ at a cell (x, y) in the height map. In the elevation map frame M, the height measurements are approximated by a Gaussian probability distribution as $\tilde{p} \sim \mathcal{N}(p, \sigma_p^2)$ with mean p and variance σ_p^2. A single measurement, given as the position ${}_S\boldsymbol{r}_{SP}$ in the sensor frame S, can be transformed to the corresponding height measurement p with

$$p = \boldsymbol{P}\left(\boldsymbol{\Phi}_{SM}^{-1}({}_S\boldsymbol{r}_{SP}) - {}_M\boldsymbol{r}_{SM}\right) \ . \tag{3.2}$$

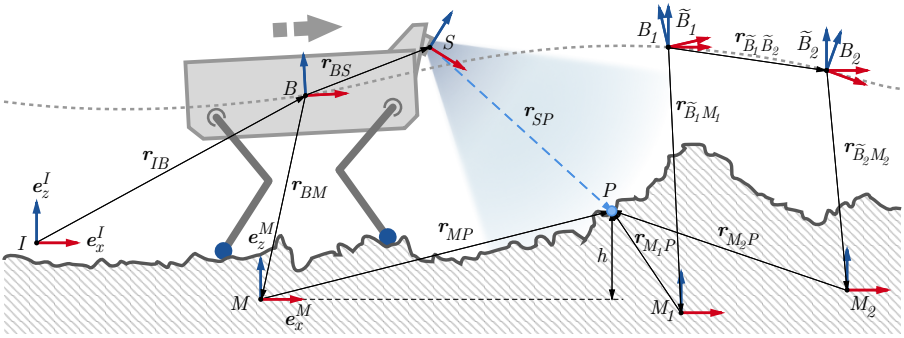


Figure 3.3 Illustration of the coordinate frames used for elevation mapping. The relation between the inertial frame I and base frame B is given by the state estimation of the robot. The sensor frame S is associated with the base B through a fixed transformation. Note that in our robot-centric formulation the map frame M is defined relative to the base frame B (as opposed to the inertial frame I). The point P is measured in the sensor frame S and incorporated in the map in frame M. The base frames $\tilde{B}$ are aligned with B in position and yaw-angle, but are not tilted as defined in (3.1) with $\boldsymbol{e}_z^I = \boldsymbol{e}_z^{\tilde{B}}$.

The projection matrix $\boldsymbol{P} = [0\ 0\ 1]$ maps the three-dimensional measurement to the scalar height measurement p (in map frame M). To obtain the variance of the height measurement σ_p^2, we derive the Jacobians for the sensor measurement $\boldsymbol{J}_S$ and the sensor frame rotation $\boldsymbol{J}_\Phi$ from (3.2) as

$$\boldsymbol{J}_S = \frac{\partial p}{\partial_S \boldsymbol{r}_{SP}} = \boldsymbol{P}\boldsymbol{C}(\Phi_{SM})^\mathsf{T} \qquad \text{and} \qquad \boldsymbol{J}_\Phi = \frac{\partial p}{\partial \Phi_{SM}} = \boldsymbol{P}\boldsymbol{C}(\Phi_{SM})^\mathsf{T}\, {}_S\boldsymbol{r}_{SP}^\times \,, \tag{3.3}$$

where $\boldsymbol{C}(\Phi)$ is used to describe the mapping to the corresponding rotation matrix (Section 1.5). The error propagation for the variance σ_p^2 is given as

$$\sigma_p^2 = \boldsymbol{J}_S \boldsymbol{\Sigma}_S \boldsymbol{J}_S^\mathsf{T} + \boldsymbol{J}_\Phi \boldsymbol{\Sigma}_{\Phi_{IS}} \boldsymbol{J}_\Phi^\mathsf{T} \,, \tag{3.4}$$

where $\boldsymbol{\Sigma}_S$ denotes the covariance matrix of the range sensor model (Chapter 2) and $\boldsymbol{\Sigma}_{\Phi_{IS}}$ the covariance matrix of the sensor rotation (submatrix of $\boldsymbol{\Sigma}_{IS}$). It is important to note that the uncertainties of the position of the sensor $\boldsymbol{\Sigma}_{\boldsymbol{r}_{IS}}$ do not need to be taken into account in this step because of our choice for the definition of the map frame M (Section 3.2.1). Furthermore, an influence of the uncertainty of the yaw-rotation of the sensor ψ (rotation around $\boldsymbol{e}_z^I$) on the measurement is excluded because of the use of the projection matrix $\boldsymbol{P}$ and our definition of frame M (Section 3.2.1).

The height measurement $(p,\, \sigma_p^2)$ is fused with the existing elevation map estimation $(\hat{h},\, \sigma_h^2)$ by means of a one-dimensional Kalman filter (Cremean and Murray, 2005):

$$\hat{h}^{+} = \frac{\sigma_p^2\ \hat{h}^{-} + \hat{\sigma}_h^{2-}\tilde{p}}{\sigma_p^2 + \hat{\sigma}_h^{2-}}\,, \qquad \sigma_h^{2+} = \frac{\hat{\sigma}_h^{2-}\sigma_p^2}{\hat{\sigma}_h^{2-} + \sigma_p^2}, \tag{3.5}$$

where estimates before an update are denoted with a $-$ superscript, respectively with a $+$ superscript if the update has already taken place.

If multiple measurements with different heights fall into the same cell (as it is in the case of a vertical wall), we employ an update rule similarly to the one presented by Kleiner and Dornhege (2007). Based on the Mahalanobis distance, the rule fuses the measurements for the highest elevation and drops measurements that fall below a certain distance from the current estimate.

Note that this method to combine multiple height measurements is identical in cases when the range measurements follow consecutively for the same area or when an area is re-traversed (e.g., when the robot turns or moves backward). In the latter case, 'old' areas will have accumulated uncertainty from the robot's motion (see Section 3.2.3), which causes (3.5) to quickly converge to the new measurements.

3.2.3 Map Update from Robot Motion

As the elevation map frame M is defined relative to the pose of the sensor/robot (Section 3.2.1), the elevation map data needs to be updated whenever a motion of the robot relative to the inertial frame I has occurred. This is necessary because the real terrain is stationary in the inertial frame I and we want to estimate the terrain in the moving map frame M. Therefore, the mean $\hat{h}$ and variance values σ_h^2 need to be updated according to the change of the pose estimate of the robot. Ideally, the variance and mean of each cell are updated depending on the uncertainty of the motion as well as on the estimates of the surrounding cells. However, performing such an update for each cell of the map is computationally prohibitive. Instead, we extend the elevation map structure with information about the spatial covariance matrix $\boldsymbol{\Sigma}_{P_i} \in \mathbb{R}^{3\times3}$ relative to the true terrain for each cell i. Herewith, we can gather the full three-dimensional uncertainty for each point of the elevation map, and postpone the computationally expensive fusion (Section 3.2.4) to the time when the map is needed by the user or another algorithm.

In the map frame M, which is defined by a known transformation relative to the base frame B (Section 3.2.1), the robot can measure the terrain part visible to the sensor accurately. Hence, if a grid cell i receives a measurement update, we set its covariance to

$$\boldsymbol{\Sigma}_{P_i} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & \sigma_{h_i}^2 \end{bmatrix} \tag{3.6}$$

with the variance of the height estimate $\sigma_{h_i}^2$ computed from Section 3.2.2. If the cell does not receive new sensor measurement updates, we continuously update the covariance $\boldsymbol{\Sigma}_{P_i}$ based on the relative motion of the robot from its previous to its current pose. We derive the robot motion based map update with a setup as shown in Figure 3.3. For the derivation, a map reference frame M_k is associated with the robots current reference frame $\tilde{B}_k$ through the transform $(\boldsymbol{r}_{\tilde{B}_k M_k}, \boldsymbol{\Phi}_{\tilde{B}_k M_k})$ for each time instance k.[2] The estimated position of a point P in the map coordinate frame M_2 for time $k = 2$, $\hat{\boldsymbol{r}}_{M_2 P}$, can be expressed in relation to the map coordinate frame M_1 at time $k = 1$ as

$$\hat{\boldsymbol{r}}_{M_2 P} = -\boldsymbol{r}_{\tilde{B}_2 M_2} - \hat{\boldsymbol{r}}_{\tilde{B}_1 \tilde{B}_2} + \boldsymbol{r}_{\tilde{B}_1 M_1} + \hat{\boldsymbol{r}}_{M_1 P} \ . \tag{3.7}$$

Expressing this relation w.r.t. reference frame M_2, it can be written as

$$\begin{aligned} {}_{M_2}\hat{\boldsymbol{r}}_{M_2 P} = &- {}_{M_2}\boldsymbol{r}_{\tilde{B}_2 M_2} - \boldsymbol{\Phi}^{-1}_{\tilde{B}_2 M_2}({}_{\tilde{B}_2}\hat{\boldsymbol{r}}_{\tilde{B}_1 \tilde{B}_2}) \\ &+ \hat{\boldsymbol{\Phi}}^{-1}_{M_1 M_2}({}_{M_1}\boldsymbol{r}_{\tilde{B}_1 M_1} + {}_{M_1}\hat{\boldsymbol{r}}_{M_1 P}) \ . \end{aligned} \tag{3.8}$$

Without loss of generality, we can choose the pose of reference frame M_2 as

$$\boldsymbol{\Phi}_{\tilde{B}_2 M_2} = \hat{\boldsymbol{\Phi}}^{-1}_{\tilde{B}_1 \tilde{B}_2} \circ \boldsymbol{\Phi}_{\tilde{B}_1 M_1} \ , \tag{3.9}$$

$${}_{M_2}\boldsymbol{r}_{\tilde{B}_2 M_2} = -\boldsymbol{\Phi}^{-1}_{\tilde{B}_2 M_2}({}_{\tilde{B}_2}\hat{\boldsymbol{r}}_{\tilde{B}_1 \tilde{B}_2}) + \hat{\boldsymbol{\Phi}}^{-1}_{M_1 M_2}({}_{M_1}\boldsymbol{r}_{\tilde{B}_1 M_1}) \tag{3.10}$$

to align the reference frames M_1 and M_2 with properties

$$\hat{\boldsymbol{\Phi}}_{M_1 M_2} = \boldsymbol{\Phi}^{-1}_{\tilde{B}_1 M_1} \circ \hat{\boldsymbol{\Phi}}_{\tilde{B}_1 \tilde{B}_2} \circ \boldsymbol{\Phi}_{\tilde{B}_2 M_2} = \boldsymbol{\Phi}_I \ , \tag{3.11}$$

$$\hat{\boldsymbol{r}}_{M_1 M_2} = -\boldsymbol{r}_{\tilde{B}_1 M_1} + \hat{\boldsymbol{r}}_{\tilde{B}_1 \tilde{B}_2} + \boldsymbol{r}_{\tilde{B}_2 M_2} = \mathbf{0} \ . \tag{3.12}$$

[2] We use here the yaw-aligned but not tilted reference frame $\tilde{B}$ as defined in Section 3.2.1 because we are interested in the states unobservable by the robot state estimator.

This way, we can refer to a common reference frame M and do not have to move any data in the map which is efficient in the implementation. The propagation of the covariance from time $k = 1$ to $k = 2$ can be written with (3.8) as follows for $\boldsymbol{r}_{M_k P} \sim \mathcal{N}(\hat{\boldsymbol{r}}_{M_k P}, \boldsymbol{\Sigma}_{P,k})$:

$$\boldsymbol{\Sigma}_{P,2} = \boldsymbol{J}_P \boldsymbol{\Sigma}_{P,1} \boldsymbol{J}_P^\mathsf{T} + \boldsymbol{J}_r \boldsymbol{\Sigma}_r \boldsymbol{J}_r^\mathsf{T} + \boldsymbol{J}_\Phi \boldsymbol{\Sigma}_\Phi \boldsymbol{J}_\Phi^\mathsf{T} \,. \tag{3.13}$$

The covariance $\boldsymbol{\Sigma}_{P,1}$ for time $k = 1$ is either initialized with (3.6) or known from the previous update. The covariances $\boldsymbol{\Sigma}_r$ and $\boldsymbol{\Sigma}_\Phi$ express the uncertainty for the estimated motion of the robot reference frames $\tilde{B}_1$ and $\tilde{B}_2$ with

$$\boldsymbol{r}_{\tilde{B}_1 \tilde{B}_2} \sim \mathcal{N}(\hat{\boldsymbol{r}}_{\tilde{B}_1 \tilde{B}_2}, \boldsymbol{\Sigma}_r) \,, \tag{3.14}$$

$$\boldsymbol{\Phi}_{\tilde{B}_1 \tilde{B}_2} \sim \mathcal{N}(\hat{\boldsymbol{\Phi}}_{\tilde{B}_1 \tilde{B}_2}, \boldsymbol{\Sigma}_\Phi) \,. \tag{3.15}$$

The transformation of the relative covariances $\boldsymbol{\Sigma}_r$ and $\boldsymbol{\Sigma}_\Phi$ from the inertial covariance $\boldsymbol{\Sigma}_{IB}$ is derived in Section A.1. The Jacobians can be evaluated to

$$\boldsymbol{J}_P = \frac{\partial_{M_2} \hat{\boldsymbol{r}}_{M_2 P}}{\partial_{M_1} \hat{\boldsymbol{r}}_{M_1 P}} = \boldsymbol{C}(\hat{\boldsymbol{\Phi}}_{M_1 M_2})^\mathsf{T} = \boldsymbol{I} \,, \tag{3.16}$$

$$\boldsymbol{J}_r = \frac{\partial_{M_2} \hat{\boldsymbol{r}}_{M_2 P}}{\partial_{\tilde{B}_2} \hat{\boldsymbol{r}}_{\tilde{B}_1 \tilde{B}_2}} = -\boldsymbol{C}(\boldsymbol{\Phi}_{\tilde{B}_2 M_2})^\mathsf{T} \,, \tag{3.17}$$

$$\boldsymbol{J}_\Phi = \frac{\partial_{M_2} \hat{\boldsymbol{r}}_{M_2 P}}{\partial \hat{\boldsymbol{\Phi}}_{\tilde{B}_1 \tilde{B}_2}} = -({}_{M_1}\hat{\boldsymbol{r}}_{\tilde{B}_1 M_1} + {}_{M_1}\hat{\boldsymbol{r}}_{M_1 P})^\times \boldsymbol{C}(\boldsymbol{\Phi}_{\tilde{B}_1 M_1})^\mathsf{T} \,. \tag{3.18}$$

We can rewrite the error propagation (3.13) for the case where (3.11) and (3.12) are met: To update the robot pose uncertainty from time k to $k+1$, evaluate the covariance of point P_i for every grid cell i of the map for the common reference frame M as

$$\boldsymbol{\Sigma}_{P_i,k+1} = \boldsymbol{\Sigma}_{P_i,k} + \boldsymbol{J}_r \boldsymbol{\Sigma}_r \boldsymbol{J}_r^\mathsf{T} + \boldsymbol{J}_\Phi \boldsymbol{\Sigma}_\Phi \boldsymbol{J}_\Phi^\mathsf{T} \tag{3.19}$$

with

$$\boldsymbol{J}_r = -\boldsymbol{C}(\boldsymbol{\Phi}_{\tilde{B}_{k+1} M})^\mathsf{T} \quad \text{and} \quad \boldsymbol{J}_\Phi = -{}_M\hat{\boldsymbol{r}}^\times_{\tilde{B}_k P_i} \boldsymbol{C}(\boldsymbol{\Phi}_{\tilde{B}_k M})^\mathsf{T} \,. \tag{3.20}$$

Note that while the change of the full robot position covariance matrix $\boldsymbol{\Sigma}_r$ is propagated on the elevation map variances, only the variances for the yaw-rotation ψ from the sensor rotation covariance matrix $\boldsymbol{\Sigma}_\Psi$ (rotation around $\boldsymbol{e}_z^I = \boldsymbol{e}_z^M$) are propagated. This is complementary to the measurement up-

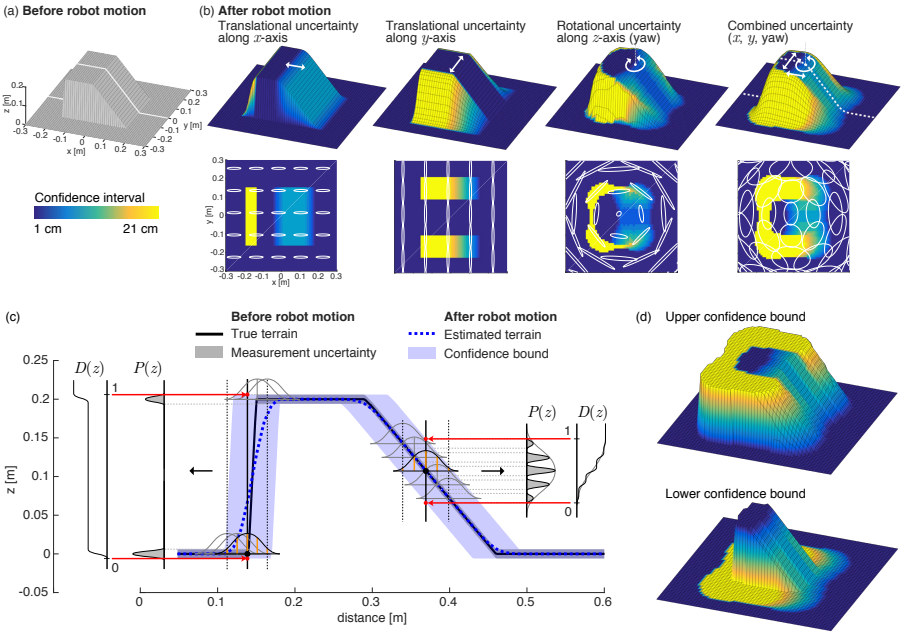


Figure 3.4 The map fusion process is illustrated for a reference terrain (a) for different uncertainty directions (b). Based on the spatial covariance Σ_{P_i} (uncertainty ellipses in the top view), an empirical cumulative distribution function $D(z)$ can be computed for each grid cell based on its neighboring cells (c). With this, a lower and upper confidence bound can be estimated for every cell in the map (d).

date (Section 3.2.2), where the sensor position covariance and the sensor yaw-rotation uncertainties are excluded from the update step.

3.2.4 Map Fusion

Whenever required, we transform the elevation map data structure $(\hat{h}_i, \boldsymbol{\Sigma}_{P_i})$ to the representation $(\hat{h}_i, h_{i,\min}, h_{i,\max})$ for every cell i. We refer to this step as the *map fusion* process. The mean height estimate $\hat{h}_i$ is computed as the weighted mean from all cells within the 2σ confidence ellipse in the xy-plane of cell i.[3] The values $h_{i,\min}$ and $h_{i,\max}$ represent the lower and upper confidence bounds of the height estimation such that the real height is within the 95 % confidence interval $h_i \in [h_{i,\min}, h_{i,\max}]$. We infer $h_{i,\min}$ and $h_{i,\max}$ from the data of the surrounding cells within the 2σ confidence ellipse (95 %). This process

[3]Depending on the application, the height estimate $\hat{h}_i$ could be also taken over from the original data without averaging to retain the surface structure information.

is illustrated in Figure 3.4c: For each cell i, we create a weighted empirical probability density function (PDF) $P(z)$ and its corresponding cumulative density function (CDF) $D(z)$ from the height $\hat{h}_j$ and variance $\sigma^2_{\hat{h}_j}$ for all cells j in the confidence ellipse. The bivariate CDF (Hogg, McKean, and Craig, 2005) in the xy-plane for cell i determines the weight for each cell j evaluated at its position relative to cell i. The lower and upper confidence bounds $h_{i,\min}$ and $h_{i,\max}$ can now be determined by sampling the minimum and maximum values of the CDF $D(z)$ (Hyndman and Fan, 1996).

Figure 3.4 shows the results of the map fusion process for a slanted block (Figure 3.4a) for uncertainty ellipsoids around different axes (Figure 3.4b). A rendering of the resulting upper and lower confidence bounds for the combined uncertainty case is shown in Figure 3.4d.

3.2.5 Extension to Dynamic Environments

As expected, the height update rule (3.5) converges after multiple measurements to a high-confidence estimate. However, this approach works poorly in dynamic environments because many measurement updates are required to reflect the new shape of the terrain, causing a significant mapping delay. To this end, we introduce an extension to the map update process which consists of two parts: The first part adds a constant noise value $\bar{\sigma}^2$ as

$$\sigma_h^{2+} = \sigma_h^{2-} + \bar{\sigma}^2 \,. \tag{3.21}$$

to map cells for which new measurements (based on the time-stamp) are recorded that fall below the current height estimate and exceed a Mahalanobis distance threshold. This improves the map adaptation time in regions where the real terrain is *lowered*, which is the case when for example an obstacle is moved away. With the choice of the value $\bar{\sigma}^2$, the user trades off between fast adaptation to a changing terrain and an increased uncertainty for cells at edges in the terrain.

In a second part, a visibility check is performed based on ray tracing as shown in Figure 3.5. We create a *visibility map* from the rays connecting the point of the height measurement $p + 3\sigma_p$ to the depth sensor's position at this measurement. This map reflects the maximal height $h_{\max}$ that each cell can have based on the visibility constraint. Consequently, cells with $\hat{h} - 3\sigma_h > h_{\max}$ violate the visibility constraint and are removed. As this visibility check is computationally intensive, it is only performed at a lower rate (e.g., 1 Hz) and in parallel to the data collection process. In this sense, it is complementary

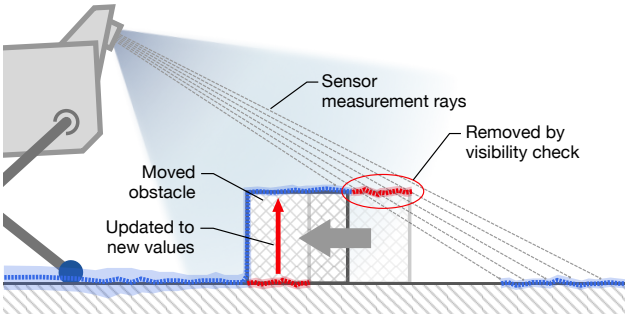


Figure 3.5 The mapping process adapts to dynamic environments through removal of cells that violate the visibility constraint. In this figure, the obstacle was moved from right to left. The previous map (red) is updated and the visibility is checked with ray tracing resulting in an updated map (blue).

to the first part of the adaptation to dynamic environments presented in this section.

3.3 Results

The described elevation mapping algorithm is implemented as a C++ library with an interface to the Robot Operating System (ROS) and is available open-source (S2). Our implementation features measurement noise models for popular range sensors from the product families Intel RealSense, Microsoft Kinect, Hokuyo, and Velodyne. For efficient data handling and operations, we have developed a universal grid map library (S1). The Grid Map library supports multiple data layers and is, for example, applicable to elevation, variance, color, surface normal, and occupancy. The underlying data storage is implemented as a two-dimensional circular buffer, which allows for non-destructive and computationally efficient shifting of the map position.

In the following, we discuss the presented mapping framework with results from simulation and experiments with robotic platforms.

3.3.1 Evaluation in Simulation

We compare the map fusion process (Section 3.2.4) for an artificial dataset to a reference map. Figure 3.6a shows the original and resulting terrain map. The reference map was created by exhaustively disturbing and sampling the original map with the given uncertainty. Figure 3.6b shows the estimated and simulated confidence intervals for three slices A–C through the map. This

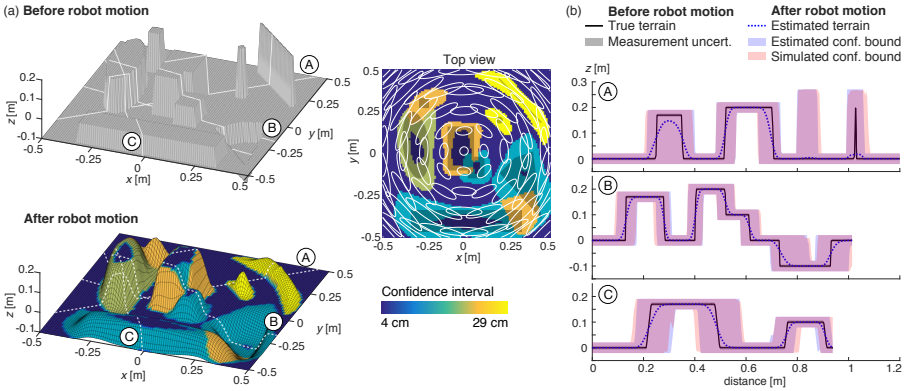


Figure 3.6 A simulated terrain (a) is used to verify the map fusion process (Section 3.2.4) for a terrain perceived under uncertainty (*after robot motion*). The three terrain profile sections A–C (b) show that the presented map fusion approximates the true (simulated) confidence bounds accurately.

comparison shows that the estimated confidence intervals describe the position of the actual terrain at high fidelity. One important observation is that large flat regions remain at a low confidence interval. In contrast, areas at the edges of objects become more uncertain in their height estimation with high confidence intervals. This is a critical feature that is exploited in our locomotion planning approach for foothold selection and collision avoidance (Chapter 5).

3.3.2 Evaluation on Hardware

We have validated our terrain mapping approach in various environments with our robotic platforms. Figure 3.7 shows a snapshot from three trials: In the first case (Figure 3.7a), the robot StarlETH walks indoors over obstacles and maps the terrain at 20 Hz at a resolution of 1 cm × 1 cm per map cell with the PrimeSense Carmine 1.08 structured-light depth camera (Section 2.2.2). Here, the robot walks in a static gait and was localized only through the proprioceptive state estimation (Bloesch et al., 2013). In the second case (Figure 3.7b), the outdoor environment is mapped with a rotating Hokuyo laser range sensor (Section 2.2.3) at 1 Hz at a resolution of 4 cm × 4 cm per map cell on the robot ANYmal. Here, ANYmal was moving with a dynamic trotting gait, and in addition to the proprioceptive state estimation, we used iterative closest point-based (ICP) localization (Pomerleau et al., 2013). Lastly, Figure 3.7c shows the terrain reconstruction for ANYmal climbing a set of outdoor stairs. In this trial, the environment is perceived with an Intel RealSense ZR300 active

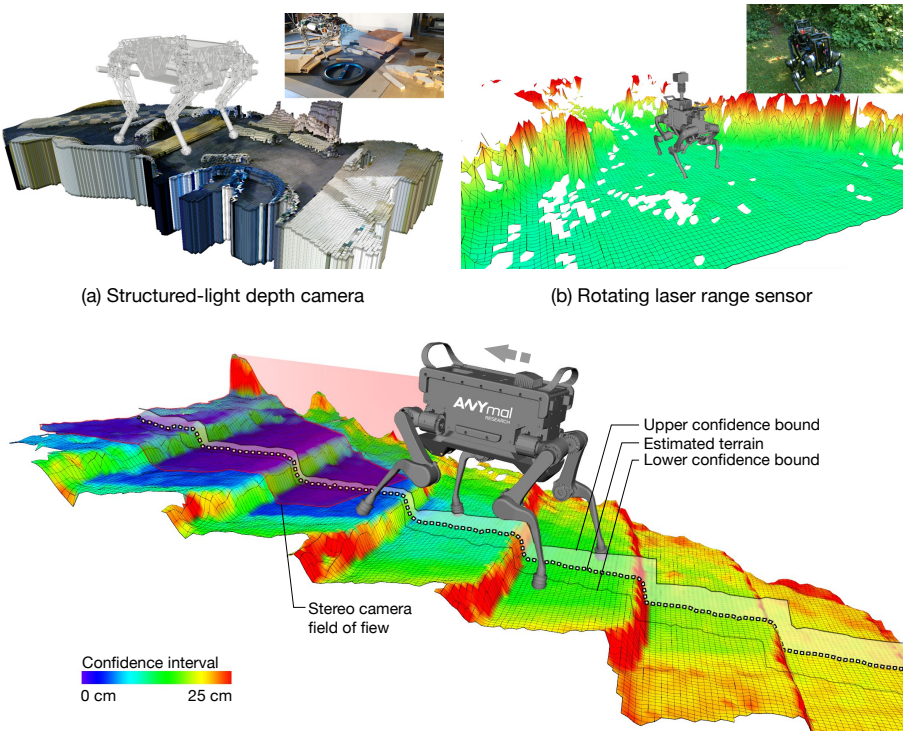


Figure 3.7 The presented mapping framework was validated for different environments and on different platforms: (a) The quadrupedal robot StarlETH walks (static gait) over obstacles and scans the terrain with a structured-light depth sensor. (b) ANYmal trots (dynamic gait) through a natural environment and perceives the terrain with rotating laser range sensor. (c) ANYmal climbs outdoor stairs with help of the terrain map generated from active stereo vision.

stereo camera (Section 2.2.5) at 10 Hz with a 2 cm × 2 cm map resolution. In all cases, our mapping framework adapts well to the different setups and provides a reliable environment reconstruction. Videos of the described mapping experiments are available at (V2), (V9), and (V13).

To evaluate the effectiveness of the proposed robot-centric mapping approach, we compare the resulting terrain reconstruction to ground truth measurements. For this, the robot StarlETH is equipped with a structured-light depth sensor (PrimeSense Carmine 1.08) and walks over obstacles while localizing itself with its proprioceptive state estimation (Bloesch et al., 2013). With this setup, the position and yaw-rotation of the robot are unobservable and drift over time as

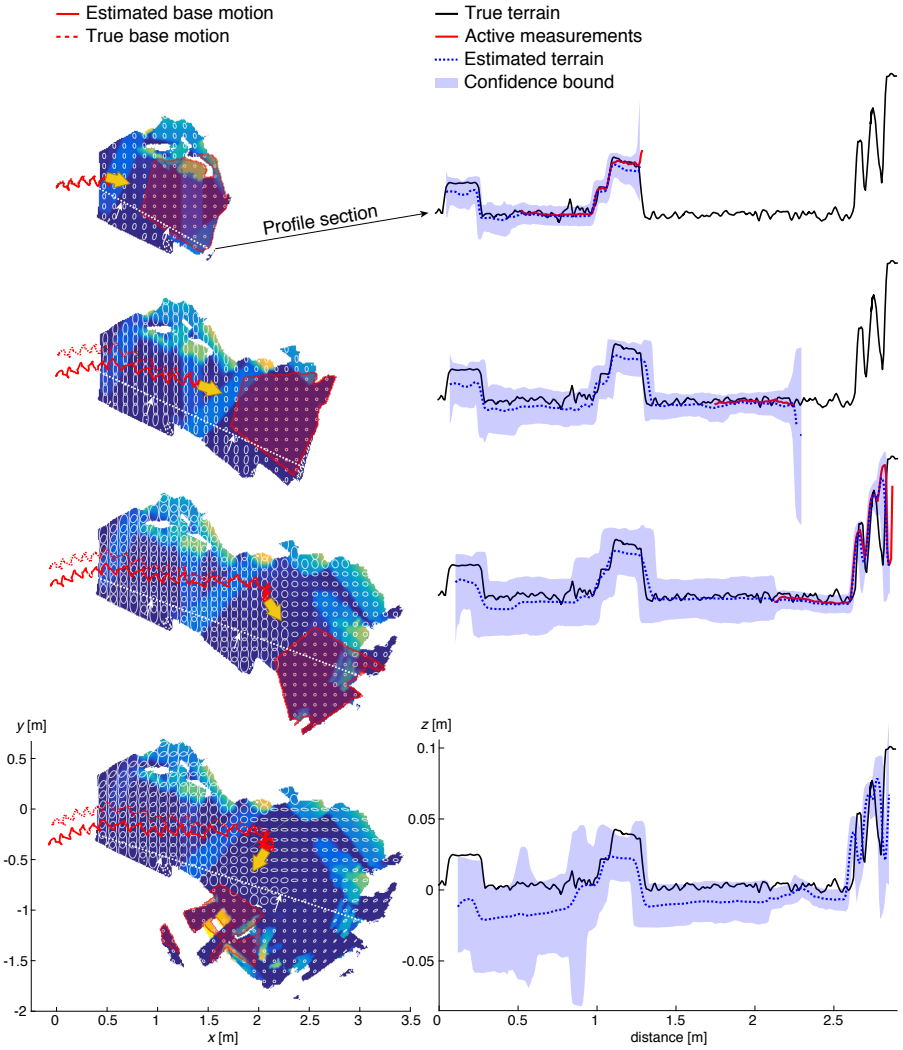


Figure 3.8 This figure shows the mapping process with a legged robot walking over obstacles for four instances. The robot uses proprioceptive state estimation to localize itself, which is affected by drift as the system moves. The top view (a) shows the onboard map creation and the comparison of the estimated robot trajectory to the real motion. The terrain profile sections (b) show that the true terrain remains within the confidence bounds.

the robot moves. As ground truth, we tracked the real position of the robot with an optical motion capture system and recorded a reference terrain with a stationary Leica Nova MS50 MultiStation geomatic laser scanner. Figure 3.8 shows different stages of the robot walking over obstacles. The robot's estimated and reference position are shown in Figure 3.8a together with a top view of the onboard terrain reconstruction. We analyze the resulting terrain map in comparison with the reference map for a terrain profile section as shown in Figure 3.8b. As expected, the drift of the state estimation causes the terrain estimates to diverge from the actual terrain for areas which have not been updated with range measurements. For these areas, we can observe an increase of the error ellipses in Figure 3.8a. Even though the unobserved regions diverge, our mapping method correctly captures the uncertainty, and the actual terrain remains within the confidence bounds as shown in Figure 3.8b.

3.3.3 Comparison

The approach presented in this thesis is compared to two alternative approaches, namely to our previous contribution (Fankhauser et al., 2014) and to the work (Kleiner and Dornhege, 2007). Figure 3.9 shows the result of the different mapping methods when applied to terrain under the uncertainty of a robot's motion. For the presented approach (a), the estimated terrain confidence bounds match closely with the true (sampled) bounds. For our earlier approach (Fankhauser et al., 2014) (b), the confidence bounds partially over and underestimate the real distribution. On the one side, the two-dimensional covariance data is a conservative approximation of the true uncertainty. On the other side, the approach in (Fankhauser et al., 2014) insufficiently captures the multi-modal distribution in the map fusion process, leading to 'overshooting' artifacts at edges. Finally, the approach used in (Kleiner and Dornhege, 2007) (c) neglects the effect of motion uncertainty in the horizontal plane. Instead, the vertical uncertainty σ_h^2 is accumulated, which leads to strong over- and underestimation in flat and structured regions of the terrain, respectively. Furthermore, no indication of the terrain-structure influenced uncertainty can be given with this method.

3.4 Summary

We have presented a novel approach to elevation mapping that addresses the problem of localization drift as it often occurs on mobile robots with proprioceptive state estimation only. The presented method estimates the elevation map in a robot-centric coordinate frame such that the process of integrating

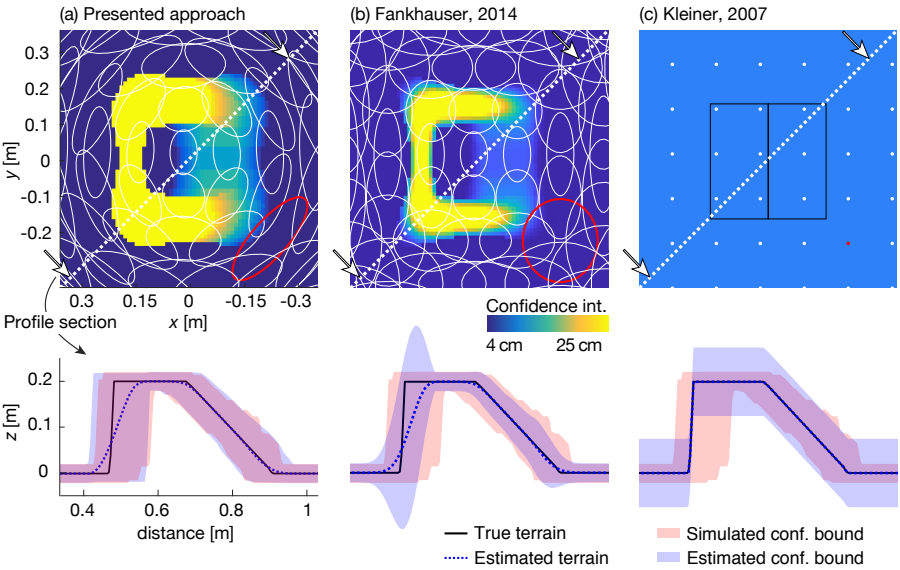


Figure 3.9 We compare the mapping method presented in this work (a) with the approaches used in (Fankhauser et al., 2014) (b) and (Kleiner and Dornhege, 2007) (c). The top row shows a top view on the map (same terrain as in Figure 3.4) where the colors indicate the confidence interval and the ellipsoids depict the (horizontal) uncertainty. The bottom row shows a diagonal profile section of the map and compares the estimated confidence bounds (blue) with the true confidence bounds (red).

new measurements into the map is only affected by the range sensor noise and the uncertainty of the observable roll- and pitch angles. The data in the map is updated based on the uncertainty of the incremental motion as the robot moves through the environment. This gives the robot at any point in time an estimate of the terrain from its local perspective. We lower the computation burden of the mapping procedure by splitting the method into a data collection (map update from range measurements and robot motion) and a map fusion step. This enables reliable real-time elevation mapping independent of an exteroceptive localization method. We illustrate how the mapping process estimates the terrain profile with a corresponding confidence interval and show how the estimated distribution matches the ground truth reference in simulated and real-world experiments. A comparison to prior work demonstrates the advantages of the proposed method.

4

Legged Motion Control

This chapter incorporates material from the following publication:

Fankhauser, P., Dario Bellicoso, C., Gehring, C., Dube, R., Gawel, A., and Hutter, M. (2016). "Free Gait – An Architecture for the Versatile Control of Legged Robots". In: *IEEE-RAS International Conference on Humanoid Robots (Humanoids)*, pp. 1052–1058

Controlling robots with a high number of degrees of freedom (DOFs) is a complex task. A significant challenge for legged robots is the generation of movements through intermittent contact while ensuring stability and avoiding collision with the environment and with the robot itself. Also, when implementing controllers for real robots, one has to account for real-time performance, safety, and robustness against uncertainties in actuation and sensing. For these reasons, developing software for legged robots is complex and coupled with a significant development overhead and learning curve for new developers.

Abstraction layers provide a tool to hide complexity when systems become too difficult to work with efficiently. One example is computer systems where hardware and applications are separated by a series of abstraction layers such as firmware, kernel, and libraries which communicate over application programming interfaces (APIs). Based on this idea, we have developed *Free Gait*, a framework for the versatile control of legged robots. The goal of Free Gait is to reduce the complexity of developing software for the motion generation of legged robots. It consists of a *whole-body abstraction layer* (Free Gait API) and several tools that are designed to interface higher level motion goals with the lower level tracking and stabilizing controllers. The separation between motion generation and execution through the Free Gait API enhances the ease of use, interoperability, and platform independence. The application of our framework ranges from intuitive teleoperation of the robot, scripting of user-defined

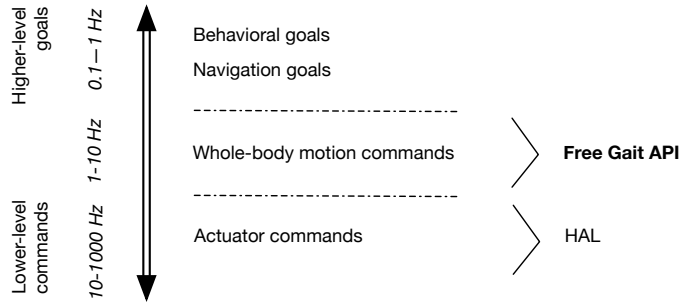


Figure 4.1 The Free Gait API is designed for motion commands involving the entire structure of the robot (whole-body abstraction layer). This differentiates it from more abstract task-level goals and from the more local hardware abstraction layer (HAL) on actuator level.

behaviors, to fully autonomous operations with the help of motion planners. It is designed with general legged robots in mind and compatible with existing whole-body controllers.

In this chapter, we discuss the relation to prior work (Section 4.1), introduce the Free Gait architecture (Section 4.2), address the issues that arise when working with real robots (Section 4.3), and describe several applications of the introduced framework (Section 4.4).

4.1 Related Work

Previous work on abstraction layers in robotics has often focused on hardware abstraction layers (HALs) (Jorg, Tully, and Albu-Schaffer, 2014; Murray, Pham, and Pirjanian, 2005). The goal here is to separate the communication with sensors and actuators from the perception and motion algorithms. From the motion generation perspective, the HAL provides the interface between the desired joint commands (such as position and torque commands) and the underlying actuator controller for electric, hydraulic, or other actuators. They are an essential tool in robotics and many robots such as the LittleDog (Figure 1.3g, Murphy et al., 2010) provide such an interface. However, a HAL captures only the local control on joint (or limb) level, whereas Free Gait is designed as an interface for whole-body motions. As illustrated in Figure 4.1, the Free Gait API can be understood as an intermediate layer between a HAL and more abstract navigation goals such as platform velocity commands as used by Gehring et al. (2013) and Kerscher et al. (2008).

Several software packages exist for the intuitive generation of robot motions, for example for the *NAO* robot (Pierris and Lagoudakis, 2009; Pot et al., 2009). These tools provide complete development environments and build upon predefined sequences for behaviors such as 'stand up' or 'walk forward'. With these tools, whole-body motions are executed as interpolation between robot pose 'snapshots', which are defined by the user as joint position configurations. The designed motions are executed as open-loop joint position trajectories, an approach that requires highly controlled environments for successful performance.

A concept similar to our work has been discussed in Germann, Hiller, and Schramm (2005) and Kohlbrecher et al. (2016). In the former, the HAL has been extended (and is limited to) to more abstract input types such as feet and base velocities relative to the ground. The velocity output of the different modules (gait generation, stability, obstacle avoidance, and body posture) was then combined as a weighted sum to command the robot. In the more recent work (Kohlbrecher et al., 2016), a structure for the intuitive remote control of a Humanoid is proposed. It is based on the definition of desired joint angles, Cartesian goals for the end-effector, and pelvis poses. These commands are either directly executed on the robot or used as a target for the joint or footstep planner. Our work extends over these concepts by generalizing to a more versatile definition of commands including position, velocity, and force command types. Furthermore, Free Gait also includes the definition of targets (time-independent), entire trajectories (time-dependent), and automatized commands such as footsteps and pose adaptation commands.

4.2 Interface Definition

The Free Gait API connects the *motion generation* algorithms with the *motion execution* control (Figure 4.2). The motion generation modules define the desired motion of the robot with various base and leg motion commands. In the *Free Gait Core* module, these commands are transformed to a *robot desired state*, which describes the desired base pose and joint states for each time step of the motion execution. The motion is typically executed in a real-time controller that implements the tracking of the desired robot state with the help of the state estimation and motor controllers. In the following, we describe the structure and the elements of the Free Gait interface.

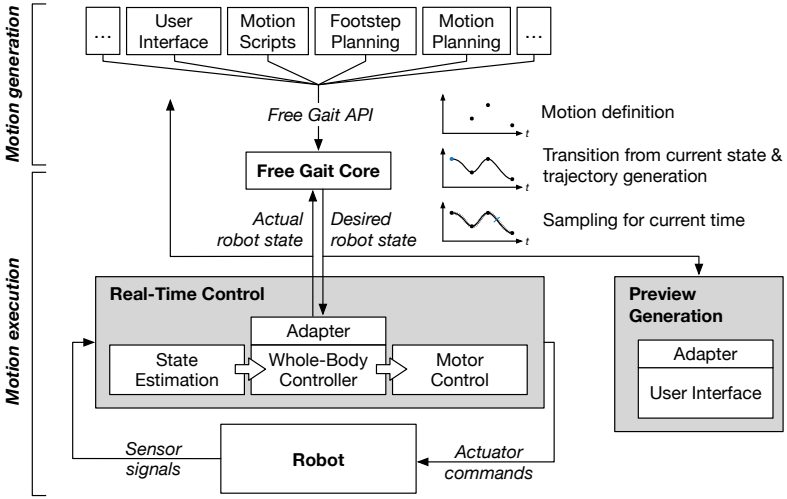


Figure 4.2 Motion goals are commanded through the Free Gait API to the whole-body motion controller. The API is suitable to be interfaced with motion generation sources such as user interfaces, motion behavior scripts, or footstep and motion planners. During motion execution, the desired robot state is sampled by the Free Gait Core and tracked by the real-time whole-body controller. Similarly, the generated robot state can be previewed for quick development feedback.

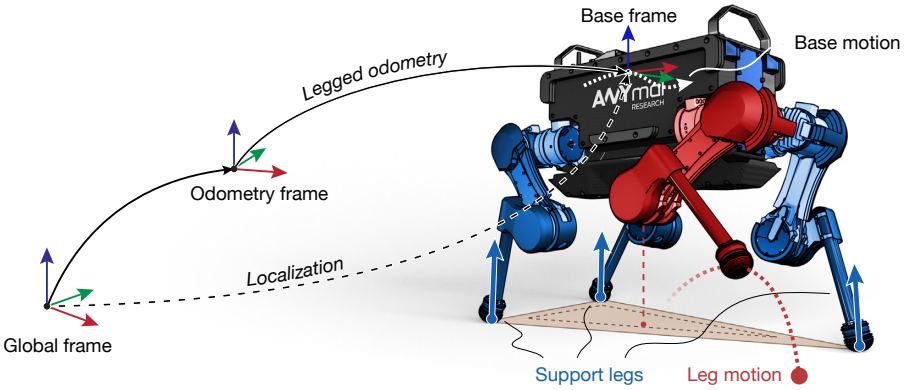


Figure 4.3 Free Gait motions are based on a combination of (possibly multiple) leg motions and a base motion per command (step). The base motion defines indirectly the motion of the support legs. Both leg and base motions can be defined a reference frame suitable to the task.

4.2.1 Command Structure

A Free Gait command (or action) is composed of a list of *steps* (of type `Step`), which are executed successively in the specified order. During execution, continuous feedback on the progress of the execution is provided to the sender of the action. Once the command is successfully finished (or in case it could not be executed or was intentionally stopped), the corresponding result is returned. At any time, the sender can stop the execution to replace it with new commands. In this case, the remaining steps are discarded and the new commands executed. This enables the implementation of receding horizon controllers with the Free Gait API.

For the command definition, we differentiate between *leg motion* and *base motion* commands as illustrated in Figure 4.3. Leg motions describe the desired motion of the legs that do not contribute to the stabilization of the base (non-support legs). Depending on the application, leg motions can be expressed as joint space commands or as Cartesian space commands of the end-effector (foot). Base motions describe the desired motion of the base, which will be executed with the help of the legs in contact with the environment (support legs). Both leg and base motions can be defined in three different ways:

Target	a single target state of a leg or the base,
Trajectory	a series of target states each associated with a timestamp,
Automatic	a command to use online motion generation/optimization for the legs (footsteps) or the base (pose adaptation).

An overview of the Free Gait commands is given in Table 4.1. Note that each step in the command's list of steps can contain a combination of different leg and base motion definitions. This allows expressing each command with the most suitable motion definition for the task at hand.

In the following, we discuss the different types of motion definitions of the interface in more detail.

4.2.2 Leg Motions in Joint Space

The joints of the robot's non-support legs can be controlled with the leg motion commands in joint space $\boldsymbol{q}(t) \in \mathbb{R}^n$ with n the number of joints per leg. In this case, a combination of a desired joint position, velocity, and torque can be defined for each joint of the leg. Combining different control modes allows

Table 4.1 Free Gait interface commands

	Target	**Trajectory**	**Automatic**
Leg motion in joint space	`JointTarget`$^{+}$	`JointTrajectory`	`LegMode`
Leg motion in Cartesian space	`EndEffectorTarget`$^{*+}$	`EndEffectorTrajectory`*	`Footstep`$^{*+}$
Base motion	`BaseTarget`$^{*+}$	`BaseTrajectory`*	`BaseAuto`$^{+}$

* With frame definition $^{+}$ With average rate of change definition

to determine the desired behavior of the leg. For example, specifying desired joint positions and velocities enables smooth motions while the combination of the joint position with a feed-forward torque is suited for interacting with the environment.

In the simplest case, a target joint state is defined as a `JointTarget` command type. Free Gait determines the duration of the motion based on the current state of the joints, the desired state, and (optional) information about the desired average rate of change (e.g., average joint velocity in case of joint position commands). Building upon a rate definition (instead of direct duration information) decouples the resulting velocities and accelerations from the current state of the robot. This makes the target definition in practice more convenient and safer to work with.

For more complex joint motions, a full trajectory can be specified as type `JointTrajectory`. This command consists of a list of joint states each associated with a time and supports variable durations between the states. Upon start of the motion, Free Gait computes a spline that connects the current state of the joints and the desired joint state knot points (see Figure 4.2). For each joint, we parameterize the trajectory as a quintic polynomial spline. Similarly, the `JointTarget` command is a joint trajectory with a start and end knot point.

For convenience, the `LegMode` command is used to determine if a leg should be used as support leg or not while keeping the current control mode and setpoints.

4.2.3 Leg Motions in Cartesian Space

As an alternative to the joint space commands (Section 4.2.2), the desired foot motion of the non-support legs can be defined in Cartesian space. In this case,

one specifies a combination of the desired position, velocity, and force of the end-effector (foot) in Cartesian space as $\boldsymbol{f}(t) \in \mathbb{R}^3$.

Conceptually similar to the joint space target definition, the `EndEffector-Target` command specifies a target state of the foot together with (optional) information about the desired average rate of change. Analogously, the `EndEffectorTrajectory` type is used to command a sequence of timed foot states. These end-effector commands are parametrized as cubic Hermite splines. Importantly, both end-effector target and trajectory are expressed in a coordinate frame that can be chosen to suit the task at hand. On one side, this makes it easy to specify leg configurations relative to the base (base frame) which are independent of the robot's pose. On the other side, it is often useful to interact with the environment and to specify the desired foot state relative to a global frame. In the latter case, Free Gait tracks the state in the specified frame during execution, enabling robust and accurate motion execution even under external disturbances such as slippage or pushes. In case the desired command cannot be tracked due to kinematics limits, the closest (in terms of Euclidian distance) feasible configuration is targeted.

As a specialized version of a foot trajectory, Free Gait implements a `Footstep` command type for stepping to desired footholds. A footstep is determined by the target position (x-, y-, and z-coordinate) and the height and profile form of the swing motion. The swing height is defined as the difference in z-direction between the trajectory apex and either the start or target location, depending on which one is higher. Currently, we have implemented triangle, square, and straight line swing profiles, which internally use three, four, and two spline knot points, respectively.

4.2.4 Base Motions

Base motion commands determine the motion of the robot's base/torso. The desired base motion is tracked with help of the support legs (Figure 4.3). Similar to leg motions, a combination of the desired pose in $\mathbb{R}^3 \times \mathrm{SO}(3)$, twist in $\mathbb{R}^6$, and net feed-forward force and torque at the base in $\mathbb{R}^6$ can be commanded. The base motion definition is independent of the number of support legs and the position of the feet. Again, both single targets as `BaseTarget` (timing determined through desired average rate of change) and full trajectories as `BaseTrajectory` are available. Internally, the base motion paths are composed of a Hermite spline for the translation and a Hermite quaternion spline for the rotation (Kim, Kim, and Shin, 1995). As with leg

motions in Cartesian space, both base targets and trajectories are associated with a reference frame.

Base motions are practical for the general locomotion of the robot where the center of mass has to be shifted to take steps. Additionally, we have found great use of them for stand up/lie down maneuvers and for positioning onboard equipment (e.g., cameras) at specific locations. In these cases, the desired base motions are intuitively defined and robustly executed even from very different feet positions and leg configurations. It is important to note that for the definition of base target and trajectory commands, the motion generation has to ensure stability and compatibility with the current position of the feet.

Online motion generation for the base motion is provided by the `BaseAuto` command type. It computes a base target pose based on the current and, if available, next contact situation. The computation bases on the notion of *feet/legs in support* given by current stance and the *feet/contacts to reach* defined by the motion generation. In the example motion shown in Figure 4.3, the stance legs form the support polygon and the contact points of the stance legs plus the target of the swing leg are the contacts to be reached. The goal is to find a pose under the constraints of stability and torque limits (defined by the support legs), kinematic limits (defined by the contacts to reach), and optionally collision constraints. Typically, many solutions exist under these constraints, and the underlying selection algorithm should be adapted for different types of robots. Our pose optimization algorithm for quadrupedal systems is described in Section 5.4.3. The user can optionally set the support margin and the base height. The base height is defined as the vertical difference between the geometrical center of the support feet and the base frame.

The `BaseAuto` command is useful for using before, together with, and after leg motion commands. When using it as a preparation step before a leg motion command, it moves the base to a position from which the feet can be unloaded safely in the next step. The usage in parallel with leg motions moves the base in the direction of the anticipated target foothold, supporting the reachability of the robot. Using the pose adaptation after a contact change moves the base taking the new contacts into account, distributing the load to all legs and hence increasing the stability margin.

4.2.5 Output and Integration of Free Gait

While the Free Gait API is designed to support various motion commands as input, the output consists of a simple description of the desired robot state.

This robot state is the setpoint for a robot's motion controller and allows for easy integration with existing robot control software. The desired robot state consists for the non-support legs of the control mode (position, velocity, and/or torque) and the desired setpoints for each joint. Similarly, the state also contains the control mode and setpoints for the base motion. The setpoints of the base motion are converted by the whole-body controller to joint commands of the support legs. Additionally, the surface normal for each contact is also contained in the robot state if it is given by the motion generation.

The implementation of Free Gait with the robot specific software is realized through adapters. An adapter is responsible for providing the actual state of the robot and to forward the desired robot state to the robot controller. Through the adapter, the execution of the Free Gait commands is controlled with a user-defined sampling time.

By separating the motion generation with the motion execution, Free Gait is robot platform agnostic, enabling to quickly port motion generation methods and algorithms to a different robot. As Free Gait is also independent of the underlying whole-body controller, implementing different control methods for the same control tasks requires only to write a new adapter without additional overhead. Furthermore, it is often useful to simulate or preview the motion plan, which can be accomplished by writing adapters for each and viewing the commands before the execution with the real robot.

4.3 Implementation

The motion generation specifies the desired motion of the robot, but in reality, the actual motion is never executed perfectly as defined. In practice, we have to deal with the problems that arise among others from inaccurate tracking, simplified models, sensor noise, drifting state estimation, and external, not modeled disturbances. To this end, we describe in the following our strategies, tools, and control methods that enable a robust motion execution with a real robot.

4.3.1 Base Motion Control

Based on our previous work (Gehring, 2017; Gehring et al., 2013), we apply virtual model control (VMC) (Pratt et al., 2001) for the robot's base motion control. Emulating a virtual spring-damper system at a certain point on the base, it is a simple but powerful approach leveraging the torque-controllability of the actuators for robust base motion tracking. Given the current robot's

base pose $\boldsymbol{q}_b = (\boldsymbol{r}_{IB}, \Phi_{IB})$ with position $\boldsymbol{r}_{IB}$ and orientation Φ_{IB} w.r.t. the inertial frame I and the desired base pose $\hat{\boldsymbol{q}}_b$, a virtual force $\boldsymbol{F}_v$ and torque $\boldsymbol{T}_v$ are computed as

$$\begin{pmatrix} \boldsymbol{F}_v \\ \boldsymbol{T}_v \end{pmatrix} = \boldsymbol{K}_p(\hat{\boldsymbol{q}}_b - \boldsymbol{q}_b) + \boldsymbol{K}_d(\dot{\hat{\boldsymbol{q}}}_b - \dot{\boldsymbol{q}}_b) + \boldsymbol{K}_i \int (\hat{\boldsymbol{q}}_b - \boldsymbol{q}_b)\mathrm{d}t + \begin{pmatrix} \boldsymbol{F}_G \\ \boldsymbol{T}_G \end{pmatrix} . \quad (4.1)$$

The terms $\boldsymbol{K}$ are diagonal gain matrices, and $\boldsymbol{F}_G$ and $\boldsymbol{T}_G$ are summed gravity compensation forces and torques, respectively. The $-$ operator is used to denote the generalized minus operation which also extends to rotations as introduced in Section 1.5. To track the base pose $\boldsymbol{q}_b$, the virtual force $\boldsymbol{F}_v$ and torque $\boldsymbol{T}_v$ have to be effectuated by the contact forces $\boldsymbol{F}_{c,i}$ at position $\boldsymbol{r}_{F_1}$ each support leg i. This can be formulated as an optimization problem for the stacked contact force vector $\boldsymbol{F}_c = (\boldsymbol{F}_{c,1}, \ldots, \boldsymbol{F}_{c,N})$ with

$$\begin{aligned} \text{minimize} \quad & (\boldsymbol{A}\boldsymbol{F}_c - \boldsymbol{b})^\mathsf{T} \boldsymbol{Q} (\boldsymbol{A}\boldsymbol{F}_c - \boldsymbol{b}) + \boldsymbol{F}_c^\mathsf{T} \boldsymbol{R} \boldsymbol{F}_c \\ \text{subject to} \quad & F_{c,i,\min} \leq F_{c,i}^n \leq F_{c,i,\max} \\ & |\tau_j(\boldsymbol{F}_{c,i})| \leq \tau_{i,\max} \\ & \left\| \boldsymbol{F}_{c,i}^t \right\|_2 \leq \mu F_{c,i}^n \\ & \forall i = 1, \ldots, N \, , \end{aligned} \quad (4.2)$$

with

$$\boldsymbol{A} = \begin{bmatrix} \boldsymbol{I} & \ldots & \boldsymbol{I} \\ \boldsymbol{r}_{F_1}^\times & \ldots & \boldsymbol{r}_{F_N}^\times \end{bmatrix} \quad \text{and} \quad \boldsymbol{b} = \begin{pmatrix} \boldsymbol{F}_v \\ \boldsymbol{T}_v \end{pmatrix} . \quad (4.3)$$

The matrix $\boldsymbol{Q}$ is a weighting matrix and $\boldsymbol{R}$ is used as regularizer. The contact force optimization is constrained by minimum and maximum values $F_{c,i,\min}$ and $F_{c,i,\max}$ for the contact force in terrain normal direction $F_{c,i}^n$. Similarly, the torque τ_j for each joint j is limited to the maximum absolute torque $\tau_{j,\max}$. Finally, Coulomb friction constraints with coefficient μ are enforced for the norm of the terrain tangential contact force $\boldsymbol{F}_{c,i}^t$. Approximating these friction cones as pyramids allows to formulate the optimization as a quadratic programming (QP) which can be solved efficiently at 400 Hz. From the desired contact forces, the corresponding joint torques can be obtained through Jacobi mapping as shown by Gehring (2017).

4.3.2 Swing Leg Control

An adequate swing leg controller is required to achieve fast and accurate coordination of the robot's joints. We employ an inverse dynamics controller

from which robust tracking for the full range of the joints can be obtained. A feedback control on the desired position and velocity additionally acts as an impedance controller (Buchli et al., 2009; Hogan, 1985) to stabilize the motion tracking by compensating for model errors. Note that the following formulation assumes that the base is moving slowly such that its motion can be neglected.

Given a legs' desired joint positions $\hat{\boldsymbol{q}}_i$, velocities $\hat{\dot{\boldsymbol{q}}}_i$, and accelerations $\hat{\ddot{\boldsymbol{q}}}_i$, the dynamically consistent desired joint torques $\hat{\boldsymbol{\tau}}_i$ are given by

$$\hat{\boldsymbol{\tau}}_i = \boldsymbol{M}_i(\boldsymbol{q}_i)\hat{\ddot{\boldsymbol{q}}}_i + \boldsymbol{h}_i \ , \tag{4.4}$$

for leg i with mass matrix $\boldsymbol{M}_i$ and with the non-linear forces including Coriolis terms captured in $\boldsymbol{h}_i$. For leg motion commands in joint space (Section 4.2.2), the joint velocities and accelerations are derived directly from the spline parametrization. For Cartesian end-effector commands (Section 4.2.3), the corresponding joint space values have to be obtained. We use an iterative inverse kinematics scheme to find the joint angles $\hat{\boldsymbol{q}}_i$ that correspond to the desired Cartesian foot position $\hat{\boldsymbol{r}}_{F_i} = \boldsymbol{r}(\hat{\boldsymbol{q}}_i)$ (Buss, 2004). Starting from the current joint positions, the joint angles $\hat{\boldsymbol{q}}_i$ are updated at every iteration until convergences with

$$\hat{\boldsymbol{q}}_i = \hat{\boldsymbol{q}}_i + \Delta\hat{\boldsymbol{q}}_i \ , \tag{4.5}$$

$$\Delta\hat{\boldsymbol{q}}_i = \boldsymbol{J}_{F_i}^{+}(\hat{\boldsymbol{r}}_{F_i} - \boldsymbol{r}(\hat{\boldsymbol{q}}_i)) \ . \tag{4.6}$$

The pseudo-inverse ($^+$) of the Jacobian $\boldsymbol{J}_{F_i} = \frac{\partial \boldsymbol{r}_{F_i}}{\partial \boldsymbol{q}_i}$ is calculated with the adaptive damped least squares method presented by Chiaverini, Siciliano, and Egeland (1994) to smoothly handle singular joint configurations. This approach allows executing Cartesian leg motion commands while flexibly adapting to the current leg configuration (knees inwards or outwards). Likewise, the joint velocities $\hat{\dot{\boldsymbol{q}}}_i$ and accelerations $\hat{\ddot{\boldsymbol{q}}}_i$ are computed from the Cartesian commands as

$$\hat{\dot{\boldsymbol{q}}}_i = \boldsymbol{J}_{F_i}^{+}\hat{\dot{\boldsymbol{r}}}_{F_i} \quad \text{and} \quad \hat{\ddot{\boldsymbol{q}}}_i = \boldsymbol{J}_{F_i}^{+}(\hat{\ddot{\boldsymbol{r}}}_{F_i} - \dot{\boldsymbol{J}}_{F_i}\hat{\dot{\boldsymbol{q}}}_i) \ . \tag{4.7}$$

The hereby obtained desired joint positions $\hat{\boldsymbol{q}}_i$, velocities $\hat{\dot{\boldsymbol{q}}}_i$, and torques $\hat{\boldsymbol{\tau}}_i$ are updated at 400 Hz and tracked by the cascaded controllers of the motor

electronics. On the highest level, the internal control of the motor electronics generates the desired output torque for one joint as

$$\hat{\tau}_m = P(\hat{q} - q) + I \int (\hat{q} - q)\, \mathrm{d}t + D(\hat{\dot{q}} - \dot{q}) + \hat{\tau}\ , \tag{4.8}$$

where $\hat{q}$ and q is the desired and actual joint position, $\hat{\dot{q}}$ and $\dot{q}$ the desired and actual joint velocity, and $\hat{\tau}$ the desired feed-forward torque from (4.4). The parameters P, I, and D are control parameters, where we set $I = 0$ for this application, and P and D are tuned such that accurate tracking for the full range of the joints is obtained. The desired motor output torque $\hat{\tau}_m$ is then internally tracked by the series-elastic actuator (SEA) controller (Hutter et al., 2016).

4.3.3 State Transitions

The desired/commanded state of the robot is often offset to the actual/measured state of the robot. This tracking error can be found for both leg and base motions and can range from a few millimeters/degrees to multiple centimeter and degrees. To enable a smooth transition from the actual state to the desired motion, one can use the actual state as the first knot point when generating the spline trajectories just before they are needed for execution. However, strictly following this strategy leads to the problem that a steady state error is canceled at the time of transition leading to an undesired jump of the actuator reference value. To this end, we employ the following strategy: In case the control mode is the same before and after the transition from one step to the next, the desired setpoint is used as starting point. In case the control mode changes, the actual state is used. This results in smooth transitions from the current state of the robot to the start of the commanded motions.

4.3.4 Contact Transitions

A legged robot moves by changing the contact situation at each step, making it important that a planned contact be established before the leg can be loaded to support the robot. Due to the errors associated with calibration, sensing, and tracking, the contact transitions typically occur offset to the planned transition time and position. We differentiate between *early* or *late* closing of contacts, depending on if the foot touches down before or after the planned transition time.

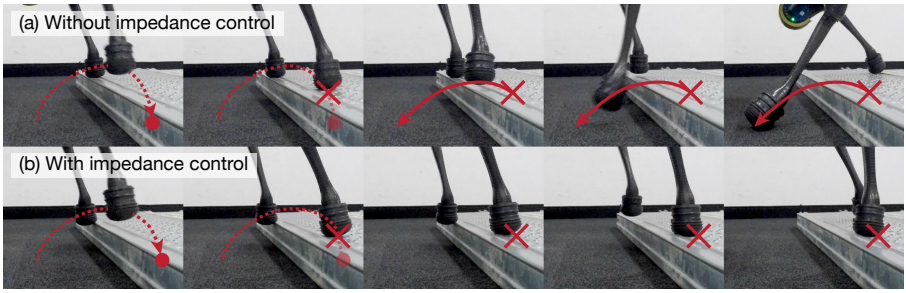


Figure 4.4 The image sequence shows the right fore leg stepping onto an unperceived obstacle with 7 cm height. The foot establishes contact with the obstacle during the swing motion because the planned step target lies below the obstacle. Without impedance control (a), the foot bounces of the obstacle and destabilizes the motion execution. Impedance control (b) stabilizes the motion execution with position and velocity feedback.

Early Touchdown

In case a foot contact is established earlier than planned, our implementation switches the leg to become a support leg before the planned transition. This has been shown to work well for locomotion on flat terrain (Gehring, 2017), but leads to mediocre results for rough terrain. Figure 4.4a shows a step onto an unperceived obstacle. As the desired target foothold is planned for flat terrain below the obstacle, the foot touches down earlier than expected at a high velocity causing the foot to bounce off the terrain. At this moment, the leg is controlled as a support leg with torque control (Section 4.3.1) but the missing ground reaction force causes the leg to sway away from the touchdown position. To counteract this destabilizing issue, we employ, additionally to the base motion torque command, a feedback for joint position and velocity as impedance controller as described in Section 4.3.2 for swing legs. In this case, we set the P and D controller gains of (4.8) as low as possible, while still ensuring that early touchdown situations are robustly controlled as shown in Figure 4.4b, where the foot remains close to the touchdown point.

Late Touchdown

For situations where the foot has no contact although it should have according to plan, the transition to the next Free Gait command is paused, and a behavior is triggered to gain contact. This behavior consists of a motion of the foot from its current position towards the surface along the surface normal. A base motion with the help of the pose optimization is used to support the leg motion extend the reach of the foot. If the contact is be closed during this

maneuver, the execution of the commands continues. Upon reaching the maximum extension of the leg without establishing contact, the motion execution is stopped and the result return as canceled.

4.3.5 Reference Frames

The Free Gait commands for leg motions in Cartesian space and the base motions are given relative to a reference frame, as illustrated in Figure 4.3. We differentiate between three types of reference frames: *body*-, *odometry*-, and *localization*-type frames. Motions in body-type coordinate frames are relative to a frame fixed on a body (e.g., base) of the robot. These motions can be controlled independent of the robot's pose knowledge, i.e., without a functioning pose estimator. Odometry-type coordinate systems are defined relative to a frame associated with odometry-based state estimation (e.g., kinematics and inertial sensors). In these frames, the pose of the robot is typically provided as a smooth trajectory with a high update rate but might accumulate drift relative to the real motion of the robot. Localization-type reference frames are defined relative to GPS, simultaneous localization and mapping (SLAM), or laser-based localization measurements. These often work drift-free but at lower rates and might undergo sudden, discrete displacements (jumps).

The sudden displacements of the robot's pose estimate associated with localization-type frames are problematic for the control of the robot. They can cause sudden changes in the actuator signals, leading to oscillation of the system. We achieve a smooth execution of motions defined in a localization-type reference frame by transforming the motion plan into an odometry-type frame. This transformation is updated after every Free Gait step command (Section 4.2.1), assuming that the state estimation from odometry does not undergo significant drift during one step. With this, the robot can execute motions defined in a frame provided by localization methods, which is vital for an accurate interaction with the environment.

4.3.6 Software Tools

Our software implementation of Free Gait consists of the core module which processes the action definitions and several tools that support the development, execution, and monitoring of whole-body motions. We have implemented Free Gait as a C++ library with Python support classes and made it available open-source (S3). Our implementation supports actions written in any language with a Robot Operating System (ROS) client library. For simple motion definitions, Free Gait supports motions defined in the YAML format (Ben-Kiki, Evans,

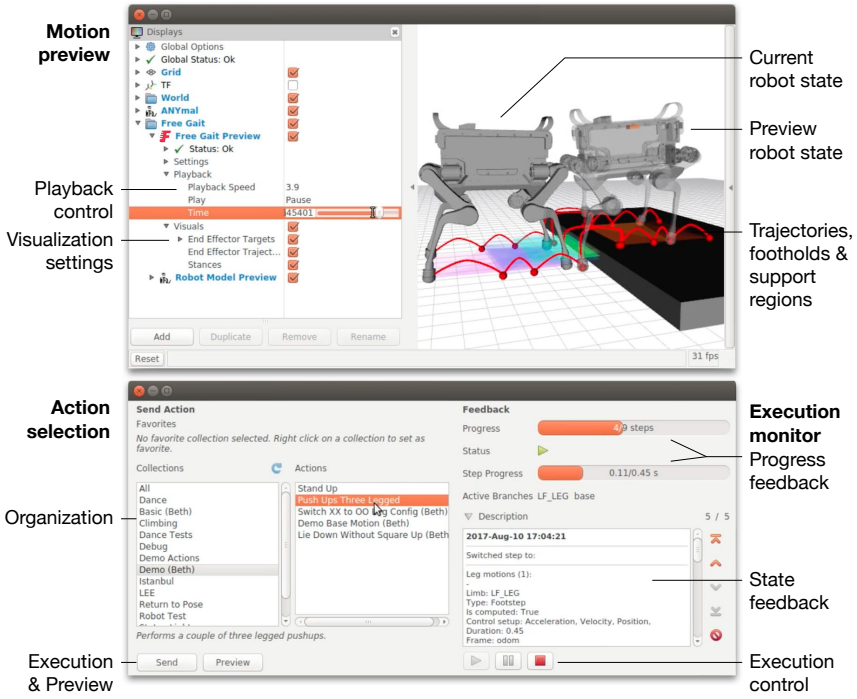


Figure 4.5 The Free Gait frontend provides the user with a user interface to facilitate the development, execution, and monitoring of motion commands.

and Ingerson, 2009). We have developed a graphical user interface to facilitate the execution, analysis, and organization of motion definitions (Figure 4.5). Motion definitions are loaded through a plugin mechanism and organized for quick selection and execution. Action previews can be generated and visualized to analyze the results and facilitate developments. During execution with the robot, the progress can be monitored, paused, and stopped if required. A video overview of the Free Gait user interface is available at (V6).

4.4 Results

Thanks to its flexible interface and structure, the Free Gait framework can be applied to a broad range of motion tasks. Ranging from straight-forward definitions of motion templates to the execution of receding horizon whole-body motion plans, the implementation allows for robust motion control in

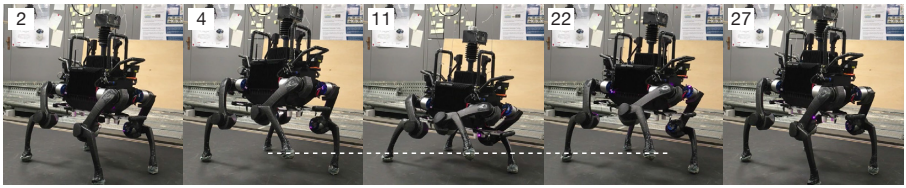


Figure 4.6 A three-legged push-up motion to demonstrate the ease of use of the Free Gait API (total 35 lines of code). The numbers on the top left indicate the corresponding step command in the code example of Section 4.4.1.

real-world environments. This section gives an overview of the results that have been achieved by applying the Free Gait framework.

4.4.1 Introductory Examples

We illustrate the definition of a motion with three basic examples:

Three-Legged Push-Ups

The goal of this task is to perform three-legged push-ups while tracking a spatially fixed position with the non-supporting foot as shown in Figure 4.6 and (V11). The following YAML motion definition generates the described motion:

```
steps:
 - step:
    - base_auto:
 - step:
    - end_effector_target:
        name: RF_LEG
        ignore_contact: true
        target_position:
          frame: footprint
          position: [0.39, -0.24, 0.20]
 - step:
    - base_auto:
        height: 0.38
        ignore_timing_of_leg_motion: true
    - end_effector_target: &foot
        name: RF_LEG
        ignore_contact: true
        ignore_for_pose_adaptation: true
        target_position:
          frame: footprint
          position: [0.39, -0.24, 0.20]
 - step:
```

```
23      - base_auto:
24          height: 0.45
25          ignore_timing_of_leg_motion: true
26      - end_effector_target: *foot
27    - step:
28      - footstep:
29          name: RF_LEG
30          profile_type: straight
31          target:
32            frame: footprint
33            position: [0.32, -0.24, 0.0]
34    - step:
35      - base_auto:
```

In the first step, the `BaseAuto` command on Line 2 moves the base to a position such that the leg in the next step can be safely lifted off ground. In the next step on Line 4, the right fore (RF) foot is moved to a target position with height 0.2 m defined in the footprint frame. The footprint frame is defined as the geometric center of the stance feet (odometry-type frame). The statement `ignore_contact: true` (Line 7) instructs that no contact is expected at the end of the step. Line 11 describes the step to lower the base to a height of 0.38 m (relative to the mean height of the feet) while keeping the right front foot in the air at same position.[1] The Line 14 `ignore_timing_of_leg_motion: true` states that the timing of the `BaseAuto` command is not adapted to the leg motion, causing it the use the default average velocity value to compute the duration of the motion. The statement on Line 18 `ignore_for_pose_adaptation: true` instructs the `BaseAuto` command to ignore the right front leg in the pose adaptation, causing the base to remain straight instead of tilting towards the elevated leg. The upward motion of the push-up to a height of 0.45 m is described in the fourth step (Line 22). In this step, the `EndEffectorTarget` is reused from Line 15 with the YAML syntax for anchor `&` and reference `*`. Finally, the right front leg is lowered again (Line 27) and the base is moved to the center of all four legs (Line 34).

Switching Leg Configurations

With their extensive range of motion, ANYmal's legs can be configured with the knee joints inwards (X-configuration) and outwards (O-configuration). Figure 4.7 shows the sequence of switching each leg in sequence from X- to the O-configuration in standing posture. The motion is accomplished with the following commands and is repeated for each leg:

[1] If the `EndEffectorTarget` command (Line 15) were not included, the foot would not be moved relative to the base and move down together with the base motion.

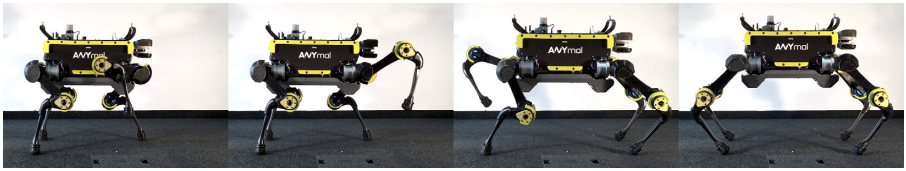


Figure 4.7 ANYmal's range of motion allows it to work with different leg configurations. Switching leg configuration is achieved with a Free Gait command consisting of a joint trajectory and a footstep.

```
1  - step:
2    - base_auto:
3       height: 0.43
4       support_margin: 0.09
5  - step:
6    - joint_trajectory:
7       name: RF_LEG
8       ignore_contact: true
9       trajectory:
10       joint_names: [HAA, HFE, KFE]
11       knots:
12         - time: 2.5
13           positions: [deg(0), deg(290), deg(-270)]
14  - step:
15    - footstep:
16       name: RF_LEG
17       target:
18        frame: footprint
19        position: [0.5, -0.23, 0.0]
20       profile_type: straight
```

Similarly to the previous example, the `BaseAuto` command on Line 2 prepares the base position for a subsequent leg motion with a increased support margin for high robustness. The `JointTrajectory` command on Line 6 causes the joints of the RF leg to rotate from their current position to the desired position in O-configuration. Using a joint trajectory command (instead of a join target command) allows us to define the desired timing and to add via points if required. Finally, the footstep motion on Line 15 establishes the contact of the foot with the ground to switch it from swing to a support leg.

Stand Up & Lie Down

At deployment, the robot is started from a configuration where the legs are folded up, and the base is in contact with the ground. Standing up from this

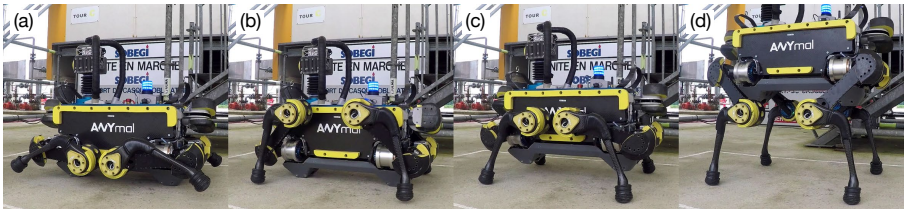


Figure 4.8 Standing up from the ground is a often used maneuver at the beginning of a task. This motion consists of a sequence joint, end-effector, and base motion commands to accommodate the initialization of the state estimator during the execution.

configuration and returning to it is an often executed maneuver. The difficulty with standing up is that the robot's state estimator is not yet initialized at the start with the feet not contacting the ground (Bloesch et al., 2013). Figure 4.8 shows the four steps of our approach. First, the joints are turned in with a join trajectory the stand-up start position (a)→(b). All four legs are now commanded a downwards motion in the base frame (body-type frame) (b)→(c), which circumvents the issue of missing robot pose information:

```
- step:
  - end_effector_trajectory:
     name: LF_LEG
     ignore_contact: true
     ignore_for_pose_adaptation: true
     trajectory:
      frame: base
      knots:
       - time: 2.0
         position: [0.28, 0.25, -0.3]
```

While the base is slightly lifting off the ground, the weight is loaded on the legs triggering the initialization of the state estimator. As soon as pose information is available, the base is lifted to a default hight with a `BaseAuto` command (c)→(d). The lie-down maneuver is very similar but using the motions in reverse order.

4.4.2 Motion Templates

In structured environments such as buildings and industrial sites, we can decompose the locomotion into categories such as stepping up, stepping down, stepping over, and stepping under an obstacle. The motions required to overcome obstacles of the same type are typically similar with the exception of spe-

(a) Step down (b) Step over (c) Stairs up

Figure 4.9 For the autonomous inspection of oil & gas sites, a database of parameterized motion templates was applied for overcoming various obstacles. A motion template guides the robot over a type of obstacle and adapts to the specific dimensions.

cific parameters. To this end, we have developed a database of parametrized Free Gait motion templates to overcome various types of obstacles.

Locomotion in Structured Rough Terrain

Figure 4.9 illustrates the application of template based rough terrain locomotion at the ARGOS Challenge (Fankhauser et al., 2016a; Hutter et al., 2017a). A video overview of ANYmal at the ARGOS Challenge is given in (V7) and (V10). The structure for these motions consists of a sequence of `Footstep` and `BaseAuto` commands. We choose a pre-defined sequence of footstep positions for each of the obstacle types parametrized over the key parameters, such as the height in the case of a step up and down.

A robust execution of the defined templates requires accurate alignment of the motion with the environment. With the knowledge of the position and dimensions of these obstacles, the motion templates are transformed to an environment-fixed frame (localization-type frame) before execution. At the ARGOS Challenge, we used a localization method based on scan matching with iterative closest point (ICP) (Pomerleau et al., 2013) to determine the robot's position w.r.t. the environment.

Figure 4.10 shows the motion of stepping up on an obstacle defined in four trials starting from different positions. During execution, we have perturbed the robot by manually pushing on the base. In one trial, we have blocked the foot from reaching the desired footstep with an obstacle, causing the foot to

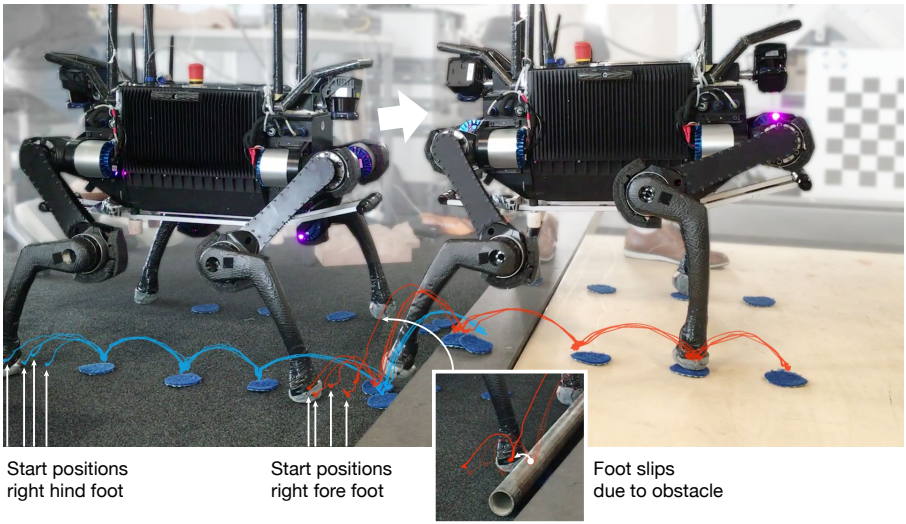


Figure 4.10 ANYmal climbs a 12 cm step with a motion template for four times from different start positions. During execution, we pushed the robot several times, and in one trial, the foothold was blocked by an obstacle causing the foot to slip. With help of the onboard localization, the motion definition can be executed robustly and accurately.

slip. The motion tracking of the feet during the climbing maneuver illustrates the precision and robustness against disturbances of stepping.

Figure 4.11 depicts the foot trajectories of one trial in the global frame from ICP localization, in the odometry frame from the legged state estimation, and in the combined approach as executed by Free Gait (Section 4.3.5). The trajectory of the ICP localization is prone to undergo discrete jumps as shown in Figure 4.11a, which are problematic for control. While the odometry based trajectories are smooth, they accumulate an error of $\sim$5 cm over the walked distance due to drift as shown in Figure 4.11b. The combined approach unifies smoothness and accuracy and has shown to work to our satisfaction in our experiments.

Locomotion in Confined Spaces

Legged robots can adapt their height and mode of locomotion to move through confined spaces. For the motion shown in Figure 4.12a, we have lowered the height of the `BaseAuto` command to 10 cm to walk in crouched posture while the execution adapts the joint coordination. Changing the leg configuration allows ANYmal to move through even tighter spaces. In Figure 4.12b and (V8)

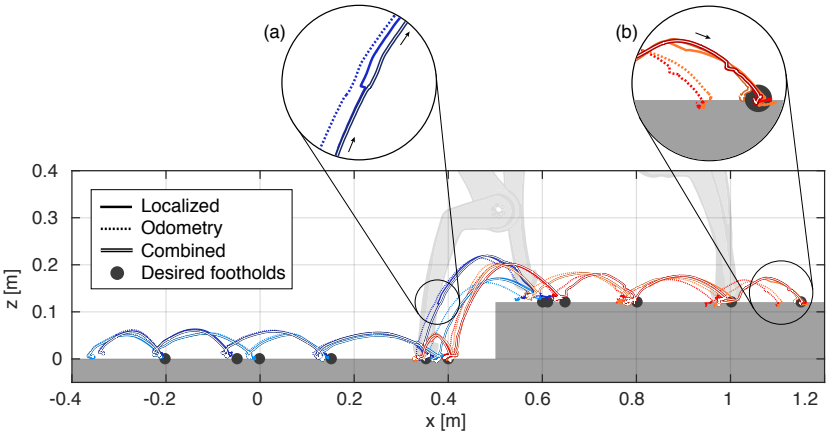
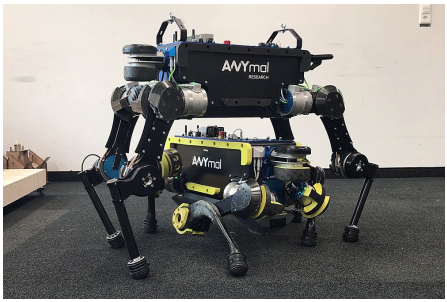


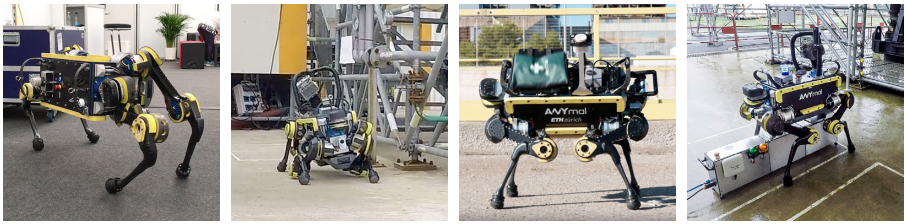
Figure 4.11 The foot trajectories for one trial of the step up motion template are shown. Internally, the movement is generated such that 'jumps' of low-frequency pose updates from localization are rejected (a) to allow for smooth but accurate motion tracking (combined approach) (b).



(a) Crouched walking

(b) Crawling

Figure 4.12 Moving through tight spaces is enabled by lowering the base height (a) or switching to an alternative leg configuration for crawling with help alternating base and leg contacts (b).



(a) 90° tilt (b) Inspection (c) Payload delivery (d) Docking

Figure 4.13 The base commands of Free Gait allow for flexible and robust posture adaptions which are useful for various applications.

we demonstrate ANYmal crawling through a narrow tunnel. In this motion, the robot repeats the motion of simultaneously reaching with all four legs forward (`EndEffectorTrajectory` in the base frame) and then lifting and shifting the base forward (`BaseTrajectory`).

Posture Adaptation

Adapting the base pose is often useful in practical applications. The `BaseTarget` and `BaseTrajectory` commands allows for robust and flexible base motion commands. ANYmal's construction allows it to change height up to 0.5 m and to change its orientation as shown in Figure 4.13a. We have applied the posture adaptation for positioning the inspection head to achieve an optimal point of view without an additional arm (V7) (Figure 4.13b, Hutter et al., 2017a). Equipped with a release mechanism, the change of posture can also be applied to deliver a payload (Figure 4.13c, Bellicoso et al., 2018b). For automated charging, a base motion command is used to precisely maneuver the base onto the docking station (Figure 4.13d, Kolvenbach and Hutter, 2017).

Basic Manipulation

Leveraging the versatility and range of motion ANYmal's legs, basic manipulation tasks can be achieved. The image sequence in Figure 4.14 and (V4) shows how a button at the height of 93 cm is pressed. The position of the button is visually determined with the help of an *AprilTag* (Olson, 2011). The first part of the motion template consists of a `JointTrajectory` command to move the foot close to the target. In a second step, an `EndEffectorTarget` command with position and velocity command in April Tag-frame (localization-type frame) is used to push the button. The target velocity is used to generate an impact velocity when pressing the button.

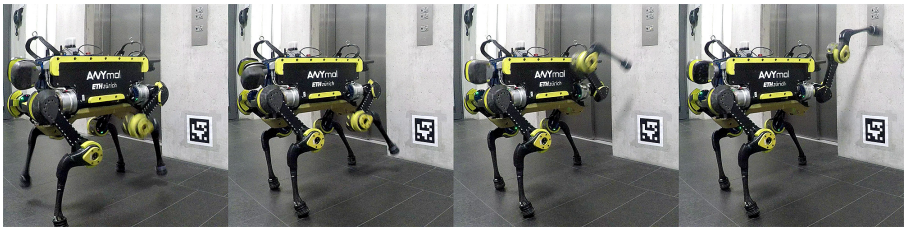


Figure 4.14 Moving the legs above the base allows simple manipulation such as pressing a button. The button is visually localized with help of an AprilTag (Olson, 2011).

4.4.3 Motion Planning

Separating motion generation and execution, the Free Gait API connects motion planners with the motion controllers. In comparison to motion templates (Section 4.4.2) which consist of a few higher-level motion commands, motion planners often output entire robot states at high frequencies. We discuss in this section three examples of the integration of motion planners with Free Gait and refer to Chapter 5 for the integration with a locomotion planner.

Kinematic Whole-Body Motion Planning

For cases where stepping with the feet is not sufficient, Wiedebach (2016) has developed a kinematic whole-body motion planner that takes contact with all links into account. This way, climbing maneuvers can be executed that allows the robot to overcome substantial obstacles safely using support points on the base and knees. A high-level climbing strategy is specified by the operator which consists of a sequence of contact situations. These contact situations can contain arbitrary points on the robot's body as support points. Based on information on the geometry of the environment, a motion planner finds the kinematic plan to connect the contact situations. The planner accounts for collision with the environment, self-collision, static stability, and the kinematic limits of the robot (Zucker et al., 2013). After the kinematic motion has been planned, gravity compensation is computed taking all contact points into account. The planner finds the motion at a time resolution of 10 Hz and transmits the motions as `JointTrajectory` with joint position and torque command to Free Gait for execution. After each contact situation, the whole-body motion planner computes the next transition to account for the current position of the robot.

Figure 4.15 and (V7) show the whole-body motion planner being applied to climb industrial stairs (inclination >50°) with ANYmal. The robot approaches

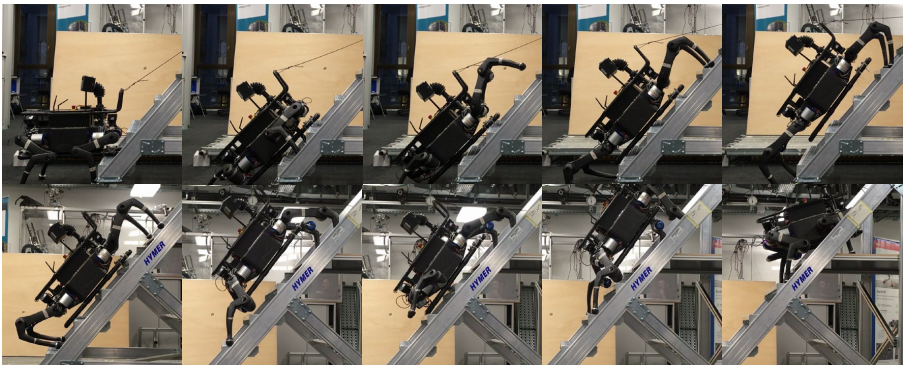


Figure 4.15 ANYmal scales an industrial stair with an inclination of >50° with the help of hooks on the base. A whole-body motion planner (Wiedebach, 2016) computes the joint trajectories to be executed.

the stairs by relying on contact between the base and the stairs to move the legs to a crab-like configuration. With the help of hooks at the lower part of the base, ANYmal then climbs up the stairs by alternating between base contact and contact at all feet to move the base to the next step. At the upper part of the stairs, a similar maneuver is performed to get off the stairs and stand up.

Real-Time Dance Generation

For entertainment purposes, we have implemented the generation of dance motions to live music as shown in Figure 4.16 and (V1) (Bi et al., 2018). The robot carries a microphone to analyze the music for real-time beat extraction. The algorithm then creates a choreography by picking dance motions from a pre-defined library based on the tempo of the music and the previously shown motions. Dance motions include various stepping and base motions. To minimize the offset between music and motion, the execution of the robot is compared to a reference motion generated from a Free Gait preview trajectory. From this comparison, the time offset is computed and minimized by adapting the timing of the dance moves over the course of the performance. We have sown in demonstrations with playback music and real musicians a fast convergence to synchronous motion execution even at tempo changes.

Highly Dynamic Maneuvers: Jumping & Rearing

Similar to our previous work on single-legged hopping (Fankhauser et al., 2013c), we have implemented highly-dynamic maneuvers for ANYmal which

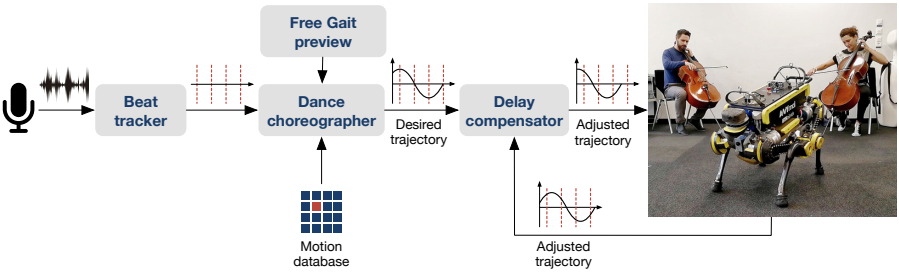


Figure 4.16 The robot listens to live music with an onboard microphone, extracts the current beat, and generates varying dance motions with its legs and base. Online delay compensation ensures synchronous motions to the perceived music. (Musicians: Mr. & Mrs. Cello)

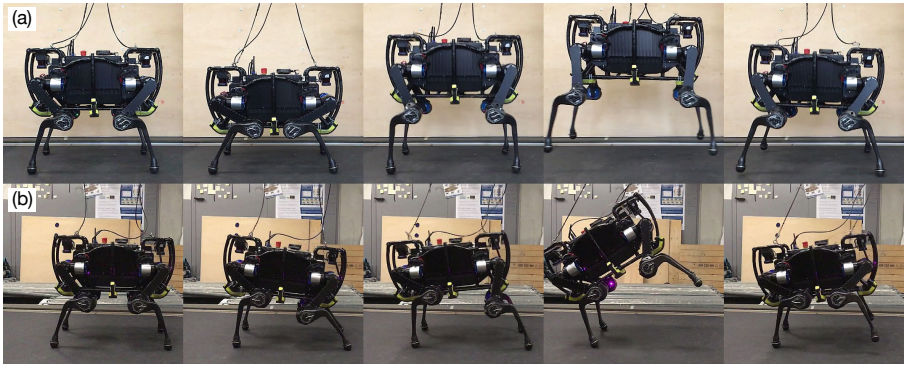


Figure 4.17 Highly dynamic maneuvers are executed with the Free Gait framework. The squat jump reaches through an explosive base motion a jump height of 20 cm (a). During the rearing motion the robot is stabilized by the hind legs (b).

are executed with the Free Gait framework. Figure 4.17 shows the execution of a squat jump and a rearing motion (V11). Both motions are parametrized with a `BaseTrajectory` for the launch-phase, a `EndEffectorTrajectory` for each leg during flight phase, and `BaseAuto` command to stabilize the landing. The motions were hand-crafted for this demonstration but could be generated with learning or numerical optimization (Gehring, 2017).

4.5 Summary

With Free Gait, we have realized a tool for the versatile and platform-agnostic control of legged robots to support the development of modular motion generation algorithms. The motion commands provide a range of command types

that adapt to the complexity of the tasks. With the support of different coordinate frames, motions can be formulated intuitively and efficiently. Our implementation for the robot ANYmal provides a robust motion execution that manages the issues that arise in real-world applications such as state and contact transitions. The user interface complements the framework with tools for organization, development, execution, and analysis of motion plans. We have presented a variety of different applications ranging from basic motions, template-based locomotion and manipulation to the integration with motion planners for static and dynamic motions.

5

Legged Locomotion Planning

This chapter incorporates material from the following publication:

Fankhauser, P., Bjelonic, M., Bellicoso, C. D., Miki, T., and Hutter, M. (2018). "Robust Rough-Terrain Locomotion with a Quadrupedal Robot". In: *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 5761–5768

Legged robots provide many advantages for navigation tasks in challenging environments. To provide the highest flexibility in their usage, our goal is to equip the system with the capability to travel over previously unseen rough terrain. To this end, we combine our work on sensor modeling (Chapter 2), state estimation (Bloesch et al., 2013), terrain mapping (Chapter 3), and motion control (Chapter 4) to a fully integrated system. A successful application of a robot in real-world environments poses demanding requirements on the performance and robustness of each component and the integration as a whole. A chief requirement is that no previous knowledge nor any external equipment is available, which can be outlined as follows:

- The environment is previously unseen, and the robot has to perceive, plan, and adapt its motion as it travels. The robot might be disturbed, and the environment might change during locomotion.
- All sensing and computation is performed onboard, and the robot runs fully self-contained. Sensors and actuators are corrupted by noise, drift, and delay and we take no assumptions on any 'perfect' data.
- All algorithms run real-time in the sense that the robot never stops to 'think' about the next steps.

- Commands can be in the form of desired velocity or position targets. If localization is available, a global target position must be reached if possible.

For quadrupedal climbing, we employ a gait where one leg is in motion while the feet of the other three legs are in contact with the ground. This static walking gait (also referred to as creeping gait) is well suited for rough-terrain locomotion since it permits a quadruped to remain statically stable during the entire locomotion cycle (McGhee and Frank, 1968). In our approach, the acquired elevation map is used to differentiate between safe/valid and unsafe/invalid regions to step on and to plan collision-free trajectories over and around obstacles. The framework replans at every footstep to adapt to updated terrain maps and possible divergence from the previous plan. With the integration of the motion plans over the Free Gait API (application programming interface), the controller can robustly track the motion plan and accommodate significant disturbances. We evaluate the performance of the proposed framework in a range of experiments with obstacles up to 40 % of the robot's leg length including steps, stairs, inclined and moving surfaces.

After discussing related work in Section 5.1, we give an overview of the proposed approach in Section 5.2. We introduce the map processing steps in Section 5.3 and present the locomotion planner in Section 5.4. Our experimental evaluation with the robot ANYmal are described in Section 5.5 and we conclude with a discussion of our approach in Section 5.6.

5.1 Related Work

Considering the high degree of freedom (DOF) of legged robots (e.g., ∼18 DOF for a quadruped with three joints per leg) and all possible foothold locations, the direct motion planning problem is in general intractable (Wettergreen, 1995). One approach to decompose the planning problem is to first consider the rough motion of the robot through the map as a sequence of robot poses and plan the leg motions in a second step (Eldershaw and Yim, 2001; Perrin, 2012). The drawback of this method becomes apparent when two neighboring poses cannot be connected with the leg planner, and replanning is necessary. A reversed approach was followed by Bretl (2006), Escande, Kheddar, and Miossec (2006, 2013), and Hauser et al. (2008), where first the foot and hand placements are planned and then a motion trajectory is synthesized that follows those placements. This approach is chosen since in very challenging ter-

rain the transitions between stances are the most constrained configurations along a multi-step path.

While many researchers have focused on evaluating their algorithms in simulation, an extensive test of quadrupedal locomotion over rough terrain has been conducted within the scope of the Learning Locomotion (L2) program. The focus of the challenge was set on the development of control and planning algorithms while all other aspects were greatly facilitated. Interestingly, the solutions of most teams have eventually converged to a similar structure (Kalakrishnan et al., 2010; Kolter and Ng, 2011; Neuhaus, Pratt, and Johnson, 2011; Zucker et al., 2011). A typical planning approach was to first conduct global planning of the center of mass (COM) across the terrain. Then, a foothold planner based on a regular static gait would carefully choose each step. Here, the main difficulty was to choose safe footholds that would inhibit the robot from tumbling over. Since the provided hardware platform possessed limited torque control capabilities, the missing compliance made it difficult to recover from foot slipping from non-ideal contacts (Buchli et al., 2009). In this context, Kalakrishnan et al. (2010) and Neuhaus, Pratt, and Johnson (2011) demonstrated the importance of real-time re-planning to achieve robust, rough terrain locomotion. While impressive performances were shown during the L2 program, the pre-scanned terrain models, external pose measurements with motion capture, and preparative computation time before the run restrict the direct transfer of these results to real-world use cases.

Recent advances towards a fully integrated system were published for the *Messor II* hexapod robot (Figure 1.3i) by Belter, Łabęcki, and Skrzypczyński (2016). This work uses an A* guided rapidly-exploring random tree (RRT)-based (Kuffner and LaValle, 2000) planner to find a kinematic plan, integrating foothold selection and motion planning in one sampling-based planner. While some results have been shown in combination with online state estimation and mapping, the robot is tethered to an external computation and power source, and no information about obstacle height and execution speed is given. Their approach relies on accurate sensing of the environment and precise self-localization of the robot, which are reported to be the most common sources for failures (Belter, Łabęcki, and Skrzypczyński, 2016).

Closest to our approach is the work on the hydraulic quadruped *HyQ* (90 kg, 1 m height, Figure 1.3k, Semini et al., 2011). While still tethered to an external power source, the robot has shown autonomous locomotion over previously unseen obstacles. The approach by Havoutis et al. (2013) focusses on maneuvering over flat horizontal surfaces by selecting the most suitable foothold from

a mask with nine candidates located around the nominal step. Localization is based on visual frequency-based matching of the ground texture, which is difficult to achieve robustly in many situations. In a similar approach (Barasuol et al., 2015), HyQ was shown to step over obstacles with a dynamic trotting gait. However, the author's fell back on an external motion capture system for drift-free localization. In the work of Mastalli et al. (2015), the authors showed online graph-based footstep planning on flat horizontal obstacles (also with help of an external motion capture system), but no velocity results are reported. Similarly, the work of Winkler et al. (2015) builds upon the architecture of Kalakrishnan et al. (2010) and extends it among others with online state estimation and terrain mapping. With this, the authors report a maximum achieved step height of 15 cm (20 % leg length) at an average speed of 7–13 cm/s. An alternative method was presented by Mastalli et al. (2017b), which combines the foothold and 2D COM planning as a coupled, non-linear optimization. This was shown to increase the locomotion speed (Mastalli et al., 2017a), but at the cost of prohibitive computation times of up to 10–15 min. The problem of local minima in non-linear optimizations was tackled by Aceituno-Cabezas et al. (2018) in a mixed-integer convex formulation based on Deits and Tedrake (2014). So far, the trajectories have been optimized offboard and it is unclear how well the segmentation of the terrain with convex regions will extend to realistic environments (currently hand-labeled). In their latest work (Focchi et al., 2018), the authors emphasize the importance of continuous replanning and adaptive foot placement and reflexes for degraded vision. They use a one-step foothold adaptation to set the swing leg target and height according to an elevation map, but do not differentiate between different foothold qualities. With this, HyQ is shown to traverse boulder with obstacles up to 10 cm diameter (13 % leg length) at a relatively moderate speed.

5.2 Overview of the Approach

Our proposed method consists of several modules that are all interconnected but run in parallel at different update rates. Figure 5.1 shows an overview of the modules and the information flow. Starting with a desired final goal pose (defined by a position and a yaw-orientation angle) or desired velocity command, a next intermediate goal pose is computed. This is to ensure that the robot does not walk long distances in a direction in which no range sensor is mounted. The range sensor continuously measures the distance to the terrain and is used together with the robot pose estimate to create an elevation map around the robot at 10–20 Hz depending on the type of range sensor. The

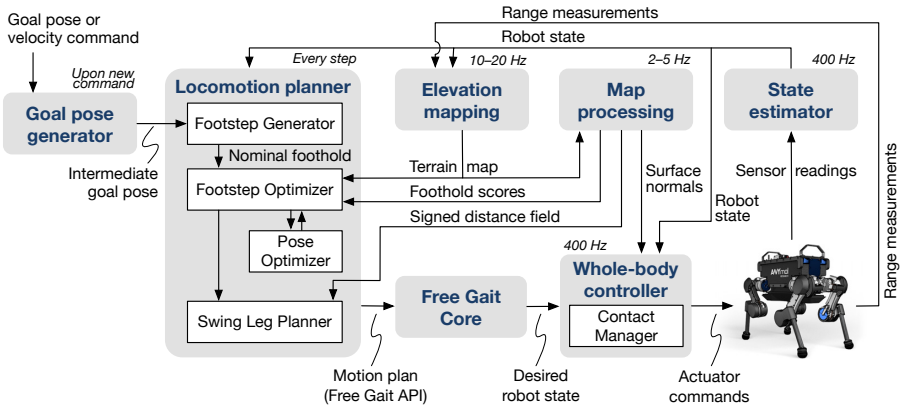


Figure 5.1 Based on a goal pose or velocity, the locomotion planning framework generates a new motion plan which is tracked by the whole-body controller. Sensory information is used to estimate the robot's pose and to build a map of the surrounding terrain. The terrain map is processed, and valid footholds are identified. At every step, new foothold locations and swing leg trajectories are planned to account for disturbances and changing environments.

terrain map is shifted along with the motion of the robot and stores the height information beneath and around the robot and fuses existing data with new measurements. At a lower rate of 2–5 Hz, the elevation map is processed to compute foot score values, surface normal vectors, and a three-dimensional signed distance field (SDF). The foot score values reflect the suitability of an area in the map to place a foot on.

The terrain information is used in the locomotion planner to find a feasible motion of the robot over the perceived terrain with a one-step look-ahead. The final output of the planner is the target foothold together with the swing leg motion for the next step. First, based on the goal pose and the current stance (position of the legs), a sequence of (planar) footsteps is generated. These footsteps do not consider the terrain but represent the nominal gait sequence and step lengths. To find a feasible footstep, the area around the nominal footstep position is searched for the closest valid foothold. Since valid footholds are not necessarily reachable, a pose optimizer is used to check if a stable robot configuration can be found which respects static stability and kinematic constraints. Once a candidate foothold is verified by the pose optimizer, the swing trajectory can be planned. Based on the terrain's SDF, a collision-free swing trajectory is planned which connects the current foot position and the optimized foothold target. As the map is updated continuously and because the robot might not reach previously planned footholds (due to

slip or compliant ground), this sequence of steps in the locomotion planner is performed at every footstep to adapt to these changes.

Based on the target foothold and the swing leg motion plan, the Free Gait Core module synthesizes a whole-body motion. Internally, it uses the same pose optimizer for the generation of the base motion as previously in the locomotion planning process, ensuring that the planned leg motions are feasible. Because the robot lifts one leg at the time, we assume that the statically stable start and target pose can be connected such that the projected COM trajectory lies within the (convex) support polygon. Given the full motion plan, the whole-body controller executes and stabilizes the robot's behavior around the desired trajectories. The controller runs at 400 Hz to generate the actuator commands from a closed-loop with the state estimator. It uses the terrain surface normal information from the map to generate contact forces that respect the boundaries of the friction cones. Rather than executing the trajectories regardless of the current state, the controller ensures that the desired contact situations are reached (contact manager). Because the motion plans always contain errors, this is a crucial step in the robust tracking of the motion.

5.3 Map Processing

In Chapter 3, we presented an approach to create 2.5D elevation maps from the combination of pose estimate and a range measurement sensor. In this representation, each cell of the grid holds a height and a lower and upper confidence bound. Our approach takes the uncertainty of the pose estimate and the range measurements explicitly into account to obtain consistent maps even without requiring an additional localization system such as GPS or vision- or laser-based registration. The elevation mapping framework works with different range sensors such as laser range sensors, time-of-flight, structured-light, and (active) stereo cameras. Given the continuously updated robot-centric elevation map, the terrain is processed to compute surface normals, find valid footholds, and build a three-dimensional SDF used for planning of collision-free swing trajectories.

5.3.1 Surface Normal Vectors

We use a standard way to extract the surface normals from the noisy surface representation based on the principal component analysis (PCA)(Klasing et al., 2009). Given the surface points in a circular region and subtracting their mean position, the surface normal can be estimated from the Eigen vector

associated to the smallest Eigen value of the subset. As the nearest neighbors for each cell in the grid can be accessed directly, the surface normals can be computed efficiently for the entire map.

5.3.2 Foothold Classification

Each cell in the elevation map is associated with a foothold score $s(x, y) \in [0, 1]$. This value represents the quality of the foothold, where a cell with $s = 1$ is a safe foothold, and a cell with $s = 0$ might cause the foot to slip. Similarly to other work (Kolter, Rodgers, and Ng, 2008; Wermelinger et al., 2016; Winkler et al., 2015), we have found that a linear combination of different quality measures

$$s(x, y) = \max \left(1 - \sum_i w_i \frac{v_i(x, y)}{v_{\text{crit}}}, 0 \right) \tag{5.1}$$

provides a simple but reliable foothold score. The variable w_i represents the weight factor for each quality measure i with value $v_i(x, y)$ and maximum allowed/critical value v_{crit}. As foothold quality measures, we use the following geometrical features:

- **Slope** describes the tilt angle of a local surface patch. We compute it from the surface normal vector $\boldsymbol{n}(x, y)$ (Section 5.3.1) as $v_{\text{slope}}(x, y) = \arccos n_z(x, y)$.
- **Curvature** represents the local deviation of the terrain shape from a plane. We use the curvature to characterize for examples convex or concave areas and compute it with a local fit of a fourth-order polynomial.
- **Roughness** is used to describe the local terrain deviations in the direction of the normal vector. We can compute the roughness with an approach from image processing where the difference between the original elevation map and a radially blurred version of it is computed.
- **Uncertainty** is computed as the local difference between the upper and lower confidence bound of the elevation map (Section 3.2.4). It is used as a quality measure to enable robust foothold selection even under the influence of drift from the state estimation of the robot.

These geometrical characteristics are computed, where applicable, for circular patches of the size of the foot of the robot (6 cm diameter for ANYmal). To further simplify the foothold selection process, we create a binary foothold

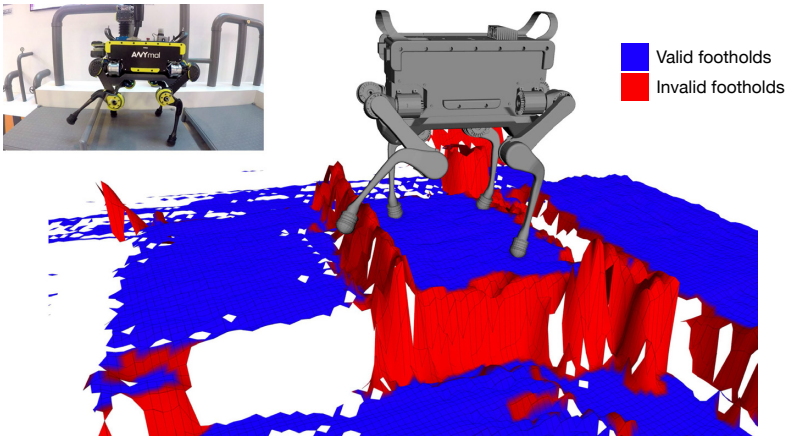


Figure 5.2 The elevation map is processed to differentiate regions with valid (blue) and invalid footholds (red). The foothold classifications is based on local terrain features such as slope, curvature, roughness, and uncertainty of the map data.

score $\hat{s}$ based on a threshold to map s to either 0 or 1. Figure 5.2 shows an elevation map where the color represents the binary foothold score $\hat{s}$ for valid and invalid footholds.

5.3.3 Signed Distance Field

Planning collision-free trajectories can be solved with the help of a SDF map representation. We generate a three-dimensional SDF from the elevation map, where each voxel holds the distance to the nearest surface. The SDF can be efficiently computed with the approach presented in (Felzenszwalb and Huttenlocher, 2012).

5.4 Map-Based Motion Planning

Based on the current state of the robot, the goal pose, and the terrain information (Section 5.3), the goal is to find a feasible sequence of contacts and whole-body motions to let the robot overcome rough terrain. Our approach is a motion planner that, starting from a nominal footstep, finds the next feasible foothold (one-step planning horizon) with the help of the combination of the binary foothold score map and a robot pose optimizer. Once a foothold is verified to be feasible from both the foothold score map and the pose optimizer, the swing leg is planned in a second step. Finally, the desired motion plan is executed while accounting for errors in sensing and actuation.

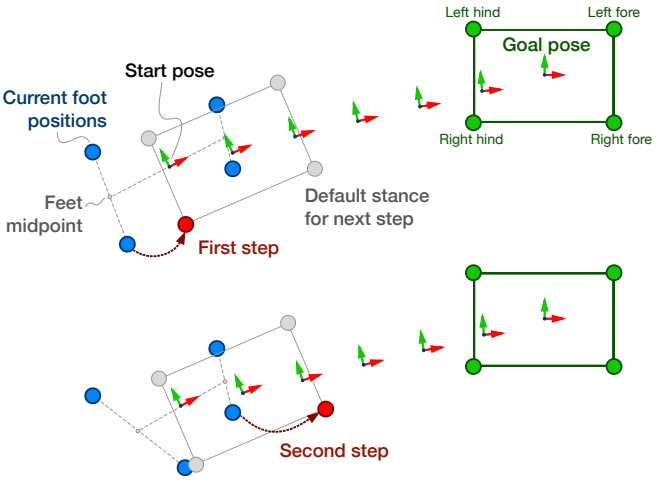


Figure 5.3 Illustration of the first two steps of the nominal footstep generation. At every step, a sequence of default stances is (re-)computed that guide the robot from the start pose to the goal. The foot position in the default stance of the subsequent pose is selected as the target for the current swing leg. This procedure converges to a skew between the left and right legs of the robot, which is important for stability and speed.

5.4.1 Footstep Generation

For generating the nominal footsteps, we employ a geometrical approach which is recomputed at each step (Figure 5.3): First, the current stance position and orientation (*start pose*) is determined based on the current foot positions. The start position is given by the geometrical center of the feet and the start orientation is given by the orientation of the connecting line between the midpoints between hind and forefeet. The *goal pose* is either given by a navigation algorithm (Wermelinger et al., 2016) or a velocity command (e.g., from a joystick) is used to project a goal pose from the start pose. Based on the difference between the start and goal pose, the main direction (longitudinal, lateral, and yaw rotation) is determined. Given the main direction and the default step length, the path between the start and goal pose is linearly sampled with n equal *default stances*. Here we use a priori knowledge of a default step length for longitudinal, lateral, and yaw orientation steps. In case of a velocity command, the default step length is scaled according to the desired speed. Each stance is a representation of how much the robot travels for a full cycle in the main direction with the default step length. The leg sequence is determined with the main direction and is given by the optimality analysis by McGhee and Frank (1968). For the forward direction, the leg sequence is right-hind,

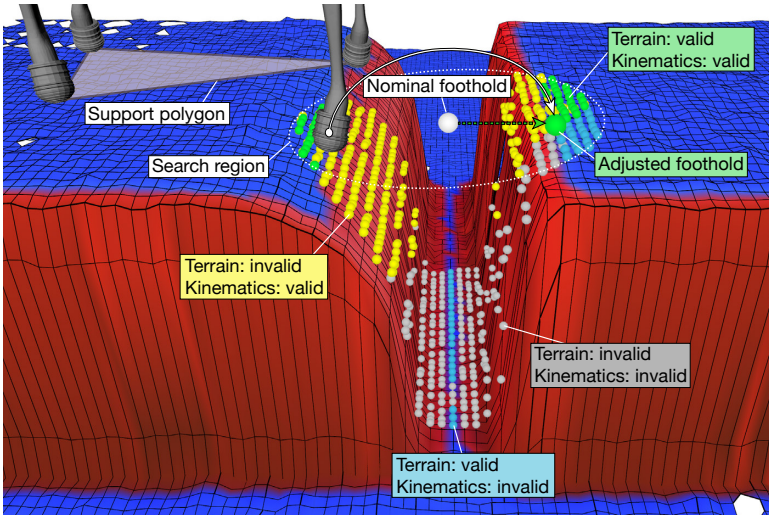


Figure 5.4 Starting with the nominal foothold (Section 5.4.1), candidate footholds in the search region are checked for their validity. Terrain validity is indicated by the foothold scores $\hat{s}$ based on the local geometric features (Section 5.3.2). Kinematic validity is checked with the solution of a pose optimization (Section 5.4.3). The nominal foothold is adjusted to the closest candidate foothold which is valid for both terrain and kinematics.

right-fore, left-hind, left-fore. The new footstep location is given by the position of the corresponding foot in the first default stance after the start pose. If the main direction changes, the leg with the largest first step is chosen as the new start of the sequence.

This entire procedure is repeated after every step and provides a suitable default foothold pattern. Apart from being simple and extremely quick to compute, there are several important properties of the approach: The algorithm can start from any configuration of the feet and always ends up in the default stance at the goal pose. Since the algorithm is recomputed every step, this method converges to a foothold pattern with a skew between the left and right legs. This skew is essential for speed and stability and reduces the required base motion (Neuhaus, Pratt, and Johnson, 2011). Finally, this approach is robust against any deviation from the original plan and allows the robot to re-compute the next step towards the goal at each step.

Algorithm 1 Footstep optimization

Input:

- Nominal foothold position $\overline{\boldsymbol{r}}_F = (x_F, y_F)$
- Search region R
- Terrain height map function $z = f(x, y)$
- Binary foothold score map $s^* = f(x, y)$
- Pose optimization function $f(\boldsymbol{r}_{IB}, \Phi_{IB})$

Output:

Closest foothold $\hat{\boldsymbol{r}}_F = (\hat{x}, \hat{y}, \hat{z})$ satisfying terrain and kinematics constraints if exists within search region

function FOOTSTEPOPTIMIZATION($\overline{\boldsymbol{r}}_F$, $z(x, y)$, $s^*(x, y)$)
  $\mathcal{F} \leftarrow \langle \overline{\boldsymbol{r}}_1, \ldots, \overline{\boldsymbol{r}}_n \rangle \quad \forall\, \overline{\boldsymbol{r}} \in R \quad \big| \quad \left\| \overline{\boldsymbol{r}}_i - \overline{\boldsymbol{r}}_F \right\|_2 \leq \left\| \overline{\boldsymbol{r}}_{i+1} - \overline{\boldsymbol{r}}_F \right\|_2$
  for $\overline{\boldsymbol{r}}_i$ in $\mathcal{F}$ **do**
    if $s^*(\overline{\boldsymbol{r}}_i) = 1$ **then**
      $\boldsymbol{r}_i \leftarrow (\overline{\boldsymbol{r}}_i, z(\overline{\boldsymbol{r}}_i))$
      if argmin $f(\boldsymbol{r}_{IB}, \Phi_{IB}) \neq \emptyset$ for current stance and $\boldsymbol{r}_i$ **then**
        return $\hat{\boldsymbol{r}}_F \leftarrow \boldsymbol{r}_i$
      end if
    end if
  end for
  return $\emptyset$
end function

5.4.2 Footstep Optimization

The footstep determined by the nominal footstep generator is verified for kinematic and terrain feasibility. If the nominal footstep is not a feasible foothold location, the nearest neighbor valid foothold location is found as outlined in Algorithm 1 and illustrated in Figure 5.4. The positions around the nominal foothold are iterated through with increasing planar Euclidian distance from the nominal foothold. Each position is checked for its validity by checking the binary foothold score for the cells in a patch around the candidate position. We choose a circular patch with a diameter of 8 cm, which is slightly larger than the foot size of ANYmal to account for foot rolling during locomotion. If all binary foothold scores in the patch are valid, a candidate foothold is found.

Although the candidate foothold might be a feasible position based on the terrain, the robot might not be able to reach it due to kinematic constraints. For this reason, every candidate foothold is checked for its reachability by

running the pose optimizer with the current and the candidate foothold as input. If the pose optimizer does not find a solution, the iteration continuous. Once the search region is exhausted, the procedure is stopped, and the method signalizes that no valid footstep could be found.

5.4.3 Pose Optimization

The pose optimization is a crucial part of our approach to enable climbing up and down high obstacles. When reaching for places that are far apart from the previous foot location, the robot needs to adapt its pose such that the new foothold can be reached without sacrificing stability or running into joint position limits. The pose optimization is applied for evaluating the kinematic feasibility of candidate footholds in Section 5.4.2. It is also used in the Free Gait's `BaseAuto` command (Section 4.3.4), which adapts the robot's pose for the current and next contact situation. With this, the pose optimization is a essential component for the contact recovery approach (late touchdown, Section 4.3.4), motion templates (Section 4.4.2), and motion planning (Section 4.4.3 and Section 5.4).

The pose optimization task can be formulated as an optimization problem as follows: Given N foot locations $\boldsymbol{r}_{F_i}$, the goal is to find a robot base position $\boldsymbol{r}_{IB}$ and orientation $\boldsymbol{\Phi}_{IB}$ which maximizes reachability and the stability margin under the constraint of static stability and allowed joint limits. We encode the reachability in the cost function $f(\boldsymbol{r}_{IB}, \boldsymbol{\Phi}_{IB})$ by penalizing the difference between a default foot position $\hat{\boldsymbol{r}}_{F_i}$ and the foot position resulting from the optimized posed $\boldsymbol{r}_{F_i}(\boldsymbol{r}_{IB}, \boldsymbol{\Phi}_{IB})$. Additionally, we maximize the static stability margin by adding a term that penalizes the distance between the planar projection of the centroid of the support polygon (SP) $\bar{\boldsymbol{r}}_{\mathrm{SP}}$ and the planar projection of the position of the COM $\bar{\boldsymbol{r}}_{\mathrm{COM}}(\boldsymbol{r}_{IB}, \boldsymbol{\Phi}_{IB})$:

$$\begin{aligned} \text{minimize} \quad & f(\boldsymbol{r}_{IB}, \boldsymbol{\Phi}_{IB}) = \sum_{i=0}^{N} \left\| \hat{\boldsymbol{r}}_{F_i} - \boldsymbol{r}_{F_i}(\boldsymbol{r}_{IB}, \boldsymbol{\Phi}_{IB}) \right\|_2^2 \\ & \qquad + w_{\mathrm{COM}} \left\| \bar{\boldsymbol{r}}_{\mathrm{SP}} - \bar{\boldsymbol{r}}_{\mathrm{COM}}(\boldsymbol{r}_{IB}, \boldsymbol{\Phi}_{IB}) \right\|_2^2 \qquad (5.2) \\ \text{subject to} \quad & \boldsymbol{A}_{\mathrm{SP}}\, \boldsymbol{r}_{\mathrm{COM}}(\boldsymbol{r}_{IB}, \boldsymbol{\Phi}_{IB}) \leq \boldsymbol{b}_{\mathrm{SP}} \qquad (5.3) \\ & l_{\min,i} \leq \left\| \boldsymbol{l}_i(\boldsymbol{r}_{IB}, \boldsymbol{\Phi}_{IB}) \right\|_2 \leq l_{\max,i} \quad \forall i \in N\ . \qquad (5.4) \end{aligned}$$

We use the support polygon as the area defined by the convex hull of the support feet positions projected along the direction of gravity. The robot is statically stable if the projection along gravity of the COM $\bar{\boldsymbol{r}}_{\mathrm{COM}}$ lies with the

support polygon. As the COM position ${}_B\boldsymbol{r}_{\text{COM}}$ is a function of the joint state vector $\boldsymbol{q}$, we employ the inverse kinematics approach described in Section 4.3.2 to find $\boldsymbol{q}$ from ${}_B\boldsymbol{r}_{F_i}$ for each leg i. We encode the static stability in (5.3) where $\boldsymbol{A}_{\text{SP}}$ and $\boldsymbol{b}_{\text{SP}}$ are the linear constraints spanned by the support polygon. Optionally, we add a support margin which offsets the support polygon inwards to increase robustness (as illustrated in Figure 4.3). The joint limits are expressed with the inequality constraint (5.4) with $\boldsymbol{l}_i$ the leg length given by the distance from the hip joint to the foot for leg i. By adjusting the weighting term w_{COM} in (5.2), we can achieve a suitable trade-off between maximizing kinematic reachability and the stability margin.

We can solve this non-linear optimization problem with sequentiell quadratic programming (SQP) with the analytical derivation of the gradients and Hessians of the cost function (5.2) and inequality constraints (5.3) & (5.4) given in Section A.2 Due to the non-linearity of this problem, the success of finding a solution is dependent on the initial guess. To this end, we use an initial guess based on a least square optimization of the kinematic error for leg i

$$\boldsymbol{e}_i(\boldsymbol{r}_{IB}, \boldsymbol{\Phi}_{IB}) = {}_I\boldsymbol{r}_{IB} + \boldsymbol{\Phi}_{IB}({}_B\hat{\boldsymbol{r}}_{F_i}) - {}_I\boldsymbol{r}_{F_i} \ . \tag{5.5}$$

This error can be minimized by transforming it to an Eigen value problem when parametrizing the rotation $\boldsymbol{\Phi}_{IB}$ as a unit quaternion (Bloesch and Hutter, 2017).

Figure 5.5 shows to solution of the pose optimization for different values of the weighting term w_{COM} in (5.2). For low values of w_{COM} (e.g., $w_{\text{COM}} = 0.5$), the reachability is dominant, and the projected COM position is pushed closer towards the edges of the support polygon. This means also that the contact forces (red arrows Figure 5.5) are distributed unequally, increasing the load on legs closer to the COM projection. A more homogeneous load distribution can be achieved with higher values (e.g., $w_{\text{COM}} = 8.0$) at the sacrifice of moving the legs closer to the kinematic limits. For ANYmal, a balanced trade-off is achieved with $w_{\text{COM}} = 2.0$ which we use in our work.

Figure 5.6 depicts the solution of the pose optimization formulation for situations with increasing difficulty (a)→(d). In most cases when the footholds are well within reach of the robot such as in Figure 5.6a, the pose optimization returns a solution within the constraint without the need for further iterations of the SQP algorithm. For more demanding cases, several iterations are required until a solution is found. For example in Figure 5.6b, the base height has to be lowered to satisfy the stability constraint and the kinematic constraint for the

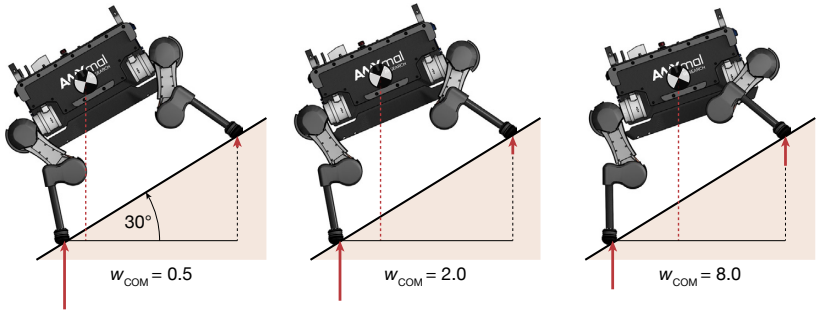


Figure 5.5 Given a set of footholds and a support area, the pose optimizer finds a base pose that fulfills the stability and kinematic constraints (if it exists). Among the set of valid solutions, prioritization is given with the cost weighting term w_{COM} on either the kinematic reachability (left) or the stability margin (right).

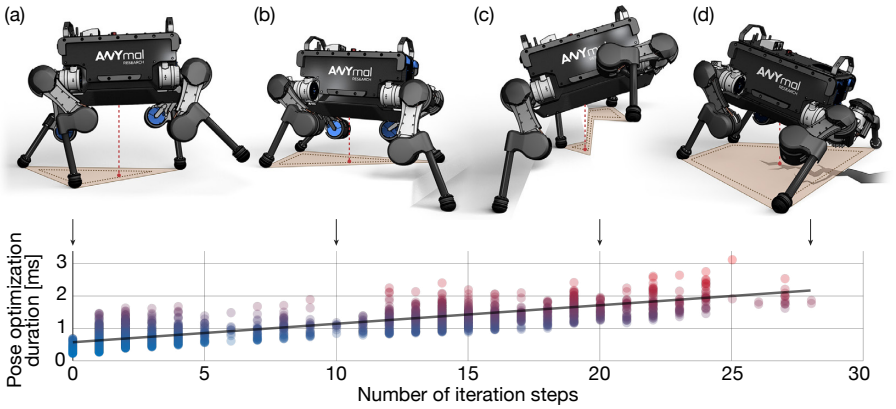


Figure 5.6 Our pose optimization formulation efficiently finds the full base pose (6D position and orientation) for given contact situations. In (a)→(d), we show the solution of the pose optimizer for increasingly more demanding problems. While simple problems (such as regular walking) is optimized at the first iteration in ∼0.5 ms, even complex tasks (such as stair climbing) are solved within 3 ms. The black line depicts the linear fit for the computation durations taken from 7500 trials.

right fore (RF) leg. Similarly, the base is lowered and pitched in the situation shown in Figure 5.6c to satisfy the constraints. In Figure 5.6d, the two front legs are at their kinematic limits, and the pose optimizer finds a solution by pitching the base in the front.

We have evaluated the optimization duration for 7500 problems ranging from simple to complex situations. Note that we update the COM position in (5.2) only at the beginning and after the first iteration in the base frame as ${}_B\bar{\boldsymbol{r}}_{\mathrm{COM}}(\boldsymbol{q})$. This leads to an increased computational efficiency without significant errors for our system. Figure 5.6 shows the measured duration of the pose optimization for our implementation[1]. A solution is found in $\sim$0.5 ms for simple cases and the computation duration stays below 3.0 ms for even the most difficult situations. We were unable to produce problems that require more than 30 iterations to find a valid solution if one exists. Therefore, we use this threshold to signalize the existence of a solution as used in Section 5.4.2.

5.4.4 Swing Leg Planning

Given the start and target foothold position, the trajectory of the swing leg needs to be planned to find the shortest collision-free trajectory. We reduce this task to the problem of finding a trajectory of the foot only and limit the collision checking with the terrain to a sphere (diameter 6 cm) around the foot. Inspired by the CHOMP trajectory planner (covariant Hamiltonian optimization for motion planning, Zucker et al., 2013), we formulate the planning problem as an optimization. Since our swing leg trajectories are parametrized as cubic Hermite splines (Section 4.2.3), we choose to directly optimize over the knot points (except for the start and target). This way, the optimization yields continuous paths that can be smoothly tracked by the robot. We stack the three-dimensional knot positions as a vector $\boldsymbol{u}$ and the position on the spline for time t is given by $\boldsymbol{r}(\boldsymbol{u}, t)$. The optimization cost function is expressed as a weighted sum of length and collisions costs, $f_l(\boldsymbol{u})$ and $f_c(\boldsymbol{u})$, respectively:

$$\begin{aligned} \text{minimize} \quad f(\boldsymbol{u}) &= w_l f_l(\boldsymbol{u}) + w_c f_c(\boldsymbol{u}) \\ &= w_l \int_{t=t_0}^{t_f} \left\| \dot{\boldsymbol{r}}_F(\boldsymbol{u}, t) \right\|_2 \mathrm{d}t + w_c \int_{t=t_0}^{t_f} c(\boldsymbol{r}_F(\boldsymbol{u}, t)) \left\| \dot{\boldsymbol{r}}_F(\boldsymbol{u}, t) \right\|_2 \mathrm{d}t \,, \end{aligned} \tag{5.6}$$

$$\text{subject to} \quad z_{\min} \leq u_{i,z} \leq z_{\max} \quad \forall \boldsymbol{u}_i \in \boldsymbol{u} \,. \tag{5.7}$$

[1] 2.5 GHz Intel Core i7, 16 GB RAM, quadratic programming (QP) solved with the *Quad-Prog++* (Di Gaspero, 2007–18) active set implementation

We constrain the knot points z-coordinate $u_{i,z}$ to lie within a minimum clearance $z_{\min}$ and maximum height $z_{\max}$. The function $c(\boldsymbol{r}_F)$ represents a collision field based on the terrain map

$$c(\boldsymbol{r}) = c_t(\boldsymbol{r}) c_{t_0}(\boldsymbol{r}) c_{t_f}(\boldsymbol{r}) \; . \tag{5.8}$$

The function $c_t(\boldsymbol{r})$ is a smooth potential function for distance to an obstacle based on the distance to an obstacle $d(\boldsymbol{r})$ acquired from the SDF (Zucker et al., 2013):

$$c_t(\boldsymbol{r}) = \begin{cases} -d(\boldsymbol{r}) + \frac{1}{2}\varepsilon & \text{if } d(\boldsymbol{r}) < 0 \; , \\ \frac{1}{2\varepsilon}\left(d(\boldsymbol{r}) - \varepsilon\right)^2 & \text{if } 0 < d(\boldsymbol{r}) \leq \varepsilon \; , \\ 0 & \text{otherwise} \; . \end{cases} \tag{5.9}$$

Note that we use the upper confidence bound of the elevation map for the generation of the SDF as illustrated in Figure 5.7, where ε is the allowable distance threshold. This ensures that the foot does not collide with the terrain even if the knowledge of the terrain is coupled with uncertainties. The functions $c_{t_0}(\boldsymbol{r})$ and $c_{t_f}(\boldsymbol{r})$ create funnels in the collision function to allow the foot to create collision-free contact at the start t_0 and end t_f of the swing motion. They are expressed as a smooth function based on the dead zone d_0 (approx. foot diameter) and the desired funnel angle δ (Figure 5.7a):

$$c_{t_{0,f}}(\boldsymbol{r}_F(t)) = \frac{1}{2}\left(1 + \tanh\left(\frac{d_{0,f}(\boldsymbol{r}_F(t))}{d_2}\right)\right) \; , \tag{5.10}$$

$$d_{0,f}(\boldsymbol{r}_F(t)) = \left\| \overline{\boldsymbol{r}}_F(t) - \overline{\boldsymbol{r}}_F(t_s, t_f) \right\|_2 - d_0 \; , \tag{5.11}$$

$$d_2 = \frac{d_1}{\tan\delta} \; . \tag{5.12}$$

The collision field $c(\boldsymbol{r})$ shapes the resulting swing trajectory based on the magnitude of the uncertainty (difference between terrain estimate and upper confidence bound) at the start and target position. The higher the uncertainty, the more vertical will the foot approach/leave the ground to prevent any potential collision (Figure 5.7b). With little uncertainty, the planning will result in shorter paths due to the lower step height (Figure 5.7c).

We solve the swing motion optimization problem with SQP and derive the gradients and Hessians numerically. For the integrals in (5.6) we use a time resolution of $\Delta t = 50\,\text{ms}$ to provide a dense resolution for collision checking. As this optimization might get stuck in a local minimum, the optimization restarts with an increased number of knots until a solution is found or a maximum knot

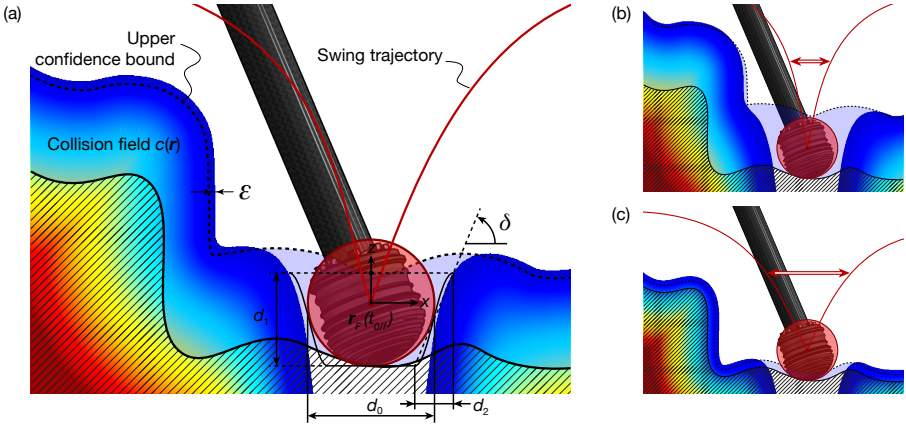


Figure 5.7 Collision-free swing leg trajectories are planned with help of the 3D collision fields (here shown in 2D). The collision field represents the upper confidence bound of the map (a), to take the uncertainty of the mapping process into account. The higher the uncertainty (b), the higher and 'more careful' the robot will step (red arrow). With more accurate knowledge (c), shorter and more deliberate steps are taken.

count is reached.[2] If still no feasible solution is found, we fall back on a line search for the z-position of one intermediate knot point.

5.4.5 Integration with Motion Control

For the reliable execution of the generated motion plan, we build upon our Free Gait controller framework (Chapter 4). The locomotion planner outputs the desired footsteps as foot swing trajectory knot points (`EndEffectorTrajectory`, Section 4.2.3) in the odometry frame in which the terrain map is represented (Chapter 3). With this information, the controller can track the motion relative to the terrain and any slip or error of the robot base motion is compensated for.

Additionally to the swing leg commands, the locomotion planner adds `BaseAuto` commands (Section 4.3.4) together and in-between the leg motion commands. These trigger the generation of base motions that allow the robot to unload legs without violating static stability while using the base to support the leg motions for extended reach.

Finally, the contact management of our motion controller (Section 4.3.4) allows for robust handling of contact changes that occur prematurely or delayed

[2] We use a maximum of three intermediate knot points as threshold.

according to the motion plan. Additionally, the controller smoothly interpolates between the last motion plan and the new incoming plans, such that the planner does not have to care about the delay between planning and execution. In situations where no map is available (e.g., due to dysfunctional range sensor), the architecture of the locomotion framework allows the robot to continue. In this case, the locomotion planner uses the nominal footsteps (with an increased swing height anticipating obstacles) with reactive contact handling as described in Section 4.3.4.

5.5 Results

The following describes the experiments we have conducted with the robot ANYmal to verify and evaluate our approach described in this chapter. All the presented algorithms run on two onboard PCs[3] which are powered by the onboard battery. As range sensors, we either use a rotating Hokuyo UTM-30LX-EW laser range sensor (Section 2.2.3) or an Intel RealSense ZR300 active stereo camera (Section 2.2.5) which are mounted in the front and back of the robot. For stereo processing, we use the OpenCV (Bradski, 2000) implementation of the semi-global matching algorithm by Hirschmuller (2008). The results are accompanied by the video (V5).

5.5.1 Steps

We have commanded ANYmal to walk over a flat step at various heights, which includes a step up and a step down maneuver. For comparison, we have conducted the experiments for the pure reactive walking without exteroceptive perception (*without map*) and with active range sensing (*with map*). At each trial, the robot started from a different position. For the experiments without mapping, we have globally adjusted the step height to match the obstacle. The success rate out of ten trials for each type of combination is summarized in Table 5.1. The average velocity is measured as the total covered horizontal distance divided the duration that the robot takes to clear the obstacle.

For a step height of 7 cm (13 % of leg length), both approaches never failed during our trails. For the step with 14 cm (25 % of leg length), stepping up without perception led to some failure cases, where the footstep would be placed to close to the step and experienced a collision during the following step. This was successfully prevented with active perception. For the 21 cm

[3]Intel Core i5 2.3 GHz, 16 Gb RAM for control and planning / Intel Core i7 3.1 GHz, 16 Gb RAM for terrain mapping

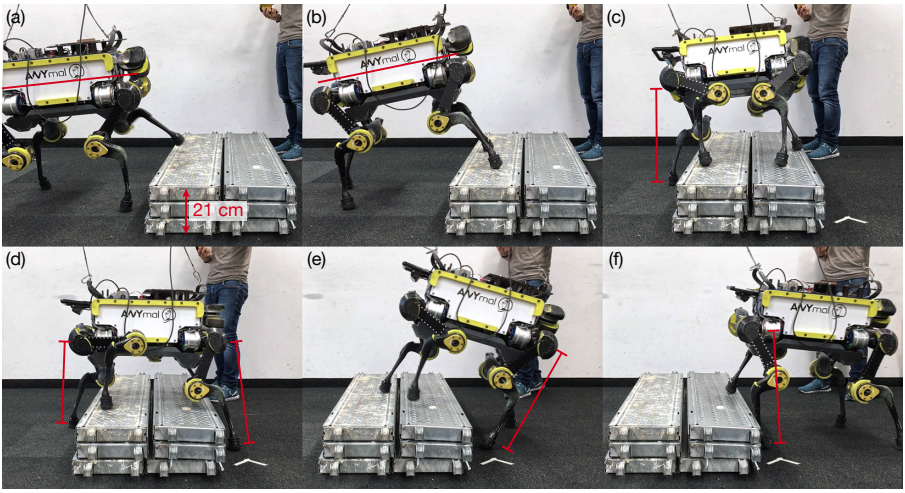


Figure 5.8 The locomotion planner is used to climbing over a step of 21 cm with active terrain mapping with the frontal laser range sensor. The planning framework finds a sequence of safe footholds, base poses, and swing trajectories that guide the robot over the obstacle in 20 s for visible sequence (average 8 cm/s).

step (38 % of leg length), stepping up without perception was not possible, due to the fact that reaching this height without tilting the pose cannot be achieved. However, stepping down was still mostly successful due to our step recovery approach for a late touchdown (Section 4.3.4). Stepping up to 21 cm was again significantly improved with active perception. Figure 5.8 shows snapshots of a trial on the step with 21 cm height with active mapping. One can observe how the base is tilted to facilitate stepping up Figure 5.8a, b. At later stages Figure 5.8c–f, the leg length constraint in the pose optimizer prevents over-extension of the legs.

Table 5.1 Success rates for stepping up and down a step (10 trials each)

Step height		Avg. velocity	w/o mapping	w/ mapping
Step up 7 cm Step down -7 cm	13 % leg length	15 cm/s	100 % 100 %	100 % 100 %
Step up 14 cm Step down -14 cm	25 % leg length	11 cm/s	70 % 90 %	100 % 100 %
Step up 21 cm Step down -21 cm	38 % leg length	8 cm/s	0 % 80 %	90 % 80 %

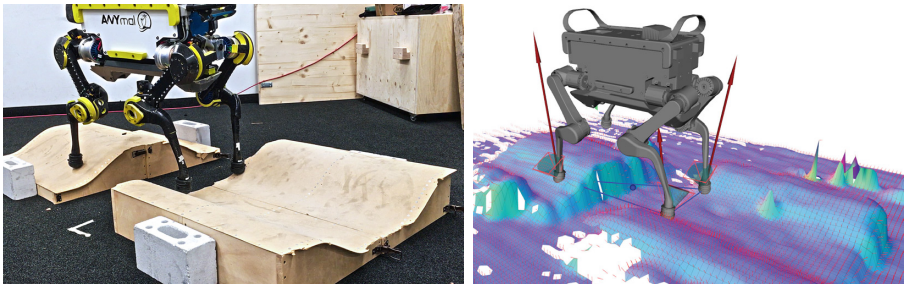


Figure 5.9 Surface normals are computed from the terrain map to enhance the controller performance on inclined surfaces. Based on the orientation of the friction cones, the contact forces are regulated to respect the friction constraints preventing the feet from slipping.

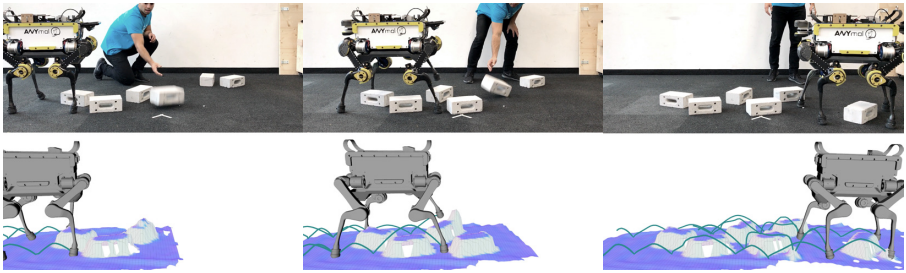


Figure 5.10 As the robot is commanded to walk forward, obstacles are placed in front of it. The mapping framework continuously updates the map which allows the locomotion planner to adapt to the changing terrain.

5.5.2 Inclined Surfaces

With the help of the torque-control capability of ANYmal, we make use of the knowledge of the surface normals from the elevation map (Section 5.3.1). The controller takes the tilted friction cones in the force distribution (4.2) as constraints into account. This extends ANYmal capability to overcome difficult terrain with inclined surfaces. Figure 5.9 shows an image from an experiment on a concave and a convex structure. In this situation, the contact forces are regulated such that they push 'inwards' or 'outwards' to prevent slippage. In our experiments, walking over the same obstacles without taking the surface normals into account was not possible.

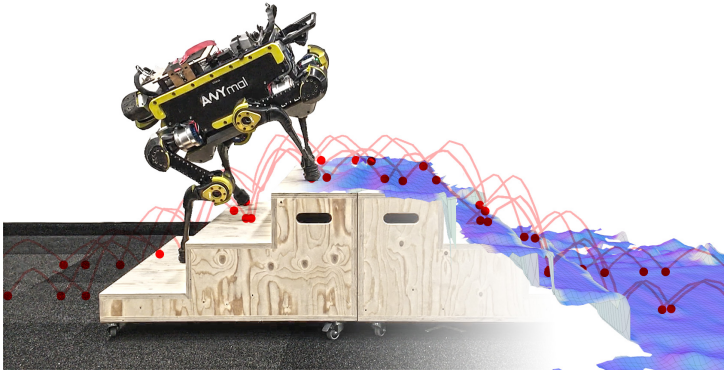


Figure 5.11 The robot ANYmal climbing a set of standardized stairs (17 cm × 29 cm). With help of onboard sensors, a terrain map is generated to plan the footholds (red dots) and base and leg motions (red lines).

5.5.3 Dynamic Environments

Demonstrating the reactivity of our approach, we have exposed ANYmal to a quickly changing environment. As depicted in Figure 5.10, we have commanded forward velocity while placing cinder blocks in front of the robot. The continuous map updates and step re-planning allowed successful planning of the footholds and trajectories around the obstacles.

5.5.4 Stairs

Stairs can be seen as a concatenation of multiple steps in both upward and downward direction. However, they are significantly more demanding because the front and hind legs are often multiple steps apart which makes it difficult to find a motion that fulfills the kinematic, stability, and joint torque limitations. We have evaluated our approach on a standardized set of stairs with a rise of 17 cm and a run of 29 cm as shown in Figure 5.11. The system has no prior knowledge about the shape of the stairs and the planning framework deals with it like with any other obstacle. ANYmal was able to climb the stairs upwards and downwards multiple times. However, climbing the stairs up turned out to be more robust than climbing down, which is explained with the following observations: When descending, the viewpoint at the stairs causes significant shadowing such that only a part of the steps is visible to the range sensor (similar to the shadowing illustrated in Figure 2.1). This means that less valid regions are available to the planner to place the feet in comparison to climbing upwards. Furthermore, lifting a foot is, in general, more robust than touching

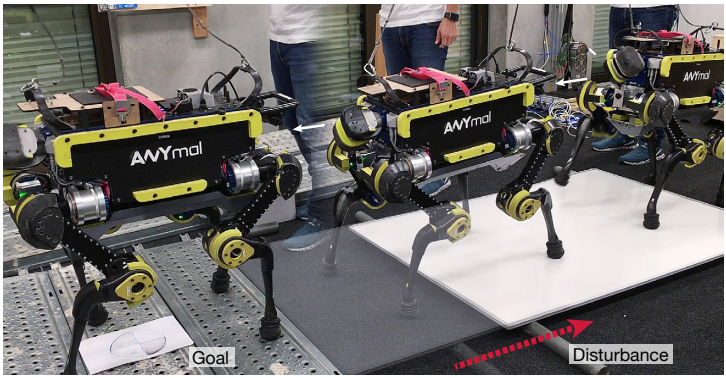


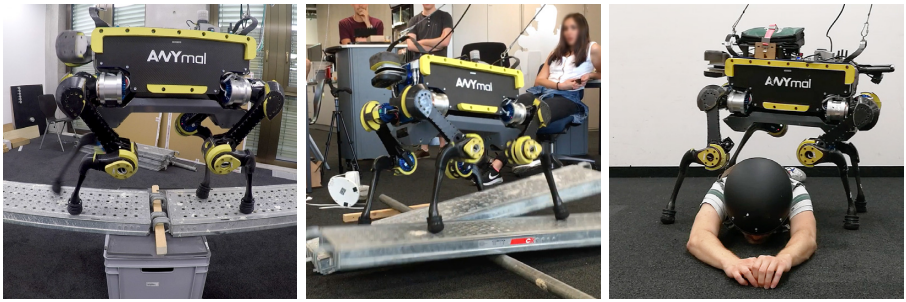
Figure 5.12 The robot replans the motion to adapt to the disturbance which has been introduced by relocating the board during execution. In this demonstration, the robot localizes itself with the rotating laser range sensor and can robustly track and reach the desired goal location.

down. The reason is that the location of the contact point is known more accurately when releasing a contact due to the kinematic information. When closing the contact, the contact location has to be estimated from the terrain map, which can vary depending on the quality of the map. In the case of stair climbing, the most demanding situations occur when one leg has to reach or leave a step which is further down as the other occupied steps. This situation is illustrated in Figure 5.11 and Figure 5.6 for the right hind (RH) leg. For ascending, the leg is lifted in these situations as opposed to touched-down when descending. This causes stair climbing in a downwards direction to be more demanding.

5.5.5 Robustness

We have conducted several experiments to illustrate the robustness of our approach. In multiple trials we have disturbed the execution of approaching a goal, requiring the robot to adapt and replan the motion. For the experiment shown in Figure 5.12, we have moved the whiteboard ∼1 m while the robot was walking over it. In this case, the robot localized w.r.t. the environment with iterative closest point (ICP) (Pomerleau et al., 2013) such that the navigation goal could be tracked in a global frame. While moving the board, the robot successfully kept balance, updated the terrain map, and replanned the footsteps such that it could reach the goal.

Other experiments further demonstrate the robustness of the framework: Figure 5.13 and video (V5) show walking over a thin ledge and climbing over a



(a) Ledge (b) Seesaw (c) Human (Linus)

Figure 5.13 Various experiments demonstrating the versatility and robustness of our approach.

seesaw and a human. In all these trials, the successful completion was enabled by the interaction of terrain mapping, legged locomotion planning, and whole-body control.

5.6 Discussion

Several assumptions underlie the introduced legged locomotion planning framework. This section discusses the design decisions and outlines our experience from practical work with the robot.

Terrain Mapping: With the employed range sensors (Section 2.2), the terrain map represents soft and hard surfaces equally. With this, our approach relies on a purely geometrical reconstruction of the terrain, meaning that the planner cannot differentiate between rigid and deformable objects. The presented method is well suited for hard surface and relies on the introduced contact recovery approach (Section 4.3.4) for compliant surfaces. In our experience, this strategy has shown to work robustly to walk on soft ground such as sand and low grass, and recovers the contact well on deformable objects such as a mattress or an empty bag. However, the higher the difference between the perceived surface and actual contact, the less efficient the gait becomes, as the feet are lifted unnecessarily high. The method might eventually fail for terrain such as high grass, where the feet cannot be lifted sufficiently for a collision-free trajectory.

Foothold Scoring: We employ a simple terrain processing method to classify terrain patches as valid and invalid regions for foot placement (Section 5.3.2), similar to the categories used by Neuhaus, Pratt, and Johnson (2011). This is a strong simplification over other approaches (e.g., Belter and Skrzypczyński (2011), Hoepflinger (2013), and Kalakrishnan et al. (2010)), where a foothold score is determined by a numerical value. In these works, the foothold scoring method is learned in simulation, with experiments, or via human feedback, and is based on a high-resolution, precise model of the terrain. However, in practice we can only retrieve terrain maps of significantly lower quality, rendering a fine-grained foothold scoring method unsuitable. Our approach is motivated by the robot requiring to avoid hazardous regions that could lead to a fall. All other regions are regarded as suitable, regardless of their exact shape. This leads to a more straightforward classification problem which facilitates and robustifies the map processing and foothold selection. Applied in practice, our method successfully avoids hazardous areas while occasional foot slipping is recovered by the controller framework (Section 4.3).

Foothold Planning: Our primary assumption for foothold planning is that we can find a feasible motion by planning the next step while executing the previously planned step. This approach is limited naturally by the greedy behavior of the one-step planning horizon. However, we have found that this approach works reliably for the tested terrains described in Section 5.5 including stairs. A similar experience was reported by Kalakrishnan et al. (2010) and Mitchell, Hofmann, and Williams (2007), where multi-step look-aheads were only necessary for obstacles larger than $\sim$50 % of the leg length. In fact, a valuable property of the presented method is that when climbing a relatively large obstacle, the front legs take a smaller step to avoid the edge, letting the hind legs get closer to the front legs before stepping up with the front legs. This is a behavior that supports the climbing motion, which naturally emerges from the employed re-planning scheme.

Base Motion Planning: We use the pose optimization (Section 5.4.3) to generate a single target pose for the base motion. Our approach relies on the assumption that, given two statically stable base poses (start and target), all poses on the spline interpolation (Section 4.3.4) between the two poses are statically stable. With our quadrupedal robots, where a large part of the mass is at the base, and the COM is close to the base center, the projected COM

trajectory lies close to the connecting line between start and target. As the support polygon is convex, this assumption holds for all practical cases.

For the generation of the base motion, we further assume that the stability of the robot is reflected by constraining the projected COM position to the support polygon. For static stability, this assumption is only valid for horizontal surfaces and more advanced (and computationally more costly) approaches are required to find the correct support region (Bretl and Lall, 2008). Increasing the locomotion speed, the quasi-static motion assumption does not hold, and the stability has to be judged with the center of pressure (COP) or zero-moment point (ZMP) (Vukobratović and Borovac, 2004). For the presented results, we have circumvented the computational complexity of the more accurate approaches by working with a safety margin for the support polygon of 2–5 cm (Section 5.4.3) which is scaled depending on the desired velocity.

In the proposed pose optimization method (Section 5.4.3), the formulation of the cost function (5.2) implicitly takes the joint torques into account depending in the term w_{COM} and the desired default foot position ${}_B\hat{\boldsymbol{r}}_{F_i}$. However, as illustrated in Figure 5.5, prioritizing the distribution of the leg loads comes at the price of decreased kinematic margins. While this is mostly not an issue, but can be problematic in cases when the hardware is pushed to its limits (e.g., stair climbing).

Finally, we omit any collisions of the base with the environment during the planning. Due to the relatively high ground clearance of ANYmal, collisions of the base typically do not occur. If unwanted base collisions happen, for example during climbing of steep stairs, the compliant whole-body control (Section 4.3) suppresses this disturbance.

Swing Leg Motion Planning: Similar to other works (Neuhaus, Pratt, and Johnson, 2011; Winkler et al., 2015), we apply a sequential approach in which the swing leg trajectory is planned after the foothold selection. Thereby, it is not guaranteed that the swing leg planner can find a feasible solution. However, we have found that with the help of the base motion towards to next stance during the swing motion and the large range of motion of our hardware, this is almost never an issue. We also reduce the collision checking to the geometry of the foot, and we neglect other parts of the leg. This simplification still yields satisfactory results in practice, as the unique form of ANYmal's shanks avoid most of the shin collisions as can be observed for example in Figure 5.8d.

5.7 Summary

We have presented a locomotion planning architecture for a quadrupedal robot to climb over rough terrain. All sensing and computation are run onboard the robot in real-time without any prior knowledge of the terrain. Based on the acquired elevation map, the framework finds a sequence of suitable footholds and generates base and swing leg trajectories that guide the motion of the robot over the obstacles. A core element of the framework is the proposed pose optimizer which efficiently finds full robot poses for given foot placements. The planner is tightly integrated with the terrain mapping, robot controller, and localization, allowing it to continuously re-plan the motion while robustly accommodating disturbances and changes in the environment. We have evaluated the effectiveness of our approach in experiments including climbing over steps, inclined surfaces, stairs, and various disturbance tests with changing environments. The results are summarized in video (V5) and show ANYmal climbs over obstacles with a height up to 38 % of the leg length at speeds of 8–15 cm/s.

6

Collaborative Navigation for Flying and Walking Robots

This chapter incorporates material from the following publication:

Fankhauser, P., Bloesch, M., Krusi, P., Diethelm, R., Wermelinger, M., Schneider, T., Dymczyk, M., Hutter, M., and Siegwart, R. (2016). "Collaborative Navigation for Flying and Walking Robots". In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 2859–2866

Flying and ground vehicles, such as walking robots, possess complementary beneficial properties for autonomous navigation tasks. Flying robots can quickly cover large areas and have a favorable viewpoint for situational assessment. On the other hand, a ground vehicle has extended autonomy, can carry a substantial payload, and can actively interact with the environment. Working with a heterogeneous team of flying and ground vehicles opens new possibilities to combine these unique features. In this chapter, we present our work on a fully integrated approach to collaborative navigation for flying and walking robots in unknown and challenging terrain. Our approach is to have the (faster) flying robot explore and map the environment (see Figure 6.1), while the (slower) walking robot utilizes the shared data to localize itself and plan a global path to the goal position.

In our work, we introduce a range of methods which interact as follows: A flying robot (e.g., a hexacopter or similar) is guided to fly over an unknown area of interest to collect visual data with a monocular camera. The visual data is used to create a global map of features (landmarks) and simultaneously localize itself against it (SLAM). Furthermore, the collected monocular image stream is used to create depth images with the help of a semi-dense

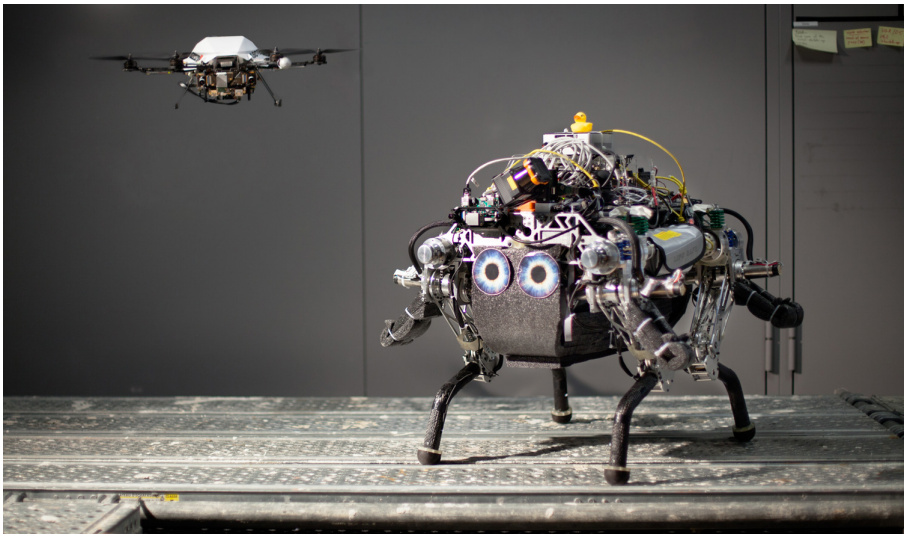


Figure 6.1 A heterogeneous team consisting of a flying and a legged robot is used to navigate in rough terrain. The flying robot complements the ground robot with a higher viewpoint to efficiently map, localize, and move through an unknown environment.

reconstruction pipeline. These depth-images are used to generate a probabilistic elevation map representation of the terrain. The information about the terrain profile is leveraged by the walking robot to plan a global path to the desired goal position. This is achieved by interpreting the elevation map as a traversability map, judging the easiness/safety to walk over a particular region. While following the planned path, the pose of the robot relative to the global landmark map is tracked with an onboard camera. The absolute pose references and the high-rate data from joint states and Inertial Measurement Unit (IMU) are fused in the legged state estimator providing accurate, fast, and drift-free pose estimations for the locomotion controller. As the legged robot moves through the environment, the original elevation and traversability maps are continuously updated with the help of a laser range sensor. The walking robot performs continuous replanning of the path to account for the new data acquired in the map. This allows the walking robot to navigate through a previously unknown environment while preventing collision with unforeseen obstacles. A video demonstrating our work is available at (V12).

We relate the main contribution of this work in Section 6.1. Section 6.2 introduces the different methods applied in the navigation framework and de-

scribes their interrelation. Section 6.3 presents the experimental setup and evaluation of the proposed framework. In Section 6.4, we extend the sequential deployment of the flying and walking robot with a concurrent operation of the platforms.

6.1 Related Work

Prior work in collaboration between aerial and ground robots often addressed the navigation problem in large-scale outdoor environments (Grocholsky et al., 2004; Hsieh et al., 2007; Kelly et al., 2006; Stentz et al., 2002; Vidal-Calleja et al., 2011). These approaches rely on the (at least partial) availability of satellite-based localization, which precludes their application in indoor environments and GPS-denied outdoor areas such as dense forests or narrow urban canyons. Alternatively, several approaches have been proposed for GPS-independent *relative* localization of aerial and ground robots. A flying robot is tracked using a sensor mounted on a ground robot (Heppner, Roennau, and Dillman, 2013; Pratama, 2017; Rudol et al., 2008), or vice versa (Harik et al., 2015; Mueggler et al., 2014; Shen et al., 2017), while mapping is accomplished by only one of the agents. The two robots are required to stay close to each other at all times since the tracked agent can only be localized as long as it is in the field of view (FOV) of the other robot. In a navigation scenario, this prevents the aerial robot from exploring areas further away from the ground robot, which is essential for efficient global guidance. Moreover, tracking-based systems may not easily be scalable to groups of more than two robots.

Our localization and mapping system is independent of both external localization (such as GPS) and relative observations between the robots. In contrast, the robots build and localize within a single map of visual features using onboard cameras. Hence, the system is readily extendable to groups of more than two robots. The approaches by Michael et al. (2012), Forster, Pizzoli, and Scaramuzza (2013), and Käslin et al. (2016) are conceptually similar to ours, yet based on geometric registration, which is known to be sensitive to initial guesses. We experimentally demonstrate that our feature-based visual localization and mapping framework enables reliable operation of both aerial and ground robots using a single map — despite the strongly varying viewpoints of the different agents, and without requiring prior information about the relative poses of the robots.

Moreover, unlike Michael et al., 2012 and Forster, Pizzoli, and Scaramuzza, 2013, we integrate localization and mapping with a complete system for autonomous navigation of the ground robot, comprising collaborative terrain

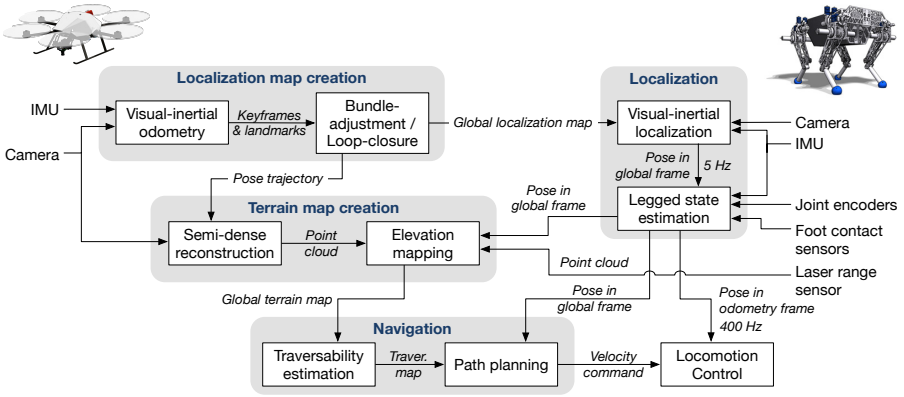


Figure 6.2 Overview of the collaborative navigation framework: The hexacopter's onboard IMU and camera are used to localize, construct a global sparse map of landmarks (global localization map) and to provide a semi-dense reconstruction of the surroundings. The global localization map is used by the legged robot to localize itself and is fused in the state estimation with data from IMU and leg kinematics. A joint terrain map is created from visual data (flying robot) and laser data (legged robot). Finally, control and navigation of the legged robot rely on the state estimation and traversability map to move through previously unseen rough terrain.

mapping using both cameras and laser scanners (multi-modal mapping), as well as traversability estimation, path planning, and motion control. To this end, we also integrate the high-level localization and mapping tools into our high-bandwidth but drift-affected extended Kalman filter (EKF)-based state estimation fusing kinematics and inertial measurements. Together, this provides the control and planning framework with an accurate, drift-free, global state estimation, with sufficient bandwidth such that the legged robot can stabilize itself and precisely navigate through the environment.

6.2 Methods

We describe the methods that we have developed for the collaborative navigation of flying and walking robots. Although our approach can be used for multiple flying and walking robots in the same mission, we will assume that the tasks involve one robot of each type (Figure 6.1).

For the walking robot, we work with *StarlETH* (Hutter et al., 2014b) and the flying robot is an *Asctec Firefly* hexacopter. Both machines can operate autonomously as they have onboard batteries and computers. Each robot has

a network for internal data transfer, and the robots can share information through a dedicated WLAN network.

For global localization and mapping, the flying and walking robots are equipped with a VI-Sensor (Nikolic et al., 2014) containing two global-shutter, wide-VGA 1/3 inch cameras in a front-parallel stereo configuration. For this work, the stereo configuration is not made use of (although could readily be used) because a single camera is sufficient and the adaptive/larger baseline can provide better accuracy. The mounted lenses have a diagonal field of view of 120°. The intrinsics and extrinsics of the sensor are factory calibrated for a standard pinhole camera model and a radial-tangential distortion model. StarlETH is additionally equipped with a front-facing rotating Hokuyo laser range sensor. Figure 6.2 provides an overview of the system and the workflow of the involved components.

6.2.1 Localization and Mapping

For achieving a proper collaboration between flying and legged robots, it is essential to localize both robots within a common global reference frame. To this end, we propose the use of a framework based on Lynen et al. (2015). The idea is to provide a statistically consistent interface between visual-inertial odometry and a mapping backend. Both parts rely on the extraction of visual point features and corresponding descriptors.

The mapping backend merges the visual-inertial data from multiple independent robot trajectories into a single global map representation. Each trajectory is composed of a graph of visual-inertial keyframes and a set of 3D landmarks. The map merging process establishes landmark correspondences between the different trajectories based on visual appearance-matching (Lynen et al., 2014). Subsequently, it performs a joint visual-inertial bundle-adjustment to achieve a globally consistent map (global localization map). Finally, landmarks with low information for localization are pruned from the map (compression step) to obtain a reduced set of landmarks which can be used by the online localization (Dymczyk et al., 2015).

The visual-inertial localization is based on a non-linear fixed-lag smoother. It can track its own set of *SLAM landmarks* (simultaneous localization and mapping) and simultaneously integrate global localization information by tracking the *map landmarks* provided by the backend. This is enabled by co-estimating the relative transformation between the global reference frame, in which the map landmarks are represented, and the odometry frame of reference for local SLAM landmarks. Only a fixed number of keyframes is kept within the

optimization window to keep the computational complexity tractable. Local SLAM landmarks get marginalized once the corresponding keyframe leaves the temporal window. This approach enables accurate visual-inertial localization w.r.t. a global reference frame. At each localization step, the SLAM landmarks are queried in the database of the global map landmarks to establish the feature matches. This means that no initial guess about the pose of the walking robot is required.

6.2.2 Legged State Estimation

The discussed localization and mapping framework provides an accurate pose estimation (see Lynen et al., 2015, for an evaluation), but the obtained bandwidth (5 Hz) is insufficient for the walking robot's feedback controller for stabilizing the system. In order to overcome the above shortcoming, we extend the on-board legged state estimation (Bloesch et al., 2013) with the incorporation of the external pose measurements (loose coupling). This provides the controller and planner processes a unified, high accuracy, high bandwidth, gravity aligned estimate of the robot pose and its derivatives. Additionally, the external pose measurements improve the quality of the legged state estimator by limiting the drift of the unobservable states and by facilitating the estimation of online calibration parameters like inertial measurement unit (IMU) biases.

6.2.3 Terrain Mapping

Range measurements for terrain reconstruction are processes from both robots. They originate either from estimating the image depth from the camera motion of the flying robot or directly from the distance measurements from the rotating laser range sensor on the walking robot. For the flying robot, we employ a semi-dense reconstruction pipeline. Assuming an accurate estimation of the camera trajectory, the pipeline performs a patch warping and alignment between selected camera frames and thereby estimates the corresponding depth. Given a frame of interest, a matching frame is heuristically selected to allow an accurate estimation of the disparity. The baseline of the selected camera frames is adapted to the mean scene distance which improves depth estimation for varying flying altitudes of the robot. In both cases for the walking and flying robot, we are interested in the terrain profile below the walking robot and remove all distance measurements which are above its current base.

For all data generated by the flying and the walking robot, we use the elevation mapping method presented in Chapter 3 to probabilistically fuse the measurements. For the entire process, one elevation map is maintained and

updated with data from the different robots. The noise of the sensor measurements is approximated with the measurement errors models from Pomerleau et al. (2012) for the laser range sensor and an error model based on Lemaire et al. (2007) for the depth estimation from the camera. Note that because in this application the robots can localize themselves accurately and drift-free, we skip the method of propagating the robot pose uncertainty into the elevation map in contrast to the method described in Chapter 3.

6.2.4 Traversability Estimation

Finding a safe path for the walking robot through the environment requires judging the suitability of the terrain for safe locomotion. To this end, we interpret the generated elevation map regarding *traversability*, a local measure expressing the safety to pass the terrain at different locations. Based on our previous work (Wermelinger et al., 2016), we use the three local terrain characteristics slope, roughness, and step height to estimate the traversability for each cell. The slope value reflects the local surface normal within a small radius around the cell. The roughness is computed as the standard deviation of the height values of the neighboring cells. The difference between the maximum and minimum height of the neighboring cells is captured in the step height value. These terrain characteristics are normalized with a constant for the critical value and summed up to the final traversability value as a weighted mean. The traversability is expressed in the interval $[0, 1]$, where a value of 1 indicates full traversability (flat and smooth ground) and a value of 0 means that the terrain is locally untraversable. If one of the terrain characteristics exceeds its critical value, the traversability is set to 0.

6.2.5 Navigation Planning

We employ a sampling-based RRT* planner (Karaman and Frazzoli, 2011) to find the shortest valid path from the walking robot's start pose to the goal as presented by Wermelinger et al. (2016). The planner searches for a trajectory of the x- and y-position of the robot's footprint. Since a legged robot works omnidirectionally, we ignore the yaw-orientation of the robot in the planning process and adapt the yaw-angle such that the robot walks forward towards the next node on the path. To guarantee a rotation-independent traversability of the terrain, we approximate the footprint conservatively with a circumscribed circle. When checking the connectivity between two nodes in the planning process, we check if all cells within the convex hull of the two circles of the footprint are traversable. We define the path cost between two nodes as the

weighted sum of the length and traversability cost. The weighting parameters determine how much short paths over safe paths are preferred.

Since the map is permanently updated, the planner is automatically restarted on a regular basis to update the plan from the current position. In the recurring planning steps, new information in the updated map is taken into account which makes sure that previously unmapped obstacles are avoided, and potential shortcuts are taken.

6.2.6 Path Execution

The robot receives walking commands by iterating through the nodes of the planned path as the robot walks. A proportional-integral-derivative (PID) controller determines the desired velocity from the error between the current and the next desired pose. The desired velocity contains the translational velocities in x- and y-direction and the yaw rotational velocity. For long distances, we prefer to walk forward and orient the robot towards the goal pose while for short distances, the controller is also allowed to step sideways. The commands are executed by the walking robot by placing its feet in a static walking gait corresponding to the desired velocity (Gehring et al., 2013).

As the navigation planning updates the path on a regular basis, the planning time might cause the new path to start behind the current robot location. To tackle this issue, a simple algorithm iteratively removes the first pose on the path as long as the angle between its connections to the second and current robot pose is smaller than 90°.

6.3 Results

We have successfully verified and analyzed the proposed approach with multiple experiments involving the entire algorithmic pipeline. In the following, we describe our results for the mission shown in Figure 6.3 and (V12). The walking robot is deployed in a start area from which the direct line of sight to the desired goal position is blocked by a big obstacle. The only feasible path to the goal consists of a scaffolding with narrow passages (90 cm width) and a ramp which leads down to the floor. During the execution of the mission with the walking robot, a previously unmapped obstacle is put on to the pathway and forces the walking robot to maneuver around it.

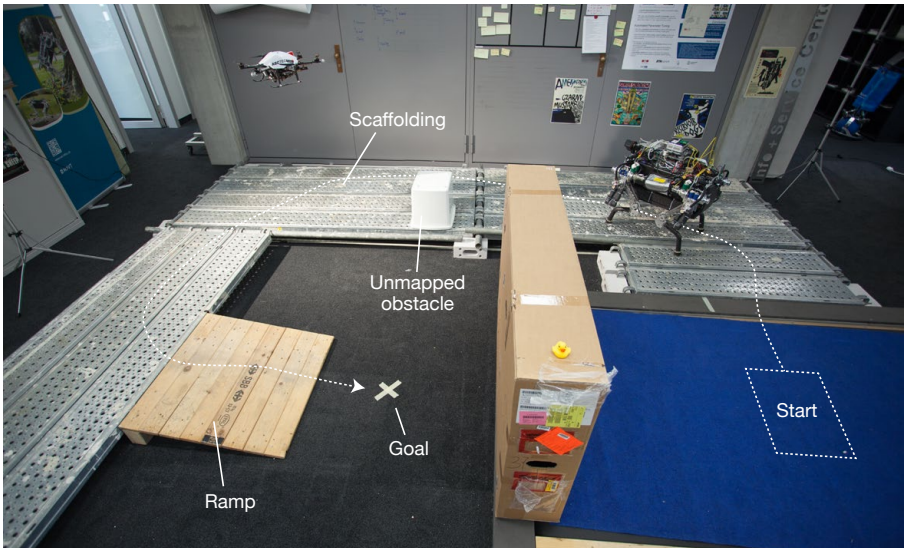
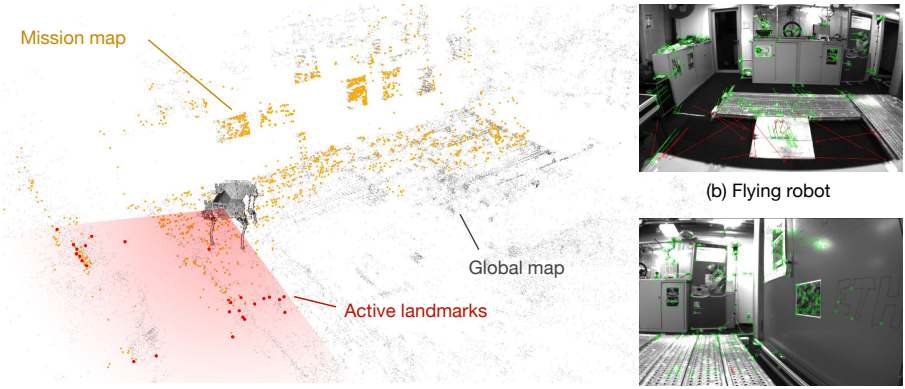


Figure 6.3 Overview of the mission in which the walking robot is asked to autonomously find a path from the start region to the goal. In an exploration flight, the hexacopter covers the area to provide global maps of the environment in which the quadruped robot can plan a path and localize against. During execution a previously unmapped obstacle is added, forcing the walking robot to adapt its plan.

6.3.1 Localization and Mapping

For mapping the environment with the flying robot, we let it cover multiple rounds over our experimental area to detect and track the visual landmarks (see Figure 6.4b). The captured data are transmitted to the walking robot. With its higher computational capacity (Intel Core i7 4500U dual-core 1.8 GHz and 8 Gb RAM), the generation of the global localization map takes several seconds. After the exploration with the flying robot from the air, the walking robot can localize itself within the global map. We have tested different start poses of the legged robot without an initial guess for the localization. The proposed method was able to reliably localize even though the landmarks are observed from a different viewpoint (see Figure 6.4c) than initially recorded.

Once the robot starts walking, the localization against the global map continuously tracks the pose of the robot (localization runs in real-time with $\sim$20 ms processing time per query). Figure 6.4 shows the successfully matched features of the walking robot as it moves through the global map. The pose correction updates to the legged state estimator helped to safely track the path on the scaffolding without causing any instabilities of the walking controller.



(a) Landmarks

(b) Flying robot

(c) Walking robot

Figure 6.4 Visualization of the landmarks (a) as the walking robot executes the mission and localizes itself with the mission map against the global map from the flying robot. The tracked features are shown for the flying robot (b) and the walking robot (c). Despite the change in viewpoints both frames are successfully registered to a common global model.

6.3.2 Elevation Mapping

Similar to the localization map, the data from the flying robot are used to generate dense vision depth estimates which are fused in an elevation map on the computer of the walking robot (processing in approx. real-time). Figure 6.5a shows a snapshot of the elevation map creation during the exploration phase of the flying robot. The dense reconstruction from the camera of the flying robot delivers accurate depth measurements at a frame rate of 5 Hz. This enables it to generate an elevation map with a grid size of 4 cm with only a few holes. After the exploration flight, the entire area is well covered as elevation map (see Figure 6.5b).

As the legged robot moves through the environment, the elevation map is continuously updated from the rotating laser range sensor as shown in Figure 6.5c. The moved obstacle is successfully captured by the map updates (A in Figure 6.5). Errors in the pose estimation of the walking robot can cause an offset between the previous map from the flying robot and new data from the walking robot. While this has a more significant effect on map regions further away from the walking robot (such as B in Figure 6.5), these offsets diminish for close regions and the critical area in front of the robot is represented with more detail in comparison to the original map (C in Figure 6.5).

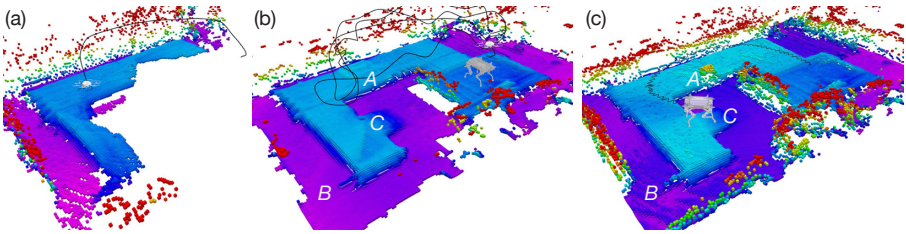


Figure 6.5 (a) The elevation map is built incrementally from the dense depth estimates of the flying robot's camera. (b) The map after the flying robot has observed the full scene is used for an initial navigation plan for the legged robot. (c) The final elevation map is retrieved with the additional fusion of the range measurements from the walking robot. Point *A* marks the point where an obstacle has been added after the initial mapping of the flying robot.

6.3.3 Traversability Estimation and Navigation Planning

Figure 6.6 shows the interpreted elevation map (after the full mission) as traversability map. We can observe that all safe areas including the ramp are correctly marked as traversable while the borders of the scaffolding are correctly interpreted as dangerous. Thanks to the high resolution of the map and the locally bounded interpretation of the traversability, the navigation planner can successfully find a feasible path even in narrow regions.

The first plan based on the map from the exploration of the flying robot (no updates from the walking robot yet) is shown in Figure 6.7a. The planner can find a short path while maintaining a minimal distance to untraversable regions. As the robot starts tracking the planned path, the planner continuously replans the trajectory to take into account the refined map and a changing environment. Figure 6.7b shows how the updated plan successfully guides the robot around the previously unmapped obstacle. Furthermore, we can observe that our navigation controller approach smoothly guides the robot along the path without causing irregularities because of plan updates.

6.4 Concurrent Operation of Walking and Flying Robots

In the experimental evaluation of Section 6.3, we have let the flying robot first fully explore the environment before sharing the data with the walking robot (sequential operation). In the following, we describe the main differences that have to be considered for an operation of the platforms at the same time (concurrent operation).

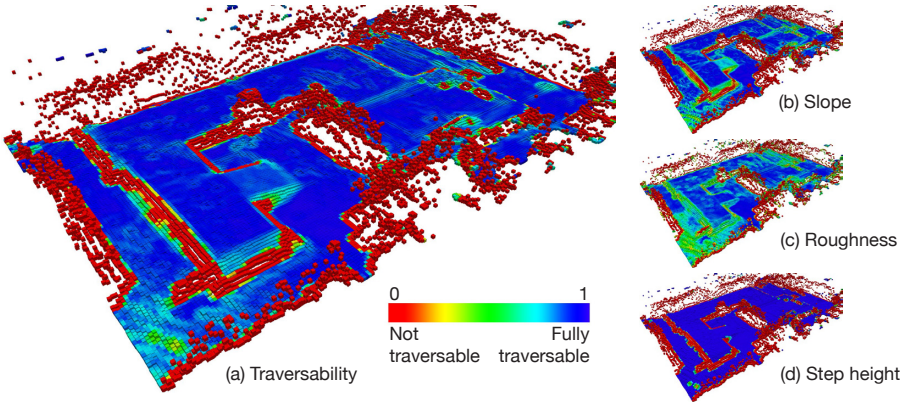
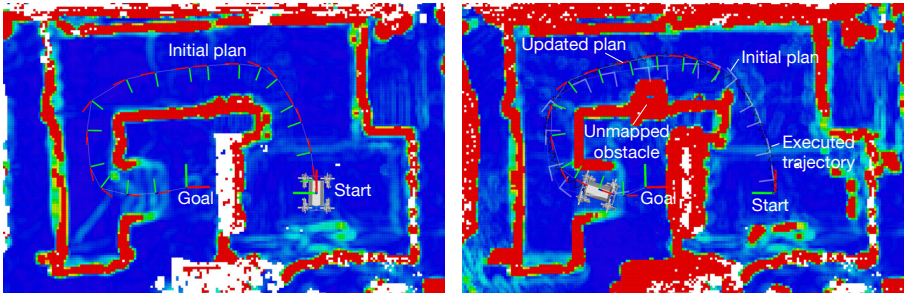


Figure 6.6 The traversability of the terrain evaluates whether a certain region is safe to walk over for the legged robot. The total traversability (a) consists of a combination of the local characteristics taking slope (b), roughness (c), and step height (d) into account.



(a) Initial plan (b) Final plan

Figure 6.7 Top-view on the traversability map and the planned paths for the walking robot. The initial path is based on the traversability map acquired from the flying robot (a). The updated path accounts for the additional obstacle (b).

We enable a simultaneous exploration and navigation of the robots by updating the shared map data between the robots in a batch-based approach. After the flying robot has transferred the first batch of the (incomplete) map, the walking robot starts to localize against the global localization map and uses the provided terrain data to plan its path and initiate execution. As the operation continues, new map data is provided by the flying robot and is merged with the existing data to extend the range in which the legged robot can operate within the global frame.

6.4.1 Localization and Mapping

As the flying robot operates, the environment is explored on an increasing scale. To maintain global consistency, we apply bundle-adjustment (Section 6.2.1) on the trajectory which was previously included in the last batch and the newly acquired trajectory data. We maintain a computationally scalable map by pruning features with the least visual information.

As the global map grows with each new batch, the legged robot can localize in more areas. Figure 6.8 shows an example of how the walking robot can localize with increased certainty as the map coverage and quality increases with each batch update A→D. We quantify the global localization quality as the number of feature correspondences between the global map and the features from the current view of the walking robot (vertical axis of Figure 6.8). The lower limit for the number of features for a (long-term) reliable localization is 4 and for performance reasons we threshold the maximum number at 25. Although some viewpoints of the walking robot contain large areas with little visual features (e.g., at time 80 s and 155 s), the localization can still be performed reliably.

6.4.2 Legged State Estimation

With each update of the global map from the flying robot, the localization of the walking robot can vary. Figure 6.9 shows a top-down view of the trajectories for the localization with the different map batches A→D. For each update, the resulting trajectory for the walking robot transits to the latest (and more accurate) localization. This means that at each update A→D the perceived robot's position can 'jump' instantaneously. We decouple these discontinuities from the control framework with the loosely coupled approach described in Section 6.2.2.

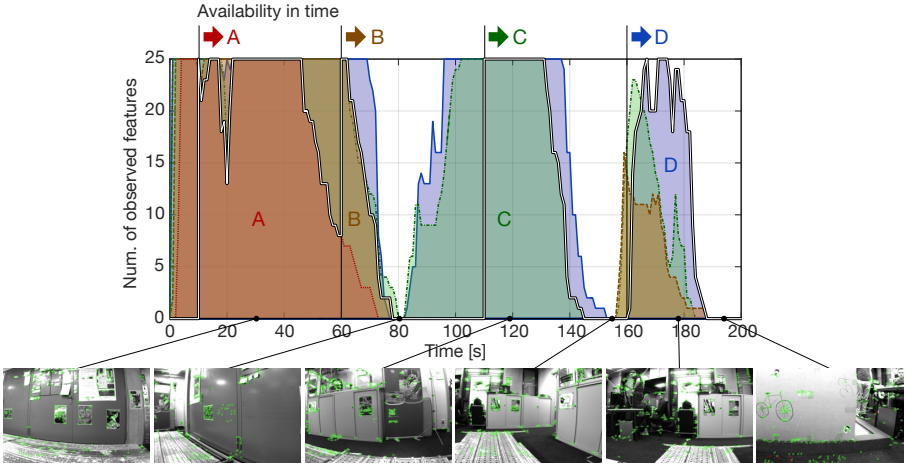


Figure 6.8 Number of observed features over time for the legged robot's camera within the global localization map. The colored areas represent the theoretical number of observable features given the localization map at four different stages (A→D) of the flying robot's exploration trajectory. After 10 s map A is provided to the robot and the number of observed features (bold black-white) coincides with the observable features of map A. The coverage of the map gets increasingly better as more data is included.

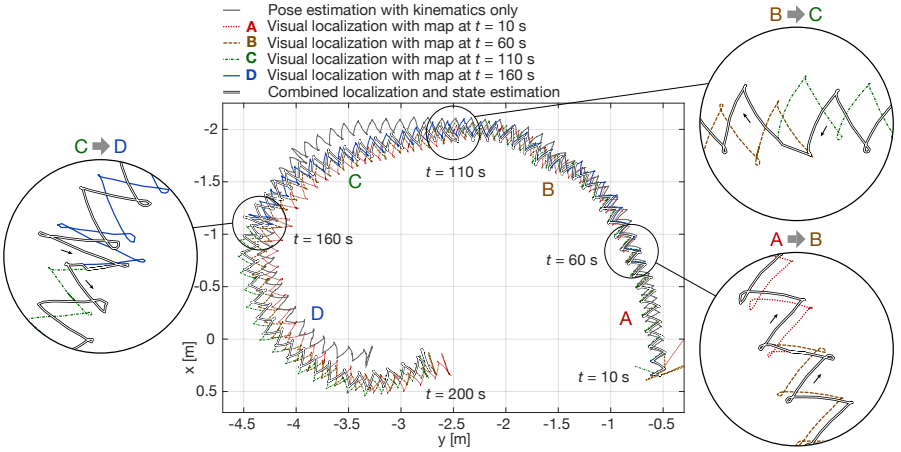


Figure 6.9 Top-down view on the localization trajectories of the legged robot for the different map updates A→D. The resulting trajectory for the walking robot transits to the latest trajectory at each map update.

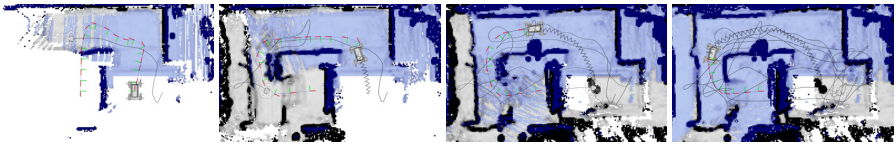


Figure 6.10 The terrain map and associated motion plans for different execution stages (A→D). From left to right the coverage of the semi-dense visual map is increasing (grey) as the hexacopter flies over the area (smooth black line). Simultaneously, the legged robot adds its perception data (laser, blue) to the elevation map. The estimated traversability (brightness of the terrain), the planned (colored coordinate frames), and the executed path (black zigzag line) of the legged robot are depicted.

6.4.3 Mapping and Navigation Planning

The walking robot can utilize the map updates from the flying robot to increase the range and precision of the path planning. At each map update, empty cells in the global terrain map are filled in where new data is available while existing cells are not changed. Where no data is provided from the flying robot and is not yet visible by the onboard sensors, our planner assumes a traversability with value 0.1. Figure 6.10 shows snapshots of the concurrent mapping and navigation planning process of the two platforms. The terrain and the walking robot's position changes slightly with each localization and terrain map update from the flying robot. These changes are taken into account in our navigation planning method (Section 6.2.5) which adapts the trajector locally to changes in the environment and replans the global path on a regular basis.

6.5 Summary

We have presented the design and methods for a collaborative localization and mapping framework for a heterogeneous team of flying and ground robots. The methods tightly integrate with each other and allow the ground robot to navigate with significantly increased performance than without the help of the flying robot. We have experimentally verified our approach with a full integration with a hexacopter and a quadrupedal robot, in which the hexacopter enables the quadrupedal robot to navigate over previously unknown, rough terrain in an environment of $\sim 9\,\mathrm{m} \times 5\,\mathrm{m}$. A summary of the experiment is given in video (V12). Our implementation demonstrates a successful functional interaction of many complex navigation tasks. We have additionally shown that our framework naturally extends to concurrent operation of the robots.

7

Conclusion & Future Directions

In this dissertation, we have studied and developed the principles of quadrupedal rough terrain locomotion with an autonomous robot. This problem lends itself to be structured in the perception and locomotion aspect, both of which are highly relevant in a broader context. Instead of focussing on isolated results for each of these areas, we have set out to approach the task in unison and regarding real applications. We encode robustness into every element of our work with the guiding principles of robust design (see P1–P3 in Section 1.3). For example, modeling the error characteristics of the employed range sensors, we can incorporate the resulting uncertainty in the representation of the map. Similarly, instead of relying on precise localization, we have formulated the terrain reconstruction from a robot-centric perspective to capture the drift from the state estimation. Another critical simplification is achieved through the task-oriented separation of motion generation and execution. This way, the result of motion planning is not merely tracked 'blindly' but instead treated as a guiding plan which is robustly executed under constant feedback of sensory signals. Finally, compliant torque-controlled joints greatly facilitate the problem of rough terrain locomotion. This was also reported by Escande, Kheddar, and Miossec (2006), Kolter and Ng (2011), and Neuhaus, Pratt, and Johnson (2011), where the authors argue that force controllable actuators become more critical as the knowledge about the terrain decreases.

With these and with other architectural choices, we were able to validate the performance of our approach in a series of experiments including the practical application for industrial inspection and search and rescue. Figure 7.1 and video (V2) demonstrate how the contributions of this thesis enable the robot ANYmal to autonomously climb stairs in rainy weather in an urban environment. In this example, the presented mapping, planning, and control approaches are shown to robustly integrate with the (water-proof) ANYmal

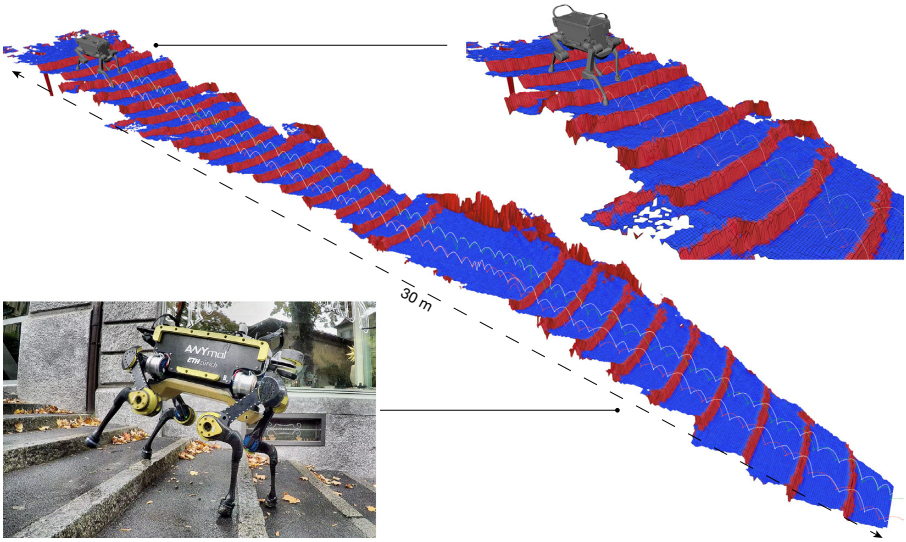


Figure 7.1 This experiment demonstrates the application of our work on mapping, planning, and control for legged locomotion in challenging terrain. The robot ANYmal is commanded to autonomously climb a set of stairs in an urban environment over a length of ∼30 m. The robot creates a terrain map with the measurements of an onboard active stereo camera and plans a motion for the footholds and trajectories to climb up the stairs safely.

platform to enable this performance. We discuss the contributions of each element of our work in the remainder of this chapter and highlight the directions for future work.

7.1 Evaluation and Modeling of Range Sensors

We have outlined the performance and non-performance criteria for the principled study of range sensors with regards to close-range mapping with a mobile robot. Having applied the evaluation to four different sensors, we identified the performance profile for each type of sensor technology. Sensor setups with very different specifications were made comparable with the introduction of the scan flux and the presented model to estimate the density of the resulting terrain map. With this, our contribution lays the basis for the evaluation and comparison of new sensor devices and new measurement technologies. For the Kinect v2 time-of-flight (TOF) sensor, we have analyzed its performance for aspects, such as short-range performance and ambient light resilience, that are important to our application. Additionally, the derived an empirical noise

model of the sensor serves as a prerequisite for statistical environment mapping.

In the future, we will continue integrating new range sensing technologies and devices on our robots. Thanks to the technological push in unmanned aerial vehicles (UAVs) autonomous driving, commercial interest will continue to spark new developments towards more capable, smaller, and cheaper sensors. As we want to operate the system in increasingly harsher conditions, the evaluation will be expanded to include the sensors' resilience to rain, smoke, and fog. We are also investigating the use of more sensors around the robot to widen the perception to all sides. With the increasing numbers of sensors on a robot and with the general more common use of robots, the time-synchronization and online self-calibration of the sensors will have to be addressed (Maye, 2014).

7.2 Terrain Mapping

We apply standard grid-based elevation mapping for terrain modeling. However, instead of relying on an accurate localization method, we derived an approach in which errors from a drifting state estimation are propagated to the map data. We have shown that our method allows the robot to robustly perceive its environment while expressing the belief of the terrain profile with a lower and upper confidence bounds. Due to the simplicity of the grid format, further processing of the data for surface normal estimation, foothold selection, and collision checking is highly efficient. We made our mapping software available open-source (S2) along with the underlying grid map library (S1). These were used by our and other teams for various applications such as navigation planning (Kohlbrecher and Stryk, 2016; Primatesta, Rizzoa, and La Cour-Harbo, 2018; Wermelinger et al., 2016), autonomous excavation (Jud et al., 2017), landing spot detection for UAVs (Forster et al., 2015), co-localization through elevation maps (Käslin et al., 2016), aerial mapping (Delmerico et al., 2017; Hinzmann et al., 2017), coverage planning (Jia et al., 2016), swarm robotics (Kumar et al., 2017), and road segmentation for autonomous vehicles (Lyu, Bai, and Huang, 2018).

For the future, we are planning to port our terrain mapping software to an implementation running on a graphics processing unit (GPU). This step will help to increase the processing speed even if more data from multiple sensors is processed. It would also be valuable to extend our robot-centric formulation to a full 3D map representation, such as a signed distance field (SDF) (e.g. Oleynikova et al., 2017). This would allow for motion planning that includes

overhanging obstacles, such as crouching into a tunnel. One of the main limitations of our approach is the missing meta-level understanding of the scene. For example, when walking through undergrowth or down steep stairs, one needs to understand the context to choose footholds that might not be visible from the current standpoint. This leaves many opportunities for future work to develop the capability to infer the support surface from models, experience, or learning.

7.3 Legged Motion Control

In our work on the motion control of legged robots, we demonstrated the importance of the well-designed separation between motion generation and motion execution. Our Free Gait control framework provides an interface for the flexible and task-orientation formulation of whole-body motion plans. The implementation ensures accurate tracking in the coordinate frame essential to the successful execution of the task. Automatic contact handling achieves robust switching of contact situations. This approach has allowed us to implement various motion tasks for different projects in a straight-forward way. These range from templated motions defined as motion scripts to whole-body motion planners working in a receding horizon fashion. With this, Free Gait has become a core element of our software framework.

Current work focusses on the implementation of a whole-body controller based on hierarchical optimization (Bellicoso et al., 2016; Bellicoso et al., 2017) for the motion execution to improve tracking of dynamic motions. Next iterations of Free Gait are focussed on both the efficient and user-friendly work with the robot as well as the tight integration with planners for complex and dynamic motions. For example, we have started the work on a versatile robot remote control user interface along the lines of Fallon et al. (2015), Johnson et al. (2014), and Johnson et al. (2015) in which the Free Gait preview plays a significant role. Other work involves the extension of footstep target positions to target areas such that reactive re-stepping to safe areas under disturbances is possible. Although we have only demonstrated Free Gait with quadrupedal systems, the architecture is built for any number of legs. We also foresee extending Free Gait for the use in robotic manipulation involving gripping tasks. While future iterations of the software are required to cover more robot morphologies and applications, the underlying principles of the architecture will continue to help in the implementation of versatile control of legged robots.

7.4 Legged Locomotion Planning

In this thesis, we have presented a locomotion planner for quadrupedal rough terrain locomotion. With the design of a tailored architecture, we have achieved efficient and robust locomotion based on online mapping for a fully self-contained robot. A core element of the framework is the proposed pose optimization which efficiently finds robot poses that satisfy the stability and kinematic constraints. We have demonstrated in several experiments that the continuous re-planning strategy is well suited to adapt to disturbances and changing environments. By including the terrain surface normals from the map, we additionally exploit the torque-controllability of our robots to generate feasible contact forces on inclined surfaces that prevent slippage.

We are working on further improving the performance of our locomotion planner in several areas. Our goal is to increase the locomotion speed of ANYmal by combining the presented pose optimization with the dynamic base trajectory generation based on the zero-moment point (ZMP) stability criterion as shown in our group's previous work (Bellicoso et al., 2017; Bellicoso et al., 2018a). To increase the height at which obstacles can be negotiated (>50 % of leg length), we will extend the foothold selection to a multi-step horizon. This extension will come along with more extensive collision checking and hardware changes with increased joint torques. In a longer term, more inclusive approaches such as non-linear trajectory optimization (Farshidian et al., 2017; Gehring et al., 2017; Mordatch, Todorov, and Popović, 2012; Neunert et al., 2018; Posa and Tedrake, 2013; Winkler et al., 2017) and reinforcement learning with deep neural networks (Heess et al., 2017; Peng, Berseth, and Panne, 2016; Peng et al., 2017) are considered. These methods have shown to discover complex locomotion behaviors, and we will focus on transferring these skills to robots in realistic applications. One of the main restrictions of the presented approach is that it relies on a geometrical representation of the terrain. This makes it difficult for the robot to judge about the physical terrain properties, such as viscosity, stiffness, and friction. In the future, other sensing modalities such as the visible and near-infrared light spectrum could be used to assess the terrain.

7.5 Collaborative Navigation

Bringing our work on rough terrain locomotion into a broader context, we presented a framework for the collaborative navigation of flying and ground robots. The team of heterogeneous robots has the advantage that flying robots

can map large areas quickly and help ground robots with a higher payload capacity to efficiently reach the target. This task poses many open research questions and our work has shown a first successful implementation at this level of complexity.

Further research and engineering is required to bring this approach from a demonstration to a practical approach. In current work, the goal is to extend the presented localization method to an extended team of robots (e.g., Cieslewski et al., 2015; Schneider et al., 2018), but reliable wireless communication is still a critical issue. In a next step, the data flow should be targeted in both directions including the exchange from the ground to the flying robot, for example, to pick a safe landing post. Eventually, the legged robot could also be equipped with a landing platform, where the flying robot could rest and charge until its action is required.

7.6 Outlook

This thesis addresses a fundamental part for legged robots to start solving real-world problems. However, many more aspects need to be considered before a wide-spread practical application becomes a reality. Imagine the situation when a robot is required to reach the upper deck on an offshore platform by climbing a steep and slippery industrial stair during the exposure to heavy winds and rainfall. Even in this situations, a high success rate has to be achieved. To reach this level of performance, several technical challenges need to be solved, such as the robust and accurate self-localization, damage-free falling and self-righting, resilience against extreme environmental conditions, and many more. Also, several additional challenges have to be considered with the actual application task of the robot. Furthermore, a commercial solution will require certification (such as *ATEX* for explosive environments) and very likely a cost optimization.

While these are difficult engineering problems, we believe that the robotics community will solve them to a satisfying extent within the next few years. A strong indication for the technical feasibility and commercial interest is that several companies such as *Agility Robotics*, *Boston Dynamics*, *Motiv Robotics*, *PAL Robotics*, and *Unitree Robotics* have recently targeted to provide legged robots to the market. At the company *ANYbotics*, we are currently working with partner companies on test installations of ANYmal for several different applications. To this end, the robot carries different payload modules which include RGB zoom and thermal cameras, microphones, and gas detection sensors. For example, in tight underground tunnels built for conveyor belts or in

sewage systems, an autonomous robot can perform a visual and acoustic inspection to check for regular operation and to signalize any irregularities such as cracks, water ingress, or accumulation of dangerous gasses. Because these tunnels are often filled with debris, and significant steps have to be overcome, legged robots provide a unique advantage over wheeled- and tracked systems. In urban environments, a patrolling robot can survey buildings and check for safety-relevant aspects such as windows, doors, unexpected heat sources (e.g., indication for potential cable fire) and objects. With a legged robot, a wide range of environments including staircases can be covered. In the energy and process industry, regular maintenance of machinery and infrastructure is efficiency and safety-relevant. An autonomous legged robot can perform reliable inspection in an environment made for humans without requiring any changes to the infrastructure. Our robot ANYmal will be in the next months foreseeably the first autonomous robot operating on an industrial offshore site.

A

Appendix

A.1 Computation of Relative Position Uncertainties

Localizations methods typically specify the uncertainty of the robot's pose B relative to an inertial frame I as

$$\boldsymbol{x}_{IB} = ({}_I\boldsymbol{r}_{IB}, \Phi_{IB}) \;, \quad \boldsymbol{\Sigma}_{IB}^{6\times 6} = \mathrm{cov}(\boldsymbol{x}_{IB}) \;. \tag{A.1}$$

However, in our formulation of the robot motion update process (Section 3.2.3), the uncertainty of the pose is expressed relative from a pose at time k to time $k+1$ as

$$\boldsymbol{x}_{\tilde{B}\tilde{B}} = ({}_{\tilde{B}_{k+1}}\boldsymbol{r}_{\tilde{B}_k\tilde{B}_{k+1}}, \psi_{\tilde{B}_{k+1}\tilde{B}_k}) \;, \quad \boldsymbol{\Sigma}_{\tilde{B}\tilde{B}}^{4\times 4} = \mathrm{cov}(\boldsymbol{x}_{\tilde{B}\tilde{B}}) \;. \tag{A.2}$$

In the following, we derive the relative covariance $\boldsymbol{\Sigma}_{\tilde{B}\tilde{B}}$ for the case that the absolute covariance $\boldsymbol{\Sigma}_{IB}$ is available.

A.1.1 State Reduction

In a first step, we need to reduce the state representation of the covariance matrix from the full robot pose B with state vector $\boldsymbol{x}_{IB} = (\boldsymbol{r}_{IB}, \Phi_{IB})$ to the z-aligned robot pose $\tilde{B}$ with state vector $\boldsymbol{x}_{I\tilde{B}} = (\boldsymbol{r}_{IB}, \psi)$ with $\boldsymbol{r}_{IB} = \boldsymbol{r}_{I\tilde{B}}$ and $\psi = \psi_{I\tilde{B}} = \psi_{IB}$. We can achieve this with

$$\boldsymbol{\Sigma}_{I\tilde{B}}^{4\times 4} = \boldsymbol{J}\boldsymbol{\Sigma}_{IB}^{6\times 6}\boldsymbol{J}^{\mathsf{T}} \tag{A.3}$$

$$\text{with} \quad \boldsymbol{J} = \begin{bmatrix} \boldsymbol{I}^{3\times 3} & \boldsymbol{0}^{3\times 3} \\ \boldsymbol{0}^{1\times 3} & \frac{\partial\psi}{\partial\Phi} \end{bmatrix} \tag{A.4}$$

$$\text{and} \quad \frac{\partial\psi}{\partial\Phi_{IB}(\varphi,\theta,\psi)} = \begin{pmatrix} \frac{\cos\psi\sin\theta}{\cos\theta} & \frac{\sin\theta\sin\psi}{\cos\theta} & 1 \end{pmatrix} \;. \tag{A.5}$$

A.1.2 Covariance Transformation

We assume that the state estimate can be modeled as Gaussian random walk, which is the case for example for odometry-based localization methods:

$$\boldsymbol{x}_{I\tilde{B},k+1} = \begin{pmatrix} {}_I\boldsymbol{r}_{I\tilde{B},k+1} \\ \psi_{k+1} \end{pmatrix} = \begin{pmatrix} {}_I\boldsymbol{r}_{I\tilde{B},k} \\ \psi_k \end{pmatrix} + \begin{bmatrix} \boldsymbol{C}_{I\tilde{B}}(\psi_k) & \boldsymbol{0} \\ \boldsymbol{0} & 1 \end{bmatrix} \begin{pmatrix} {}_{\tilde{B}}\boldsymbol{v}_k + \boldsymbol{n}_{v,k} \\ \omega_k + n_{\omega,k} \end{pmatrix} \Delta t \tag{A.6}$$

with translational velocity of the robot $\boldsymbol{v}$ and rotational velocity ω around the ${}_I\boldsymbol{e}_z = {}_{\tilde{B}}\boldsymbol{e}_z$ axis with zero mean multivariate Gaussian noise vector $\boldsymbol{n} = (\boldsymbol{n}_v, n_\omega)^\mathsf{T}$. The covariance propagation can be written as

$$\boldsymbol{\Sigma}_{I\tilde{B},k+1} = \boldsymbol{F}\boldsymbol{\Sigma}_{I\tilde{B},k}\boldsymbol{F}^\mathsf{T} + \boldsymbol{G}\boldsymbol{Q}_{k+1}\boldsymbol{G}^\mathsf{T} \ , \tag{A.7}$$

with

$$\boldsymbol{F} = \frac{\partial \boldsymbol{x}_{I\tilde{B},k+1}}{\partial \boldsymbol{x}_{I\tilde{B},k}} = \begin{bmatrix} \boldsymbol{I}^{3\times 3} & \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}^\times \boldsymbol{C}_{I\tilde{B}}(\psi_k)\,{}_{\tilde{B}}\boldsymbol{v}_k\Delta t \\ \boldsymbol{0}^{1\times 3} & 1 \end{bmatrix} , \tag{A.8}$$

$$\boldsymbol{G} = \frac{\partial \boldsymbol{x}_{I\tilde{B},k+1}}{\partial \boldsymbol{n}_k} = \begin{bmatrix} \boldsymbol{C}_{I\tilde{B}}(\psi_k)\Delta t & \boldsymbol{0}^{3\times 1} \\ \boldsymbol{0}^{1\times 3} & \Delta t \end{bmatrix} . \tag{A.9}$$

The relative motion of robot from time k to $k+1$ can be expressed in the $\tilde{B}$ frame as

$$\begin{aligned} \boldsymbol{x}_{\tilde{B}\tilde{B}} &= \begin{pmatrix} {}_{\tilde{B}_{k+1}}\boldsymbol{r}_{\tilde{B}_k\tilde{B}_{k+1}} \\ \psi_{\tilde{B}_{k+1}\tilde{B}_k} \end{pmatrix} \\ &= \begin{pmatrix} \boldsymbol{C}_{\tilde{B}I}(\psi_k)({}_I\boldsymbol{r}_{I\tilde{B},k+1} - {}_I\boldsymbol{r}_{I\tilde{B},k}) \\ \psi_{k+1} - \psi_k \end{pmatrix} \\ \text{with (A.6)} \qquad &= \begin{pmatrix} {}_{\tilde{B}}\boldsymbol{v}_k + \boldsymbol{n}_{v,k} \\ \omega_k + n_{\omega,k} \end{pmatrix} \Delta t \ . \end{aligned} \tag{A.10}$$

With this we can express $\Sigma_{\tilde{B}\tilde{B}}$ as

$$\boldsymbol{\Sigma}_{\tilde{B}\tilde{B}} = \mathrm{cov}(\boldsymbol{x}_{\tilde{B}\tilde{B}}) = \boldsymbol{Q}_{k+1}\Delta t^2 \ . \tag{A.11}$$

With (A.7) we get

$$\boldsymbol{Q}_{k+1} = \boldsymbol{G}^{-1}(\boldsymbol{\Sigma}_{I\tilde{B},k+1} - \boldsymbol{F}\boldsymbol{\Sigma}_{I\tilde{B},k}\boldsymbol{F}^\mathsf{T})(\boldsymbol{G}^\mathsf{T})^{-1} \tag{A.12}$$

and finally

$$\boldsymbol{\Sigma}_{\tilde{B}\tilde{B}} = \boldsymbol{G}^{-1}(\boldsymbol{\Sigma}_{I\tilde{B},k+1} - \boldsymbol{F}\boldsymbol{\Sigma}_{I\tilde{B},k}\boldsymbol{F}^\mathsf{T})(\boldsymbol{G}^\mathsf{T})^{-1}\Delta t^2 \tag{A.13}$$

with

$$\boldsymbol{G}^{-1} = \begin{bmatrix} \frac{\boldsymbol{C}_{I\tilde{B}}(\psi_k)^\mathsf{T}}{\Delta t} & \boldsymbol{0}^{3\times 1} \\ \boldsymbol{0}^{1\times 3} & \frac{1}{\Delta t} \end{bmatrix} . \tag{A.14}$$

This can be simplified assuming the property $\boldsymbol{v}_k \Delta t = \boldsymbol{r}_{I\tilde{B},k+1} - \boldsymbol{r}_{I\tilde{B},k}$ to

$$\boldsymbol{\Sigma}_{\tilde{B}\tilde{B}} = \hat{\boldsymbol{G}}(\boldsymbol{\Sigma}_{I\tilde{B},k+1} - \hat{\boldsymbol{F}}\boldsymbol{\Sigma}_{I\tilde{B},k}\hat{\boldsymbol{F}}^{\mathsf{T}})\hat{\boldsymbol{G}}^{\mathsf{T}} \tag{A.15}$$

with

$$\hat{\boldsymbol{F}} = \begin{bmatrix} \boldsymbol{I}^{3\times 3} & \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}^{\times} \boldsymbol{C}_{I\tilde{B}}(\psi_k)(\boldsymbol{r}_{I\tilde{B},k+1} - \boldsymbol{r}_{I\tilde{B},k}) \\ \boldsymbol{0}^{1\times 3} & 1 \end{bmatrix}, \tag{A.16}$$

$$\hat{\boldsymbol{G}} = \begin{bmatrix} \boldsymbol{C}_{I\tilde{B}}(\psi_k)^{\mathsf{T}} & \boldsymbol{0}^{3\times 1} \\ \boldsymbol{0}^{1\times 3} & 1 \end{bmatrix}. \tag{A.17}$$

For this to be used in (3.19), we need to get $\boldsymbol{\Sigma}_r$ and $\boldsymbol{\Sigma}_R$ from $\boldsymbol{\Sigma}_{\tilde{B}\tilde{B}}$. From the definition (A.2), $\boldsymbol{\Sigma}_r$ can be read directly from

$$\boldsymbol{\Sigma}_{\tilde{B}\tilde{B}} = \begin{bmatrix} \boldsymbol{\Sigma}_r & 0 \\ \boldsymbol{0} & \sigma^2_{\psi_{\tilde{B}_{k+1}\tilde{B}_k}} \end{bmatrix}. \tag{A.18}$$

The expression for $\boldsymbol{\Sigma}_R$ can be retrieved as

$$\boldsymbol{\Sigma}_R = \boldsymbol{J}_\psi \sigma^2_{\psi_{\tilde{B}_{k+1}\tilde{B}_k}} \boldsymbol{J}_\psi^{\mathsf{T}} \tag{A.19}$$

with

$$\boldsymbol{J}_\psi = \frac{\partial \Phi(\varphi, \theta, \psi)}{\partial \psi} = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}. \tag{A.20}$$

A.2 Pose Optimization

The pose optimization cost function (5.2) can be rewritten as

$$\begin{aligned} f(\boldsymbol{x}) = &\sum_{i=0}^{N} \left\| {}_I\boldsymbol{r}_{IB} + \boldsymbol{\Phi}_{IB}({}_B\hat{\boldsymbol{r}}_{BF_i}) - {}_I\boldsymbol{r}_{F_i} \right\|_2^2 \\ &+ w_{\mathrm{COM}} \left\| {}_I\bar{\boldsymbol{r}}_{\mathrm{SP}} - \left(\overline{{}_I\boldsymbol{r}_{IB} + \boldsymbol{\Phi}_{IB}({}_B\boldsymbol{r}_{\mathrm{COM}})} \right) \right\|_2^2 . \end{aligned} \tag{A.21}$$

Multiplying out and using $\boldsymbol{\Phi}_{AB}^{-1} \circ \boldsymbol{\Phi}_{AB} = \boldsymbol{\Phi}_I$ with identity element $\boldsymbol{\Phi}_I$:

$$\begin{aligned} f(\boldsymbol{x}) = &\sum_{i=0}^{N} \Big({}_I\boldsymbol{r}_{IB}{}^2 + {}_B\hat{\boldsymbol{r}}_{BF_i}^2 + {}_I\boldsymbol{r}_{F_i}{}^2 + 2{}_I\boldsymbol{r}_{IB}^{\mathsf{T}}\boldsymbol{\Phi}_{IB}({}_B\hat{\boldsymbol{r}}_{BF_i}) \\ &\quad -2\boldsymbol{\Phi}_{IB}({}_B\hat{\boldsymbol{r}}_{BF_i})^{\mathsf{T}}{}_I\boldsymbol{r}_{F_i} - 2{}_I\boldsymbol{r}_{F_i}^{\mathsf{T}}{}_I\boldsymbol{r}_{IB} \Big) \\ &+ w_{\mathrm{COM}} \left({}_I\bar{\boldsymbol{r}}_{\mathrm{SP}}^2 + {}_I\bar{\boldsymbol{r}}_{IB}^2 + {}_B\bar{\boldsymbol{r}}_{\mathrm{COM}}^2 - 2{}_I\bar{\boldsymbol{r}}_{\mathrm{SP}}^{\mathsf{T}}{}_I\bar{\boldsymbol{r}}_{IB} \right. \\ &\quad \left. +2{}_I\bar{\boldsymbol{r}}_{IB}^{\mathsf{T}}\overline{\boldsymbol{\Phi}_{IB}({}_B\boldsymbol{r}_{\mathrm{COM}})} - 2\overline{\boldsymbol{\Phi}_{IB}({}_B\boldsymbol{r}_{\mathrm{COM}})}^{\mathsf{T}}{}_I\bar{\boldsymbol{r}}_{\mathrm{SP}} \right) . \end{aligned} \tag{A.22}$$

A.2.1 Gradient

With help of (A.34) (derivation shown in Section A.3), the gradient of (A.22) can be computed as

$$\begin{aligned} \left(\frac{\partial f(\boldsymbol{x})}{\partial \boldsymbol{x}} \right)^{\mathsf{T}} = &2\sum_{i=0}^{N} \begin{pmatrix} {}_I\boldsymbol{r}_{IB} + \boldsymbol{\Phi}_{IB}({}_B\hat{\boldsymbol{r}}_{BF_i}) - {}_I\boldsymbol{r}_{F_i} \\ \boldsymbol{\Phi}_{IB}({}_B\hat{\boldsymbol{r}}_{BF_i})^{\times}{}_I\boldsymbol{r}_{IB} - \boldsymbol{\Phi}_{IB}({}_B\hat{\boldsymbol{r}}_{BF_i})^{\times}{}_I\boldsymbol{r}_{F_i} \end{pmatrix} \\ &+ 2w_{\mathrm{COM}} \begin{pmatrix} {}_I\bar{\boldsymbol{r}}_{IB} - {}_I\bar{\boldsymbol{r}}_{\mathrm{SP}} + \overline{\boldsymbol{\Phi}_{IB}({}_B\boldsymbol{r}_{\mathrm{COM}})} \\ \overline{\boldsymbol{\Phi}_{IB}({}_B\boldsymbol{r}_{\mathrm{COM}})}^{\times}{}_I\bar{\boldsymbol{r}}_{IB} - \overline{\boldsymbol{\Phi}_{IB}({}_B\boldsymbol{r}_{\mathrm{COM}})}^{\times}{}_I\bar{\boldsymbol{r}}_{\mathrm{SP}} \end{pmatrix} . \end{aligned} \tag{A.23}$$

A.2.2 Hessian

Similarly, the Hessian of (A.22) is computed with (A.43) as

$$\begin{aligned} \frac{\partial^2 f(\boldsymbol{x})}{\partial \boldsymbol{x}^2} = &2\sum_{i=0}^{N} \begin{bmatrix} \boldsymbol{I} & -\boldsymbol{\Phi}_{IB}({}_B\hat{\boldsymbol{r}}_{BF_i})^{\times} \\ \boldsymbol{\Phi}_{IB}({}_B\hat{\boldsymbol{r}}_{BF_i})^{\times} & \boldsymbol{D}_i \end{bmatrix} \\ &+ 2w_{\mathrm{COM}} \begin{bmatrix} \bar{\boldsymbol{I}} & -\overline{\boldsymbol{\Phi}_{IB}({}_B\boldsymbol{r}_{\mathrm{COM}})}^{\times} \\ \overline{\boldsymbol{\Phi}_{IB}({}_B\boldsymbol{r}_{\mathrm{COM}})}^{\times} & \boldsymbol{E} \end{bmatrix} \end{aligned} \tag{A.24}$$

with

$$\begin{aligned}\boldsymbol{D}_i = &\frac{1}{2}\left({}_I\boldsymbol{r}_{IB}^{\times}\boldsymbol{\Phi}_{IB}({}_B\hat{\boldsymbol{r}}_{BF_i})^{\times} + \boldsymbol{\Phi}_{IB}({}_B\hat{\boldsymbol{r}}_{BF_i})^{\times}{}_I\boldsymbol{r}_{IB}^{\times}\right.\\ &\left. - {}_I\boldsymbol{r}_{F_i}^{\times}\boldsymbol{\Phi}_{IB}({}_B\hat{\boldsymbol{r}}_{BF_i})^{\times} - \boldsymbol{\Phi}_{IB}({}_B\hat{\boldsymbol{r}}_{BF_i})^{\times}{}_I\boldsymbol{r}_{F_i}^{\times}\right) ,\end{aligned} \tag{A.25}$$

$$\begin{aligned}\boldsymbol{E} = &\frac{1}{2}\left({}_I\bar{\boldsymbol{r}}_{IB}^{\times}\overline{\boldsymbol{\Phi}_{IB}({}_B\boldsymbol{r}_{\mathrm{COM}})}^{\times} + \overline{\boldsymbol{\Phi}_{IB}({}_B\boldsymbol{r}_{\mathrm{COM}})}^{\times}{}_I\bar{\boldsymbol{r}}_{IB}^{\times}\right.\\ &\left. - {}_I\bar{\boldsymbol{r}}_{\mathrm{SP}}^{\times}\overline{\boldsymbol{\Phi}_{IB}({}_B\boldsymbol{r}_{\mathrm{COM}})}^{\times} - \overline{\boldsymbol{\Phi}_{IB}({}_B\boldsymbol{r}_{\mathrm{COM}})}^{\times}{}_I\bar{\boldsymbol{r}}_{\mathrm{SP}}^{\times}\right) .\end{aligned} \tag{A.26}$$

A.2.3 Inequality Constraints

The constraints (5.3) and (5.4) can be rewritten as

$$\boldsymbol{A}_{\mathrm{SP}}\left({}_I\boldsymbol{r}_{IB} + \boldsymbol{\Phi}_{IB}({}_B\boldsymbol{r}_{\mathrm{COM}})\right) \leq \boldsymbol{b}_{\mathrm{SP}} \tag{A.27}$$

$$l_{\min,i} \leq \left\|{}_I\boldsymbol{r}_{IB} + \boldsymbol{\Phi}_{IB}({}_B\boldsymbol{r}_{BH_i}) - {}_I\boldsymbol{r}_{F_i}\right\|_2 \leq l_{\max,i} \quad \forall i \in N . \tag{A.28}$$

For the SQP optimization, the first derivatives of the constraints are derived as

$$\frac{\partial g(\boldsymbol{x})}{\partial \boldsymbol{x}} = \begin{bmatrix} \boldsymbol{A}_{\mathrm{SP}} & -\boldsymbol{A}_{\mathrm{SP}}\boldsymbol{\Phi}_{IB}({}_B\boldsymbol{r}_{\mathrm{COM}})^{\times} \\ \boldsymbol{g}_i^{\mathsf{T}} & -\boldsymbol{g}_i^{\mathsf{T}}\boldsymbol{\Phi}_{IB}({}_B\boldsymbol{r}_{BH_i})^{\times} \\ \vdots & \vdots \\ \boldsymbol{g}_N^{\mathsf{T}} & -\boldsymbol{g}_N^{\mathsf{T}}\boldsymbol{\Phi}_{IB}({}_B\boldsymbol{r}_{BH_N})^{\times} \end{bmatrix} \tag{A.29}$$

with

$$\boldsymbol{g}_i = \frac{{}_I\boldsymbol{r}_{IB} + \boldsymbol{\Phi}_{IB}({}_B\boldsymbol{r}_{BH_i}) - {}_I\boldsymbol{r}_{F_i}}{\left\|{}_I\boldsymbol{r}_{IB} + \boldsymbol{\Phi}_{IB}({}_B\boldsymbol{r}_{BH_i}) - {}_I\boldsymbol{r}_{F_i}\right\|_2} . \tag{A.30}$$

A.3 $\Phi(\boldsymbol{a})^\mathsf{T}\boldsymbol{b}$ Derivates

We are looking for the first and second partial derivative of $f(\Phi) = \Phi(\boldsymbol{a})^\mathsf{T}\boldsymbol{b}$ with respect to the orientation Φ. We can write Φ as sum of the basic orientation $\bar{\Phi} \in SO(3)$ and the difference of orientation $\boldsymbol{\phi} \in \mathbb{R}^3$ as

$$\Phi = \bar{\Phi} \boxplus \boldsymbol{\phi} \,. \tag{A.31}$$

With (14) from Bloesch et al. (2016):

$$f(\Phi) = \Phi(\boldsymbol{a})^\mathsf{T}\boldsymbol{b} = \left((\bar{\Phi} \boxplus \boldsymbol{\phi})(\boldsymbol{a})\right)^\mathsf{T} \boldsymbol{b} = \left((\exp(\boldsymbol{\phi}) \circ \bar{\Phi})(\boldsymbol{a})\right)^\mathsf{T} \boldsymbol{b} \overset{\phi=0}{=} \bar{\Phi}(\boldsymbol{a})^\mathsf{T}\boldsymbol{b} \,. \tag{A.32}$$

A.3.1 Gradient

Using the chain rule with and (27) and (31) from Bloesch et al. (2016) yields

$$\frac{\partial \Phi(\boldsymbol{a})^\mathsf{T}\boldsymbol{b}}{\partial \boldsymbol{\phi}} = -\boldsymbol{b}^\mathsf{T} \left((\exp(\boldsymbol{\phi}) \circ \bar{\Phi})(\boldsymbol{a})\right)^\times \boldsymbol{\Gamma}(\boldsymbol{\phi}) \overset{\phi=0}{=} -\boldsymbol{b}^\mathsf{T}\bar{\Phi}(\boldsymbol{a})^\times \,, \tag{A.33}$$

which gives us the row vector for the gradient as

$$\frac{\partial f(\Phi)}{\partial \Phi} = \frac{\partial \Phi(\boldsymbol{a})^\mathsf{T}\boldsymbol{b}}{\partial \Phi} = -\boldsymbol{b}^\mathsf{T}\Phi(\boldsymbol{a})^\times \,. \tag{A.34}$$

A.3.2 Hessian

We can express the Hessian as derivative of the gradient w.r.t. Φ as

$$\frac{\partial^2 f}{\partial \Phi^2} = \frac{\partial \left(\frac{\partial f}{\partial \Phi}\right)^\mathsf{T}}{\partial \Phi} \,. \tag{A.35}$$

With $\boldsymbol{a}^\times\boldsymbol{b} = -\boldsymbol{b}^\times\boldsymbol{a}$ and (1) from Bloesch et al. (2016), we get from (A.33)

$$\left(\frac{\partial \Phi(\boldsymbol{a})^\mathsf{T}\boldsymbol{b}}{\partial \boldsymbol{\phi}}\right)^\mathsf{T} = \boldsymbol{\Gamma}(\boldsymbol{\phi})^\mathsf{T}\boldsymbol{b}^\times(\exp(\boldsymbol{\phi}) \circ \bar{\Phi})(\boldsymbol{a}) \,, \tag{A.36}$$

which can be approximated with a Taylor expansion ((34) and (22) from Bloesch et al. (2016)) as

$$\left(\frac{\partial \Phi(\boldsymbol{a})^\mathsf{T}\boldsymbol{b}}{\partial \boldsymbol{\phi}}\right)^\mathsf{T} \approx \left(\boldsymbol{I} - \frac{1}{2}\boldsymbol{\phi}^\times\right) \boldsymbol{b}^\times(\boldsymbol{I} + \boldsymbol{\phi}^\times)\boldsymbol{C}(\bar{\Phi})\boldsymbol{a} \,, \quad \|\boldsymbol{\phi}\| \approx 0 \,. \tag{A.37}$$

Applying the product rule and $\boldsymbol{a}^\times \boldsymbol{b} = -\boldsymbol{b}^\times \boldsymbol{a}$ yields

$$\begin{aligned}\frac{\partial \left(\frac{\partial \Phi(\boldsymbol{a})^\mathsf{T}\boldsymbol{b}}{\partial \boldsymbol{\phi}}\right)^\mathsf{T}}{\partial \boldsymbol{\phi}} &= \frac{\partial}{\partial \boldsymbol{\phi}_1}\left(-\frac{1}{2}\boldsymbol{\phi}_1^\times \boldsymbol{b}^\times (\boldsymbol{I} + \boldsymbol{\phi}_2^\times)\boldsymbol{C}(\bar{\Phi})\boldsymbol{a}\right) \\ &\quad + \frac{\partial}{\partial \boldsymbol{\phi}_2}\left(\left(\boldsymbol{I} - \frac{1}{2}\boldsymbol{\phi}_1^\times\right)\boldsymbol{b}^\times \boldsymbol{\phi}_2^\times \boldsymbol{C}(\bar{\Phi})\boldsymbol{a}\right) \qquad \text{(A.38)} \\ &= \frac{1}{2}\left(\boldsymbol{b}^\times (\boldsymbol{I} + \boldsymbol{\phi}^\times)\boldsymbol{C}(\Phi)\boldsymbol{a}\right)^\times - \left(\boldsymbol{I} - \frac{1}{2}\boldsymbol{\phi}^\times\right)\boldsymbol{b}^\times \left(\boldsymbol{C}(\Phi)\boldsymbol{a}\right)^\times , \qquad \text{(A.39)}\end{aligned}$$

and applying $(\boldsymbol{a} \times \boldsymbol{b})^\times = \boldsymbol{a}^\times \boldsymbol{b}^\times - \boldsymbol{b}^\times \boldsymbol{a}^\times$ yields

$$\begin{aligned}\frac{\partial \left(\frac{\partial \Phi(\boldsymbol{a})^\mathsf{T}\boldsymbol{b}}{\partial \boldsymbol{\phi}}\right)^\mathsf{T}}{\partial \boldsymbol{\phi}} &\overset{\boldsymbol{\phi}=0}{=} \frac{1}{2}\left(\boldsymbol{b}^\times \boldsymbol{C}(\Phi)\boldsymbol{a}\right)^\times - \boldsymbol{b}^\times \left(\boldsymbol{C}(\Phi)\boldsymbol{a}\right)^\times \qquad \text{(A.40)} \\ &\overset{\boldsymbol{\phi}=0}{=} \frac{1}{2}\left(\boldsymbol{b}^\times (\boldsymbol{C}(\Phi)\boldsymbol{a})^\times - (\boldsymbol{C}(\Phi)\boldsymbol{a})^\times \boldsymbol{b}^\times\right) - \boldsymbol{b}^\times \left(\boldsymbol{C}(\Phi)\boldsymbol{a}\right)^\times \qquad \text{(A.41)} \\ &\overset{\boldsymbol{\phi}=0}{=} -\frac{1}{2}\left(\boldsymbol{b}^\times (\boldsymbol{C}(\Phi)\boldsymbol{a})^\times + (\boldsymbol{C}(\Phi)\boldsymbol{a})^\times \boldsymbol{b}^\times\right) . \qquad \text{(A.42)}\end{aligned}$$

Finally, the Hessian is

$$\frac{\partial^2 f}{\partial \Phi^2} = \frac{\partial^2 \Phi(\boldsymbol{a})^\mathsf{T}\boldsymbol{b}}{\partial \Phi^2} = -\frac{1}{2}\left(\boldsymbol{b}^\times \Phi(\boldsymbol{a})^\times + \Phi(\boldsymbol{a})^\times \boldsymbol{b}^\times\right) . \qquad \text{(A.43)}$$

List of Figures

1.1 ANYmal demonstration at the World Petroleum Congress . . . 2
1.2 Challenges of legged rough terrain locomotion 3
1.3 Overview of legged robots 5
1.4 Quadrupedal robots StarlETH and ANYmal 11

2.1 Characteristics of range sensors on a mobile robot 18
2.2 Overview of the Kinect v2 TOF camera 25
2.3 Depth distortion error 27
2.4 Experimental setup for noise evaluation 28
2.5 Example data from the Kinect v2 29
2.6 Measurements and fitted models for the axial noise σ_z 30
2.7 Lateral noise model for indoors 31
2.8 Short range data quality 32
2.9 Outdoor example data 33
2.10 Data quality in sunlight 34
2.11 Axial noise model for direct sunlight 35
2.12 Noise model comparison for depth sensors 36
2.13 Scan pattern for active stereo camera and laser range sensor . . 38

3.1 World- and robot-centric mapping approach 43
3.2 Overview of the elevation mapping processing steps 45
3.3 Coordinate frames for the mapping process 47
3.4 Illustration and examples for the map fusion process 51
3.5 Visibility cleanup for dynamic environments 53
3.6 Evaluation in simulation 54
3.7 Results of the elevation mapping for different sensors 55
3.8 Ground truth comparison 56
3.9 Comparison between presented and alternative approaches . . . 58

4.1 Overview of robot API 60
4.2 Free Gait control scheme 62
4.3 Motion types and coordinate frames 62

4.4 Handling of early foot contact transition 71
4.5 Free Gait user interface 73
4.6 Three-legged push-up motion 74
4.7 Switching leg configuration 76
4.8 Stand up and lie down maneuvers 77
4.9 Motion templates for rough terrain locomotion 78
4.10 Motion tracking under disturbances 79
4.11 Foot trajectories for motion tracking under disturbances 80
4.12 Legged motion through confined spaces 80
4.13 Posture adaptation 81
4.14 Example for basic manipulation (elevator) 82
4.15 Whole-body stair climbing 83
4.16 Real-time dance generation to music 84
4.17 Jumping and rearing maneuvers 84

5.1 Overview of the locomotion planning approach 91
5.2 Foothold classification 94
5.3 Nominal footstep generation 95
5.4 Footstep optimization 96
5.5 Post optimization cost values 100
5.6 Pose optimization examples and computation times 100
5.7 Terrain-based swing leg planning 103
5.8 Perception based locomotion over steps 105
5.9 Locomotion over included surfaces 106
5.10 Locomotion in dynamic environments 106
5.11 Locomotion over stairs 107
5.12 Global locomotion goal tracking under disturbances 108
5.13 Demonstration of robustness of the approach 109

6.1 Heterogeneous team consisting of a flying and a legged robot . 114
6.2 Overview of the collaborative navigation framework 116
6.3 Mission environment and task 121
6.4 Collaborative landmark-based visual localization 122
6.5 Elevation mapping 123
6.6 Traversability evaluation 124
6.7 Navigation planning for the legged robot 124
6.8 Visual features for concurrent operation 126
6.9 Legged robot localization for concurrent operation 126
6.10 Terrain mapping for concurrent operation 127

7.1 Autonomous stair climbing in an urban environment 130

List of Tables

2.1 Range sensors specifications 21

4.1 Free Gait interface commands 64

5.1 Success rates for stepping up and down a step 105

Bibliography

Aceituno-Cabezas, B., Mastalli, C., Dai, H., Focchi, M., Radulescu, A., Caldwell, D., Cappelletto, J., Grieco, J., Fernández-López, G., and Semini, C. (2018). "Simultaneous Contact, Gait, and Motion Planning for Robust Multilegged Locomotion via Mixed-Integer Convex Optimization". *IEEE Robotics and Automation Letters (RA-L)* 3.3, pp. 2531–2538.

Amon, C. and Fuhrmann, F. (2014). "Evaluation of the spatial resolution accuracy of the Windows Face Tracking SDK in combination with the Kinect V1 and the Kinect V2". In: *Congress of Alps-Adria Acoustics Assosiation*. October, pp. 9–12.

Bajracharya, M., Ma, J., Malchano, M., Perkins, A., Rizzi, A. A., and Matthies, L. (2013). "High Fidelity Day / Night Stereo Mapping with Vegetation and Negative Obstacle Detection for Vision-in-the-Loop Walking". In: *IEEE/RSJ International Conference on Intelligent Robot and Systems (IROS)*, pp. 3663–3670.

Barasuol, V., Camurri, M., Bazeille, S., Caldwell, D. G., and Semini, C. (2015). "Reactive Trotting with Foot Placement Corrections through Visual Pattern Classification". In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 5734–5741.

Bellicoso, C. D., Gehring, C., Hwangbo, J., Fankhauser, P., and Hutter, M. (2016). "Perception-less Terrain Adaptation through Whole Body Control and Hierarchical Optimization". In: *IEEE-RAS International Conference on Humanoid Robots (Humanoids)*, pp. 558–564.

Bellicoso, C. D., Jenelten, F., Fankhauser, P., Gehring, C., Hwangbo, J., and Hutter, M. (2017). "Dynamic Locomotion and Whole-Body Control for Quadrupedal Robots". In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 3359–3365.

Bellicoso, C. D., Jenelten, F., Gehring, C., and Hutter, M. (2018a). "Dynamic Locomotion Through Online Nonlinear Motion Optimization for Quadrupedal Robots". *IEEE Robotics and Automation Letters (RA-L)* 3.3, pp. 2261–2268.

Bellicoso, C. D., Bjelonic, M., Wellhausen, L., Holtmann, K., Guenther, F., Tranzatto, M., Fankhauser, P., and Hutter, M. (2018b). "Advances in Real-World Applications for Legged Robots". *(in review) Journal of Field Robotics*.

Belter, D., Labcki, P., and Skrzypczynski, P. (2012). "Estimating Terrain Elevation Maps from Sparse and Uncertain Multi-Sensor Data". In: *IEEE International Conference on Robotics and Biomimetics (ROBIO)*, pp. 715–722.

Belter, D., Łabęcki, P., and Skrzypczyński, P. (2016). "Adaptive Motion Planning for Autonomous Rough Terrain Traversal with a Walking Robot". *Journal of Field Robotics* 33.3, pp. 337–370.

Belter, D. and Skrzypczyński, P. (2011). "Rough Terrain Mapping and Classification for Foothold Selection in a Walking Robot". *Journal of Field Robotics* 28.4, pp. 497–528.

Belter, D., Skrzypczyński, P., Walas, K., Fankhauser, P., Gehring, C., Hutter, M., Hoepflinger, M. A., and Siegwart, R. (2014). "Dynamic Simulation of Legged Robots Using a Physics Engine". In: *International Conference on Climbing and Walking Robots (CLAWAR)*, pp. 567–574.

Belter, D., Labecki, P., Fankhauser, P., and Siegwart, R. (2016). "RGB-D Terrain Perception and Dense Mapping for Legged Robots". *International Journal of Applied Mathematics and Computer Science (AMCS)* 26.1, pp. 81–97.

Ben-Kiki, O., Evans, C., and Ingerson, B. (2009). *YAML Ain't Markup Language (YAML)*. Tech. rep.

Bi, T., Fankhauser, P., Bellicoso, D., and Hutter, M. (2018). "Real-Time Dance Generation to Music for a Legged Robot". In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*.

Bjelonic, M., Homberger, T., Kottege, N., Borges, P., Chli, M., and Beckerle, P. (2017). "Autonomous Navigation of Hexapod Robots with Vision-Based Controller Adaptation". In: *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 5561–5568.

Blake, J., Echtler, F., and Kerl, C. (2013–2018). *libfreenect2: Open source drivers for the Kinect for Windows v2 device*. `https://github.com/OpenKinect/libfreenect2`, accessed August 18, 2018.

Blöchliger, F., Blösch, M., Fankhauser, P., Hutter, M., and Siegwart, R. (2016). "Foot-Eye Calibration of Legged Robot Kinematics". In: *International Conference on Climbing and Walking Robots (CLAWAR)*, pp. 420 –427.

Bloesch, M., Omari, S., Fankhauser, P., Sommer, H., Gehring, C., Hwangbo, J., Hoepflinger, M., Hutter, M., and Siegwart, R. (2014). "Fusion of Optical Flow and Inertial Measurements for Robust Egomotion Estimation". In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 3102–3107.

Bloesch, M. and Hutter, M. (2017). "Technical Implementations of the Sense of Balance". In: *Humanoid Robotics: A Reference*. Ed. by P. Vadakkepat and A. Goswami. Springer Netherlands.

Bloesch, M., Hutter, M., Hoepflinger, M. A., Leutenegger, S., Gehring, C., Remy, C. D., and Siegwart, R. (2012). "State Estimation for Legged Robots - Consistent Fusion of Leg Kinematics and IMU". In: *Robotics: Science and Systems Conference (RSS)*, pp. 17–24.

Bloesch, M., Gehring, C., Fankhauser, P., Hutter, M., Hoepflinger, M. A., and Siegwart, R. (2013). "State Estimation for Legged Robots on Unstable and Slippery Terrain". In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 6058–6064.

Bloesch, M., Sommer, H., Laidlow, T., Burri, M., Nützi, G., Fankhauser, P., Bellicoso, D., Gehring, C., Leutenegger, S., Hutter, M., and Siegwart, R. (2016). *A Primer on the Differential Calculus of 3D Orientations*. Tech. rep. arXiv: 1606.05285.

Bradski, G. (2000). "The OpenCV Library". *Dr. Dobb's Journal of Software Tools*.

Bretl, T. (2006). "Motion Planning of Multi-Limbed Robots Subject to Equilibrium Constraints: The Free-Climbing Robot Problem". *The International Journal of Robotics Research* 25.4, pp. 317–342.

Bretl, T. and Lall, S. (2008). "Testing Static Equilibrium for Legged Robots". *IEEE Transactions on Robotics* 24.4, pp. 794–807.

Breuer, T., Bodensteiner, C., and Arens, M. (2014). "Low-cost Commodity Depth Sensor Comparison and Accuracy Analysis". In: *SPIE Security + Defence*. Ed. by G. Kamerman, O. Steinvall, G. J. Bishop, A. Killey, and J. D. Gonglewski. Vol. 9250. International Society for Optics and Photonics.

Buchli, J., Kalakrishnan, M., Mistry, M., Pastor, P., and Schaal, S. (2009). "Compliant Quadruped Locomotion Over Rough Terrain". In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 814–820.

Buss, S. R. (2004). *Introduction to Inverse Kinematics with Jacobian Transpose, Pseudoinverse and Damped Least Squares methods*. Tech. rep.

Butkiewicz, T. (2014). "Low-cost Coastal Mapping using Kinect v2 Time-of-Flight Cameras". In: *Oceans - St. John's*. IEEE, pp. 1–9.

Chiaverini, S., Siciliano, B., and Egeland, O. (1994). "Review of the Damped Least-Squares Inverse Kinematics with Experiments on an Industrial Robot Manipulator". *IEEE Transactions on Control Systems Technology* 2.2, pp. 123–134.

Cieslewski, T., Lynen, S., Dymczyk, M., Magnenat, S., and Siegwart, R. (2015). "Map API - Scalable Decentralized Map Building for Robots". In: *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 6241–6247.

Cremean, L. and Murray, R. M. (2005). "Uncertainty-based Sensor Fusion of Range Data for Real-time Digital Elevation Mapping (RTDEM)". In: *IEEE International Conference on Robotics and Automation (ICRA)*.

Cruz, L., Lucio, D., and Velho, L. (2012). "Kinect and RGBD Images: Challenges and Applications". In: *SIBGRAPI Conference on Graphics, Patterns and Images Tutorials*, pp. 36–49.

Deits, R. and Tedrake, R. (2014). "Footstep Planning on Uneven Terrain with Mixed-Integer Convex Optimization". In: *IEEE-RAS International Conference on Humanoid Robots (Humanoids)*, pp. 279–286.

Delmerico, J., Mueggler, E., Nitsch, J., and Scaramuzza, D. (2017). "Active Autonomous Aerial Exploration for Ground Robot Path Planning". *IEEE Robotics and Automation Letters* 2.2, pp. 664–671.

Devereux, D., Mitra, B., Holland, O., and Diamond, A. (2011). "Using the Microsoft Kinect to model the environment of an anthropomimetic robot". In: *IASTED International Confonference on Robotics (Robo2011)*.

Di Gaspero, L. (2007–18). *QuadProg++*. `https://github.com/liuq/QuadProgpp`, accessed August 18, 2018.

Dymczyk, M., Lynen, S., Bosse, M., and Siegwart, R. (2015). "Keep it Brief: Scalable Creation of Compressed Localization Maps". In: *IEEE/RSJ International Conference on Intelligent Robot and Systems (IROS)*, pp. 2536–2542.

El-Iaithy, R. A., Huang, J., and Yeh, M. (2012). "Study on the Use of Microsoft Kinect for Robotics Applications". In: *Position Location and Navigation Symposium (PLANS)*. IEEE/ION, pp. 1280–1288.

Eldershaw, C. and Yim, M. (2001). "Motion Planning of Legged Vehicles in an Unstructured Environment". In: *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 3383–3389.

Escande, A., Kheddar, A., and Miossec, S. (2006). "Planning Support Contact-Points for Humanoid Robots and Experiments on HRP-2". In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 2974–2979.

– (2013). "Planning Contact Points for Humanoid Robots". *Robotics and Autonomous Systems*.

Fallon, M., Kuindersma, S., Karumanchi, S., Antone, M., Schneider, T., Dai, H., D'Arpino, C. P., Deits, R., DiCicco, M., Fourie, D., Koolen, T., Marion, P., Posa, M., Valenzuela, A., Yu, K. T., Shah, J., Iagnemma, K., Tedrake, R., and Teller, S. (2015). "An Architecture for Online Affordance-Based Perception and Whole-Body Planning". *Journal of Field Robotics* 32.2, pp. 229–254.

Fankhauser, P., Bloesch, M., and Hutter, M. (2018). "Probabilistic Terrain Mapping for Mobile Robots with Uncertain Localization". *IEEE Robotics and Automation Letters (RA-L)* 3.4, pp. 3019–3026.

Fankhauser, P. and Hutter, M. (2015). "Localization, Terrain Mapping, and Navigation with StarlETH". In: *Workshop on Perception and Planning for Legged Robot Locomotion in Challenging Domains, IEEE International Conference on Intelligent Robots and Systems (IROS)*.

– (2016). "A Universal Grid Map Library: Implementation and Use Case for Rough Terrain Navigation". In: *Robot Operating System (ROS): The Complete Reference (Volume 1)*. Ed. by A. Koubaa. Springer International Publishing. Chap. 5, pp. 99–120.

Fankhauser, P., Hutter, M., Bloesch, M., and Siegwart, R. (2013a). "Optimizing Robotic Single Legged Locomotion with Reinforcement Learning". In: *Dynamic Walking Meeting*.

Fankhauser, P., Hutter, M., Gehring, C., and Siegwart, R. (2013b). "Optimizing Robotic Single Legged Locomotion with Reinforcement Learning". In: *Workshop on Novel Methods for Learning and Optimization of Control Policies and Trajectories for Robotics, IEEE Internation Conference on Robotics and Automation (ICRA)*.

Fankhauser, P., Hutter, M., Gehring, C., Bloesch, M., Hoepflinger, M. A., and Siegwart, R. (2013c). "Reinforcement Learning of Single Legged Locomotion". In: *IEEE/RSJ International Conference on Intelligent Robot and Systems (IROS)*, pp. 188–193.

Fankhauser, P., Bloesch, M., Gehring, C., Hutter, M., and Siegwart, R. (2014). "Robot-Centric Elevation Mapping with Uncertainty Estimates". In: *International Conference on Climbing and Walking Robots (CLAWAR)*, pp. 433–440.

Fankhauser, P., Bloesch, M., Rodriguez, D., Kaestner, R., Hutter, M., and Siegwart, R. (2015). "Kinect v2 for Mobile Robot Navigation: Evaluation and Modeling". In: *International Conference on Advanced Robotics (ICAR)*. IEEE, pp. 388–394.

Fankhauser, P., Diethelm, R., Bachmann, S., Gehring, C., Wermelinger, M., Bellicoso, D., Tsounis, V., Lauber, A., Bloesch, M., Leemann, P., Hottiger, G., Jud, D., Kaestner, R., Isler, L., Hoepflinger, M., Siegwart, R., and Hutter, M. (2016a). "ANYmal at the ARGOS Challenge – Tools and Experiences from the Autonomous Inspection of Oil & Gas Sites with a Legged Robot". In: *ROSCon*. Open Source Robotics Foundation.

Fankhauser, P., Bloesch, M., Krusi, P., Diethelm, R., Wermelinger, M., Schneider, T., Dymczyk, M., Hutter, M., and Siegwart, R. (2016b). "Collaborative Navigation for Flying and Walking Robots". In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 2859–2866.

Fankhauser, P., Dario Bellicoso, C., Gehring, C., Dube, R., Gawel, A., and Hutter, M. (2016c). "Free Gait – An Architecture for the Versatile Control of Legged Robots". In: *IEEE-RAS International Conference on Humanoid Robots (Humanoids)*, pp. 1052–1058.

Fankhauser, P., Bellicoso, D., Gehring, C., Wiedebach, G., and Hutter, M. (2016d). "Free Gait: Versatile Control of Legged Robots". In: *Workshop on Dynamic Locomotion and Manipulation*.

Fankhauser, P., Jud, D., Wermelinger, M., and Hutter, M. (2017). "Programming for Robotics – Introduction to ROS". Lecture Notes. ETH Zurich, Switzerland.

Fankhauser, P., Bjelonic, M., Bellicoso, C. D., Miki, T., and Hutter, M. (2018). "Robust Rough-Terrain Locomotion with a Quadrupedal Robot". In: *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 5761–5768.

Farshidian, F., Neunert, M., Winkler, A. W., Rey, G., and Buchli, J. (2017). "An Efficient Optimal Planning and Control Framework for Quadrupedal Locomotion". In: *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 93–100.

Felzenszwalb, P. F. and Huttenlocher, D. P. (2012). "Distance Transforms of Sampled Functions". *Theory of Computing* 8.1, pp. 415–428.

Focchi, M., Orsolino, R., Camurri, M., Barasuol, V., Mastalli, C., Caldwell, D. G., and Semini, C. (2018). *Heuristic Planning for Rough Terrain Locomotion in Presence of External Disturbances and Variable Perception Quality*. Tech. rep. arXiv: 1805.10238.

Foix, S., Alenya, G, and Torras, C (2011). "Lock-in Time-of-Flight (ToF) Cameras: A Survey". *IEEE Sensors Journal* 11.9, pp. 1917 –1926.

Forster, C., Faessler, M., Fontana, F., Werlbeger, M., and Scaramuzza, D. (2015). "Continuous On-Board Monocular-Vision–based Elevation Mapping Applied to Autonomous

Landing of Micro Aerial Vehicles". In: *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 664–671.

Forster, C., Pizzoli, M., and Scaramuzza, D. (2013). "Air-Ground Localization and Map Augmentation Using Monocular Dense Reconstruction". In: *IEEE International Conference on Intelligent Robots and Systems (IROS)*, pp. 3971–3978.

Frank, A. A. (1968). *Automatic Control Systems for Legged Locomotion*. Tech. rep. University of Southern California.

Fukuoka, Y., Kimura, H., and Cohen, A. H. (2003). "Adaptive Dynamic Walking of a Quadruped Robot on Irregular Terrain Based on Biological Concepts". *The International Journal of Robotics Research* 22.3-4, pp. 187–202.

Fürsattel, P., Placht, S., Balda, M., Schaller, C., Hofmann, H., Maier, A., and Riess, C. (2016). "A Comparative Error Analysis of Current Time-of-Flight Sensors". *IEEE Transactions on Computational Imaging* 2.1, pp. 27–41.

Gehring, C., Coros, S., Hutter, M., Bloesch, M., Fankhauser, P., Hoepflinger, M., and Siegwart, R. (2014). "Towards Automatic Discovery of Agile Gaits for Quadrupedal Robots". In: *IEEE International Conference on Robotics and Automation*, pp. 4243–4248.

Gehring, C. (2017). "Planning and Control for Agile Quadruped Robots". Doctoral Thesis. ETH Zurich.

Gehring, C., Coros, S., Hutter, M., Bloesch, M., Hoepflinger, M. A., and Siegwart, R. (2013). "Control of Dynamic Gaits for a Quadrupedal Robot". *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 3287–3292.

Gehring, C., Bellicoso, C. D., Coros, S., Bloesch, M., Fankhauser, P., Hutter, M., and Siegwart, R. (2015). "Dynamic Trotting on Slopes for Quadrupedal Robots". In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 5129–5135.

Gehring, C., Coros, S., Hutter, M., Bellicoso, C. D., Heijnen, H., Diethelm, R., Bloesch, M., Fankhauser, P., Hwangbo, J., Hoepflinger, M. A., and Siegwart, R. (2016). "Practice Makes Perfect: An Optimization-Based Approach to Controlling Agile Motions for a Quadruped Robot". *IEEE Robotics & Automation Magazine* 23.1, pp. 34–43.

Gehring, C., Dario Bellicoso, C., Fankhauser, P., Coros, S., and Hutter, M. (2017). "Quadrupedal Locomotion Using Trajectory Optimization and Hierarchical Whole Body Control". In: *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 4788–4794.

Germann, D., Hiller, M., and Schramm, D. (2005). "Design and Control of the Quadruped Walking Robot ALDURO". *International Symposium on Automation and Robotics in Construction (ISARC)*.

Gonon, D., Jud, D., Fankhauser, P., and Hutter, M. (2017). "Safe Self-Collision Avoidance for Versatile Robots based on Bounded Potentials". In: *International Conference on Field and Service Robotics (FSR)*.

Grocholsky, B., Bayraktar, S., Kumar, V., and Pappas, G. (2004). "UAV and UGV Collaboration for Active Ground Feature Search and Localization". In: *AIAA "Unmanned Unlimited" Technical Conference, Workshop and Exhibit*. September.

Haldane, D. W., Fankhauser, P., Siegwart, R., and Fearing, R. S. (2014). "Detection of Slippery Terrain with a Heterogeneous Team of Legged Robots". In: *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 4576–4581.

Harik, E. H. C., Guerin, F., Guinand, F., Brethe, J. F., and Pelvillain, H. (2015). "UAV-UGV Cooperation for Objects Transportation in an Industrial Area". In: *IEEE International Conference on Industrial Technology*, pp. 547–552.

Hauser, K., Bretl, T., Latombe, J.-C., Harada, K., and Wilcox, B. (2008). "Motion Planning for Legged Robots on Varied Terrain". *The International Journal of Robotics Research* 27.11-12, pp. 1325–1349.

Havoutis, I., Ortiz, J., Bazeille, S., Barasuol, V., Semini, C., and Caldwell, D. G. (2013). "Onboard Perception-Based Trotting and Crawling with the Hydraulic Quadruped Robot (HyQ)". In: *IEEE/RSJ International Conference on Intelligent Robot and Systems (IROS)*, pp. 6052–6057.

Heess, N., Sriram, S., Lemmon, J., Merel, J., Wayne, G., Tassa, Y., Erez, T., Wang, Z., Eslami, A., Riedmiller, M., et al. (2017). *Emergence of Locomotion Behaviours in Rich Environments*. Tech. rep. arXiv: 1707.02286.

Heppner, G., Roennau, A., and Dillman, R. (2013). "Enhancing Sensor Capabilities of Walking Robots Through Cooperative Exploration with Aerial Robots". *Journal of Automation, Mobile Robotics & Intelligent Systems* 7, pp. 5–11.

Herbert, M., Caillas, C., Krotkov, E., Kweon, I., and Kanade, T. (1989). "Terrain Mapping for a Roving Planetary Explorer". In: *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 997–1002.

Hinzmann, T., Schönberger, J. L., Pollefeys, M., and Siegwart, R. (2017). "Mapping on the Fly: Real-time 3D Dense Reconstruction, Digital Surface Map and Incremental Orthomosaic Generation for Unmanned Aerial Vehicles". In: *International Conference on Field and Service Robotics (FSR)*.

Hirose, S. (1984). "A Study of Design and Control of a Quadruped Walking Vehicle". *The International Journal of Robotics Research* 3.2, pp. 113–133.

Hirose, S. and Kunieda, O. (1991). "Generalized Standard Foot Trajectory for a Quadruped Walking Vehicle". *The International Journal of Robotics Research* 10.1, pp. 3–12.

Hirschmuller, H. (2008). "Stereo Processing by Semi-Global Matching and Mutual Information". *IEEE Transactions on Pattern Analysis and Machine Intelligence* 30.2, pp. 328–341.

Hoepflinger, M. A. (2013). "Haptic Foothold Robustness Identification and Estimation for Legged Robots". Ph.D. Dissertation. ETH Zurich.

Hogan, N. (1985). "Impedance Control : An Approach to Manipulation". *Journal of Dynamic Systems, Measurements, and Control* 107, pp. 304–313.

Hogg, R. V., McKean, J., and Craig, A. T. (2005). *Introduction to Mathematical Statistics*. 6th ed. Pearson Education.

Horaud, R., Hansard, M., Evangelidis, G., and Ménier, C. (2016). "An Overview of Depth Cameras and Range Scanners Based on Time-of-Flight Technologies". *Machine Vision and Applications* 27.7, pp. 1005–1020.

Hornung, A., Wurm, K. M., Bennewitz, M., Stachniss, C., and Burgard, W. (2013). "OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees". *Autonomous Robots* 34.3, pp. 189–206.

Hsieh, M. A., Cowley, A., Keller, J. E., Chaimowicz, L., Grocholsky, B., Kumar, V., Taylor, C. J., Endo, Y., Arkin, R. C., Jung, B., Wolf, D. F., Sukhatme, G. S., and MacKenzie, D. C. (2007). "Adaptive Teams of Autonomous Aerial and Ground Robots for Situational Awareness". *Journal of Field Robotics* 24.11-12, pp. 991–1014.

Hutter, M., Gehring, C., Bloesch, M., Hoepflinger, M. A., Remy, C. D., and Siegwart, R. (2012). "StarlETH: A Compliant Quadrupedal Robot for Fast, Efficient, and Versatile Locomotion". In: *International Conference on Climbing and Walking Robots (CLAWAR)*, pp. 483–490.

Hutter, M., Gehring, C., Bloesch, M., Hoepflinger, M. A., Fankhauser, P., and Siegwart, R. (2014a). "Excitation and Stabilization of Passive Dynamics in Locomotion using Hierarchical Operational Space Control". In: *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 2977–2982.

Hutter, M., Gehring, C., Hoepflinger, M., Bloesch, M., and Siegwart, R. (2014b). "Towards combining Speed, Efficiency, Versatility and Robustness in an Autonomous Quadruped". *IEEE Transactions on Robotics* 30.6, pp. 1427–1440.

Hutter, M., Gehring, C., Jud, D., Lauber, A., Bellicoso, C. D., Tsounis, V., Hwangbo, J., Bodie, K., Fankhauser, P., Bloesch, M., Diethelm, R., Bachmann, S., Melzer, A., and Hoepflinger, M. (2016). "ANYmal – A Highly Mobile and Dynamic Quadrupedal Robot". In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*.

Hutter, M., Gehring, C., Lauber, A., Gunther, F., Bellicoso, C. D., Tsounis, V., Fankhauser, P., Diethelm, R., Bachmann, S., Bloesch, M., Kolvenbach, H., Bjelonic, M., Isler, L., and Meyer, K. (2017a). "ANYmal – Toward Legged Robots for Harsh Environments". *Advanced Robotics*. Advanced Robotics 31.17, pp. 918–931.

Hutter, M., Diethelm, R., Bachmann, S., Fankhauser, P., Gehring, C., Tsounis, V., Lauber, A., Guenther, F., Bjelonic, M., Isler, L., Kolvenbach, H., Meyer, K., and Hoepflinger, M. (2017b). "Towards a Generic Solution for Inspection of Industrial Sites". In: *International Conference on Field and Service Robotics (FSR)*.

Hwangbo, J., Gehring, C., Bellicoso, D., Fankhauser, P., Siegwart, R., and Hutter, M. (2015). "Direct State-to-Action Mapping for High DOF Robots Using ELM". In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*.

Hwangbo, J., Bellicoso, C. D., Fankhauser, P., and Hutter, M. (2016). "Probabilistic Foot Contact Estimation by Fusing Information from Dynamics and Differential/Forward Kinematics". In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, pp. 3872 –3878.

Hyndman, R. J. and Fan, Y. (1996). "Sample Quantiles in Statistical Packages". *The American Statistician* 50.4, pp. 361–365.

iFixit (2013). *Xbox One Kinect Teardown*. `https://www.ifixit.com/Teardown/Xbox+One+Kinect+Teardown/19725`, accessed August 18, 2018.

Jia, D., Wermelinger, M., Diethelm, R., Krusi, P., and Hutter, M. (2016). "Coverage Path Planning for Legged Robots in Unknown Environments". In: *International Symposium on Safety, Security and Rescue Robotics (SSRR)*, pp. 68–73.

Johnson, M., Bradshaw, J. M., Feltovich, P. J., Jonker, C. M., Van Riemsdijk, M. B., and Sierhuis, M. (2014). "Coactive Design: Designing Support for Interdependence in Joint Activity". *Journal of Human-Robot Interaction* 3.1, pp. 43–69.

Johnson, M., Shrewsbury, B., Bertrand, S., Calvert, D., Wu, T., Duran, D., Stephen, D., Mertins, N., Carff, J., Rifenburgh, W., Smith, J., Schmidt-Wetekam, C., Faconti, D., Graber-Tilton, A., Eyssette, N., Meier, T., Kalkov, I., Craig, T., Payton, N., McCrory, S., Wiedebach, G., Layton, B., Neuhaus, P., and Pratt, J. (2017). "Team IHMC's Lessons Learned from the DARPA Robotics Challenge: Finding Data in the Rubble". *Journal of Field Robotics* 34.2, pp. 241–261.

Johnson, M. J., Shrewsbury, B., Bertrand, S., Wu, T., Duran, D., Floyd, M., Abeles, P., Stephen, D., Mertins, N., Lesman, A., Carff, J., Rifenburgh, W., Kaveti, P., Straatman, W., Smith, J., Griffioen, M., Layton, B., Boer, T. de, Koolen, T., Neuhaus, P., and Pratt, J. (2015). "Team IHMC's Lessons Learned from the DARPA Robotics Challenge Trials". *Journal of Field Robotics* 32.2, pp. 192–208.

Jorg, S., Tully, J., and Albu-Schaffer, A. (2014). "The Hardware Abstraction Layer - Supporting Control Design by Tackling the Complexity of Humanoid Robot Hardware". In: *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 6427–6433.

Jud, D., Hottiger, G., Leemann, P., and Hutter, M. (2017). "Planning and Control for Autonomous Excavation". *IEEE Robotics and Automation Letters* 2.4, pp. 2151–2158.

Kalakrishnan, M., Buchli, J., Pastor, P., Mistry, M., and Schaal, S. (2010). "Learning, Planning, and Control for Quadruped Locomotion over Challenging Terrain". *The International Journal of Robotics Research* 30.2, pp. 236–258.

Karaman, S. and Frazzoli, E. (2011). "Sampling-based Algorithms for Optimal Motion Planning". *The International Journal of Robotics Research* 30.7, pp. 846–894.

Karumanchi, S., Edelberg, K., Baldwin, I., Nash, J., Reid, J., Bergh, C., Leichty, J., Carpenter, K., Shekels, M., Gildner, M., Newill-Smith, D., Carlton, J., Koehler, J., Dobreva, T., Frost, M., Hebert, P., Borders, J., Ma, J., Douillard, B., Backes, P., Kennedy, B., Satzinger, B., Lau, C., Byl, K., Shankar, K., and Burdick, J. (2017). "Team RoboSimian: Semi-autonomous Mobile Manipulation at the 2015 DARPA Robotics Challenge Finals". *Journal of Field Robotics* 34.2, pp. 305–332.

Käslin, R., Fankhauser, P., Stumm, E., Taylor, Z., Mueggler, E., Delmerico, J., Scaramuzza, D., Siegwart, R., and Hutter, M. (2016). "Collaborative Localization of Aerial and Ground Robots through Elevation Maps". In: *International Symposium on Safety, Security, and Rescue Robotics (SSRR)*.

Kelly, A., Amidi, O., Bode, M., Happold, M., Herman, H., Pilarski, T., Rander, P., Stentz, A., Vallidis, N., and Warner, R. (2006). "Toward Teliable Off Toad Autonomous Vehicles

Operating in Challenging Environments". *The International Journal of Robotics Research* 25.5-6, pp. 449–483.

Kerscher, T., Roennau, A., Ziegenmeyer, M., Gassman, B., Zolelner, J., and Dillman, R. (2008). "Behaviour-based control of the six-legged walking machine Lauron IVc". In: *International Conference on Climbing and Walking Robots (CLAWAR)*, pp. 736–743.

Keselman, L., Woodfill, J. I., Grunnet-Jepsen, A., and Bhowmik, A. (2017). "Intel (R) RealSense (TM) Stereoscopic Depth Cameras". In: *IEEE Computer Vision and Pattern Recognition Workshops (CVPRW)*, pp. 1267–1276.

Khoshelham, K. and Elberink, S. O. (2012). "Accuracy and Resolution of Kinect Depth Data for Indoor Mapping Applications". *Sensors* 12.2, pp. 1437–1454.

Kim, M.-J., Kim, M.-S., and Shin, S. Y. (1995). "A General Construction Scheme for Unit Quaternion Curves with Simple High Order Derivatives". In: *Conference on Computer Graphics and Interactive Techniques*, pp. 369–376.

Kimura, H., Fukuoka, Y., and Cohen, a. H. (2007). "Adaptive Dynamic Walking of a Quadruped Robot on Natural Ground Based on Biological Concepts". *The International Journal of Robotics Research* 26.5, pp. 475–490.

Klasing, K., Althoff, D., Wollherr, D., and Buss, M. (2009). "Comparison of Surface Normal Estimation Methods for Range Sensing Applications". *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 3206–3211.

Klein, C. A., Olson, K. W., and Pugh, D. R. (1983). "Use of Force and Attitude Sensors for Locomotion of a Legged Vehicle over Irregular Terrain". *The International Journal of Robotics Research* 2.2, pp. 3–17.

Kleiner, A. and Dornhege, C. (2007). "Real-Time Localization and Elevation Mapping within Urban Search and Rescue Scenarios". *Journal of Field Robotics* 24.8-9, pp. 723–745.

Kohlbrecher, S. and Stryk, O. v. (2016). "From RoboCup Rescue to Supervised Autonomous Mobile Robots for Remote Inspection of Industrial Plants". *KI - Künstliche Intelligenz* 30.3, pp. 311–314.

Kohlbrecher, S., Stumpf, A., Romay, A., Schillinger, P., Stryk, O. von, and Conner, D. C. (2016). "A Comprehensive Software Framework for Complex Locomotion and Manipulation Tasks Applicable to Different Types of Humanoid Robots". *Frontiers in Robotics and AI* 3, p. 31.

Kolter, J. Z. and Ng, a. Y. (2011). "The Stanford LittleDog: A learning and rapid replanning approach to quadruped locomotion". *The International Journal of Robotics Research* 30.2, pp. 150–174.

Kolter, J. Z. and Ng, A. Y. (2009). "Stereo Vision and Terrain Modeling for Quadruped Robots". *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 1557–1564.

Kolter, J. Z., Rodgers, M. P., and Ng, A. Y. (2008). "A Control Architecture for Quadruped Locomotion Over Rough Terrain". *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 811–818.

Kolvenbach, H. and Hutter, M. (2017). “Life Extension: An Autonomous Docking Station for Recharging Quadrupedal Robots”. In: *International Conference on Field and Service Robotics (FSR)*.

Krotkov, E. and Simmons, R. (1996). “Perception, Planning, and Control for Autonomous Walking With the Ambler Planetary Rover”. *The International Journal of Robotics Research* 15.2, pp. 155–180.

Kuffner, J. and LaValle, S. (2000). “RRT-connect: An Efficient Approach to Single-Query Path Planning”. In: *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 995–1001.

Kuindersma, S., Deits, R., Fallon, M., Valenzuela, A., Dai, H., Permenter, F., Koolen, T., Marion, P., and Tedrake, R. (2016). “Optimization-based locomotion planning, estimation, and control design for the atlas humanoid robot”. *Autonomous Robots* 40.3, pp. 429–455.

Kumar, A. S., Manikutty, G., Bhavani, R. R., and Couceiro, M. S. (2017). “Search and Rescue Operations using Robotic Darwinian Particle Swarm Optimization”. In: *International Conference on Advances in Computing, Communications and Informatics (ICACCI)*, pp. 1839–1843.

Kweon, S. and Kanade, T. (1992). “High Resolution Terrain Map from Multiple Sensor Data”. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 14.2.

Lachat, E., Macher, H., Mittet, M.-A., Landes, T., and Grussenmeyer, P. (2015). “First Experiences With Kinect V2 Sensor for Close Range 3D Modelling”. In: *International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences (ISPRS)*, pp. 93–100.

Lefloch, D., Nair, R., Lenzen, F., Schäfer, H., Streeter, L., Cree, M. J., Koch, R., and Kolb, A. (2013). “Technical Foundation and Calibration Methods for Time-of-Flight Cameras”. *Time-of-Flight and Depth Imaging. Sensors, Algorithms, and Applications.* Pp. 3–24.

Lemaire, T., Berger, C., Jung, I.-K., and Lacroix, S. (2007). “Vision-Based SLAM: Stereo and Monocular Approaches”. *International Journal of Computer Vision* 74.3, pp. 343–364.

Lynen, S., Bosse, M., Furgale, P., and Siegwart, R. (2014). “Placeless Place-Recognition”. In: *International Conference on 3D Vision (3DV)*, pp. 303–310.

Lynen, S., Sattler, T., Bosse, M., Hesch, J., Pollefeys, M., and Siegwart, R. (2015). “Get Out of My Lab: Large-scale, Real-Time Visual-Inertial Localization”. In: *Robotics: Science and Systems (RSS)*.

Lyu, Y., Bai, L., and Huang, X. (2018). “ChipNet: Real-Time LiDAR Processing for Drivable Region Segmentation on an FPGA”. *arXiv preprint: 1808.03506*.

Mallick, T., Das, P. P., and Majumdar, A. K. (2014). “Characterizations of Noise in Kinect Depth Images: A Review”. *IEEE Sensors Journal* 14.6, pp. 1731–1740.

Mastalli, C., Havoutis, I., Winkler, A., Caldwell, D., and Semini, C. (2015). “On-line and On-board Planning and Perception for Quadrupedal Locomotion”. In: *IEEE International Conference of Technologies for Practical Robot Applications (TEPRA)*.

Mastalli, C., Havoutis, I., Focchi, M., Caldwell, D., and Semini, C. (2017a). *Motion Planning for Quadrupedal Locomotion: Coupled Planning, Terrain Mapping and Whole-Body Control*. Tech. rep.

Mastalli, C., Focchi, M., Havoutis, I., Radulescu, A., Calinon, S., Buchli, J., Caldwell, D. G., and Semini, C. (2017b). "Trajectory and Foothold Optimization using Low-Dimensional Models for Rough Terrain Locomotion". In: *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 1096–1103.

Matthies, L., Litwin, T., Owens, K., Rankin, A., Murphy, K., Coombs, D., Gilsinn, J., Hong, T. H. T., Legowik, S., Nashman, M., and Yoshimi, B. (1998). "Performance Evaluation of UGV Obstacle Detection with CCD/FLIR Stereo Vision and LADAR". In: *IEEE ISIC/CIRA/ISAS Joint Conference*, pp. 658–670.

Maye, J. (2014). "Online Self-Calibration for Robotic Systems". Doctoral Thesis. ETH Zurich, Switzerland.

McGhee, R. B. (1967). "Finite State Control of Quadruped Locomotion". *Simulation* 9.3, pp. 135–140.

McGhee, R. B. and Frank, A. (1968). "On the Stability Properties of Quadruped Creeping Gaits". *Mathematical Biosciences* 351, pp. 331–351.

Michael, N., Shen, S., Mohta, K., Mulgaonkar, Y., Kumar, V., Nagatani, K., Okada, Y., Kiribayashi, S., Otake, K., Yoshida, K., Ohno, K., Takeuchi, E., and Tadokoro, S. (2012). "Collaborative Mapping of an Earthquake-Damaged Building via Ground and Aerial Robots". *Journal of Field Robotics* 29.5, pp. 832–841.

Mitchell, B., Hofmann, A. G., and Williams, B. C. (2007). "Search-based Foot Placement for Quadrupedal Traversal of Challenging Terrain". In: *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 1461–1466.

Mordatch, I., Todorov, E., and Popović, Z. (2012). "Discovery of Complex Behaviors Through Contact-Invariant Optimization". *ACM Transactions on Graphics* 31.4, p. 43.

Mueggler, E., Faessler, M., Fontana, F., and Scaramuzza, D. (2014). "Aerial-guided Navigation of a Ground Robot among Movable Obstacles". In: *IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR)*.

Murphy, M. P., Saunders, A., Moreira, C., Rizzi, A. A., and Raibert, M. (2010). "The LittleDog Robot". *The International Journal of Robotics Research* 30.2, pp. 145–149.

Murray, T. J., Pham, B. N., and Pirjanian, P (2005). *Hardware Abstraction Layer for a Robot*.

Neuhaus, P. D., Pratt, J. E., and Johnson, M. J. (2011). "Comprehensive summary of the Institute for Human and Machine Cognition's experience with LittleDog". *The International Journal of Robotics Research* 30.2, pp. 216–235.

Neunert, M., Stäuble, M., Giftthaler, M., Bellicoso, C. D., Carius, J., Gehring, C., Hutter, M., and Buchli, J. (2018). "Whole-Body Nonlinear Model Predictive Control Through Contacts for Quadrupeds". *IEEE Robotics and Automation Letters (RA-L)* 3.3, pp. 1458–1465.

Nguyen, C. V., Izadi, S., and Lovell, D. (2012). "Modeling Kinect Sensor Noise for Improved 3D Reconstruction and Tracking". In: *International Conference on 3D Imaging, Modeling, Processing, Visualization and Transmission*, pp. 524–530.

Nikolic, J., Rehder, J., Burri, M., Gohl, P., Leutenegger, S., Furgale, P., and Siegwart, R. (2014). "A Synchronized Visual-Inertial Sensor System with FPGA Pre-Processing for Accurate Real-Time SLAM". *IEEE Internation Conference on Robotics and Automation (ICRA)*, pp. 431–437.

Oleynikova, H., Taylor, Z., Fehr, M., Siegwart, R., and Nieto, J. (2017). "Voxblox: Incremental 3D Euclidean Signed Distance Fields for On-Board MAV Planning". In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 1366–1373.

Olson, E. (2011). "AprilTag: A robust and flexible visual fiducial system". In: *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 3400–3407.

Peng, X. B., Berseth, G., and Panne, M. van de (2016). "Terrain-Adaptive Locomotion Skills Using Deep Reinforcement Learning". *ACM Transactions on Graphics* 35.4.

Peng, X. B., Berseth, G., Yin, K. K., and Panne, M. van de (2017). "DeepLoco: Dynamic Locomotion Skills Using Hierarchical Deep Reinforcement Learning". *ACM Transactions on Graphics* 36.4.

Perrin, N. (2012). "From Discrete to Continuous Motion Planning". In: *Workshop on the Algorithmic Foundations of Robotics Foundations (WAFR)*.

Pierris, G. and Lagoudakis, M. G. (2009). "An Interactive Tool for Designing Complex Robot Motion Patterns". In: *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 4013–4018.

Pinto, A. M., Costa, P., Moreira, A. P., Rocha, L. F., Veiga, G., and Moreira, E. (2015). "Evaluation of Depth Sensors for Robotic Applications". In: *IEEE International Conference on Autonomous Robot Systems and Competitions*, pp. 139–143.

Pomerleau, F. (2013). "Applied Registration for Robotics". Doctoral Thesis. ETH Zurich.

Pomerleau, F., Breitenmoser, A., Liu, M., Colas, F., and Siegwart, R. (2012). "Noise Characterization of Depth Sensors for Surface Inspections". In: *International Conference on Applied Robotics for the Power Industry (CARPI)*. IEEE, pp. 16–21.

Pomerleau, F., Colas, F., Siegwart, R., and Magnenat, S. (2013). "Comparing ICP variants on real-world data sets". *Autonomous Robots* 34.3, pp. 133–148.

Posa, M. and Tedrake, R. (2013). "Direct Trajectory Optimization of Rigid Body Dynamical Systems through Contact". In: *Algorithmic Foundations of Robotics X*, pp. 527–542.

Pot, E., Monceaux, J., Gelin, R., Maisonnier, B., and Robotics, A. (2009). "Choregraphe: a Graphical Tool for Humanoid Robot Programming". In: *IEEE International Workshop on Robot and Human Interactive Communication*. November, pp. 46–51.

Pratama, R. A. (2017). "State Estimation and Obstacle-free Navigation of a Small Air Robot from a Mobile Ground Base Equipped with a 3D Laser Scanner". Master's Thesis. University of Tsukuba.

Pratt, G. and Manzo, J. (2013). "The Darpa Robotics Challenge [competitions]". *IEEE Robotics & Automation Magazine* 20.2, pp. 10–12.

Pratt, G. A. and Williamson, M. M. (1995). "Series Elastic Actuators". In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 3137–3181.

Pratt, J., Chew, C.-M., Torres, A., Dilworth, P., and Pratt, G. (2001). "Virtual Model Control: An Intuitive Approach for Bipedal Locomotion". *The International Journal of Robotics Research* 20.2, pp. 129–143.

Primatesta, S., Rizzoa, A., and La Cour-Harbo, A. (2018). "Ground Risk Map for Unmanned Aircraft in Urban Environments". *Reliability Engineering and System Safety (Pre-Print)*.

Rafibakhsh, N., Gong, J., Siddiqui, M. K., Gordon, C., and Lee, H. F. (2012). "Analysis of XBOX Kinect Sensor Data for Use on Construction Sites: Depth Accuracy and Sensor Interference Assessment". In: *Congress on Construction Research*, pp. 848–857.

Rankin, A., Huertas, A., and Matthies, L. (2005). "Evaluation of Stereo Vision Obstacle Detection Algorithms for Off-Road Autonomous Navigation". In: *AUVSI Symposium on Unmanned Systems*.

Roennau, A, Kerscher, T, Ziegenmeyer, M, Zoellner, J. M., and Dillmann, R (2009). "Six-legged walking in rough terrain based on foot point planning". In: *International Conference on Climbing and Walking (CLAWAR)*, pp. 591–598.

Rudol, P., Wzorek, M., Conte, G., and Doherty, P. (2008). "Micro Unmanned Aerial Vehicle Visual Servoing for Cooperative Indoor Exploration". *IEEE Aerospace Conference*.

Rusu, R. B., Sundaresan, A., Morisset, B., Hauser, K., Agrawal, M., Latombe, J.-C., and Beetz, M. (2009). "Leaving Flatland: Efficient Real-Time Three-Dimensional Perception and Motion Planning". *Journal of Field Robotics* 26.10, pp. 841–862.

Schneider, T., Dymczyk, M., Fehr, M., Egger, K., Lynen, S., Gilitschenski, I., and Siegwart, R. (2018). "Maplab: An Open Framework for Research in Visual-Inertial Mapping and Localization". *IEEE Robotics and Automation Letters (RA-L)* 3.3, pp. 1418–1425.

Schwarz, M., Rodehutskors, T., Droeschel, D., Beul, M., Schreiber, M., Araslanov, N., Ivanov, I., Lenz, C., Razlaw, J., Schüller, S., Schwarz, D., Topalidou-Kyniazopoulou, A., and Behnke, S. (2017). "NimbRo Rescue: Solving Disaster-response Tasks with the Mobile Manipulation Robot Momaro". *Journal of Field Robotics* 34.2, pp. 400–425.

Sell, J and O'Connor, P (2014). "The Xbox One System on a Chip and Kinect Sensor". *IEEE Micro* 34.2, pp. 44–53.

Semini, C., Tsagarakis, N. G., Guglielmino, E., Focchi, M., Cannella, F., and Caldwell, D. G. (2011). "Design of HyQ - A Hydraulically and Electrically Actuated Quadruped Robot". *Journal of Systems and Control Engineering* 225.6, pp. 831–849.

Shen, C., Zhang, Y., Li, Z., Gao, F., and Shen, S. (2017). "Collaborative Air-Ground Target Searching in Complex Environments". In: *IEEE International Symposium on Safety, Security and Rescue Robotics (SSRR)*.

Siciliano, B. and Khatib, O. (2016). *Springer Handbook of Robotics*. Springer.

Smith, R., Self, M., and Cheeseman, P. (1987). "A Stochastic Map For Uncertain Spatial Relationships". In: *International Symposium on Robotic Research*, pp. 467–474.

Stefanik, K. V., Gassaway, J. C., Kochersberger, K., and Abbott, A. L. (2011). "UAV-Based Stereo Vision for Rapid Aerial Terrain Mapping". *GIScience & Remote Sensing* 48.1, pp. 24–49.

Stelzer, A., Hirschmuller, H., and Gorner, M. (2012). "Stereo-vision-based navigation of a six-legged walking robot in unknown rough terrain". *The International Journal of Robotics Research* 31.4, pp. 381–402.

Stentz, A., Herman, H., Kelly, A., Meyhofer, E., Haynes, G. C., Stager, D., Zajac, B., Bagnell, J. A., Brindza, J., and Dellin, C. (2015). "CHIMP, the CMU Highly Intelligent Mobile Platform". *Journal of Field Robotics* 32.2, pp. 209–228.

Stentz, T., Kelly, A., Herman, H., Rander, P., and Amidi, O. (2002). "Integrated Air / Ground Vehicle System for Semi- Autonomous Off-Road Navigation". In: *AUVSI Symposium on Unmanned Systems*.

Stroom, J. C., De Boer, H. C. J., Huizenga, H., and Visser, A. G. (1999). "Inclusion of Geometrical Uncertainties in Radiotherapy Treatment Planning by Means of Coverage Probability". *International Journal of Radiation Oncology Biology Physics* 43.4, pp. 905–919.

Stumpf, A., Kohlbrecher, S., Conner, D. C., and Stryk, O. V. (2014). "Supervised Footstep Planning for Humanoid Robots in Rough Terrain Tasks using a Black Box Walking Controller". In: *IEEE-RAS International Conference on Humanoid Robots (Humanoids)*, pp. 287–294.

Takahiro, M., Fankhauser, P., and Bjelonic, M. (2017). "Error Analysis of the RealSense ZR300, Obstacle Removal for Elevation Mapping, and Swing Trajectory Planning for ANYmal". Student's Project. ETH Zurich, Switzerland.

Thrun, S. (2002). "Robotic Mapping: A Survey". In: *Exploring artificial intelligence in the new millennium*. Ed. by G. Lakemeyer and B. Nebel.

Tretyakov, V. and Linder, T. (2011). "Range sensors evaluation under smoky conditions for robotics applications". In: *IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR)*, pp. 215–220.

Vidal-Calleja, T. A., Berger, C., Solà, J., and Lacroix, S. (2011). "Large Scale Multiple Robot Visual Mapping with Heterogeneous Landmarks in Semi-Structured Terrain". *Robotics and Autonomous Systems* 59.9, pp. 654–674.

Vukobratović, M. and Borovac, B. (2004). "Zero-moment point – thirty five years of its life". *International Journal of Humanoid Robotics* 1.01, pp. 157–173.

Wagner, L., Fankhauser, P., Bloesch, M., and Hutter, M. (2016). "Foot Contact Estimation for Legged Robots in Rough Terrain". In: *International Conference on Climbing and Walking Robots (CLAWAR)*.

Wermelinger, M., Fankhauser, P., Diethelm, R., Krüsi, P., Siegwart, R., and Hutter, M. (2016). "Navigation Planning for Legged Robots in Challenging Terrain". In: *IEEE International Conference on Intelligent Robots and Systems (IROS)*, pp. 1184–1189.

Wettergreen, D. (1995). “Robotic walking in natural terrain: Gait planning and behavior-based control for statically-stable walking robots”. Doctoral Thesis. Carnegie Mellon University, Pittsburgh, PA, United States.

Wiedebach, G. (2016). “Whole Body Climbing with Legged Robots”. Master’s thesis. ETH Zurich, Switzerland.

Wiedemeyer, T. (2014–2018). *IAI Kinect2*. `https://github.com/code-iai/iai_kinect2`, accessed August 18, 2018.

Winkler, A. W., Mastalli, C., Focchi, M., Caldwell, D. G., and Havoutis, I. (2015). “Planning and Execution of Dynamic Whole-Body Locomotion for a Hydraulic Quadruped on Challenging Terrain”. In: *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 5148–5154.

Winkler, A. W., Farshidian, F., Neunert, M., Pardo, D., and Buchli, J. (2017). “Online Walking Motion and Foothold Optimization for Quadruped Locomotion”. In: *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 5308–5313.

Wooden, D., Malchano, M., Blankespoor, K., Howard, A., Rizzi, A., and Raibert, M. (2010). “Autonomous Navigation for BigDog”. In: *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 4736–4741.

Yang, L., Zhang, L., Dong, H., Alelaiwi, A., and Saddik, A. E. (2015). “Evaluating and Improving the Depth Accuracy of Kinect for Windows v2”. *IEEE Sensors Journal* 15.8, pp. 4275–4285.

Zanuttigh, P., Mutto, C. D., Minto, L., Marin, G., Dominio, F., and Cortelazzo, G. M. (2016). “Operating Principles of Structured Light Depth Cameras”. In: *Time-of-Flight and Structured Light Depth Cameras: Technology and Applications*. Springer. Chap. 2, pp. 43–79.

Zennaro, S. (2014). “Evaluation of Microsoft Kinect 360 and Microsoft Kinect One for robotics and computer vision applications”. Master’s thesis. University of Padova, Italy.

Zucker, M., Ratliff, N., Stolle, M., Chestnutt, J., Bagnell, J. A., Atkeson, C. G., and Kuffner, J. (2011). “Optimization and learning for rough terrain legged locomotion”. *The International Journal of Robotics Research* 30.2, pp. 175–191.

Zucker, M., Ratliff, N., Dragan, a. D., Pivtoraiko, M., Klingensmith, M., Dellin, C. M., Bagnell, J. a., and Srinivasa, S. S. (2013). “CHOMP: Covariant Hamiltonian optimization for motion planning”. *The International Journal of Robotics Research* 32.9-10, pp. 1164–1193.

Zucker, M., Joo, S., Grey, M. X., Rasmussen, C., Huang, E., Stilman, M., and Bobick, A. (2015). “A General-purpose System for Teleoperation of the DRC-HUBO Humanoid Robot”. *Journal of Field Robotics* 32.3, pp. 336–351.

Zug, S., Penzlin, F., Nguyen, T. T., and Albert, S. (2012). “Are laser scanners replaceable by Kinect sensors in robotic applications?” In: *Robotic and Sensors Environments (ROSE)*. IEEE.

Curriculum Vitae

Péter Fankhauser (born 1987 in Aarau, Switzerland) is co-founder and Chief Business Development Officer of the Zurich based startup company ANYbotics, developing autonomous mobile robots for harsh environments such as the legged robot ANYmal. During his doctoral degree at ETH Zurich under the supervision of Prof. Dr. Roland Siegwart and Prof. Dr. Marco Hutter, Péter developed mapping, planning, and control strategies for legged robots to climb over rough terrain (2013–2018). Péter holds a Bachelor's degree in Mechanical Engineering and a Master's degree in Robotics, Systems and Control from ETH Zurich (2007–2010, 2010–2012). During his studies, he worked at the Florida Institute for Human & Machine Cognition (IHMC) with Dr. Jerry Pratt and Dr. Peter Neuhaus on human exoskeletons and the DARPA Robotics Challenge and took courses at the Massachusetts Institute of Technology (MIT) and the Tohoku University in Japan. Previous work includes the modeling and control of unmanned co-axial helicopters and the development of the ball-balancing robot Rezero, which he presented at TED Global in 2011.

List of Publications

This section provides a list of publications that were published or submitted during these doctoral studies. For an up-to-date list, please see

https://purl.org/pfankhauser/publications

First Author Publications

- Fankhauser, P., Bloesch, M., and Hutter, M. (2018). "Probabilistic Terrain Mapping for Mobile Robots with Uncertain Localization". *IEEE Robotics and Automation Letters (RA-L)* 3.4, pp. 3019–3026
- Fankhauser, P., Bjelonic, M., Bellicoso, C. D., Miki, T., and Hutter, M. (2018). "Robust Rough-Terrain Locomotion with a Quadrupedal Robot". In: *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 5761–5768
- Fankhauser, P., Dario Bellicoso, C., Gehring, C., Dube, R., Gawel, A., and Hutter, M. (2016). "Free Gait – An Architecture for the Versatile Control of Legged Robots". In: *IEEE-RAS International Conference on Humanoid Robots (Humanoids)*, pp. 1052–1058
- Fankhauser, P. and Hutter, M. (2016). "A Universal Grid Map Library: Implementation and Use Case for Rough Terrain Navigation". In: *Robot Operating System (ROS): The Complete Reference (Volume 1)*. Ed. by A. Koubaa. Springer International Publishing. Chap. 5, pp. 99–120
- Fankhauser, P., Bloesch, M., Krusi, P., Diethelm, R., Wermelinger, M., Schneider, T., Dymczyk, M., Hutter, M., and Siegwart, R. (2016). "Collaborative Navigation for Flying and Walking Robots". In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 2859–2866
- Fankhauser, P., Bloesch, M., Rodriguez, D., Kaestner, R., Hutter, M., and Siegwart, R. (2015). "Kinect v2 for Mobile Robot Navigation: Evaluation and Modeling". In: *International Conference on Advanced Robotics (ICAR)*. IEEE, pp. 388–394
- Fankhauser, P., Bloesch, M., Gehring, C., Hutter, M., and Siegwart, R. (2014). "Robot-Centric Elevation Mapping with Uncertainty Estimates". In: *International Conference on Climbing and Walking Robots (CLAWAR)*, pp. 433–440

- Fankhauser, P., Hutter, M., Gehring, C., Bloesch, M., Hoepflinger, M. A., and Siegwart, R. (2013). "Reinforcement Learning of Single Legged Locomotion". In: *IEEE/RSJ International Conference on Intelligent Robot and Systems (IROS)*, pp. 188–193

Co-Authored Publications

- Bellicoso, C. D., Bjelonic, M., Wellhausen, L., Holtmann, K., Guenther, F., Tranzatto, M., Fankhauser, P., and Hutter, M. (2018). "Advances in Real-World Applications for Legged Robots". *(in review) Journal of Field Robotics*

- Bi, T., Fankhauser, P., Bellicoso, D., and Hutter, M. (2018). "Real-Time Dance Generation to Music for a Legged Robot". In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*

- Hutter, M., Diethelm, R., Bachmann, S., Fankhauser, P., Gehring, C., Tsounis, V., Lauber, A., Guenther, F., Bjelonic, M., Isler, L., Kolvenbach, H., Meyer, K., and Hoepflinger, M. (2017). "Towards a Generic Solution for Inspection of Industrial Sites". In: *International Conference on Field and Service Robotics (FSR)*

- Hutter, M., Gehring, C., Lauber, A., Gunther, F., Bellicoso, C. D., Tsounis, V., Fankhauser, P., Diethelm, R., Bachmann, S., Bloesch, M., Kolvenbach, H., Bjelonic, M., Isler, L., and Meyer, K. (2017). "ANYmal – Toward Legged Robots for Harsh Environments". *Advanced Robotics*. Advanced Robotics 31.17, pp. 918–931

- Bellicoso, C. D., Jenelten, F., Fankhauser, P., Gehring, C., Hwangbo, J., and Hutter, M. (2017). "Dynamic Locomotion and Whole-Body Control for Quadrupedal Robots". In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 3359–3365

- Gehring, C., Dario Bellicoso, C., Fankhauser, P., Coros, S., and Hutter, M. (2017). "Quadrupedal Locomotion Using Trajectory Optimization and Hierarchical Whole Body Control". In: *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 4788–4794

- Gonon, D., Jud, D., Fankhauser, P., and Hutter, M. (2017). "Safe Self-Collision Avoidance for Versatile Robots based on Bounded Potentials". In: *International Conference on Field and Service Robotics (FSR)*

- Käslin, R., Fankhauser, P., Stumm, E., Taylor, Z., Mueggler, E., Delmerico, J., Scaramuzza, D., Siegwart, R., and Hutter, M. (2016). "Collaborative Localization of Aerial and Ground Robots through Elevation Maps". In: *International Symposium on Safety, Security, and Rescue Robotics (SSRR)*
- Wermelinger, M., Fankhauser, P., Diethelm, R., Krüsi, P., Siegwart, R., and Hutter, M. (2016). "Navigation Planning for Legged Robots in Challenging Terrain". In: *IEEE International Conference on Intelligent Robots and Systems (IROS)*, pp. 1184–1189
- Belter, D., Labecki, P., Fankhauser, P., and Siegwart, R. (2016). "RGB-D Terrain Perception and Dense Mapping for Legged Robots". *International Journal of Applied Mathematics and Computer Science (AMCS)* 26.1, pp. 81–97
- Bloesch, M., Sommer, H., Laidlow, T., Burri, M., Nützi, G., Fankhauser, P., Bellicoso, D., Gehring, C., Leutenegger, S., Hutter, M., and Siegwart, R. (2016). *A Primer on the Differential Calculus of 3D Orientations*. Tech. rep. arXiv: `1606.05285`
- Bellicoso, C. D., Gehring, C., Hwangbo, J., Fankhauser, P., and Hutter, M. (2016). "Perception-less Terrain Adaptation through Whole Body Control and Hierarchical Optimization". In: *IEEE-RAS International Conference on Humanoid Robots (Humanoids)*, pp. 558–564
- Hutter, M., Gehring, C., Jud, D., Lauber, A., Bellicoso, C. D., Tsounis, V., Hwangbo, J., Bodie, K., Fankhauser, P., Bloesch, M., Diethelm, R., Bachmann, S., Melzer, A., and Hoepflinger, M. (2016). "ANYmal – A Highly Mobile and Dynamic Quadrupedal Robot". In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*
- Gehring, C., Coros, S., Hutter, M., Bellicoso, C. D., Heijnen, H., Diethelm, R., Bloesch, M., Fankhauser, P., Hwangbo, J., Hoepflinger, M. A., and Siegwart, R. (2016). "Practice Makes Perfect: An Optimization-Based Approach to Controlling Agile Motions for a Quadruped Robot". *IEEE Robotics & Automation Magazine* 23.1, pp. 34–43
- Wagner, L., Fankhauser, P., Bloesch, M., and Hutter, M. (2016). "Foot Contact Estimation for Legged Robots in Rough Terrain". In: *International Conference on Climbing and Walking Robots (CLAWAR)*
- Hwangbo, J., Bellicoso, C. D., Fankhauser, P., and Hutter, M. (2016). "Probabilistic Foot Contact Estimation by Fusing Information from Dy-

namics and Differential/Forward Kinematics". In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, pp. 3872 –3878

- Blöchliger, F., Blösch, M., Fankhauser, P., Hutter, M., and Siegwart, R. (2016). "Foot-Eye Calibration of Legged Robot Kinematics". In: *International Conference on Climbing and Walking Robots (CLAWAR)*, pp. 420 –427

- Gehring, C., Bellicoso, C. D., Coros, S., Bloesch, M., Fankhauser, P., Hutter, M., and Siegwart, R. (2015). "Dynamic Trotting on Slopes for Quadrupedal Robots". In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 5129–5135

- Hwangbo, J., Gehring, C., Bellicoso, D., Fankhauser, P., Siegwart, R., and Hutter, M. (2015). "Direct State-to-Action Mapping for High DOF Robots Using ELM". in: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*

- Haldane, D. W., Fankhauser, P., Siegwart, R., and Fearing, R. S. (2014). "Detection of Slippery Terrain with a Heterogeneous Team of Legged Robots". In: *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 4576–4581

- Gehring, C., Coros, S., Hutter, M., Bloesch, M., Fankhauser, P., Hoepflinger, M., and Siegwart, R. (2014). "Towards Automatic Discovery of Agile Gaits for Quadrupedal Robots". In: *IEEE International Conference on Robotics and Automation*, pp. 4243–4248

- Bloesch, M., Omari, S., Fankhauser, P., Sommer, H., Gehring, C., Hwangbo, J., Hoepflinger, M., Hutter, M., and Siegwart, R. (2014). "Fusion of Optical Flow and Inertial Measurements for Robust Egomotion Estimation". In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 3102–3107

- Belter, D., Skrzypczyński, P., Walas, K., Fankhauser, P., Gehring, C., Hutter, M., Hoepflinger, M. A., and Siegwart, R. (2014). "Dynamic Simulation of Legged Robots Using a Physics Engine". In: *International Conference on Climbing and Walking Robots (CLAWAR)*. Poznan, Poland, pp. 567–574

- Hutter, M., Gehring, C., Bloesch, M., Hoepflinger, M. A., Fankhauser, P., and Siegwart, R. (2014). "Excitation and Stabilization of Pas-

sive Dynamics in Locomotion using Hierarchical Operational Space Control". In: *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 2977–2982

- Bloesch, M., Gehring, C., Fankhauser, P., Hutter, M., Hoepflinger, M. A., and Siegwart, R. (2013). "State Estimation for Legged Robots on Unstable and Slippery Terrain". In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 6058–6064

Workshops

- Fankhauser, P., Diethelm, R., Bachmann, S., Gehring, C., Wermelinger, M., Bellicoso, D., Tsounis, V., Lauber, A., Bloesch, M., Leemann, P., Hottiger, G., Jud, D., Kaestner, R., Isler, L., Hoepflinger, M., Siegwart, R., and Hutter, M. (2016). "ANYmal at the ARGOS Challenge – Tools and Experiences from the Autonomous Inspection of Oil & Gas Sites with a Legged Robot". In: *ROSCon*. Seoul, South Korea: Open Source Robotics Foundation
- Fankhauser, P., Bellicoso, D., Gehring, C., Wiedebach, G., and Hutter, M. (2016). "Free Gait: Versatile Control of Legged Robots". In: *Workshop on Dynamic Locomotion and Manipulation*. Zurich, Switzerland
- Fankhauser, P. and Hutter, M. (2015). "Localization, Terrain Mapping, and Navigation with StarlETH". in: *Workshop on Perception and Planning for Legged Robot Locomotion in Challenging Domains, IEEE International Conference on Intelligent Robots and Systems (IROS)*. Hamburg, Germany
- Fankhauser, P., Hutter, M., Bloesch, M., and Siegwart, R. (2013). "Optimizing Robotic Single Legged Locomotion with Reinforcement Learning". In: *Dynamic Walking Meeting*. Pittsburgh, United States
- Fankhauser, P., Hutter, M., Gehring, C., and Siegwart, R. (2013). "Optimizing Robotic Single Legged Locomotion with Reinforcement Learning". In: *Workshop on Novel Methods for Learning and Optimization of Control Policies and Trajectories for Robotics, IEEE Internation Conference on Robotics and Automation (ICRA)*. Karlsruhe, Germany

Lectures

- Fankhauser, P., Jud, D., Wermelinger, M., and Hutter, M. (2017). "Programming for Robotics – Introduction to ROS". Lecture Notes. ETH Zurich, Switzerland: ETH Zurich, Switzerland

List of Videos

This section provides a list of videos that were published during these doctoral studies. An online collection of the videos is available at

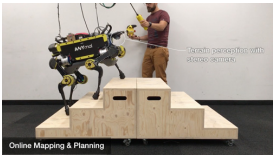
`https://purl.org/pfankhauser/phd-videos`

(V1) **Robot ANYmal Dancing to Live Music**
March, 2018
`https://youtu.be/kHBLaw5nfzk`

(V2) **Legged Robot ANYmal Climbing Stairs in Zurich**
January, 2018
`https://youtu.be/vSveQrJLRTo`

(V3) **ANYmal on Snow**
January, 2018
`https://youtu.be/O7Wge90mQ0c`

(V4) **ANYmal Using an Elevator**
October, 2017
`https://youtu.be/gM1z60aeunU`

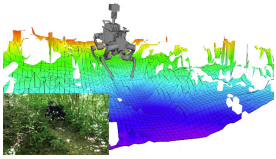


(V5) **Robust Rough-Terrain Locomotion with a Quadrupedal Robot (ANYmal)**
September, 2017
`https://youtu.be/CpzQu25iLa0`

(V6) **Free Gait Software Overview**
August, 2017
`https://youtu.be/PNJZvCCOmD8`

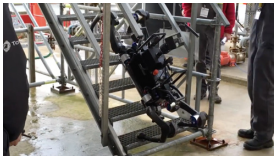
(V7) **ANYmal at ARGOS Finals – Highlights**
April, 2017
`https://youtu.be/2RQDp0Q2vSo`

(V8) **ANYmal for Search & Rescue**
November, 2016
`https://youtu.be/VAuEEJhQxF0`

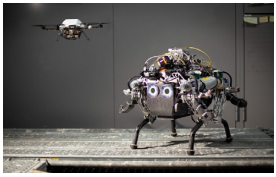


(V9) **Outdoor Terrain Mapping with ANYmal**
September, 2016
`https://youtu.be/iVMsQPTM65M`

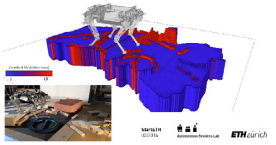
(V10) **ANYmal for the Oil & Gas Industry**
July, 2016
`https://youtu.be/SR5OJ-vklIs`



(V11) **ANYmal Free Gait Motions**
May, 2016
`https://youtu.be/EI1zBTYpXW0`



(V12) **Collaborative Navigation for Flying and Walking Robots**
October, 2015
`https://youtu.be/9PprNdIKRaw`



(V13) **Robot-Centric Elevation Mapping with StarlETH**
March, 2014
`https://youtu.be/I9eP8GrMyNQ`

List of Software

This section provides a list of open-source software that was published during these doctoral studies.

(S1) Grid Map

`https://github.com/anybotics/grid_map`

This is a C++ library with ROS interface to manage two-dimensional grid maps with multiple data layers. It is designed for mobile robotic mapping to store data such as elevation, variance, color, friction coefficient, foothold quality, surface normal, traversability, etc. The implementation supports a range of convenience functions for the simple and efficient handling of the data. Visualizations, filter pipelines, conversions from and to images, point clouds, polygonal meshes, signed distance fields (SDFs), occupancy grids, and more allow for flexible integration in existing software. A complete software usage description, tutorials, and use-case studies are given in Fankhauser and Hutter (2016).

(S2) Robot-Centric Elevation Mapping

`https://github.com/anybotics/elevation_mapping`

This is a ROS package developed for dense elevation mapping with a mobile robot. The software is designed for (local) navigation tasks with robots which are equipped with a pose estimation (e.g., IMU & odometry) and a distance sensor (e.g., structured-light, laser range sensor, stereo camera, etc.). The provided elevation map is limited around the robot and reflects the pose uncertainty that is aggregated through the motion of the robot (robot-centric mapping). This method is developed to explicitly handle drift of the robot pose estimation.

(S3) Free Gait

`https://github.com/leggedrobotics/free_gait`

Free Gait is a software framework for the versatile, robust, and task-oriented control of legged robots. The Free Gait interface defines a whole-body abstraction layer to accommodate a variety of task-space control commands such as end effector, joint, and base motions. The defined motion tasks are tracked with a feedback whole-body controller to ensure accurate and robust motion execution even under slip and external disturbances. The application of this framework includes intuitive teleoperation of the robot, efficient scripting of behaviors, and fully autonomous operation with motion and footstep planners.

List of Supervised Student Projects

This section provides a list of student projects that were supervised by the author as part of the doctoral studies.

Master's Theses

Six months full-time research project

- Homberger, T. (2018): "Support Surface Estimation for Legged Robots in Obstructive and Rugged Terrain"
- Roos, J. and Sütterlin, M. (2018): "Rezero with Arm: A Three Degrees of Freedom Manipulator Balancing on a Ball"
- Egger, K. (2016): "Collaborative Localisation for a Quadrupedal Robot Following a Person"
- Gonon, D. (2015): "Safe Collision Avoidance with Artificial Potentials and Energy Limiting"
- Gostimir, H. (2015): "Kinesthetic Teaching for a Torque Controllable Quadrupedal Robot"
- Henein, M. (2015): "Kinematic Planning for Challenging Maneuvers with a Legged Robot"
- Wermelinger, M. (2015): "Navigation Planning for Legged Robots in Challenging Terrain"
- Wiedebach, G. (2015): "Whole Body Climbing with Legged Robots"
- Heijnen, H. (2014): "Learning Jumping Maneuvers for a Quadrupedal Robot"

Semester Projects

Four months part-time research project

- Baumann, T. (2017): "Map-Based Dynamic Locomotion Over Uneven Terrain with a Quadrupedal Robot"
- Miki, T. (2017): "Elevation Mapping Improvements for Trajectory Planning"

- Jenelten, F. (2016): “Design of a Dynamic Lateral Walk for a Quadruped Robot”
- Käslin, R. (2016): “Collaborative Localization of Aerial and Ground Robots through Elevation Maps”
- Mohanadasan, T. (2016): “Autonomous Stair Climbing with Legged Robots”
- Rytz, D. (2016): “Footstep Planning for a Quadruped Robot”
- Vollprecht, W. (2016): “Deep Learning of Footholds for Rough Terrain Locomotion”
- Watanabe, S. (2016): “Design of a Control Strategy for a Quadruped Robot to Recover from Falls”
- Winter, B. (2016): “Sliding Mode Controller for a Two-Wheeled Balancing Wheelchair”
- Blöchliger, F. (2015): “Visual Calibration of Legged Robot Kinematics”
- Wagner, L. (2015): “Foot Contact Estimation for Legged Robots in Rough Terrain”
- Hottiger, G. (2014): “High Quality Terrain Scanning”
- Ko, W. (2014): “Learning To Jump With a Quadrupedal Robot”
- Rodriguez, D. (2014): “Kinect V2 Sensor for Mapping, Evaluation and Characterization”
- Wermelinger, M. (2014): “Stability Checking for Legged Rough Terrain Locomotion”
- Fotou, F. (2013): “Programming and Design of Control Methods for Human Interaction with a Ballbot”
- Schlestein, A. (2013): “Antagonistic Knee Actuator Design for StarlETH”

Bachelor's Theses

Four months part-time research project

- Bi, T. (2017): "Real-Time Dance Generation to Music for a Quadrupedal Robot"
- Gomes, C. (2015): "Localization and Navigation for a Self-Driving Wheelchair"
- Jenelten, F. and Forni, L. (2015): "Ball-on-Ball Stabilizing System"
- Käslin, R. and Winter, B. (2015): "Adaptive Transitions on and off Stairs for a Stair Climbing Wheelchair"
- Mariani, D. and Stähli, I. (2015): "System Modeling and Control for a Self-Balancing Wheelchair"
- Voellmy, M. (2015): "Stair Recognition and Automatic Alignment for a Stair Climbing Wheelchair"
- Achermann, F. (2014): "Blind Walking over Rough Terrain"
- Carius, J. (2014): "Dynamic Maneuvers with A Single Wheel Robot"
- Herbst, C. (2014): "Safe Navigation with a Heterogeneous Team of Legged Robots"
- Seyde, T. (2014): "Balancing on Two Legs with a Quadrupedal Robot"
- Lehmann, D. (2013): "Mechanical Design of a Rotor Tilt Mechanism for a VTOL UAV"
- Skrabel, C. (2013): "Mechanical Design of a Ballbot Platform"

Studies on Mechatronics

Four months part-time literature review

- Bi, T. (2016): "Stomping to Music with a Legged Robot"
- Forni, L. (2015): "Sensor Evaluation for Measuring the Position of a Moving Ball"

- Jenelten, F. (2015): "Advanced Control Techniques for the Task of Stabilizing the Two-Ball System"
- Achermann, F. (2014): "Blind Walking over Rough Terrain"
- Estandía Hentschel, A. (2014): "Online Foothold Evaluation for a Quadruped Robot"
- Carius, J. (2013): "Modeling and Design of Spherical Rolling Robots"
- Herbst, C. (2013): "Safe Navigation with a Heterogeneous Team of Legged Robots "
- Skrabel, C. (2013): "Mechanical Design of a Ballbot Platform"

Focus Project

Ten months product development project

- Gomes, C., Käslin, R., Mariani, D., Schilling, M., Stähli, I., Voellmy, M., Winter, B., Kühne, J., Gemperle, T., and Stieger, N. (2014/2015): "Scalevo – Stair Climbing Wheelchair"